From Pizza.owl to Executable Intelligence

# EXECUTABLE KNOWLEDGE ARCHITECTURE (EKA)

Mastering Ontology Engineering

*with Protégé and Pizza.owl*

## VOLUME 4b

Chapters 23-27

### SEMANTIC KNOWLEDGE ENGINEERING — PART 2

Semantic Reasoning, Executable Knowledge, and the SKDL Synthesis

**XIAOQI ZHAO**

# Chapters List -- Volume 4b


- Cover Page
- Chapter List -- This File
- Cover Chapter (README)
- ACKNOWLEDGEMENTS
- Legal & Licensing
- Foreword from Timothy W. Cook
- Foreword from Michael DeBellis
- Preface from the Author


## This Volume's Chapters (Volume 5 of 8)


- Chapter 23 -- Semantic Reasoning: Stage 6 of the Semantic Knowledge Development Lifecycle
- Chapter 24 - Stage 7 of SKDL -- Executable Knowledge (Part 1): Introduction & Boundary Control
- Chapter 25 - Stage 7 of SKDL -- Executable Knowledge (Part 2): Value Specification & Precise Binding
- Chapter 26 - Stage 7 of SKDL -- Executable Knowledge (Part 3): Data Integrity & Comprehensive Architecture
- Chapter 27 - SKDL Synthesis & Volume 4 Conclusion


## Full Book Roadmap

- Volume 1 -- Chapters 00-08: Ontology Foundations in Protégé
- Volume 2 -- Chapters 09-13: Object Properties, Characteristics, Domain and Range
- Volume 3 -- Chapters 14-16: Semantic Requirements and EKA Governance
- Volume 4a -- Chapters 17-22: Semantic Knowledge Engineering, Part 1
- **Volume 4b (this book)** -- Semantic Knowledge Engineering, Part 2 (In writing...)
- Volume 5: EKA Part 1 — Executable Knowledge Implementation (In Plan)
- Volume 6: EKA Part 2a — Advanced Reasoning & Interdisciplinary Depth (In Plan)
- Volume 7: EKA Part 2b — Synthesis & Methodology (In Plan)

## Appendix


- Appendix A - Chapter mapping with Exercises

- Contributors

Last Updated at 2026-10-17

# Executable Knowledge Architecture (EKA)

## *Mastering Ontology Engineering with Protégé and Pizza.owl*

Table of Content


- From Ontology Learning to Executable Knowledge Architecture
- Why the Pizza.owl Tutorial Matters
- Why This Book?
- About This Book
- Why Ontology Matters Today
- Ontology Within the EKA Framework
- This Book Is NOT Only About Pizza
- Who This Book Is For
- What Makes This Book Different
- How to Use This Book - The Learning Journey
- Structural Integrity
- Acknowledgements
  - Our Intellectual Heritage & Licensing
  - Special Thanks
- Related Resources
- Final Thoughts


# From Ontology Learning to Executable Knowledge Architecture

The Semantic Web has existed for more than two decades, yet for many architects, engineers, analysts, and AI practitioners, ontology engineering still feels abstract, academic, or difficult to approach systematically.

One of the reasons is simple:

> Most ontology learning materials focus either too heavily on theory or too narrowly on tool operations.

As a result, many learners can:

- remember OWL terminology,
- click through Protégé demonstrations,
- or understand isolated semantic concepts,

but still struggle to answer the most important question:

> Why does ontology engineering actually matter in modern enterprise and AI systems?

This book was created to answer that question.

# Why the Pizza.owl Tutorial Matters

Among all ontology learning materials ever created, Michael DeBellis' Pizza OWL tutorial remains one of the most influential and practical introductions to OWL ontology engineering.

Using a familiar pizza domain, the tutorial teaches learners how to:

- think semantically,
- construct ontology hierarchies,
- model relationships,
- apply OWL logic (rules), and
- understand reasoning behavior inside Protégé

What makes the Pizza tutorial special is that it gradually introduces semantic complexity in a highly structured manner.

Rather than overwhelming learners immediately with advanced ontology theory, it guides readers step by step through the evolution of a real semantic model.

That educational philosophy strongly influenced the structure of this book.

This eBook serves as the comprehensive companion guide to the Protégé OWL Pizza Tutorial Hands-on Series

# Why This Book?

Most ontology materials focus either on abstract theory or narrow tool operations. This book bridges that gap by connecting semantic engineering to the **Executable Knowledge Architecture (EKA)** framework.

It is designed to help you engineer machine-understandable meaning for AI systems, enterprise knowledge platforms, and digital twins.

# About This Book

This eBook was written as a companion knowledge guide to the YouTube playlist:

```
**Protégé OWL Pizza Tutorial Hands-on Series**
(https://www.youtube.com/playlist?list=PL6DEHvciXKeUx4P32B3hKMK1t6mC8RhsW)
```

*Since publishing the Protégé OWL Pizza Tutorial series in 2023, I've realized how valuable practical ontology learning remains for architects and AI practitioners. What started as a hands-on walkthrough of Michael DeBellis' tutorial gradually became an important foundation for my EKA (Executable Knowledge Architecture) thinking — connecting ontology, knowledge graphs, and executable intelligence. The playlist may use pizza as the example domain, but its real purpose is helping learners develop semantic thinking and understand how machine-readable knowledge is engineered.*

However, the goal is not merely to transcribe the videos.

Instead, this book expands the tutorial into a much deeper professional learning experience by combining:

- ontology engineering theory,
- hands-on Protégé practice,
- semantic modeling methodology,
- enterprise architectural thinking,
- and modern knowledge graph perspectives.

Every chapter corresponds closely to the learning progression of the original video series so readers can learn both visually and conceptually in parallel.

This synchronization is intentional.

Ontology engineering is best learned progressively.

# Why Ontology Matters Today

We are entering a new era of intelligent systems.

Modern enterprises increasingly rely on:

- AI systems,
- semantic search,
- enterprise knowledge graphs,
- digital twins,
- intelligent automation,
- contextual reasoning,
- and machine-readable knowledge.

Traditional architectures are built purely on:

- servers and databases,
- documents,
- diagrams
- and APIs

are no longer sufficient for advanced semantic intelligence.

The missing layer here is:

```
Meaning
```

Ontology engineering provides that layer.

OWL ontologies allow organizations to formally define:

- concepts,
- relationships,
- constraints,
- semantics,
- and inference logic

in a machine-readable and machine-processable form.

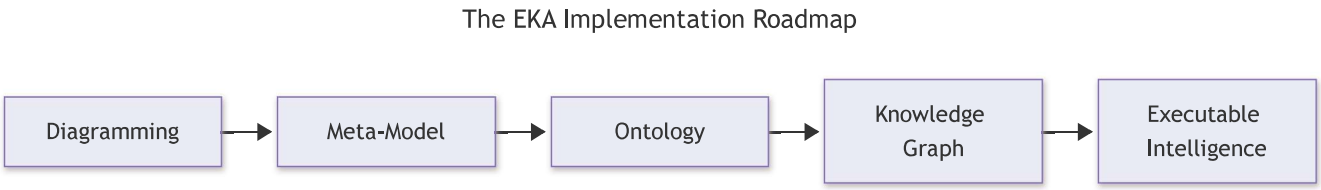
This transforms static information into executable knowledge.

# Ontology Within the EKA Framework

This book also introduces ontology engineering through the lens of the EKA (Executable Knowledge Architecture) framework.

EKA represents an architectural approach for transforming enterprise knowledge into executable intelligence systems.

Within EKA, the implementation roadmap is:




Ontology occupies the critical semantic transformation layer.

This is where:

- visual architecture,
- conceptual models,
- and enterprise abstractions

become:

- machine-readable semantics,
- reasoning structures,
- and graph-ready knowledge.

Protégé therefore becomes more than just an ontology editor.

It becomes an engineering environment for executable semantics.

This perspective fundamentally changes how ontology engineering should be understood.

# This Book Is NOT Only About Pizza

Although the tutorial domain uses pizza examples, the real purpose of the tutorial is much larger.

The Pizza ontology teaches foundational semantic engineering patterns that scale into real-world domains such as:

- enterprise architecture,
- healthcare,
- finance,
- manufacturing,
- government,

- cybersecurity,
- digital twins,
- and AI knowledge systems.

The same semantics principles used to model:

```
(Pizza)-[:hasTopping]->(CheeseTopping)
```

can later scale into:

```
(Application)-[:supportsCapability]->(businessFunction)
```

or:

```
(Device)-[:connectedTo]->(Sensor)
```

Ontology engineering is domain-independent!

The Pizza example simply provides an approachable learning environment.

# Who This Book Is For

This book is intended for:

- Enterprise Architects
- Solution Architects
- Data Architects
- Knowledge Engineers
- AI Engineers
- Semantic Web Learners
- Knowledge Graph Practitioners
- Protégé Beginners
- Meta-Modeling Practitioners
- EKA Learners
- Digital Transformation Professionals

No prior ontology experience is required.

However, readers with backgrounds in:

- UML,
- ER Modeling,
- Enterprise Architecture,
- Databases,
- Graph Technologies,
- or Software Engineering

will often recognize important conceptual parallels throughout the book.

# What Makes This Book Different

Most ontology books fall into one of two extremes:

| Style | Problem |
|---|---|
| Highly academic | Difficult for practitioners |
| Pure tool tutorials | Lack conceptual depth |

This book intentionally bridges both worlds.

The objective is to create `A Practical Ontology Engineering Handbook` that remains:

- technically rigorous,
- professionally structured,
- architecturally meaningful,
- and implementation-oriented.

The book therefore combines:

- OWL theory,
- Protégé operations,
- semantic engineering methodology,
- EKA architecture thinking,
- and knowledge graph perspectives

into a single progressive learning journey.

# How to Use This Book - The Learning Journey

The recommended learning approach is:

1. **Watch**: The corresponding YouTube video chapter.
2. **Read**: The matching eBook chapter for deeper conceptual context.
3. **Practice**: Manual Protégé exercises and hands-on semantic modeling.
4. **Reflect**: Map these semantic principles to your own enterprise architecture.

Ontology engineering is not mastered through passive reading alone.

It requires semantic thinking practice.

The goal is not merely learning Protégé.

The goal is learning how to engineer machine-understandable meaning.

# Structural Integrity

- **Foreword**: By Timothy W. Cook (Founder of SDC).
- **Core Curriculum**: Covers everything from basic class hierarchies and object properties to advanced SHACL, SPARQL, and reasoning behavior.
- **Error Tracker**: An included log to help you troubleshoot common modeling pitfalls.

# Acknowledgements

This book has benefited greatly from the generous feedback, technical reviews, and professional discussions contributed by members of the ontology and semantic technologies community.

The full acknowledgements are maintained here: ACKNOWLEDGEMENTS.md

## Our Intellectual Heritage & Licensing

This work is a direct descendant of the **Protégé 4 Tutorial (version 1.3) by Matthew Horridge**. We are deeply indebted to the original visionaries who developed the preceding versions and established the core teaching material: **Holger Knublauch, Alan Rector, Robert Stevens, Chris Wroe, Simon Jupp, Georgina Moulton, Nick Drummond, and Sebastian Brandt.**

In this modern adaptation, we have incorporated **Michael DeBellis's** revisions, which successfully bridged the transition to Protégé 5.5 and integrated advanced practices such as SWRL, SPARQL, and SHACL. Furthermore, we acknowledge the contributions of **Lorenz Buehmann, André Wolski, Dick Ooms, Colin Pilkington, Livia Pinera, Jans Aasman, Yan Xu, and the team at Franz Inc.**, whose work in applying these ontologies to real-world triple stores like AllegroGraph and Gruff has significantly shaped our current perspective on **Executable Knowledge Architecture (EKA)**.

All educational content in this eBook is licensed under **CC BY-SA 4.0**.

*For full legal details, please see the `README.md` and `LICENSE.md` files in the project root.*

## Special Thanks

We extend our sincere gratitude to **Michael DeBellis** for creating one of the most influential practical OWL learning resources available to the ontology community. (Visit Michael's homepage here - https://www.michaeldebellis.com/). We also thank him for his meticulous technical audit for this series; his expert insights -- particularly regarding the crucial engineering distinction between modeling tool and persistent graph databases -- have been instrumental in ensuring this content meets the standards of modern production-grade environments.

We are also honored to feature a foreword and ongoing review by **Timothy Cook (Founder of SDC)**. His guidance and endorsement have been vital in validating our approach, helping to bridge the gap between traditional semantic modeling and the sophisticated demands of modern enterprise architecture / knowledge architecture.

Finally, we thank **Stanford University and the Protégé community** for their enduring commitment to advancing open ontology engineering tools and Semantic Web technologies.

By preserving this attribution chain, we honor the intellectual legacy of the individuals and organizations that have sustained this field. We stand on the shoulders of these giants, aiming to provide a bridge from traditional semantic modeling to the future of `Executable Knowledge Architecture`.

# Related Resources

Official Resources:

- Protégé Official Website: https://protege.stanford.edu/
- Protégé Pizza Repository: https://github.com/yasenstar/protege_pizza/tree/main
- EKA Official Website: https://xiaoqi.com/

Learning Resources:

- YouTube Channel - Xiaoqi(Yasen) Zhao: http://www.youtube.com/@yasenzhao
- Udemy Course - Xiaoqi Zhao: https://www.udemy.com/user/xiaoqi-zhao

# Final Thoughts

Ontology engineering is NO LONGER a niche academic discipline.

It is rapidly becoming part of the foundational infrastructure for:

- AI systems,
- enterprise knowledge platforms,
- semantic interoperability,
- and executable intelligence architectures.

Understanding ontology today is increasingly similar to understanding databases or APIs twenty years ago.

It is becoming a core architectural capability.

The journey begins with a simple Pizza ontology.

But the destination is much larger:

```
Executable knowledge
```

Welcome to the World of Ontology Engineering!

---

> "You" are the learner of this book, so while you're reading, I'll say to "you" directly instead of "learner" / "reader" from now on.

---

Last updated at: 2026-10-18

# Acknowledgements

This project would not have reached its current level of maturity without the generosity of numerous members of the ontology, semantic web, enterprise architecture, and knowledge graph communities.

Many people contributed through technical discussions, detailed chapter review, engineering suggestions, issue reports, and encouragement throughout the writing process.

The following individuals deserve special recognition.

## Michael DeBellis

Author of the original *Protégé 5 New OWL Pizza Tutorial*.

This book is built upon Michael's excellent tutorial, which has served for many years as one of the most practical introductions to OWL ontology engineering.

Beyond creating the original tutorial, Michael has kindly provided valuable feedback on the direction of this book and has been supportive of extending the tutorial into a broader engineering perspective through the Semantic Knowledge Development Lifecycle (SKDL) and Executable Knowledge Architecture (EKA).

Without his original educational work, this project would not exist.

## Timothy W. Cook

Founder & CEO, Axius SDC, Inc.

Tim has generously reviewed multiple chapters of this book during its development, particularly those concerning semantic governance, ontology engineering practice, and enterprise-scale knowledge systems.

His feedback has helped strengthen several important engineering themes throughout the SKDL framework, including:

- treating ontology as a meaning-driven system rather than merely a validation system;
- distinguishing semantic inference from policy enforcement;
- clarifying the role of semantic governance within long-lived knowledge systems;
- improving the engineering narrative surrounding reusable semantic knowledge.

His perspective comes from extensive real-world work in semantic interoperability, healthcare data standards, and enterprise semantic governance.

I am deeply grateful for his thoughtful reviews, constructive criticism, and continued encouragement throughout the writing of this book.

# The Ontology Community

Many members of the broader ontology and semantic web community have contributed indirectly through discussions, questions, publications, conference presentations, open-source software, and years of accumulated research.

Protégé, OWL, RDF, Description Logic, and the Semantic Web ecosystem are the result of decades of collaborative work by thousands of researches and practitioners.

This book stands on the collective foundation.

# Readers - YOU

Finally, sincere thanks to every reader who as reported errors, suggested improvements, opened GitHub issues, starred the repository, shared the project with others, or simply spent time learning ontology engineering through this book.

Every question, correction, and discussion helps improve future editions.

Knowledge grows through reuse, collaboration, and continuous refinement -- the very principles promoted throughout this book.

Last updated at 2026-09-11

# Legal & Licensing

This eBook is a derivative work of the " `Pizza.owl` Tutorial", a foundational resource for the ontology engineering community.

We are committed to transparency and the preservation of the intellectual legacy established by the original authors.

## Licensing Terms

This work is governed by a dual-licencing strategy to balance educational openness with software best practices:

- **Documentation & Educational Content**: All textual content, diagrams, and educational explanations within this eBook are licensed under the **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** license.
  - *Requirement:* Any derivative works based on this educational content must maintain the non-commercial and Attribution-ShareAlike terms as required by the original authors.
- **Software & Scripts**: The associated code artifacts, scripts, and automation tools accompanying this eBook are provided under the **GNU General Public License v3.0 (GPL-3.0)**.

## Attribution & Intellectual Heritage

This work stands on the shoulders of the community that created the original Protégé 4 Pizza Tutorial. We acknowledge the visionary contributions of **Matthew Horridge, Holger Knublauch, Alan Rector, Robert Stevens, Chris Wroe, Simon Jupp, Georgina Moulton, Nick Drummond, and Sebastian Brandt.**

We also extend our gratitude to **Michael DeBellis** for his invaluable modern revisions, technical audit, and his ongoing dedication to providing practical OWL learning resources for the ontology community.

For further details regarding the full attribution chain, the history of this project, or the specific licensing of individual files, please refer to the README.md and LICENSE.md files in the official GitHub repository (https://github.com/yasenstar/protege_pizza).

Last updated at 2026-09-11

# Foreword from Timothy W. Cook

For more than twenty years, ontology engineering has carried a quiet reputation: powerful in theory, hard to reach in practice. Michael DeBellis' Pizza tutorial did more than almost any other resource to change that, by letting people learn semantics with their hands instead of only their heads. What Xiaoqi Zhao has done here is take that well-loved starting point and carry it all the way to where the work actually has to live, which is inside real systems, under real governance.

This book earns its subtitle. In a single progression it moves from the first class you build in Protégé to the question every enterprise eventually faces: how machine-readable meaning becomes something a system can act on safely. Along the way it never loses the teacher's discipline of one idea at a time.

I want to point readers in particular at Chapters 12 and 13, because their titles tell you something. They do not *only* say "object property characteristics" and "domain and range." They say *governing* semantic relationship behavior and *governing* semantic boundaries. That word is the whole game. A functional or transitive property is not a checkbox in a tool. It is a rule about how a relationship is allowed to behave. A domain and a range are not decoration. They are a statement about what may legitimately connect to what. Xiaoqi teaches these as governance, and that is exactly the right approach.

Chapter 13 also does something I rarely see in an introductory text: it tells the truth about a sharp edge. OWL domain and range do not reject a bad assertion the way a database constraint would. The reasoner infers, under an open-world assumption, rather than refuses, and as the chapter notes, this surprises newcomers who expected enforcement. That surprise is where academic ontology meets the enterprise information system. The moment data leaves the editor and travels between systems, a boundary that only infers is not yet a boundary you can trust. It has to be able to refuse what does not belong, deterministically, with its rules bound into the data rather than living in a separate document or a separate dashboard.

That is the work my colleagues and I have spent years on in the Semantic Data Charter. SDC treats meaning and the constraint definitions as shareable, bound assets. They are one inseparable data payload, so admissibility can be checked at the point of use, in the open, against published specifications. Readers will recognize this as related to the layer Xiaoqi names Γ in his EKA tuple: the governance layer that turns a clever model into a trustworthy one. EKA is the architecture that orchestrates the pieces; a substrate like SDC is one open, standards-based way to make that governance layer real in production. The two meet exactly where this book spends its most careful chapters.

There is a larger reason to welcome a book like this now. Ontology engineering is becoming a core architectural capability, the way databases and APIs did before it, and the field is poorly served when it stays split between texts that are rigorous but unreachable and tutorials that are reachable but shallow. Xiaoqi has taken the harder middle path: technically honest, professionally structured, and grounded in the standards that real organizations have to answer to. That is the bridge between theory and practice, and we need more people building it.

Read this book with Protégé open. Do the exercises. And when you reach those governing chapters, remember that the discipline you are learning on a pizza is the same discipline that will one day decide whether a clinical, financial, or regulatory system is allowed to act. As Xiaoqi writes, meaningful intelligence depends on meaningful boundaries. This book teaches you how to build them.

Timothy W. Cook
Founder, Axius SDC, Inc.
Creator, the Semantic Data Charter (SDC)
June 2026

Last Updated at 2026-09-11

# Preface from Michael DeBellis

I sometimes find it easy to be pessimistic about the human race. The world gives us plenty of reasons for discouragement. But one thing that illustrates some of the best in people is the open source movement.

It is remarkable how much of the software infrastructure that powers the modern internet is open source. The Linux operating system, Apache web servers, Kubernetes, Kafka, Docker containers, and countless programming languages, libraries, databases, and development tools were created as open-source projects or converted from internal projects to software freely available to all developers. Organizations and individuals understand that software becomes more valuable when it is shared openly so that the developer community can study it, improve it, and build on it. It's a very positive thing that corporations such as Google and LinkedIn are motivated to contribute software in which they've invested heavily into the open source infrastructure. Even more so that so many individuals will contribute their time to maintain and expand open source tools for the simple joy of seeing things work and providing tools that colleagues will find useful.

That generosity is not limited to code. Documentation, examples, tutorials, walkthroughs, bug reports, and teaching materials are also essential parts of the open-source ecosystem. They are just as important, because good software is wasted if no one can understand or learn to use it.

Tutorials are often underappreciated. As I said when I first created my revised version of the Protégé Pizza tutorial, no one gets a PhD for writing a tutorial. Tutorials do not usually receive the same professional recognition as research papers, software frameworks, or commercial products. Yet, for many learners, a good tutorial is the difference between a technology remaining abstract vs. becoming usable.

That is why I am pleased to see Xiaoqi Zhao's *Mastering Ontology Engineering with Protégé and Pizza.owl: From Semantic Foundations to Executable Knowledge Architecture (EKA)*. It continues a tradition that has always been central to the Semantic Web community: building on prior work, acknowledging intellectual lineage, and helping new learners move from abstract ideas to practical results.

The Pizza tutorial itself has a long history. The original tutorial was developed by members of the Protégé and ontology engineering community. It became one of the best-known introductions to OWL ontology development. Its genius was the choice of a domain that was familiar enough to get out of the learner's way. Almost everyone understands pizzas, toppings, bases, vegetarian choices, and spicy

ingredients. Because the domain is simple, learners can focus on the real subject: classes, properties, axioms, inference, queries, and rules.

My own revision of the Pizza tutorial was motivated by a practical teaching need. I was preparing a hands-on workshop and realized that the older tutorial no longer matched the current Protégé user interface closely enough for beginners. Especially since the majority of the students in that workshop were Library Science experts, not developers. Experienced OOP developers can translate instruction such as “add superclass” into revised user interfaces such as “add subclassOf”. New learners, especially those coming from fields such as library science and information architecture, should not have to do that translation while also trying to understand the concepts of OWL and other Semantic Web languages. The tutorial needed to match the actual tool.

What began as a small update became a much larger project. I updated the examples for Protégé 5, revised screenshots and instructions, and added material on topics such as SPARQL, SHACL, SWRL, IRIs, namespaces, and WebProtégé. My goal was not just to teach people where to click, but to help them understand why the clicks mattered.

Xiaoqi’s eBook takes that work in a new direction. It uses the Pizza tutorial as a foundation, but it is not simply a transcription or repetition of the older material. It connects OWL ontology engineering to a broader architectural vision that he calls Executable Knowledge Architecture (EKA).The eBook encourages learners to see ontology development not as an isolated academic exercise, but as part of a larger architecture for formal knowledge, knowledge graphs, reasoning, validation, and intelligent systems.

I’ve spent half of my career as a consultant and the other half doing research. One of the most important lessons I learned as a consultant is that the hard part isn’t writing software, the hard part is integration. That’s true whether we’re integrating rules with COBOL programs, LLMs with enterprise data, or ontologies and knowledge graphs into distributed computing environments.

That broader perspective is timely.

For a while, it seemed that the Semantic Web might remain a powerful idea that never achieved its hoped-for impact. The original vision was ambitious: not just a web of documents, but a web of data and meaning, where machines could understand and integrate information across sources. The challenge was that adding semantics requires work. It requires modeling. It requires shared identifiers. It requires explicit commitments about meaning. It requires different groups in large organizations to talk to each other and define shared concepts and domain boundaries.

Many organizations decided that this was too much effort. It was easier to publish documents, tables, and data dumps and let humans, search engines, and especially machine learning systems make sense

of them.

Ironically, the rise of Large Language Models (LLMs) has created renewed interest in the very technologies that some people thought machine learning would make unnecessary. LLMs are extraordinary tools. Used well, they can provide enormous productivity benefits. But they also have well-known limitations. Their reasoning is opaque. They can generate fluent but incorrect answers. They may struggle with provenance, controlled terminology, and domain-specific meaning.

These problems are not merely accidental bugs. They follow naturally from the fact that LLMs are statistical models rather than explicit knowledge models. Improvements in scale, training, and architecture will continue to make them more capable, but many practical applications still need complementary structures: curated data, explicit semantics, provenance, and constraints.

That is why knowledge graphs and ontologies are becoming more important. Retrieval Augmented Generation (RAG) was an early response to the problem of grounding LLMs in curated information. But as applications become more demanding, simple document retrieval is often not enough. Many domains require richly interconnected knowledge and automated reasoning. RDF, OWL, SPARQL, SHACL, and reusable ontologies such as Dublin Core, PROV-O, and SKOS provide mature standards for representing and working with exactly that kind of explicit knowledge representation. Explicit knowledge representation based on logic and set theory is the perfect complement to the powerful inductive statistical models that power LLMs.

This renewed interest also means that the word “ontology” is now used in many different ways. Some commercial systems use the term for models that are closer to enterprise object models than to ontologies in the Semantic Web sense. I understand why some researchers and ontology engineers find that frustrating. I sometimes do too. But I also see it as a sign of progress. I have seen this pattern since the early days of expert system adoption and later with the adoption of Object-Oriented Programming and Agile methods: when ideas move from research to industry, terms such as rule-based, knowledge base, *Rapid Application Development (RAD)*, object model, and ontology are stretched, simplified, and repurposed. That can create confusion, but it also shows that industry has recognized that the underlying ideas matter. Eventually as practitioners understand the core research ideas, they make educated decisions and can separate real technology from marketing hype. Tutorials such as this eBook are an essential part of that process. Learners need to understand not only that ontologies are useful, but what ontologies and other semantic technology provide that is new and why these new capabilities matter.

One of the things I appreciate about this eBook is that it tries to connect beginner-level Protégé modeling with larger architectural questions. A learner begins with classes such as *Pizza*, *PizzaTopping*, *CheeseTopping*, and *VegetableTopping*. But those beginner examples lead naturally to deeper questions. What is the difference between a class and an individual? When should a class be primitive,

and when should it be defined? What does a reasoner actually infer? Why does OWL's open-world assumption behave differently from a database constraint? When should we use OWL reasoning, and when should we use SHACL validation? How do ontologies relate to RDF graphs, triple stores, SPARQL queries, and larger distributed system architectures?

Those are not minor details. They are the conceptual foundation of ontology engineering.

My strongest advice to readers is to keep Protégé open while reading. Do the exercises. Run the reasoner frequently. That last part is really critical. For training ontologies, the reasoner executes immediately. Running it after every change means you'll see an error as soon as it is created. When that happens, the reasoner can almost always focus you on the specific problem. If you wait until there are several errors the reasoner feedback is usually much less helpful and finding errors can take hours instead of minutes.

When something surprising happens, slow down and understand why. In OWL, surprises are often the most valuable teaching moments. A reasoner that classifies something unexpectedly, fails to classify something you expected, or reports an inconsistency is showing you the difference between what you intended to model and what you actually modeled.

That distinction is at the heart of ontology engineering.

A good ontology is not just a diagram. It is not just a taxonomy. It is not just a data model. It is an explicit, formal model of meaning in a domain. Building one requires careful thought, iteration, testing, and patience. The Pizza tutorial remains useful because it gives learners a simple domain in which to practice those habits before applying them to medicine, finance, manufacturing, enterprise architecture, LLM integration and other business, engineering, and scientific domains.

I am glad to see this new contribution to the Pizza tutorial lineage. It reflects the best spirit of open educational work.

The journey begins with pizza. But the real subject is meaning.

Michael DeBellis

2 July 2026

San Francisco, CA

https://www.michaeldebellis.com/blog

Last updated at 2026-09-11

# Preface from Author

**From Static Diagrams to Executable Knowledge: A Personal Journey**

For nearly thirty years, I have lived and breathed enterprise information technology. My career began long before “knowledge graph” was a common term, working across telecom, automotive, finance, and IT outsourcing consulting. For most of those years, my primary hands-on tools for making sense of complexity was static diagramming, mostly with Visio, draw.io, or even office suite like Excel and Powerpoint. It was powerful, but it was also… silent. The diagrams described architecture, but they could not execute it. They could not reason, validate, or adapt.

That was the before.

**The shift (2021 – 2022)** began when I moved from old toolboxes to Archi, the open-source ArchiMate modeling tool. Suddenly, I was not just drawing; I was building a **repository**. I discovered the power of a shared, collaborative model – a single source of truth about an enterprise. But even Archi, as good as it is, had limitations. The model was still primarily for human consumption. It was a rich, structured "graph" database, but it was not a reasoning engine.

It was around this same time that I discovered the **Essential EAS Open Source (Ontology)** – a glimpse of what happens when an EA repository is built on formal semantics. That was the first crack in the old paradigm, learnt and practiced with its modeling tool, Protégé - the first time I met this word.

**The catalyst (2023)** was the `Pizza.owl` tutorial. Through creating my first video series on the topic, I got in touch with Michael DeBellis. More importantly, I got my hands on Protégé. For the first time, I was not just modeling structure; I was formalizing **meaning**. I was defining classes, properties, and rules that a machine could use to infer new knowledge, check for contradictions, and navigate relationships intelligently.

The pizza example was simple, but the potential was explosive. I realized that the future of enterprise architecture was not in prettier diagrams, but in **executable knowledge**.

With that learning, back to the Essential EAS tool, I realized how powerful of treating enterprise architecture from semantic perspective, and how flexible the tool provided, e.g. if you need one meaning (triple) that EAS built-in meta-model doesn't exist, just create one new from it's slots and that's now fitting the knowledge of your enterprise reality, more and more.

**The synthesis & the birth of EKA & SKDL (2023 – 2026)** became a period of intense exploration. My YouTube channel (and other video sharing platform, like BiliBili, DouYin, etc..) and online courses grew

to host not just the `Pizza.owl` series, but courses on ArchiMate, meta-modeling, diagramming as code (PlantUML, D2, OpenSCAD), Neo4j graph databases, and ontology engineering. Each course was a piece of a larger puzzle. programming taught me logic, PlantUML taught me that diagrams could be generated from text (code), Neo4j showed me how to traverse relationships at scale, and OpenSCAD reinforced the power of parametric, declarative design.

Through years of distillation, everything crystallized into two simple, powerful core concepts for readers to take away:

1. **EKA (Executable Knowledge Architecture)**: The overarching paradigm that treats enterprise architecture and knowledge graphs not as static documents, but as executable, machine-readable specifications.
2. **SKDL (Semantic Knowledge Development Lifecycle)**: The structured 7-stage approach I formulated to guide practitioners seamlessly from raw domain concepts all the way to advanced semantic reasoning and governance.

**Why this book?** This book is not a transcript of my videos. It is the theory behind the practice. It is the result of years of hands-on work, structured precisely around the SKDL stages. I wrote it because I saw a gap:

> *countless tutorials show you how to click buttons in Protégé, or write queries in Neo4j, but almost none provide a complete lifecycle methodology for architectural thinking.*

This book aims to bridge that gap, using the familiar `Pizza.owl` as a safe and repeatable domain to walk you step-by-step through the SKDL framework.

You will start with pizza. But if you follow the journey across the volumes, you will be able to model anything: an enterprise architecture, a digital twin, a regulatory framework, or an AI knowledge system.

The goal is not just to teach you a tool. The goal is to give you a new way to think about knowledge through **EKA** and **SKDL**: as something that can be **formalized, reasoned over, governed, and executed.**

Welcome to the journey.

Xiaoqi Zhao (Yasen)

Global Enterprise Architect, Founder of the EKA Framework

September, 2026 (Updated)

Last updated at 2026-09-11

# Chapter 23 - Semantic Reasoning: Stage 6 of the Semantic Knowledge Development Lifecycle

**Deriving New Knowledge Through Automated Inference**

The first five stages of the Semantic Knowledge Development Lifecycle (SKDL) transformed an abstract conceptual vocabulary into a **governed**, **reusable**, **validated** knowledge asset.

- Conceptual Modeling gave us structure.
- Semantic Description gave us meaning.
- Knowledge Reuse gave us composability.
- Semantic Governance gave us discipline.
- Semantic Validation gave us trust.

Now, as knowledge begins to flow from ontology models into production systems, a new challenge emerges:

> **"What new knowledge can we derive from what we already know?"**

Semantic Reasoning answers this question.

It is the discipline of deriving new, logical entailed knowledge from an existing ontology through automated inference.

> It transforms a static knowledge base into a dynamic, intelligent system capable of discovering relationships, classifications, and consequences that were never explicitly asserted.

Reasoning is the engine of intelligence in an ontology.

This chapter examines **Exercise 20** and **Exercise 21** from Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial* through the lens of semantic reasoning. While **Exercise 18** introduced governance mechanisms (disjointness axioms), and **Exercise 19** introduced validation (the Probe Class patter), **Exercise 20** and **Exercise 21** introduce the core distinction that makes reasoning possible:

- **Primitive Classes** (defined with `SubClassOf`) express only necessary conditions. They describe what something *must* be, but they do not enable auto-classification.
- **Defined Classes** (defined with `EquivalentTo`) express both necessary and sufficient conditions. They describe what something *is*, and they also enable the reasoner to classify individuals and classes automatically.

This distinction is the **gateway to automated reasoning**. Once a class is defined with sufficient conditions, the reasoner can recognize instances of that class without being told.

Along the way, we will examine reasoning from multiple complementary perspectives: mathematical, historical, engineering, architectural, and industrial -- revealing that

> **automated reasoning is not a modern invention, but the culmination of more than two thousand years of logical thought.**

Table of Content


- 23.1 Chapter Introduction -- From Verification to Discovery
  - 23.1.1 The Journey So Far
  - 23.1.2 The New Question
  - 23.1.3 The Central Insight
- 23.2 Learning Objectives
- 23.3 Positioning Within SKDL - Stage 6 Highlighted
- 23.4 What Is Semantic Reasoning?
  - 23.4.1 A Working Definition
  - 23.4.2 Asserted Knowledge vs. Inferred Knowledge
  - 23.4.3 Reasoning vs. Querying vs. Validation

- 23.5 Exercise 20 -- Primitive Classes and the Limits of Inference
  - 23.5.1 The Engineering Objective
  - 23.5.2 Implementation: Creating the Primitive Class
  - 23.5.3 The Result: What the Reasoner Does Not Do
  - 23.5.4 Why Primitive Classes DO Not Enable Auto-Classification
  - 23.5.5 The Engineering Value of Primitive Classes
  - 23.5.6 Engineering Perspective: Exercise 20 as a Case Study
- 23.6 Exercise 21 -- Defined Classes and the Power of Auto-Classification
  - 23.6.1 The Engineering Objective
  - 23.6.2 Implementation: Converting to a Defined Class
  - 23.6.3 Why Defined Classes Enable Auto-Classification
  - 23.6.4 The Moment of Auto-Classification
  - 23.6.5 Primitive vs. Defined Classes: A Summary
  - 23.6.6 Engineering Perspective: Exercise 21 as a Case Study
- 23.7 The Reasoner Landscape -- From Protégé to Production
  - 23.7.1 Why Reasoners Matter Beyond Protégé
  - 23.7.2 The Major RDF/OWL Reasoners
  - 23.7.3 Reasoner Selection Criteria
  - 23.7.4 The Protégé Reasoner Plugin Architecture
  - 23.7.5 Hands-On Comparison: Running the Same Ontology Through Multiple Reasoners
  - 23.7.6 When to Use Which Reasoner
  - 23.7.7 The Engineering Takeaway
- 23.8 The Mathematical Foundations of Inference
  - 23.8.1 Model-Theoretic Semantics: Interpretations and Models
  - 23.8.2 Subsumption: The Core Operation of Classification
  - 23.8.3 Entailment: The Logical Consequence Relation
  - 23.8.4 Subsumption Testing via Unsatisfiability
  - 23.8.5 The Tableau Algorithm: A Decision Procedure for Description Logic
  - 23.8.6 Decidability: The Guarantee of Termination
  - 23.8.7 Computational Complexity: The Cost of Reasoning
  - 23.8.8 From Mathematics to Engineering
- 23.9 From Aristotle to Modern Reasoners -- A Historical Perspective
  - 23.9.1 Aristotle's Syllogism: The First Reasoning Engine
  - 23.9.2 Leibniz's Calculus Ratiocinator: The Dream of Mechanical Reasoning
  - 23.9.3 Frege's Predicate Logic: Making Reasoning Formal
  - 23.9.4 Description Logic: Making Logic Computable
  - 23.9.5 The Modern Reasoner: From Theory to Tool
- 23.10 Reasoning in Industrial Contexts
  - 23.10.1 Why Reasoning Matters in Industry
  - 23.10.2 Logical Reasoning for Complex Systems
  - 23.10.3 Problem-Solving Through Reasoning
  - 23.10.4 Automated Reasoning for Error Detection and Risk Management
  - 23.10.5 Safety and Compliance Through Reasoning
  - 23.10.6 Reasoning as a Strategic Capability
  - 23.10.7 Deeper Industrial Semantic Context -- Grounding Reasoning in the ISA-95 Standard
    - 23.10.7.1 The ISA-95 Functional Hierarchy and Time Horizons
    - 23.10.7.2 Why ISA-95 Object Models Matter for Ontology Reasoning
    - 23.10.7.3 Reasoning Over Industrial Hierarchies in Practice
- 23.11 EKA Perspective -- Stage 6 Establishes the Reasoning Layer ($R$)
  - 23.11.1 Reasoning and the EKA Tuple
  - 23.11.2 The Contribution of Stage 6 to EKA
  - 23.11.3 From $K$ to $R$: The Transition to Intelligence
  - 23.11.4 Reasoning as the Bridge to Execution
- 23.12 Engineering Guidelines for Semantic Reasoning
- 23.13 Key Concepts

- 23.14 Chapter Summary
- 23.15 Looking Ahead -- Tower Executable Knowledge
- References


# 23.1 Chapter Introduction -- From Verification to Discovery

## 23.1.1 The Journey So Far

The first five stages of the Semantic Knowledge Development Lifecycle (SKDL) established the foundations of ontology engineering.

**Stage 1 -- Conceptual Modeling** identified the concepts that define a knowledge domain and organized them into a coherent semantic taxonomy.

**Stage 2 -- Semantic Description** enriched those concepts with formal meaning using Description Logic, OWL restrictions, and class expressions.

**Stage 3 -- Knowledge Reuse** transformed validated semantic knowledge into reusable assets that could be shared, extended, and specialized systematically.

**Stage 4 -- Semantic Governance** established the policies, standards, and processes required to maintain semantic integrity across an evolving knowledge ecosystem.

**Stage 5 -- Semantic Validation** verified that the ontology is logically consistent, semantically complete, and fit for purpose before deployment.

By the end of Stage 5, the `Pizza` ontology had been validated. It was consistent, satisfiable, and free of contradictions. The governance rules had been enforced. The probe class had been created, used, and removed.

The ontology was correct.

But correctness is not the same as intelligence.

A correct ontology that cannot derive new knowledge is like a well-organized library where no noe can find a needed book. It contains knowledge, but it does not *use* knowledge. It is passive, not active.

This is where **Stage 6 -- Semantic Reasoning** enters the lifecycle.

## 23.1.2 The New Question

Validation asked: **"Is this model broken?**

Reasoning asks: **"What else can this model tell us?"**

The distinction is fundamental.

Validation ensures that the ontology is logically sound. Reasoning derives new knowledge from that sound foundation.

Without validation, reasoning produces unsound inferences.

Without reasoning, validation produces a correct but static model.

Together, they produce trustworthy intelligence.

## 23.1.3 The Central Insight

The central insight of this chapter is deceptively simple:

> **Reasoning does not generate new facts -- it makes explicit what was already implicit in the axioms.**

The reasoner does not invent knowledge. It discovers the logical consequences of the axioms the engineer has written. Every inferred statement was already entailed by the ontology. The reasoner simply makes the entailment explicit.

This is the power of formal semantic:

> from a seed of **asserted knowledge**, the reasoner grows a tree of **inferred knowledge**.

## 23.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

**Knowledge:**

- Explain why Semantic Reasoning is the sixty stage of the Semantic Knowledge Development Lifecycle (SKDL)
- Distinguish between **primitive classes** ( `SubClassOf` only) and **defined classes** ( `EquivalentTo` )
- Understand the difference between **asserted knowledge** and **inferred knowledge**
- Explain how a reasoner derives new subclass relationships automatically
- Identify major RDF reasoners beyond Protégé and explain their differences
- Understand how reasoning applies in industrial contexts (problem-solving, decision-making, safety, compliance)
- Explain the mathematical foundations of reasoning: subsumption, entailment, tableau algorithm, decidability

**Practical Skills:**

- Create a primitive class ( `CheesyPizza` ) using `SubClassOf` only
- Convert a primitive class to a defined class using `Edit > Convert to defined class`
- Observe the reasoner automatically classifying individuals and classes
- Use Protégé's explanation feature to understand why an inference was made
- Compare reasoning results across different reasoners (Pellet, HermiT, ELK, JFact)
- Select the appropriate reasoner for a given task

**Engineering Perspective:**

- Understand reasoning as "automated knowledge discovery"
- Explain why defined classes enable automation while primitive classes do not
- Map reasoning activities to the EKA $R$ layer
- Apply reasoning to industrial problem-solving, safety, and compliance scenarios

**Mathematical Perspective:**

- Understand model-theoretic semantics: interpretations, models, entailment
- Explain subsumption as set inclusion and logical consequence
- Understand the tableau algorithm as a decision procedure
- Explain decidability and computational complexity
- Connect reasoning to Description Logic semantics

**Historical Perspective:**

- Trace the reasoning tradition from Aristotle's syllogism to modern automated reasoning
- Connect modern OWL reasoners to th 2,000-year quest for automated inference

## 23.3 Positioning Within SKDL - Stage 6 Highlighted

As introduced in **Chapter (17)**, ontology engineering progresses through seven stages of the Semantic Knowledge Development Lifecycle (SKDL), each stage contributing a distinct engineering objective toward the construction of executable semantic knowledge systems.

Having completed **Stage 5 -- Semantic Validation** in the previous chapter, this chapter advances to **Stage 6 -- Semantic Reasoning.**

The objective of this stage is to derive new knowledge through automated inference.

At the completion of **Stage 5**, the `Pizza` ontology contained a validated knowledge graph, governance policies, and a verified semantic model. However, the ontology had not yet been used to derive new knowledge. The reasoner had been used to **check consistency**, but not to **compute classifications.**

**Stage 6** addresses this gap by introducing:

- **Automated Classification** -- computing all inferred subclass relationships
- **Defined Classes** -- classes with necessary and sufficient conditions that enable auto-classification

- **Inferred KNowledge** -- knowledge derived by the reasoner from the asserted axioms
- **Reasoner Selection** -- choosing the appropriate reasoner for the tasks
- **Reasoning in Production** -- deploying reasoners in enterprise knowledge systems

Rather than introducing new constructs, this stage establishes the engineering discipline required to derive new knowledge from a validated ontology.

The primary deliverable from **Stage 6** is therefore:

> **not a new logical axiom, but an intelligent ontology -- one whose inferred knowledge has been computed and made available for querying, governance, and execution.**

Within the EKA framework, **Stage 6** establishes the $R$ **-- Reasoning & Rules** layer by providing the inference engine that derives new knowledge from the validated knowledge graph.

The progression across the first six stages can now be seen in full:

| SKDL Stage | Primary Outcome | EKA Contribution |
|---|---|---|
| Stage 1 -- Conceptual Modeling | Identify concepts and organize them into a coherent semantic taxonomy | $K$ -- Knowledge Graph (Structure) |
| Stage 2 -- Semantic Description | Enrich concepts with formal semantic meaning through Description Logic and OWL restrictions | $K$ (Meaning) + $R$ (Readiness) |
| Stage 3 -- Knowledge Reuse | Transform validated semantic knowledge into reusable assets | $K$ (Composable) + $\Gamma$ (Prepared) |
| Stage 4 -- Semantic Governance | Maintain semantic integrity through policies, standards, and lifecycle management | $\Gamma$ -- Governance |
| Stage 5 -- Semantic Validation | Verify correctness before deployment through consistency checking and constraint validation | $\Gamma + K$ -- Validation |
| Stage 6 -- Semantic Reasoning | Derive new knowledge through automated inference | $R$ -- Reasoning & Rules |

Each stage has built upon the previous one.

- Stage 1 gave us concepts.
- Stage 2 gave us meaning.
- Stage 3 gave us reusability.
- Stage 4 gave us governance.
- Stage 5 gave us trust.

Stage 6 now gives us intelligence.

# 23.4 What Is Semantic Reasoning?

## 23.4.1 A Working Definition

**Semantic Reasoning** is the process of deriving new, logically entailed knowledge from an existing ontology through automated inference.

It answers the question:

> ***"What else can this model tell us?"***

Not "is it syntactically well-formed?" (Protégé will let you save almost anything).

Not "is it consistent?" (validation already answered that).

But "what new knowledge can be derived from what we already know?"

This is fundamentally a **knowledge discovery** activity. It is the moment when the ontology becomes intelligent -- when it can derive conclusions that the engineer never explicitly wrote.

**Key characteristics of semantic reasoning** include:

- **Reasoning derives, not invents.** -- Every inferred statement was already entailed by the asserted axioms.
- **Reasoning is deterministic.** -- Given the same ontology, the reasoner always produces the same inferences. It does not guess. It does not hallucinate.
- **Reasoning is decidable.** -- In OWL DL, reasoning is guaranteed to terminate with a definitive answer.
- **Reasoning is explainable.** -- The reasoner can justify every inference it makes.

**Key Insight**:

> "The reasoner is not a fortune teller. It is a logical engine. It computes the consequences of your axioms -- no more, no less.
>
> - If the axioms are correct, the inferences are correct.
> - If the axioms are flawed, the reasoner reveals the flaw.

## 23.4.2 Asserted Knowledge vs. Inferred Knowledge

The distinction between asserted and inferred knowledge is fundamental to understanding reasoning.

| Type | Definition | Example |
|---|---|---|
| **Asserted Knowledge** | Statements explicitly written in the ontology by the engineer | `MargheritaPizza SubClassOf NamedPizza` |
| **Inferred Knowledge** | Statements derived by the reasoner fro the asserted axioms | `MargheritaPizza SubClassOf Pizza` (derived via transitivity) |

When the engineer writes `MargheritaPizza SubClassOf NamedPizza` and `NamedPizza SubClassOf Pizza`, the reasoner can infer `MargheritaPizza SubClassOf Pizza` -- even though the engineer never wrote that statement explicitly.

The inferred statement is not new knowledge in the sense of being invented. It was always entailed by the asserted axioms. The reasoner simply made the entailment explicit.

**Key Insight:**

> "The ontology engineer writes a small number of axioms. The reasoner discovers the logical consequences of those axioms. This is the power of formal semantics: **from a seed of asserted knowledge, the reasoner grows a tree of inferred knowledge**.

## 23.4.3 Reasoning vs. Querying vs. Validation

It is important to distinguish reasoning from querying and validation:

| Activity | Question | Timing | Tool |
|---|---|---|---|
| **Validation** | "Is this model correct?" | Before reasoning | Reasoner (consistency check) |
| **Reasoning** | "What else can this model tell us?" | After validation, before execution | Reasoner (classification) |
| **Querying** | "What does the model say about X?" | After reasoning | SPARQL / DL Query |

- **Validation** ensures that the ontology is logically sound. It answers the question: "Is this model broken?"
- **Reasoning** derives new knowledge from the validated ontology. It answers the question: " What else can this model tell us?"
- **Querying** retrieves knowledge from the ontology. It answers the question: "What does the model say about X?"

The key distinction between reasoning and querying is this: **querying retrieves asserted facts; reasoning derives new facts.**

A query over an unclassified ontology returns only what was explicitly written.

A query over a classified ontology returns both asserted and inferred knowledge.

**Key Insight:**

> "Reasoning is not querying. Querying retrieves. Reasoning derives.
>
> - A query over an unclassified ontology is like searching a library where only the books you already know about are listed.
> - A query over a classified ontology is like searching a catalog that has been enriched with connections you never knew existed."

# 23.5 Exercise 20 -- Primitive Classes and the Limits of Inference

Having established the principles of semantic reasoning, we now turn to its practical implementation through **Exercise 20** in Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial*.

This exercise represents the first hands-on activity of **Stage 6 -- Semantic Reasoning** within the Semantic Knowledge Development Lifecycle (SKDL).

Unlike previous exercises, which introduced individual OWL constructs or governance mechanisms, **Exercise 20** focuses on the distinction that makes reasoning possible:

> the difference between primitive and defined classes.

## 23.5.1 The Engineering Objective

**Exercise 20** asks us to create a class called `CheesyPizza` defined using only `SubClassOf`.

The definition is simple: a `CheesyPizza` is a `Pizza` that has cheese as a topping. In OWL Manchester Syntax:

```
Class: CheesyPizza
    SubClassOf: Pizza
    SubClassOf: hasTopping some CheeseTopping
```

At first glance, this appears to ba a complete definition. But **it is not.** This definition creates a **primitive class** -- a class defined only by necessary conditions.

The engineering objective of **Exercise 20** is not to create a fully defined class, but to demonstrate the limits of primitive classes. The exercise deliberately creates a class that the reasoner **cannot** use for auto-classification. This prepare the ground for **Exercise 21**, where the class is converted to a **defined class** and the reasoner can suddenly do something it could not do before.

**Key Insight:**

> "**Exercise 20** is a lesson in what the reasoner cannot do. It is only by understanding the limits of primitive classes that we can appreciate the power of defined classes."

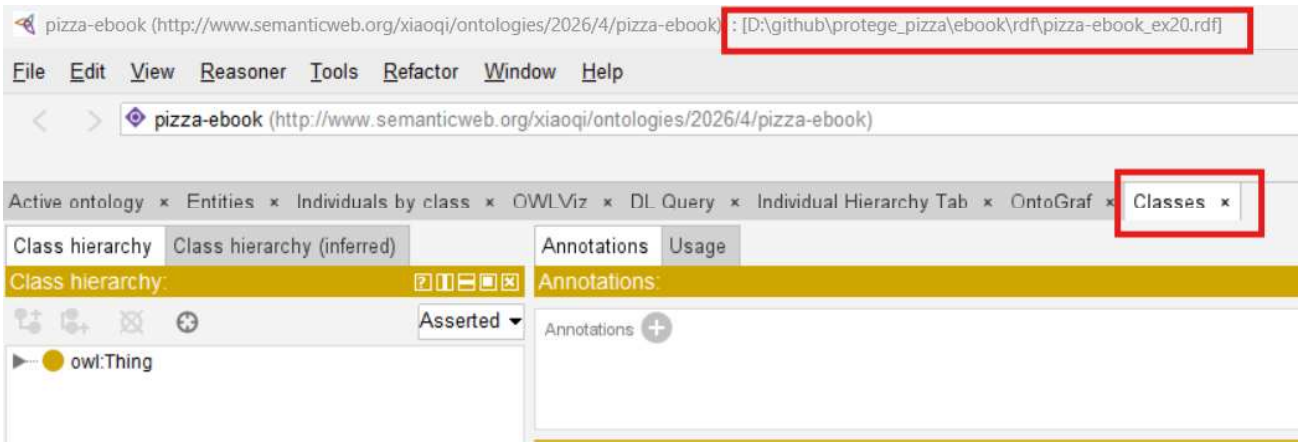
## 23.5.2 Implementation: Creating the Primitive Class

Let's walk through the implementation step by step.

> Note: you may find the RDF file `pizza-ebook_ex20.rdf` in `ebook\rdf` folder for reference.

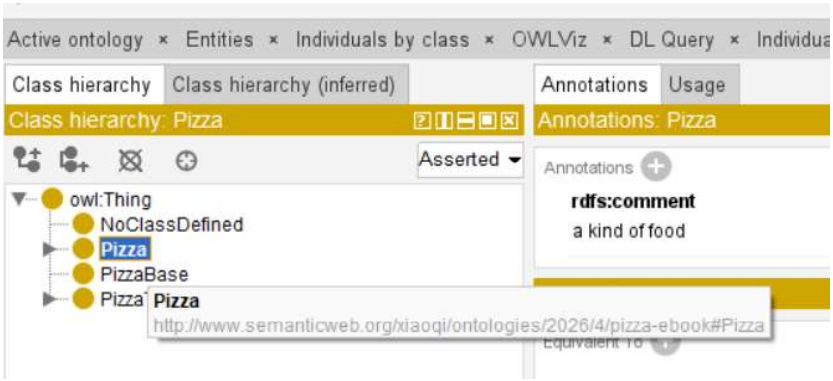
**Step 1: Navigate to the `Classes` tab in Protégé**

Open the `Pizza` ontology in Protégé. Navigate to the `Classes` tab.

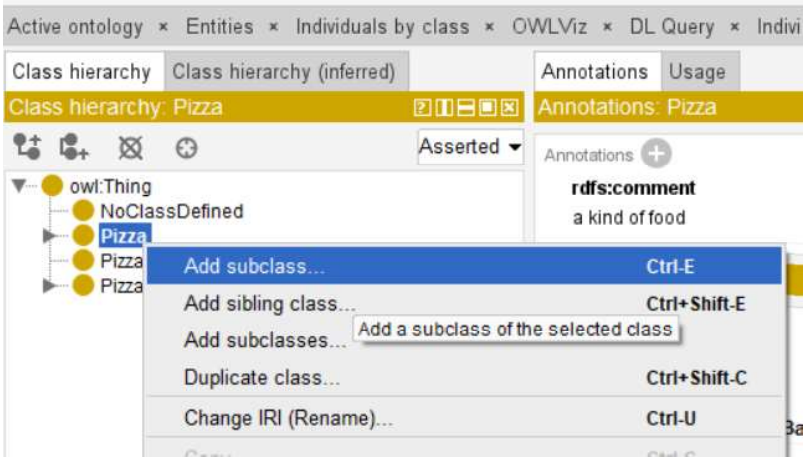



**Step 2: Create the new class**

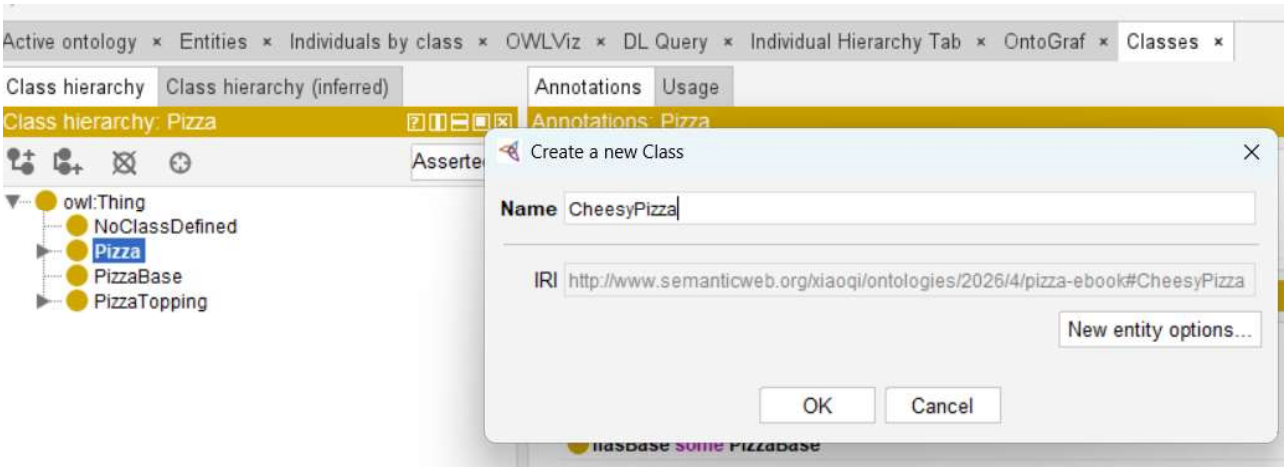
1. Select `Pizza` in the class hierarchy



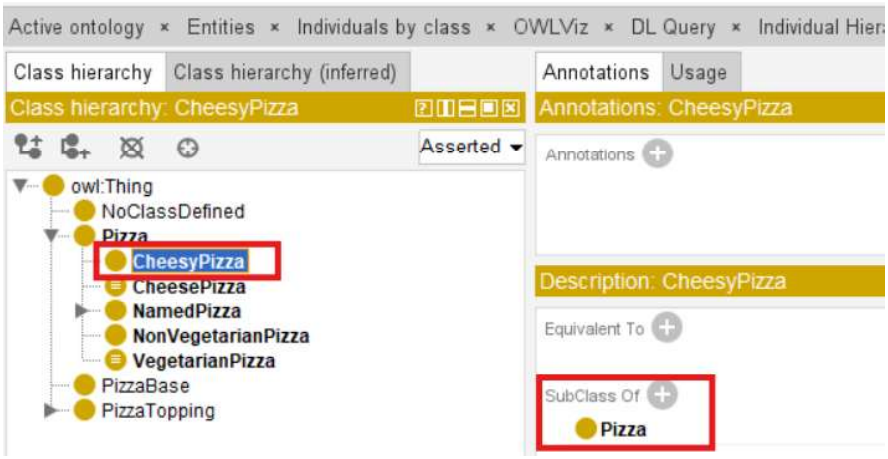

2. Click the "Add subclass` icon (or right-click and select "Add subclass")




3. Name the class `CheesyPizza`



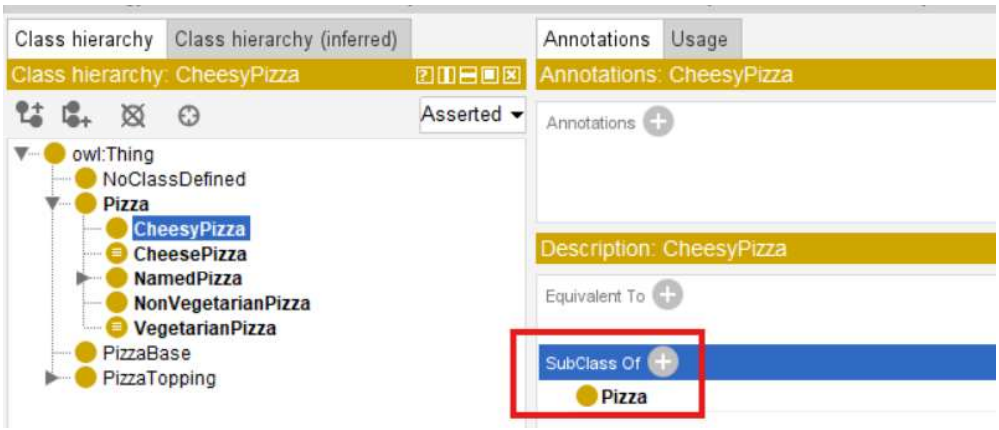

4. Click OK



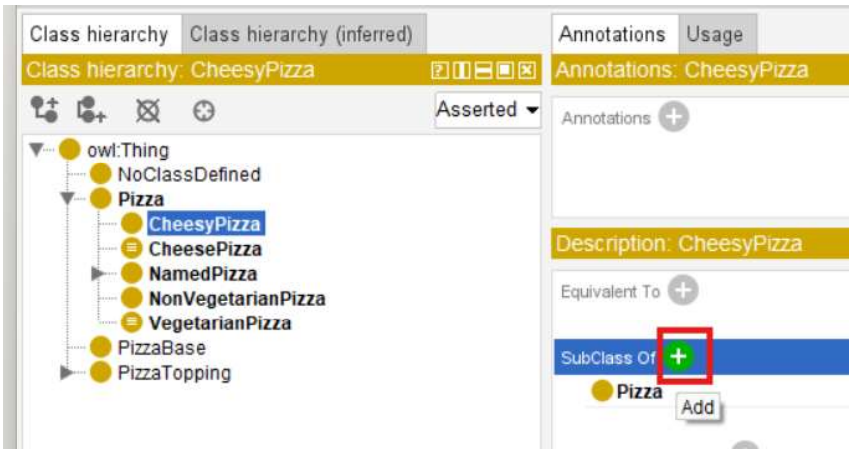

> Note: after this step, `CheesyPizza` is already `SubClass Of` the `Pizza` class.

**Step 3: Add the `SubClassOf` axioms**

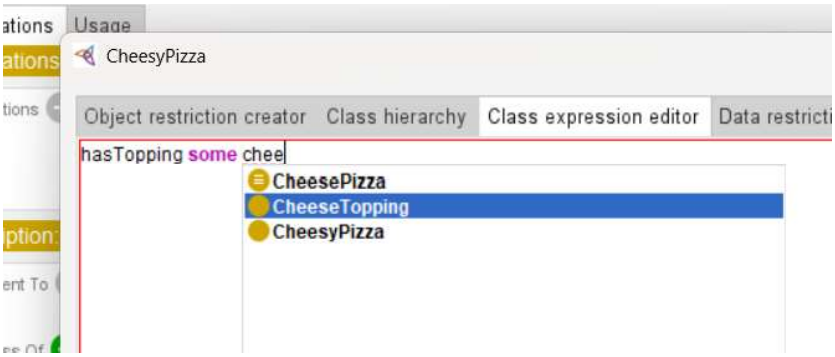
1. Select `CheesyPizza` in the class hierarchy
2. In the Description View, locate the `SubClassOf` section



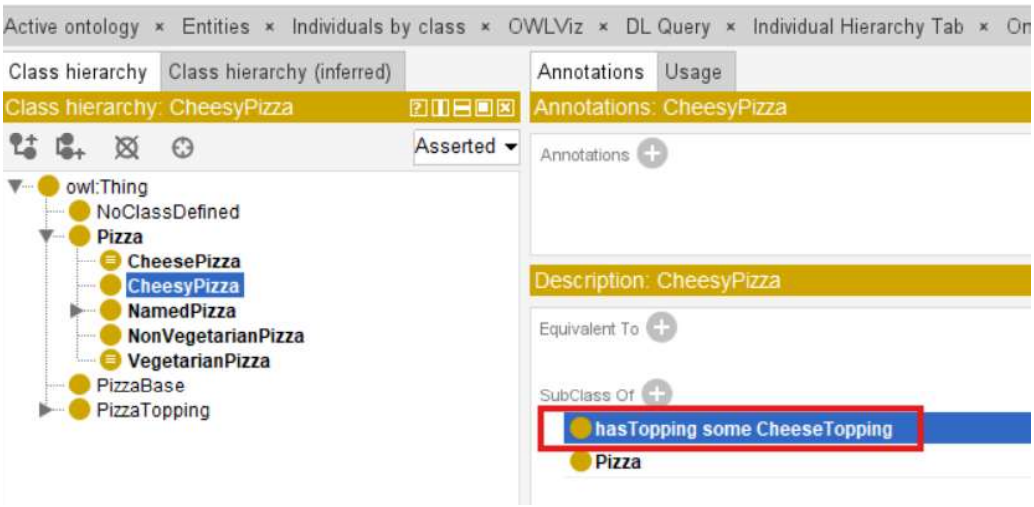

3. Click the Add icon (+)




4. In the Class Expression Editor, enter `hasTopping some CheeseTopping`




5. Click OK

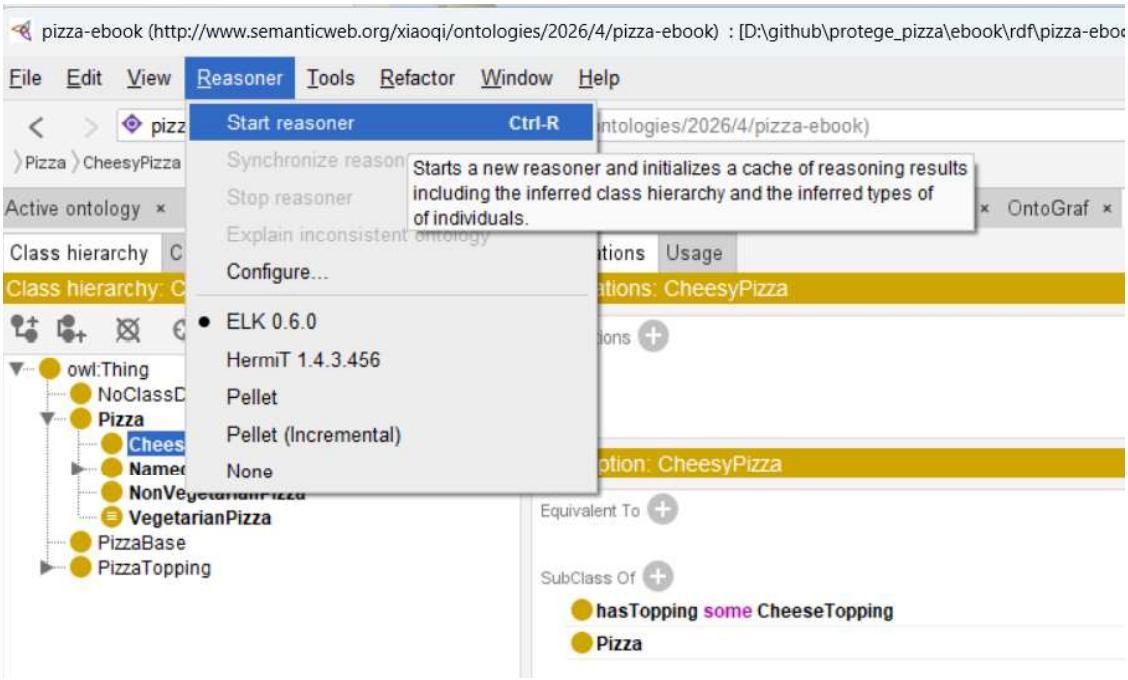



After completing these steps, `CheesyPizza` should have the following description:

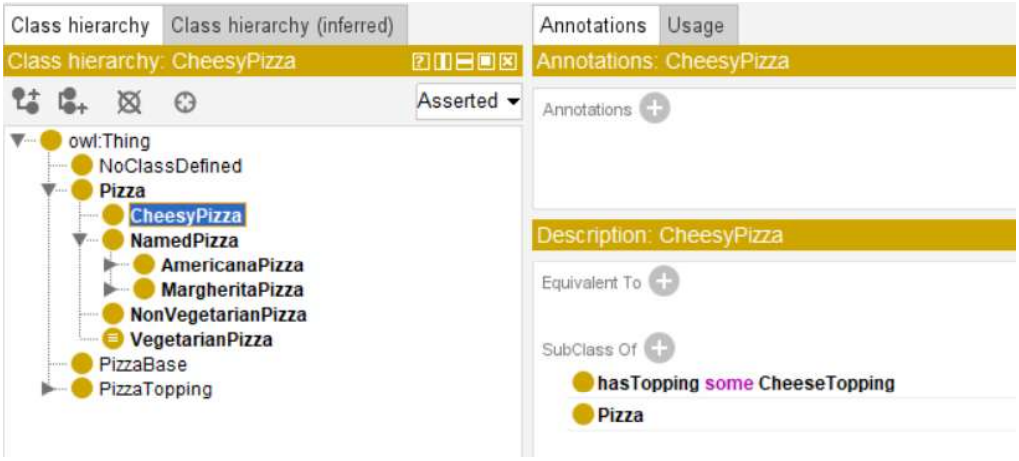
```
CheesyPizza
    SubClassOf: Pizza
    SubClassOf: hasTopping some CheeseTopping
```

**Step 4: Synchronize the reasoner**

1. Select [Reasoner] > [Start reasoner] (or `Ctrl+R` if the reasoner is already running)




2. Observe the result




## 23.5.3 The Result: What the Reasoner Does Not Do

When the reasoner is synchronized, something important - or rather, something important does ***NOT*** happen.

**What to observe:**

- `CheesyPizza` appears in the asserted hierarchy as a subclass of `Pizza`
- The inferred hierarchy is **identical** to the asserted hierarchy
- The reasoner does **NOT** auto-classify `MargheritaPizza` or other pizzas as subclasses of `CheesyPizza` -- even though they all have cheese toppings.

This is the critical observation. The reasoner has all the information it needs to conclude that `MargheritaPizza` is a `CheesyPizza` (because Margherita has mozzarella, which is a cheese topping).

But it does not make that inference!

WHY?

Because `CheesyPizza` is defined only with **necessary conditions.** The ontology says: "If something is a `CheesyPizza`, then it must be a `Pizza` that has cheese." It does *not* say: "If something is a `Pizza` that has cheese, then it is a `CheesyPizza`.

The reasoner cannot conclude that `MargheritaPizza` is a `CheesyPizza` because the ontology has not stated what is *sufficient* to qualify.

**Key Insight:**

> "A primitive class is like a club with no membership criteria. You can say 'all members must wear a hot,' but you haven't said that wearing a hat makes you a member. The reasoner cannot conclude that any particular pizza is a `CheesyPizza` , because the ontology has not stated what is sufficient to qualify."

## 23.5.4 Why Primitive Classes DO Not Enable Auto-Classification

To understand why primitive classes do not enable auto-classification, we need to examine the semantics of `SubClassOf` .

In OWL, `SubClassOf` expresses a **necessary condition**:

```
CheesyPizza SubClassOf hasTopping some CheeseTopping
```

This means: "Every instance of `CheesePizza` must have at least one cheese topping."

But it does NOT mean: "Every pizza with at least one cheese topping is a `CheesyPizza` .

The distinction is **subtle** but **fundamental**.

It is the difference between:

- **Necessary condition**: "If A, then B" (A implies B)
- **Sufficient condition**: "If B, then A" (B implies A)

`SubClassOf` expresses only the necessary condition. It tells the reasoner: "If you find a `CheesyPizza` , check that it has cheese." It does not tell the reasoner: "If you find a pizza with cheese, call is a `CheesyPizza` ."

Mathematically, the axiom is:

$$\text{CheesyPizza} \sqsubseteq \text{Pizza} \sqcap \exists \text{hasTopping}.\text{CheeseTopping}$$

This says: "Every `CheesyPizza` is a subset of the set of `Pizzas` that have at least one cheese topping."

But it does NOT say the reverse.

A pizza with cheese is not necessary a `CheesyPizza` -- it might be a `MargheritaPizza` that also happens to be a `CheesyPizza` , or it might be something else entirely.

**Key Insight:**

> "The primitive class tells the reasoner: 'If you find a `CheesyPizza` , check that it has cheese.' It does not tell the reasoner: 'If you find a pizza with cheese, call it a `CheesyPizza` .' The distinction is subtle but fundamental.

## 23.5.5 The Engineering Value of Primitive Classes

Primitive classes are not a mistake. They are an essential tool in the ontology engineer's toolkit.

**Why primitive classes matter:**

1. **Incremental modeling**: Primitive classes allow the engineer to say "this concept exists" without committing to a complete definition. This is essential during early modeling, when the domain is still being understood.
2. **Open World Assumption (OWA)**: Primitive classes support the Open World Assumption. They allow knowledge to grow incrementally without requiring the engineer to specify everything at once.
3. **Conceptual scaffolding**: Primitive classes provide the structural framework of the ontology. They define the hierarchy and the relationships between concepts, even before the concepts are fully defined.
4. **Flexibility**: Primitive classes can be refined over time.As the domain understanding matures, the engineer can add restrictions and, eventually, convert the class to a defined class.

**Key Insight:**

> "Primitive classes are the scaffolding of ontology development. They hold the structure in place while the engineer figures out the details. But scaffolding is not the building. To enable automation, we need defined classes."

### 23.5.6 Engineering Perspective: Exercise 20 as a Case Study

From the perspective of Protégé, **Exercise 20** involves creating a class and adding two restrictions. It appears to be a simple modeling activity.

From the perspective of ontology engineering, however, something more significant has occurred. The exercise has demonstrated the limits of primitive classes. It has shown that the reasoner cannot auto-classify based on necessary conditions alone.

This is a crucial lesson!

Many beginners assume that if they define a class with the right restrictions, the reasoner will automatically classify individuals into that class. **Exercise 20** shows that this is not the case.

The reasoner is not a mind reader. It does not infer what the engineer *intended*. It infers only what the axioms *entail*. If the engineer wants auto-classification, they must provide sufficient conditions -- and that means using `EquivalentTo`, not `SubClassOf`.

**Key Insight:**

> "**Exercise 20** is a lesson in humility. It teaches us that the reasoner does exactly what we tell it to do -- no more, no less. If we want auto-classification, we must give the reasoner the tools to perform it."

## 23.6 Exercise 21 -- Defined Classes and the Power of Auto-Classification

Having established the limits of primitive classes, we now turn to **Exercise 21**, which introduces defined classes and demonstrates the power of auto-classification.

This exercise represent the **"aha moment"** of ontology engineering -- the moment when the ontology becomes intelligent.

In Github repository, you may find `pizza-ebook_ex21.rdf` within `ebook/rdf` folder.

### 23.6.1 The Engineering Objective

**Exercise 21** asks us to convert `CheesyPizza` from a primitive class to a **defined class**.

This is done using `Edit > Convert to defined class` (also can from right-click pop up menu or `Ctrl+D` directly) in Protégé.

The conversion changes the class from `SubClassOf` to `Equivalentto`, given `CheesyPizza` both necessary **AND** sufficient conditions.

After conversion, the definition of `CheesyPizza` becomes:

```
Class: CheesyPizza
    EquivalentTo: Pizza and (hasTopping some CheeseTopping)
```

The engineering objective of **Exercise 21** is to demonstrate what happens when a class has sufficient conditions: the reasoner can now auto-classify individuals and classes into it.

**Key Insight:**

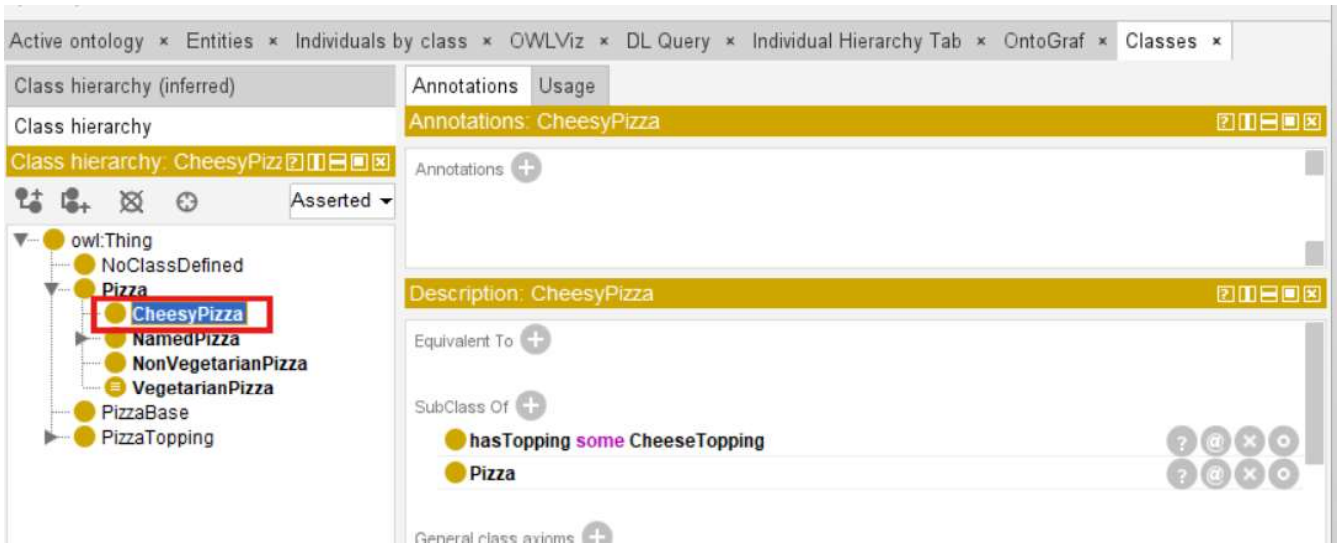
> "**Exercise 21** is the moment the ontology becomes intelligent. The reasoner can now recognize a `CheesyPizza` when it sees one. The engineer did not have a manually classify `MargheritaPizza`. The reasoner discovered it."

### 23.6.2 Implementation: Converting to a Defined Class

Let's walk through the conversion step by step.

**Step 1: Select `CheesyPizza`**

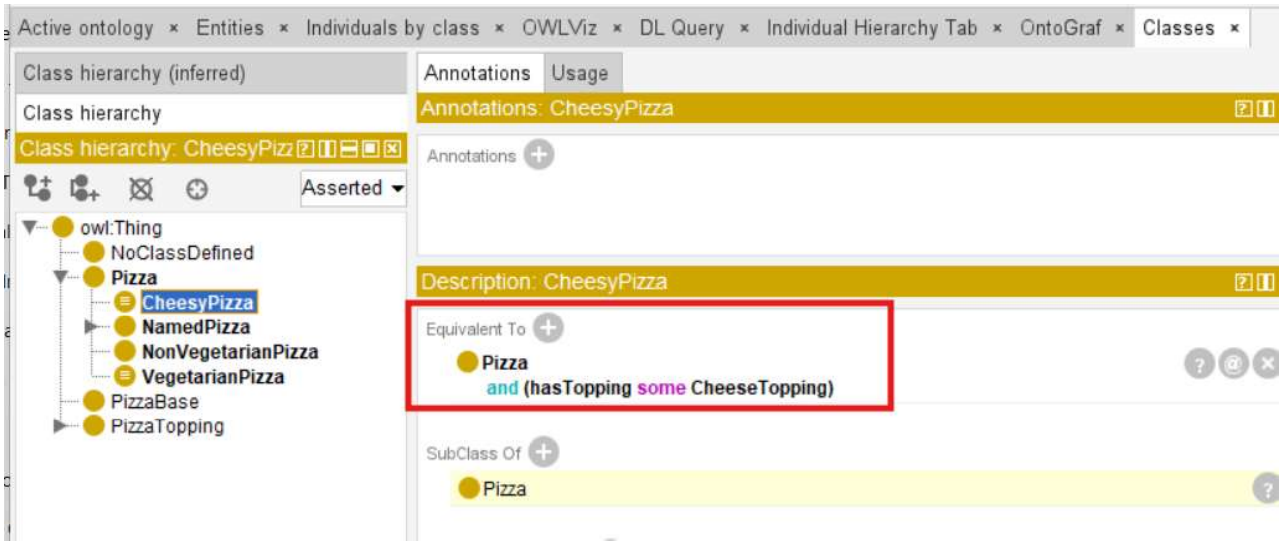
In the `Classes` tab, select `CheesyPizza`.




**Step 2: Convert to defined class**

Select `Edit > Convert to defined class`.

Observe what happens in the Description view. The `SubClassOf` axioms are replaced by an `EquivalentTo` axiom.




Before conversion:

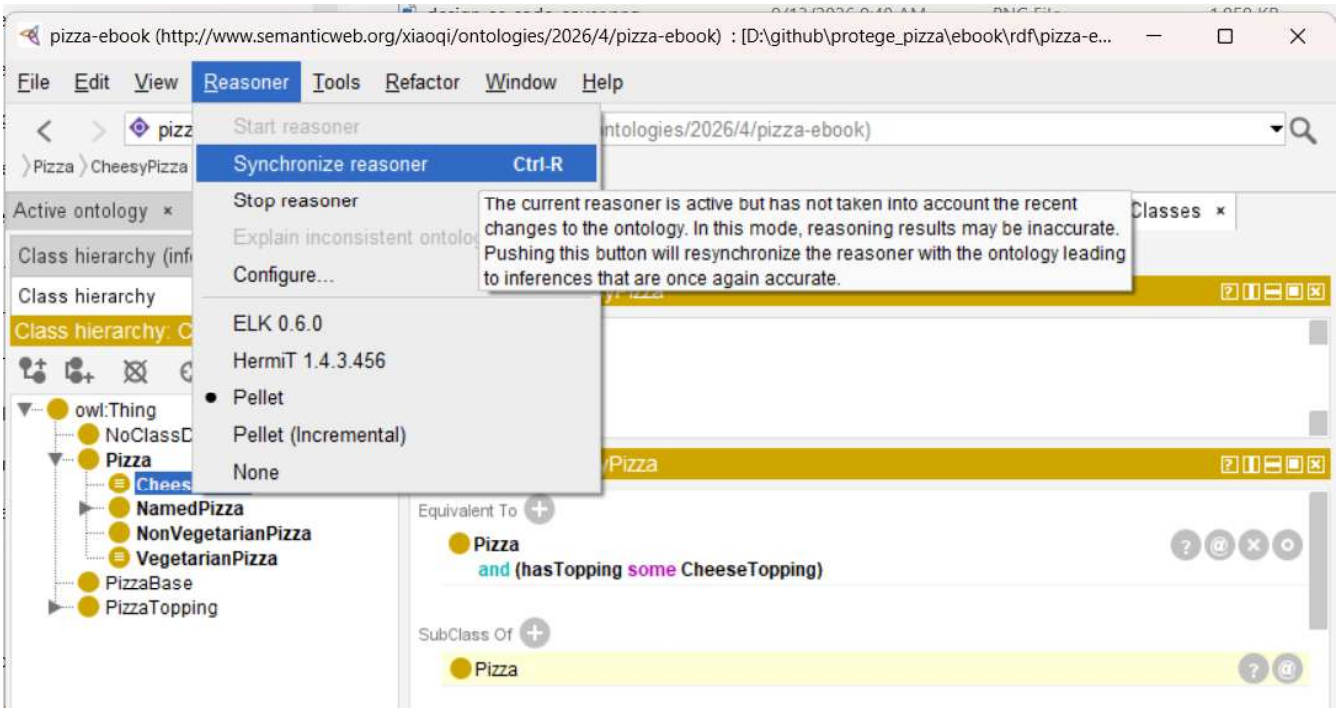
```
CheesyPizza
    SubClassOf: Pizza
    SubClassOf: hasTopping some CheeseTopping
```

After conversion:

```
CheesyPizza
    SubClassOf: Pizza
    EquivalentTo: Pizza and (hasTopping some CheeseTopping)
```

**Step 3: Synchronize the reasoner**

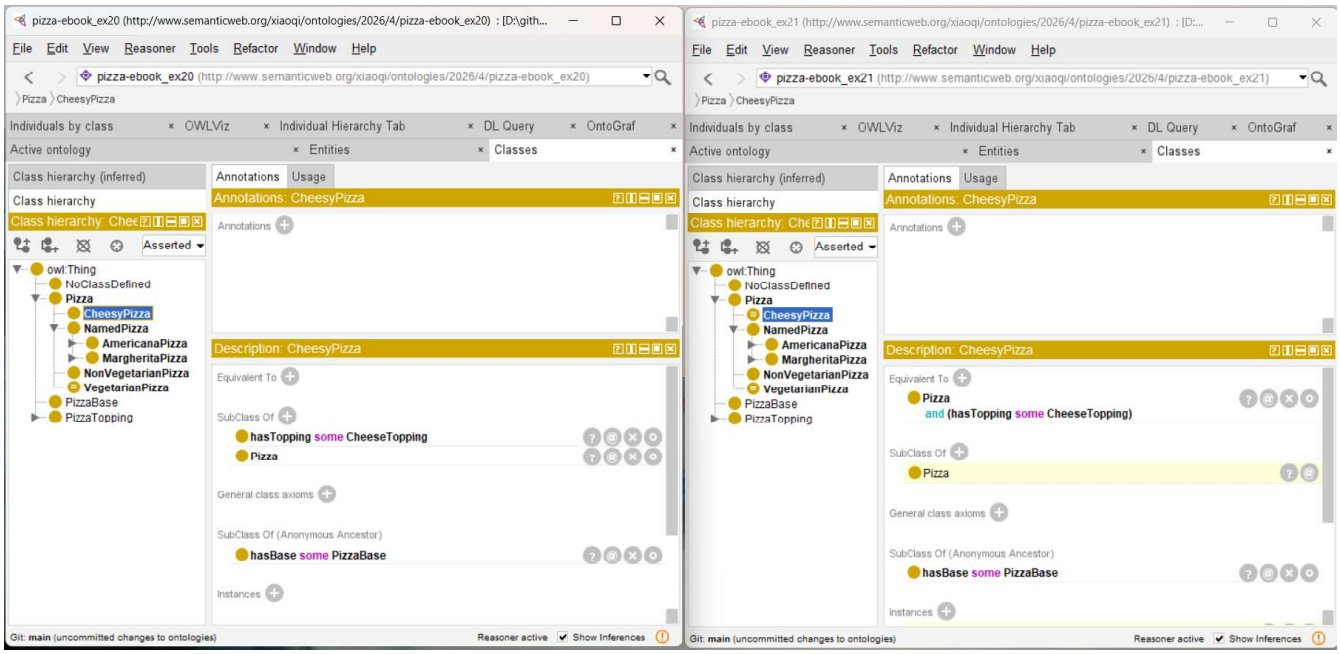
Select `Reasoner > Synchronize reasoner` or ( `Ctrl+R` ).



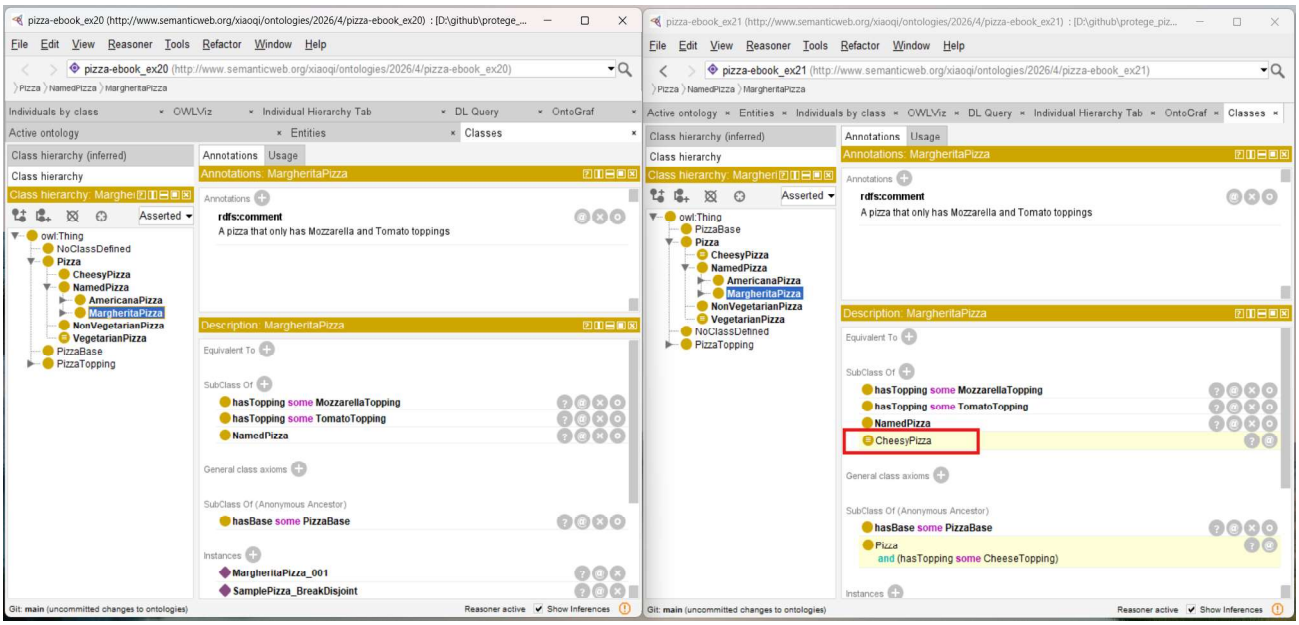

**Step 4: Observe the result**

This is the moment of truth. Teh reasoner now performs **classification** -- it computes all inferred subclass relationships.

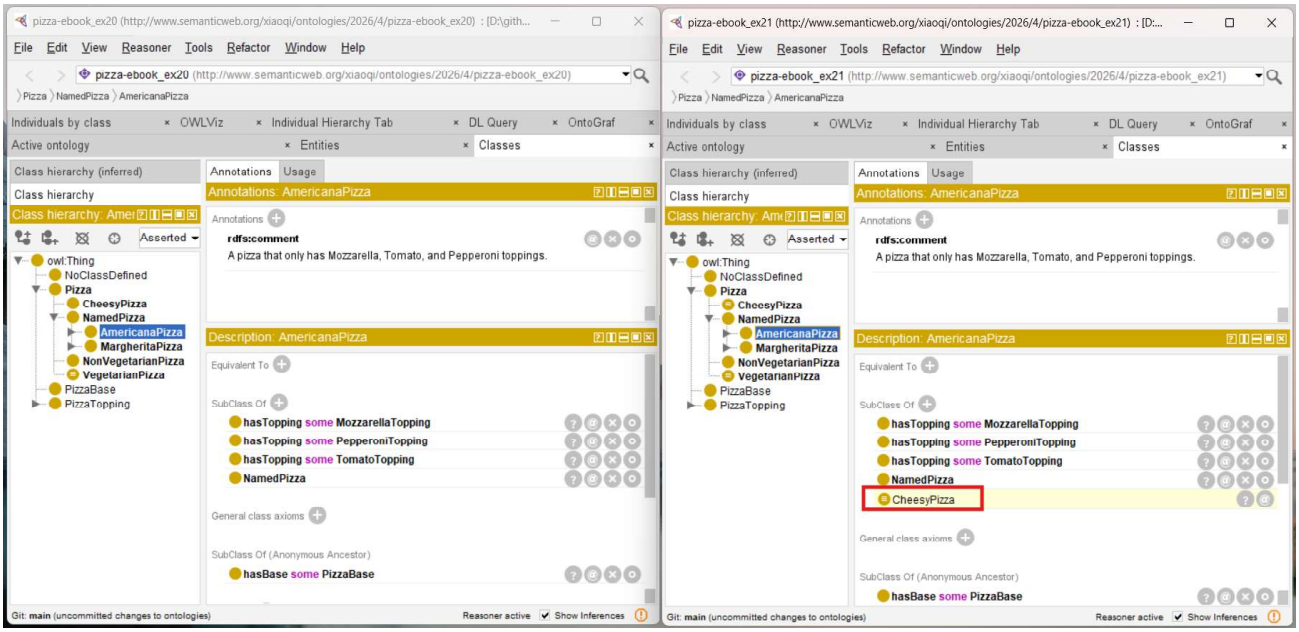
What to observe (I'm putting **Exercise 20** and **Exercise 21** side-by-side):




- The reasoner **automatically classifies** `MargheritaPizza` as a subclass of `CheesyPizza`




- Other pizzas with cheese toppings (e.g., `AmericanaPizza`) are also classified under `CheesyPizza`




- The inferred hierarchy now **differs** from the asserted hierarchy
- New subclass relationships appear that were never explicitly asserted

The reasoner has done something remarkable: it has recognized that `MargheritaPizza` is a `CheesyPizza` -- without the engineer saying so.

**Key Insight:**

> "The moment `CheesyPizza` becomes a defined class, the reasoner can do something it could not do before: it can recognize a `CheesyPizza` when it sees one. The engineer did not have to manually assert that `MargheritaPizza` is a `CheesyPizza`. The reasoner discovered it."

## 23.6.3 Why Defined Classes Enable Auto-Classification

To understand why defined classes enable auto-classification, we need to examine the semantics of `EquivalentTo`.

In OWL, `EquivalentTo` expresses **necessary and sufficient conditions**:

```
CheesyPizza EquivalentTo Pizza and (hasTopping some CheeseTopping)
```

This means: "Something is a `CheesyPizza` if and only if it is a `Pizza` that has at least one cheese topping."

The "**if and only if**" is the key.

It gives the reasoner a rule it can apply in both directions:

- If something is a `CheesyPizza`, then it is a `Pizza` with cheese (necessary condition)
- If something is a `Pizza` with cheese, then it is a `CheesyPizza` (sufficient condition)

The second direction is what enable **auto-classification.** The reasoner can now look at any pizza, check whether if satisfies the condition `Pizza and (hasTopping some CheeseTopping)`, and if it does, classify it as a `CheesyPizza`.

Mathematically, the axiom is:

$$\text{CheesyPizza} \equiv \text{Pizza} \sqcap \exists \text{hasTopping}.\text{CheeseTopping}$$

The $\equiv$ (equivalence) symbol is the key. It means tht two sides of the equation are logically identical. Any individual that satisfies the right-hand side automatically satisfies the left-hand side, and vice versa.

**Key Insight:**

> "The defined class tells the reasoner: 'If you find a pizza with cheese, it is a `CheesyPizza`.' The reasoner now has a rule it can apply automatically. This is the difference between a passive taxonomy and an active classification engine."

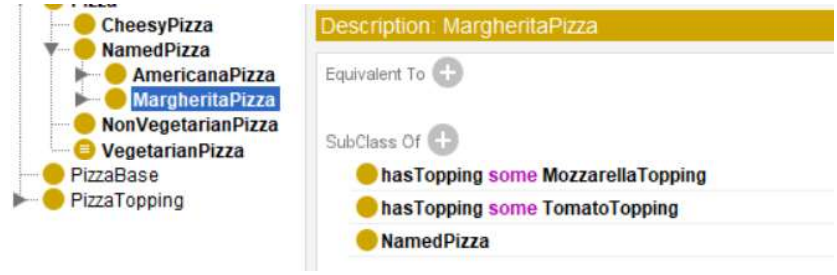
## 23.6.4 The Moment of Auto-Classification

When the reasoner is synchronized after the conversion, it performs **classification.**

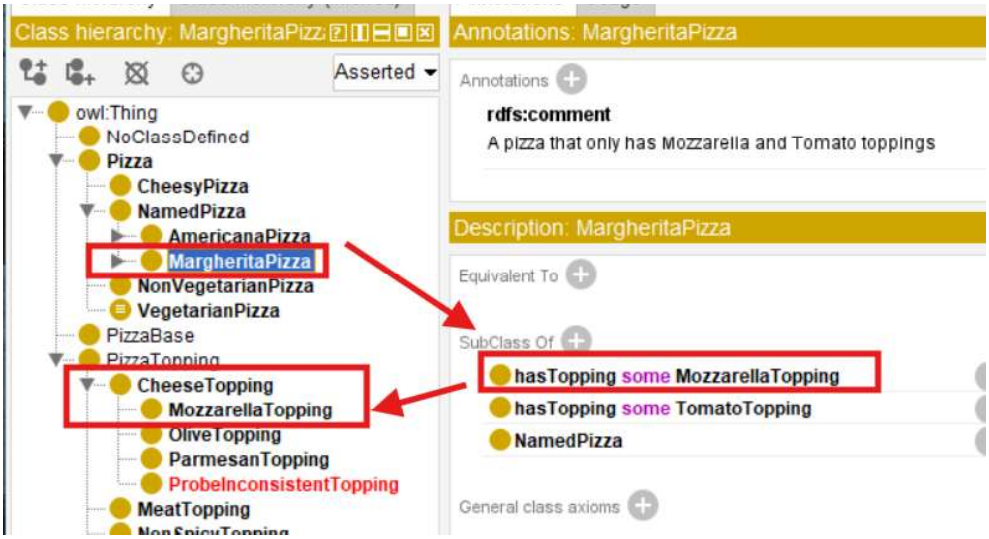
Classification is the process of computing all inferred subclass relationships. The reasoner examines every class in the ontology and determines where it belongs in the inferred hierarchy.

For `MargheritaPizza`, the reasoner performs the following reasoning:

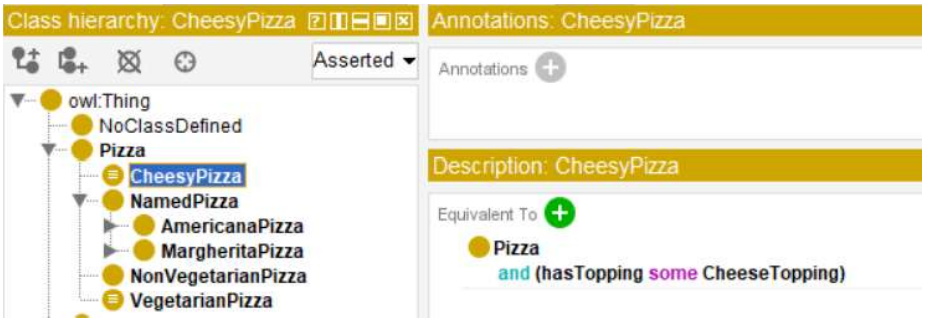
1. `MargheritaPizza` is defined as a `Pizza` that has `MozzarellaTopping` and `TomatoTopping`



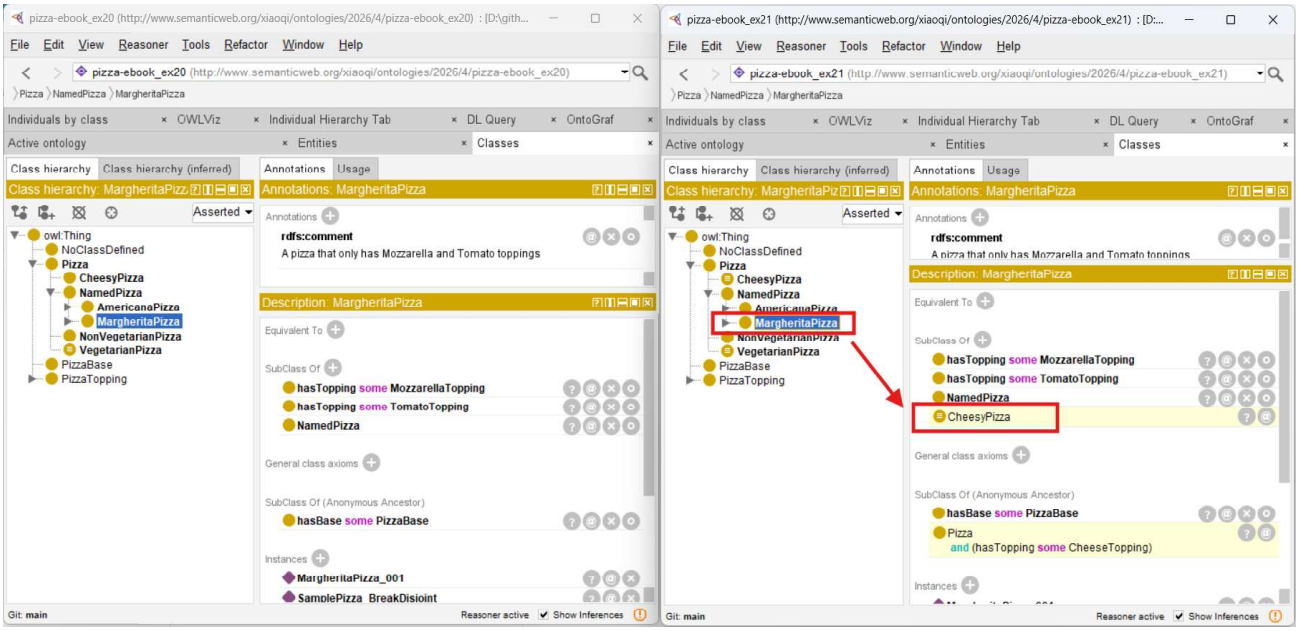

2. `MozzarellaTopping` is a subclass of `CheeseTopping`



3. Therefore, `MargheritaPizza` has at least one `CheeseTopping`
4. `CheesyPizza` is defined as a `Pizza` that has at least one `CheeseTopping`




5. Therefore, `MargheritaPizza` satisfies the definition of `CheesyPizza`
6. Therefore, `MargheritaPizza` is a subclass of `CheesyPizza`




The reasoner has derived a new subclass relationship that was never explicitly asserted. This is the power of **reasoning.**

**Key Insight:**

> "The ontology engineer does not manually classify `MargheritaPizza` as a `CheesyPizza`. The reasoner does it. This is the power of reasoning: the ontology can grow -- not in size, but in intelligence -- without additional manual effort."

## 23.6.5 Primitive vs. Defined Classes: A Summary

The distinction between primitive and defined classes is one of the most important distinctions in ontology engineering. It is the difference between a static taxonomy and an intelligent classification engine.

| Dimension | Primitive Class | Defined Class |
|---|---|---|
| **Axiom** | `SubClassOf` | `EquivalentTo` |
| **Condition** | Necessary only | Necessary and sufficient |
| **Reasoner behavior** | No auto-classification | Auto-classification |
| **Asserted vs. inferred** | Same | Might different |
| **Use case** | Early modeling, evolving concepts | Stable concepts, automation |

**When to use primitive classes:**

- During early modeling, when the domain knowledge is still being understood
- For concepts that are still evolving
- When the engineer wants to say "this concepts exists" without committing to a complete definition

**When to use defined classes:**

- When the concept has a clear, stable definition already
- When auto-classification is desired
- When the engineer is confident that the definition captures the intended meaning

**Key Insight:**

> "Primitive classes define what something must be. Defined classes define what something is. The first supports incremental modeling. The second enables automation."

### 23.6.6 Engineering Perspective: Exercise 21 as a Case Study

From the perspective of Protégé, **Exercise 21** involves a single menu command: `Edit > Convert to defined class` . It appears to be a **trivial operation**.

From the perspective of ontology engineering, however, something profound has occurred. The ontology has become intelligent. The reasoner can now derive new knowledge that the engineer never explicitly wrote.

This is the **"aha moment"** of ontology engineering. It is the moment when the engineer realizes that the ontology is not just a static representation of knowledge -- it is a dynamic system capable of discovery.

The engineering contribution of **Exercise 21** can be summarized as follows:

| Engineering Perspective | Contribution from **Exercise 21** |
|---|---|
| **SKDL Stage** | Stage 6 - Semantic Reasoning |
| **Primary Objective** | Demonstrate the power of defined classes for auto-classification |
| **Modeling Activity** | Convert `CheesyPizza` from a primitive class to a defined class |
| **Engineering Outcome** | Enable the reasoner to auto-classify `MargheritaPizza` and other cheese-topped pizzas as `CheesyPizza` |
| **Preparation for Later Chapters** | Establish the reasoning foundation required for triggers ($\Theta$) and execution ($\Phi$) |

**Key Insight:**

> "**Exercise 21** is a small operation with a large consequence. A single menu command transforms the ontology from a static taxonomy into an intelligent classification engine. This is the power of defined classes."

## 23.7 The Reasoner Landscape -- From Protégé to Production

Having explored the mechanics of reasoning through **Exercise 20** and **Exercise 21**, we now turn to the practical question:

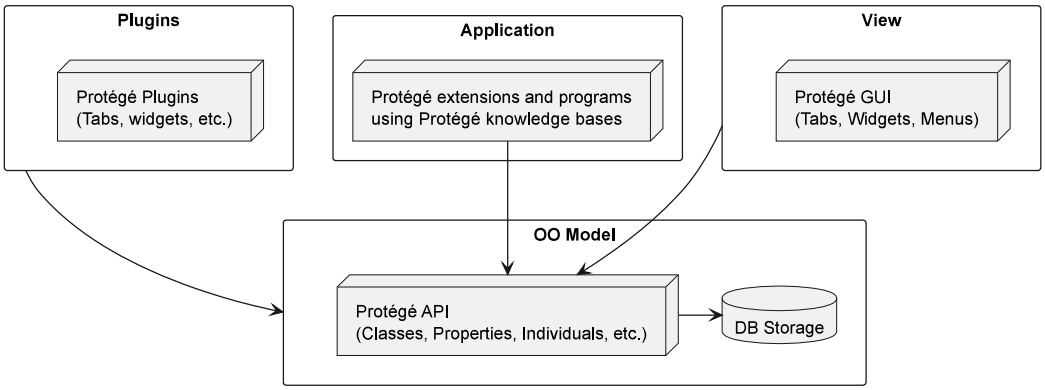
> *"Which reasoner should we use?"*

Protégé is an ontology editor, not a reasoner. It provides a plugin architecture that allows different reasoners to be integrated. However, in production systems, reasoners run as **standalone engines** -- not inside Protégé.

Understanding the landscape of available reasoners is essential for enterprise deployment.

### 23.7.1 Why Reasoners Matter Beyond Protégé

Protégé is a powerful ontology editor, but it does not include a reasoner by default. Instead it provides a **plugin architecture** that allows different reasoners to be integrated. See below Protégé architecture illustration [1]:




When you click `Start reasoner` in Protégé, you are invoking an external reasoning engine through the OWL API. The reasoner runs as a separate process (or thread), computes the inferences, and returns the results to Protégé for display.

This separation is important for several reasons:

1. **Flexibility**: Different reasoners can be plugged into Protégé without changing the ontology.
2. **Specialization**: Different reasoners are optimized for different tasks (e.g., HermiT for DL ontologies, ELK for EL ontologies).
3. **Production deployment**: In production systems, reasoners runs independently of Protégé, often as part of a larger knowledge graph infrastructure.

**Key Insight:**

> "Protégé is the workshop. The reasoner is the engine. You can sway engines without changing the workshop -- but different engines produce different results, at different speeds and characteristics, with different trade-offs."

## 23.7.2 The Major RDF/OWL Reasoners

The following table summarizes the major RDF/OWL reasoners available today.

| Reasoner | Developer | Language | OWL Profile |
|---|---|---|---|
| **Pellet** | Clark & Parsia (now Stardog) | Java | OWL 2 DL |
| **HermiT** | University of Oxford | Java | OWL 2 DL |
| **ELK** | University of Oxford | Java | OWL 2 EL |
| **JFact** | Teh OWL API community | Java | OWL 2 DL |
| **FaCT++** | University of Manchester | C++ | OWL 2 DL |
| **Konclude** | University of Ulm | C++ | OWL 2 DL |
| **OpenIlet** | Community fork of Pellet | Java | OWL 2 DL |
| **Jena** | Apache Software Foundation | Java | RDFS, OWL Lite, OWL DL (partial) |
| **DotNetRDF** | Community | .NET | RDFS, OWL (partial) |

## 23.7.3 Reasoner Selection Criteria

Choosing the right reasoner is an engineering decision.

The following criteria should be considered:

| Criterion | Consideration |
|---|---|
| **Expressivity** | Does the reasoner support the OWL profile your need? (DL, EL, Full) |

| Criterion | Consideration |
|---|---|
| **Completeness** | Does the reasoner guarantee complete inference? |
| **Explanation** | Does the reasoner support explanation of inference? |
| **Incremental Reasoning** | Can the reasoner update inferences without re-computing everything? |
| **Performance** | How large is your ontology? How complex are the axioms? |
| **Integration** | Does the reasoner integrate with your toolchain (Protégé, Jena, OWL API)? |
| **Licensing** | Is the reasoner open-source? Or commercial? |

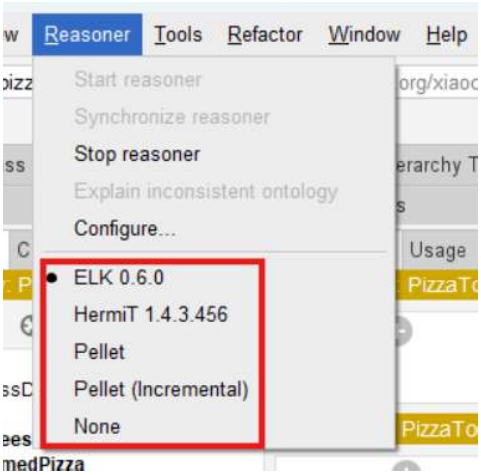
## 23.7.4 The Protégé Reasoner Plugin Architecture

Refer to the diagram in previous 23.7.1, Protégé uses the **OWL API** to interface with reasoners.

Any reasoner that implements the OWL API reasoner interface can be plugged into Protégé.

To select a reasoner in Protégé:

1. Select `Reasoner` menu
2. Choose the reasoner from the list




3. Select `Reasoner > Start reasoner` (or `Synchronize reasoner` if already running)

Protégé will use the selected reasoner for all subsequent reasoning tasks.

**Key Insight:**

> "Protégé does not have a reasoner. It has a reasoner interface (API). The reasoner is a separate engine that Protégé invokes. This separation is what allows the ontology engineering community to experiment with different reasoning strategies without changing the ontology."

## 23.7.5 Hands-On Comparison: Running the Same Ontology Through Multiple Reasoners

One of the most valuable exercises in ontology engineering is to run the same ontology through multiple reasoners and compare the results. This reveals the strengths and limitations of each reasoner.

**Steps:**

1. Open the `Pizza` ontology in Protégé (after **Exercise 21**)
2. Select `Reasoner > Configure...` (or from the list of menu)
3. Choose one reasoner (e.g., HermiT)
4. Synchronize and record the inferred hierarchy
5. Repeat with each reasoner (Pellet, ELK, JFact)
6. Compare the results

**Expected Results**:

| Comparison | Expected Outcome | Explanation |
|---|---|---|
| **HermiT vs. Pellet** | Identical inferred hierarchies | Both support OWL 2 DL; complete reasoners should agree |
| **ELK vs. HermiT/Pellet** | ELK may produce incomplete inferences if ontology uses non-EL constructs | ELK supports only OWL 2 EL; ignores fails on DL constructs |
| **JFact vs. HermiT/Pellet** | Identical inferred hierarchies | JFact is a complete OWL 2 DL reasoner |

**Interpreting Differences:**

| Difference | Possible Cause | Action |
|---|---|---|
| Missing inference in ELK | ELK does not support the required OWL construct | Use a DL reasoner (HermiT/Pellet) |
| Different explanation text | Reasoner-specific explanation algorithms | Compare explanations manually |
| Performance difference | Reasoner optimization differences | Choose the faster reasoner for the task |
| Inconsistent results | Likely a bug in one reasoner, or ontology inconsistency | Validate the ontology with multiple reasoners |

**Key Insight:**

> "Differences between reasoners are not failures. They are information. They tell you about the expressively of your ontology and the capabilities of your tools."

## 23.7.6 When to Use Which Reasoner

The following table provides guidance on reasoner selection for common scenarios.

| Scenario | Recommended Reasoner | Rationale |
|---|---|---|
| **General OWL DL reasoning** | HermiT or Pellet | Complete, mature, good explanation support |
| **Large ontologies with EL profile** | ELK | Orders of magnitude faster than DL reasoners for EL |
| **Incremental reasoning** | Pellet | Supports incremental updates without full re-computation |
| **Explanation-heavy workflows** | HermiT or Pellet | Strong explanation features in Protégé |
| **High-performance production** | Konclude | Parallelized, optimized for large-scale reasoning |
| **RDF/SPARQL-centric workflows** | Jena | Integrated with Apache Jena ecosystem |
| **.NET ecosystem** | DotNetRDF | Native .NET integration |

**Key Insight:**

> "There is no single 'best' reasoner. The best reasoner depends on the ontology, the task, and the deployment environment. Understanding the trade-offs is part of professional ontology engineering."

## 23.7.7 The Engineering Takeaway

The reasoner landscape is diverse, and no single reasoner is best for all tasks.

The professional ontology engineer should:

- **Always test your ontology with more than one reasoner.** If reasoners disagree (with different results), investigate why.
- **Choose the reasoner that best fit your expressivity and performance requirements.**
- **Document which reasoner you used and why.** This is essential for reproducibility and governance.
- **Consider the production environment.** Protégé is a development tool, but production systems may use different reasoners.

**Key Insight:**

> "Reasoner independence is a quality check. If your ontology produces the same inferences under multiple reasoners, yo can be more confident in its correctness."

# 23.8 The Mathematical Foundations of Inference

Throughout this chapter, we have explored reasoning from a practical and engineering perspective. Now we turn to the mathematical foundations that make reasoning **precise**, **reliable**, and **trustworthy**.

Understanding these foundations is not required to use a reasoner effectively, but it provides valuable insight into why reasoning works and why it can be trusted.

## 23.8.1 Model-Theoretic Semantics: Interpretations and Models

The meaning of an OWL ontology is defined by model-theoretic semantics. This is the mathematical framework that gives OWL its precise meaning.

An interpretation $I = (\Delta^I, \cdot^I)$ consists of:

- A non-empty domain $\Delta^I$ of individuals
- An interpretation function $\cdot^I$ that maps:
  - Each class $\mathrm{C}$ to a subset $\mathrm{C}^I \subseteq \Delta^I$
  - Each property $\mathrm{R}$ to a binary relation $\mathrm{R}^I \subseteq \Delta^I \times \Delta^I$
  - Each individual $\mathrm{a}$ to a element $\mathrm{a}^I \in \Delta^I$

**Satisfaction**: An interpretation $I$ **satisfies** an axiom $a$ if the axiom holds under $I$. We write $I \models a$.

**Model**: An interpretation $I$ is a **model** of an ontology $\mathcal{O}$ if $I$ satisfies every axiom in $\mathcal{O}$. We write $I \models \mathcal{O}$.

**Consistency**: An ontology $\mathcal{O}$ is **consistent** if it has at least one model.

**Key Insight:**

> "The model-theoretic semantics gives OWL its precise meaning. Every axiom is a constraint on interpretations. The reasoner searches for interpretations that satisfy all constraints -- or proves that none exist."

## 23.8.2 Subsumption: The Core Operation of Classification

The core mathematical operation in reasoning is **subsumption**.

**Definition** Class $\mathrm{C}$ is **subsumed by** class $\mathrm{D}$ if, in every model $I$ of the ontology, $\mathrm{C}^I \subseteq \mathrm{D}^I$.

**Formal definition:**

$$\mathrm{C} \sqsubseteq \mathrm{D} \text{ iff for every interpretation I: } \mathrm{C}^I \subseteq \mathrm{D}^I$$

**Subsumption as set inclusion:** Subsumption is the semantic counterpart of subset inclusion. $\mathrm{C} \sqsubseteq \mathrm{D}$ means "every instance of $\mathrm{C}$ is an instance of $\mathrm{D}$.

**Subsumption vs.** `SubClassOf` **:**

- `SubClassOf` is the syntactic axiom: $\mathrm{C}\ \mathrm{SubClass}\ \mathrm{D}$
- $\sqsubseteq$ is the semantic relation: $\mathrm{C} \sqsubseteq \mathrm{D}$
- The reasoner computes the semantic relation from the syntactic axioms

**Key Insight:**

> "Subsumption is the mathematical heart of classification. When the reasoner classifies an ontology, it is computing all valid subsumption relations. The inferred hierarchy is the set of all $\mathrm{C} \sqsubseteq \mathrm{D}$ relations that hold."

## 23.8.3 Entailment: The Logical Consequence Relation

**Definition:** An ontology $\mathcal{O}$ **entails** as statement $\phi$ (written $\mathcal{O} \models \phi$) if $\phi$ is true in every model of $\mathcal{O}$.

**Formal definition:**

$$\mathcal{O} \models \phi \text{ iff for every interpretation I: I} \models \mathcal{O}, \text{ then I} \models \phi$$

**Logical closure:** The set of all statements entailed by $\mathcal{O}$ is called the **logical closure** of $\mathcal{O}$. Reasoning is the process of computing this closure.

**Monotonicity:** Description Logics are **monotonic:** adding new axioms to an ontology can never invalidate existing entailments. If $\mathcal{O}_1 \models \phi$, then $\mathcal{O}_1 \cup \mathcal{O}_2 \models \phi$.

**Key Insight:**

> "Entailments is the mathematical definition of 'what follows from the ontology.' The reasoner computes entailments. When it classifies `MargheritaPizza` as a subclass of `CheesyPizza`, it has discovered an entailment: the ontology entails $\mathrm{MargheritaPizza} \subseteq \mathrm{CheesyPizza}$, even though this was never explicitly asserted."

## 23.8.4 Subsumption Testing via Unsatisfiability

There is a key theorem that connects subsumption to unsatisfiability:

**Theorem:** $\mathrm{C} \sqsubseteq \mathrm{D}$ if and only if $\mathrm{C} \sqcap \neg \mathrm{D}$ is unsatisfiable.

**Proof sketch:**

- If $\mathrm{C} \sqsubseteq \mathrm{D}$, then no individual can be in $\mathrm{C}$ but not in $\mathrm{D}$. Therefore, $\mathrm{C} \sqcap \neg \mathrm{D}$ has no instances -- it is unsatisfiable.
- Conversely, if $\mathrm{C} \sqcap \neg \mathrm{D}$ is unsatisfiable, then every instance of $\mathrm{C}$ must be an instance of $\mathrm{D}$. Therefore, $\mathrm{C} \sqsubseteq \mathrm{D}$.

**Engineering consequence:** Classification reduces to consistency checking. To determine whether $\mathrm{C} \sqsubseteq \mathrm{D}$, the reasoner checks whether $\mathrm{C} \sqcap \neg \mathrm{D}$ is consistent. If it is inconsistent, then $\mathrm{C} \sqsubseteq \mathrm{D}$.

**Key Insight:**

> "This theorem is the bridge between consistency checking (**Stage 5**) and classification (**Stage 6**). Classification is not a separate algorithm -- it is consistency checking applied to subsumption queries."

## 23.8.5 The Tableau Algorithm: A Decision Procedure for Description Logic

The **tableau algorithm** is a decision procedure for consistency and subsumption in Description Logic. It is the algorithm used by most OWL DL reasoners.

**How it works:**

1. **Negation normal form**: Transform the ontology into negation normal form (NNF) [2], where negation is applied only to atomic concepts.
2. **Completion graph**: Build a completion graph representing a potential model. Each node represents an individual; each edge represents a relationship.
3. **Expansion rules**: Apply expansion rules to decompose complex concepts:
   - $\sqcap$**-rule**: If $\mathrm{C} \sqcap \mathrm{D}$ is present, add both $\mathrm{C}$ and $\mathrm{D}$.
   - $\sqcup$**-rule**: If $\mathrm{C} \sqcup \mathrm{D}$ is present, create two branches: one with $\mathrm{C}$, the other with $\mathrm{D}$.
   - $\exists$**-rule**: If $\exists \mathrm{R}.\mathrm{C}$ is present, crete a new node $\mathrm{x}$ with $\mathrm{R}$-edge to it and add $\mathrm{C}$ to $\mathrm{x}$.
   - $\forall$**-rule**: If $\forall \mathrm{R}.\mathrm{C}$ is present and there is an $\mathrm{R}$-edge to $\mathrm{x}$, add $\mathrm{C}$ to $\mathrm{x}$.
4. **Clash detection**: If a node contains both $\mathrm{C}$ and $\neg \mathrm{C}$, a **clash** is detected. The branch is closed.
5. **Termination**: If all branches are closed, the ontology is inconsistent. If at least one branch remains open, the ontology is consistent.

**Example (Probe Class pattern):**

- To check whether `ProbeInconsistentTopping` is satisfiable, the tableau algorithm tries to construct a model.
- The model adds `CheeseTopping` and `Vegetabletopping` to the node (from the two `SubClassOf` axioms).
- It also adds $\neg \mathrm{CheeseTopping} \sqcup \neg \mathrm{VegetableTopping}$ (from the disjointness axiom).
- The $\sqcup$-rule creates two branches: one with $\neg \mathrm{CheeseTopping}$, one with $\neg \mathrm{VegetableTopping}$.
- Both branches contain a clash ($\mathrm{CheeseTopping}$ and $\neg \mathrm{CheeseTopping}$, or $\mathrm{VegetableTopping}$ and $\neg \mathrm{VegetableTopping}$).
- Both branches are closed. The class is unsatisfiable.

**Key Insight:**

> "The tableau algorithm is a systematic search for a model.
>
> - If it fines one, the ontology is consistent.
> - If it proves that no model exists, the ontology is inconsistent.
>
> Classification is the same algorithm applied to subsumption queries.

## 23.8.6 Decidability: The Guarantee of Termination

**Definition:** A logic is **decidable** if every reasoning task is guaranteed to terminate with a definitive answer.

**Description Logic decidability**: OWL DL is based on Description Logic, which is decidable. Every consistency check, subsumption query, and classification task terminates.

**Why decidability matters:**

- Without decidability, a reasoner might run forever.
- Decidability guarantees that reasoning is a reliable engineering tool.
- It is the reason OWL DL is used in production systems.

**Comparison with first-order logic:** Unrestricted first-order logic is **undecidable**. Description Logic is a carefully designed decidable fragment of first-order logic.

**Key Insight:**

> "Decidability is the engineering guarantee that makes reasoning practical. It is why OWL chose Description Logic over unrestricted first-order logic. The expressivity is lower, but the reliability is higher."

## 23.8.7 Computational Complexity: The Cost of Reasoning

**Complexity of OWL DL reasoning:**

- Consistency checking in OWL DL is **NExpTime-complete** in the worst case.
- Subsumption and classification have the same complexity.
- In practice, optimized reasoners handle large ontologies efficiently.

**Complexity of OWL EL reasoning:**

- Consistency checking in OWL EL is **PTIME-complete**.
- This is why ELK is orders of magnitude faster than DL reasoners for EL ontologies.

**Practical implications:**

- The worst-case complexity is rarely encountered in practice.
- Reasoner optimization (e.g., modularization, incremental reasoning) mitigates the cost.
- Ontology design decisions (e.g., avoiding unnecessary disjunction) affect performance.

**Key Insight:**

> "The complexity of reasoning is the price of expressivity. OWL DL is more expressive than OWL EL, but also more expensive. Choosing the right OWL profile is one trade-off between expressivity and performance."

## 23.8.8 From Mathematics to Engineering

The mathematical foundations of reasoning have direct engineering consequences.

The following table summarizes the connections:

| Mathematical Concept | Engineering Consequences |
|---|---|
| **Model-theoretic semantics** | Every axiom constrains interpretations; the reasoner searches for models |
| **Subsumption** | Classification computes all $\mathrm{C} \sqsubseteq \mathrm{D}$ relations |

| Mathematical Concept | Engineering Consequences |
|---|---|
| **Entailment** | Reasoning computes the logical closure of the ontology |
| **Unsatisfiability testing** | Subsumption reduces to consistency checking |
| **Tableau algorithm** | A systematic search for models; terminates with a definitive answer |
| **Decidability** | Reasoning is a reliable engineering tool |
| **Complexity** | Expressivity vs. performance trade-off; OWL profile selection matters |

**Key Insight:**

> "The mathematics of reasoning is not abstract theory. It is the foundation of every engineering decision: which OWL profile to use, which reasoner to choose, and how to design axioms for performance. Understanding the math makes you a better engineer."

# 23.9 From Aristotle to Modern Reasoners -- A Historical Perspective

The mathematical foundations examined in the previous section reveal how reasoning works.

But a deeper question remains:

> ***"Why does the human mind seek to automate reasoning?"***

The answer is not merely practical convenience. It is rooted in a philosophical tradition that stretches back more than two millennia -- a tradition that ontology engineering now brings to its logical conclusion by making reasoning not only human-readable, but machine-executable.

This section traces that intellectual lineage. It is not a diversion into abstract philosophy, but a recognition that the reasoner you invoke in Protégé is the culmination of one of humanity's oldest intellectual quests.

## 23.9.1 Aristotle's Syllogism: The First Reasoning Engine

More than two thousand years before computers existed, Aristotle (384-322 BCE) established two foundations of **formal logic.**

His most influential contribution was the **syllogism** -- a three-part argument consisting of two premises and a conclusion:

- All men are mortal.
- Socrates is a man.
- Therefore, Socrates is mortal.

The syllogism was the first formal system of **deductive reasoning.**

> It showed that valid conclusions could be derived form premises by applying rules of logic, independent of the content of the premises.

The connection to ontology engineering is direct. When the reasoner infers that `MargheritaPizza` is a subclass of `CheesyPizza`, it is performed a syllogism:

1. All pizzas with cheese are `CheesyPizza`.
2. `MargheritaPizza` is a pizza with cheese.
3. Therefore, `MargheritaPizza` is a `CheesyPizza`.

The form of reasoning is the same as Aristotle's. Only the notation has changed.

**Key Insight:**

> "When the reasoner infers that `MargheritaPizza` is a subclass of `CheesyPizza`, it is performing a syllogism. The form of reasoning is the same as Aristotle's. Only the notation has changed."

## 23.9.2 Leibniz's Calculus Ratiocinator: The Dream of Mechanical Reasoning

The next major milestone in the history of reasoning came during the seventeenth century, through the work of Gottfried Wilhelm Leibniz (1646-1716).

Leibniz dreamed of a universal symbolic language -- the **Characteristica Universalis** -- in which all human knowledge could be expressed with mathematical precision. He also proposed a **Calculus Ratiocinator** -- a symbolic reasoning system through which disagreements could be resolved by calculation rather than debate.

Leibniz famously imagined two scholars ending an argument by saying:

> ***"Let us calculate."***

Although the mathematics and computing technology necessary to realize this vision did not yet exist, the philosophical objective was remarkably similar to that of modern ontology engineering.

Today, OWL ontologies assign every concept a globally unique IRI, express semantic meaning through formal logic, and allow reasoners to derive conclusions automatically. The vision is no longer philosophical speculation. It has become practical engineering.

**Key Insight:**

> "Leibniz imagined a machine that could reason. Three centuries later, that machine exists -- not as a physical device, but as a **software reasoner**."

## 23.9.3 Frege's Predicate Logic: Making Reasoning Formal

The next major milestone occurred during the nineteenth century through the work of Gottlob Frege (1848-1925).

Where Aristotle's logic primarily analyzed categorical statements, Frege introduced **predicate logic**, providing a far more expressive mathematical language for representing knowledge.

Most importantly, predicate logic introduced **quantifiers**, allowing logical statements to describe entire collections of objects rather than individual examples.

Two quantifiers became fundamental:

| Quantifier | Meaning | Manchester OWL Equivalent |
|---|---|---|
| $\forall$ | For all | `only` |
| $\exists$ | There exists | `some` |

The existential restriction introduced in **Chapter (19)** ( `hasTopping some MozzarellaTopping` ) has deep historical roots. It expresses the Description Logic statement $\exists \text{hasTopping}.\text{MozzarellaTopping}$, whose logical origin can be traced directly to Frege's formal treatment of existential quantification.

Similarly, universal restrictions such as `hasTopping only CheeseTopping` correspond to universal quantification.

Frege supplied the mathematical langauge that eventually enabled logic to become computational rather than merely philosophical.

**Key Insight:**

> "Frege gave us the language. Description Logic gave us the computable subset. The reasoner gives us the engine!"

## 23.9.4 Description Logic: Making Logic Computable

Although predicate logic is extremely expressive, unrestricted first-order logic is computationally difficult. Many reasoning tasks become **undecidable**, meaning that no algorithm can guarantee termination for every possible knowledge base.

During the late twentieth century, researchers in knowledge representation sought a practical compromise. The result was **Description Logic (DL)**.

Rather than attempting to represent every possible logical statement, Description Logic deliberately limits the available language to carefully designed constructors that preserve **decidability**.

This engineering decision allows automated reasoners to perform important tasks such as:

- Satisfiability testing
- Consistency checking
- Subsumption computation
- Equivalence detection

- Automatic classification

In other words, Description Logic sacrifices unrestricted expressive power in exchange for reliable computation.

This balance explains why OWL is based upon Description Logic rather than unrestricted predicate logic.

**Key Insight:**

> "Description Logic is the engineering compromise between expressivity and computability. It is expressive enough to model complex domains, but restricted enough to guarantee termination."

### 23.9.5 The Modern Reasoner: From Theory to Tool

The publication of OWL (Web Ontology Language) by the World Web Consortium (W3C) represents the modern culmination of this intellectual journey.

OWL did not invent conceptual classification, logical definition, or formal reasoning.

Instead, it integrated ideas developed over more than those two thousand years into a practical engineering language for machine-understandable and machine-executable knowledge.

Modern OWL reasoners -- Pellet, HermiT, ELK, FaCT++, Konclude, etc. -- implement the tableau algorithm and its variants. They are the practical realization of centuries of logical theory.

They run in milliseconds on ontologies that would take humans hours to analyze manually.

**Key Insight:**

> "The reasoner is hte culmination of 2,000 years of logical thought. When you click `Synchronize reasoner...`, you are participating in one of humanity's oldest intellectual traditions."

## 23.10 Reasoning in Industrial Contexts

Throughout this chapter, we have explored reasoning from a theoretical and engineering perspective. But reasoning is not confined to academic exercises or Protégé demonstrations.

In industrial contexts, reasoning is essential for problem-solving and decision-making. It enables organizations to analyze complex systems, optimize processes, detect errors, manage risk, and ensure compliance with safety regulations.

This section explores how reasoning applies in industrial settings.

### 23.10.1 Why Reasoning Matters in Industry

In industrial engineering, logical reasoning helps analyze complex systems, optimize processes, and make data-driven decisions. It involves structured thinking to evaluate situations and solve problems effectively.

Reasoning is not an academic exercise -- it is a critical capability for industrial problem-solving and decision-making.

**Key Insight:**

> "In an industrial context, reasoning is not about deriving abstract truths. It is about **making better decisions**, **faster**, with **greater confidence** -- and with a **verifiable audit trail**.

### 23.10.2 Logical Reasoning for Complex Systems

Industrial systems are complex: many components, many interactions, many constraints, -- and in many circumstances, in very legacy technology stack.

Logical reasoning provides a structured approach to analyzing these systems.

It enables engineers to:

- Evaluate situations systematically

- Identify inconsistencies or conflicts
- Derive conclusions from known conditions
- Make data-driven decisions

**Industrial Example:**

> A manufacturing ontology defines `ProductionLine`, `Machine`, `MaintenanceTask` and `SafetyConstraint`. Using reasoning, the system can infer that if a machine is scheduled for maintenance, and a safety constraint prohibits operation during maintenance, then the production line must be halted. This inference is not explicitly asserted -- it is derived by the reasoner.

**key Insight:**

> "Logical reasoning in industry is not about abstract philosophy. It is about structured thinking that leads to effective problem-solving."

## 23.10.3 Problem-Solving Through Reasoning

Industrial problems are often complex and multi-faceted. Applying logical reasoning skills allows engineers to navigate complex problems, analyze data critically, and make informed decisions.

Reasoning allows engineers to:

- Analyze data critically
- Navigate complex problems methodically
- Make informed decisions based on logical consequences
- Streamline processes by automating inference

**Industrial Example:**

> A supply chain ontology defines `Supplier`, `Part`, `Inventory`, and `LeadTime`. When a supplier's lead time increases, the reasoner can infer which production schedules are affected, which orders are at risk, and which alternative suppliers could be used -- all without explicit programming.

**Key Insight:**

> "Reasoning transforms data into decisions. It is the bridge between raw information and informed action."

## 23.10.4 Automated Reasoning for Error Detection and Risk Management

Automated reasoning applies formal logic and structured knowledge to derive conclusions from known conditions.

It enhances human error detection by:

- Checking consistency across many rules
- Identifying violation sof constraints
- Detecting unsafe combinations that humans might miss

It enables organizations to manage risks effectively by:

- Providing an auditable trail of inferences
- Supporting regulatory compliance
- Automating the detection of risk conditions

**Industrial Example:**

> An industrial safety ontology defines `Worker`, `SafetyTraining`, `HazardousZone`, and `AccessPermission`. The reasoner can infer that a work without valid safety training should not have access to a hazardous zone -- and flag a violation if the permission exists. This is automated reasoning applied to safety compliance.

**Key Insight:**

> "Automated reasoning does not replace human judgment. If amplifies it -- by checking what humans cannot check at scale, and by catching what humans might overlook."

## 23.10.5 Safety and Compliance Through Reasoning

In industrial safety, reasoning is vital for understanding workplace hazards and ensuring compliance with safety regulations. It helps in identifying potential risks and implementing measures to mitigate them.

OWL ontologies can encode safety rules as axioms. The reasoner can then check compliance automatically.

**Industrial Example**:

> A regulatory ontology defines `Equipment`, `InspectionRequirement`, and `ComplianceStatus`. The reasoner can infer that any equipment whose last inspection exceeds the required interval is non-compliant -- and trigger a notification. This is reasoning applied to regulatory compliance.

**Key Insight:**

> "Safety and compliance are not checkboxes. They are logical consequences of well-designed rules. Reasoning makes those consequences explicit and actionable."

## 23.10.6 Reasoning as a Strategic Capability

Reasoning is not just a technical feature, it is a **strategic capability**.

Organizations that master reasoning can:

- Make more consistent decisions
- Respond faster to changing conditions
- Reduce risk through automated compliance
- Scale expertise beyond individual experts

**Key Insight:**

> "In the industrial context, reasoning is the difference between a system that stores data and a system that makes decisions.
>
> - The first is a database.
> - The second is an intelligent system."

## 23.10.7 Deeper Industrial Semantic Context -- Grounding Reasoning in the ISA-95 Standard

As ontology engineering expands from abstract domains into industrial manufacturing, energy, and supply chain automation, a critical challenge emerges:

> **how to connect real-time sensor data and control systems within higher-level semantic reasoning?**

An automated reasoner **cannot** evaluate operational efficiency, equipment safety, or batch quality if it operates in a vacuum. If requires a standardized framework that defines how physical assets, materials, personnel, and processes relate to one another.

In industrial automation, this framework is universally provided by **ISA-95 (ANSI/ISA-95 / IEC 62264)**, the international standard for enterprise-control system integration.

### 23.10.7.1 The ISA-95 Functional Hierarchy and Time Horizons

The ISA-95 standard structures manufacturing enterprises into a functional hierarchy spanning from physical processes up to business planning:

- **Level 0 -- The Physical Process**: The actual raw materials, chemical reactions, and physical machinery.
- **Level 1 & Level 2 -- Sensing, Manipulation, and Control**: PLCs, actuators, sensors, and SCADA systems operating in real-time milliseconds.
- **Level 3 -- Manufacturing Operations Management (MOM / MES)**: Production, quality, maintenance, and inventory management operating on event- and shift-driven horizons.
- **Level 4 -- Business Planning and Logistics (ERP)**: Enterprise resource planning operating on weekly, monthly, or quarterly horizons.

While Level 1 and Level 2 handle high-frequency, real-time control, **Level 3 and Level 4 require semantic reasoning** to aggregate, contextualize, and translate raw telemetry into business-meaningful events.

### 23.10.7.2 Why ISA-95 Object Models Matter for Ontology Reasoning

A major bottleneck in industrial AI and knowledge graph projects is the **lack of a common vocabulary.** Different systems use conflicting identifiers for the same physical assets.

ISA-95 solves this by defining standardized object models for:

- **Equipment Hierarchies**: Enterprise → Site → Area → Work Center → Work Unit → Equipment Module
- **Physical Assets**: Machinery, tools, and instruments
- **Materials**: Raw components, intermediate work-in-progress (WIP), and finished products
- **Personnel**: Roles, competencies, and labor tracking

When we build industrial ontologies within the **Executable Knowledge Architecture (EKA)** framework, mapping our classes and object properties directly to ISA-95 object models ensures that our semantic descriptions match standard industrial data structures (often serialized via B2MML or integrated through OPC UA companion specifications).

### 23.10.7.3 Reasoning Over Industrial Hierarchies in Practice

Consider an industrial scenario where a machine sensor reports an anomaly.

A flat database sees only a tag value (e.g., `Sensor_402_Temp = 95°C`).

An ISA-95-aligned ontology combined with Description Logic (DL) reasoning transforms this data point into executable knowledge, mapping to stages in SKDL:

1. **Structural Inheritance (Stage 1 & 2)**: The reasoner knows via the ISA-95 equipment hierarchy that `Reactor_01` is a member of `WorkUnit`, which belongs to `Polymerization_Area`.
2. **Semantic Description (Stage 2)**: The ontology defines that any `WorkUnit` assigned to a high-temperature process must respect specific thermal limits.
3. **Automated Reasoning (Stage 6)**: When the temperature threshold breached, the reasoner automatically classified the asset not merely as a failing sensor, but as an *Affected Production Resource* impacting the current active `productionOrder`.

By grounding industrial reasoning in ISA-95 models, the ontology bridges the gap between the shop floor and enterprise logic. It turns thousands of disparate telemetry points into a coherent queryable, and logically consistent semantic model ready for automated decision-making and operational execution.

Production platforms such as Rhize (https://rhize.com/) implement this pattern directly, using ISA-95 as the native schema of a real-time, event-driven manufacturing data hub -- a concrete example of the reasoning-to-execution pipeline described throughout this chapter (see Info Box, below).

> **Info Box -- Industrial Practice: Rhize Manufacturing Data Hub**
>
> Rhize (https://rhize.com/) is a real-time, event-driven manufacturing data hub built directly on the ISA-95 standard. Rather than treating ISA-95 as a documentation convention to be mapped after the fact, Rhize implements it as the **native schema** of its data platform: equipment hierarchies, material, personnel, and production events are all contextualized in an ISA-95-aligned model from the moment data is ingested.
>
> Three architectural choices make Rhize a compelling production-grade illustration of the reasoning principles discussed in this chapter:
>
> - **ISA-95 as the ontology, not the mapping.** Rhize's headless platform ingests data from edge to ERP to quality systems and stitches it into a standardized ISA-95 schema. This is precisely the "grounding" move described in Section 23.10.7.2 -- the ontology is not a separate artifact that must be reconciled with operational data; it ***is*** the operational data structure.
> - **Event-driven contextualization.** Rhize uses rules and custom business logic to contextualize raw telemetry into real-time events. This is the reasoning layer in operation: raw sensor readings become classified events (equipment stoppage, operator action, quality deviation) that downstream systems can act upon.
> - **GraphQL over a standardized schema.** Applications query Rhize's ISA-95-aligned model through a single GraphQL API, with query subscriptions supporting real-time feeds. This is a production illustration of the $K \rightarrow R$ pipeline that ultimately feeds $\Theta/\Phi$: the knowledge graph ($K$) is queried through reasoning-informed classifications ($R$), which trigger events ($\Theta$) and drive workflows ($\Phi$).
>
> Rhize is not a Protégé reasoner, and it does not use OWL DL in the academic sense. But it demonstrates the same engineering principle this chapter has developed: **a standardized semantic model, combined with rule-based contextualization, transforms raw data into executable knowledge.** The ISA-95 hierarchy provides the structural inheritance; the event rules provide the reasoning; the GraphQL API provide the query and subscription surface that makes the result actionable.

> For readers building industrial knowledge graphs, Rhize is a useful reference architecture: it shows what it looks like when ISA-95 is treated as a living ontology rather than a compliance document, and when reasoning is embedded in the data platform rather than bolted on as a separate reasoning service.

# 23.11 EKA Perspective -- Stage 6 Establishes the Reasoning Layer ($R$)

Throughout this chapter, we have explored reasoning from **practical**, **mathematical**, **historical**, and **industrial** perspective.

Now we turn to the architectural perspective:

> *how does reasoning fit into the Executable Knowledge Architecture?*

## 23.11.1 Reasoning and the EKA Tuple

Recall the formal EKA tuple introduced in **Chapter (00)** and reinforced throughout this book:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Where:

- $K$ -- Knowledge Graph layer
- $R$ -- Reasoning & Rules layer
- $\Theta$ -- Trigger layer
- $\Phi$ -- Execution layer
- $\Gamma$ -- Governance layer

Earlier chapters mapped their contributions to this tuple explicitly as below:

- **Chapter (18)** (Stage 1 -- Conceptual Modeling) establishes the foundational structure of $K$.
- **Chapter (19)** (Stage 2 -- Semantic Description) enriches $K$ with formal meaning and prepared the logical ground for $R$.
- **Chapter (20)** (Stage 3 -- Knowledge Reuse) strengthened $K$ and prepared $\Gamma$.
- **Chapter (21)** (Stage 4 -- Semantic Governance) establishes $\Gamma$ -- the Governance layer.
- **Chapter (22)** (Stage 5 -- Semantic Validation) verifies $\Gamma + K$ -- the correctness of the governed knowledge graph.
- **Chapter (23)** (Stage 6 -- Semantic Reasoning) now establishes $R$ -- the Reasoning & Rules layer.

## 23.11.2 The Contribution of Stage 6 to EKA

The contribution of **Stage 6** to each EKA component can be summarized as follows:

| EKA Component | Contribution of Stage 6 (Semantic Reasoning) |
|---|---|
| **$K$ -- Knowledge Graph** | The knowledge graph is enriched with inferred knowledge. The graph now contains both asserted and inferred triples. |
| **$R$ -- Reasoning & Rules** | **Primary contribution.** The reasoning layer is established. The ontology now has an active inference engine. |
| **$\Theta$ -- Trigger** | Inferred classification provide the conditions for triggers. A trigger on `HighRiskPatient` requires that `HighRiskPatient` be correctly classified by the reasoner. |
| **$\Phi$ -- Execution** | Execution decisions can now be based on inferred knowledge, not just asserted knowledge. This enables more intelligent and adaptive execution. |
| **$\Gamma$ -- Governance** | The reasoner enforces governance rules automatically. Disjointness axioms, domain/range constraints, and cardinality restrictions are all checked during classification. |

## 23.11.3 From $K$ to $R$: The Transition to Intelligence

Before **Stage 6**, the knowledge graph is **static** -- it contains only what was explicitly asserted by the engineer.

After **Stage 6**, the knowledge graph is **dynamic** -- it contains inferred knowledge that the reasoner derived.

This is the transition from **"data"** to **"intelligence.**

The reasoner is th engine that drives this transition. It takes the asserted axioms and computes their logical consequences. The result is a knowledge graph that is **richer**, **more connected**, and **more useful** than the sum of its asserted parts.

**Key Insight:**

> "The $K$ layer is the vocabulary. The $R$ layer is the grammar. Together, they produce meaningful sentences -- inferred knowledge that was never explicitly written (asserted)."

### 23.11.4 Reasoning as the Bridge to Execution

**Reasoning ($R$)** is the bridge between **knowledge ($K$)** and **triggering / execution ($\Theta$, $\Phi$)**.

Without reasoning, execution can only act on asserted knowledge. A trigger can only fire on conditions that were explicitly written. An execution rule can only reference facts that were explicitly asserted.

With reasoning, execution can act on inferred knowledge -- knowledge that the reasoner discovered itself. A trigger can fire on classifications that were derived by the reasoner. An execution rule can reference inferred facts.

This is what makes an ontology "**executable**": it can drive action based on logic, not just data.

**Key Insight:**

> "Reasoning is the engine that transforms a knowledge graph into a decision-making system. Without it, the ontology is a static library. With it, ontology is a proactive advisor."

## 23.12 Engineering Guidelines for Semantic Reasoning

The following guidelines distill the perspectives presented in this chapter into practical engineering practices.

| Guideline | Description |
|---|---|
| **Use Primitive Classes During Early Modeling** | Use primitive class ( `SubClassOf` ) for concepts that are still evolving. Convert to defined classes ( `EquivalentTo` ) once the concept has stabilized. |
| **Convert to Defined Classes When the Definition Is Stable** | Convert a primitive class to a defined class only after the definition has stabilized and the engineer is confident that it captures the intended meaning. |
| **Validate Before Reasoning** | Always validate first. Reasoning on an inconsistent ontology is dangerous. Ensure that **Stage 5 (Validation)** has been completed before relying on reasoning results. |
| **Reason Frequently, Not Just at the End** | Synchronize the reasoner after every significant change. Treat it as a design assistant, not a final exam. Frequent reasoning catches errors early. |
| **Design for Auto-Classification** | Design with the reasoner in mind. Ask: *"What do I want the reasoner to infer automatically?* Then write the definitions that make those inferences possible. |
| **Use the Explanation Feature** | When the reasoner produces an unexpected inference, use the explanation feature to understand why. This is the fastest way to diagnose modeling errors. |
| **Document Inferred Knowledge** | Document not just what the ontology asserts, but what it should infer. This makes the ontology's intended behavior explicit. |
| **Test with Multiple Reasoners** | Test your ontology with more than one reasoner. If reasoners disagree, investigate why. Document which reasoner you used and why. |
| **Choose the Right Reasoner for the Task** | Use Pellet or HermiT for DL ontologies, ELK for EL ontologies, Jena for RDF/SPARQL workflows, and Konclude for high-performance production. |

| Guideline | Description |
|---|---|
| **Apply Reasoning to Industrial Problems** | Use reasoning for error detection, safety compliance, risk management, and decision support. Reasoning is not just a technical feature -- it is a strategic capability. |

## 23.13 Key Concepts

| Concept | Description |
|---|---|
| Semantic Reasoning | The process of deriving new, logically entailed knowledge from an existing ontology through automated inference |
| Asserted Knowledge | Statements explicitly written in the ontology by the ontology engineer |
| Inferred Knowledge | Statements derived by the reasoner from the asserted axioms |
| Classification | The process of computing all inferred subclass relationships |
| Primitive Class | A class defined only through necessary conditions ( `SubClassOf` ); does not enable auto-classification |
| Defined Class | A class defined through necessary and sufficient conditions ( `EquivalentTo` ); enables auto-classification |
| Subsumption | The mathematical relation $\mathrm{C} \sqsubseteq \mathrm{D}$, meaning every instance of $\mathrm{C}$ is also an instance of $\mathrm{D}$ |
| Entailment | The relation $\mathcal{O} \models \phi$, meaning $\phi$ is true in every model of $\mathcal{O}$ |
| Model-Theoretic Semantics | The mathematical framework defining the meaning of OWL axioms |
| Interpretation | A structure $I = (\Delta^I, \cdot^I)$ consisting of a domain and an interpretation function |
| Model | An interpretation that satisfies all axioms of an ontology |
| Asserted Hierarchy | The class hierarchy as explicitly written by the engineer |
| Inferred Hierarchy | The class hierarchy as computed by the reasoner |
| Tableau Algorithm | The decision procedure used by OWL reasoners to check consistency and compute classification |
| Decidability | The property that reasoning is guaranteed to terminate with a definitive answer |
| NExpTime | The worst-case complexity class for OWL DL reasoning |
| PTIME | The complexity class for OWL EL reasoning |
| Pellet | A complete OWL 2 DL reasoner with incremental reasoning and explanation support |
| HermiT | A complete OWL 2 DL reasoner with hypertableau algorithm and strong explanation support |
| ELK | A highly optimized OWL 2 EL reasoner |
| JFact | A Java port of FaCT++, complete OWL 2 DL reasoner |
| Konclude | A high-performance C++ OWL 2 DL reasoner |
| Openllet | An open-source fork of Pellet |
| Jena | An Apache RDF framework with rule-based reasoning |
| DotNetRDF | A .NET RDF framework with basic reasoning |
| $R$ -- Reasoning & Rules Layer | The EKA layer established by SKDL Stage 6 |
| Automated Reasoning | The application of formal logic to derive conclusion from known conditions |
| Industrial Reasoning | The application of logical reasoning to industrial problem-solving, safety, and compliance |

## 23.14 Chapter Summary

This chapter introduced **Stage 6 -- Semantic Reasoning** of the Semantic Knowledge Development Lifecycle (SKDL), making the transition from verified knowledge to intelligent knowledge.

In **Chapter (22), we established **Semantic Validation** as the discipline of verifying that an ontology is logically consistent and fit for purpose. Validation gave us **trust.** But trust alone does not product **intelligence.**

This chapter addresses that gap by introducing **Semantic Reasoning** as the discipline of deriving new knowledge from a validated ontology.

Using **Exercise 20** and **Exercise 21** from Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial* as practical case studies, we explored the critical distinction between primitive classes and defined classes.

- **Exercise 20** created `CheesyPizza` as a primitive class using `SubClassOf`. The reasoner did not auto-classify any pizzas as `CheesyPizza`, because primitive classes express only necessary conditions.
- **Exercise 21** converted `CheesyPizza` to a defined class using `EquivalentTo`. The reasoner immediately auto-classified `MargheritaPizza` and other cheese-topped pizzas as subclasses of `CheesyPizza`, because defined classes express necessary and sufficient conditions.

We also explored the landscape of variable RDF reasoners beyond Protégé:

- **Pellet** and **HermiT** are complete OWL 2 DL reasoners with strong explanation support
- **ELK** is a highly optimized reasoner for OWL 2 EL ontologies
- **JFact** is a Java port of FaCT++
- **Konclude** is a high-performance C++ reasoner for large ontologies
- **Jena** and **DotNetRDF** provide RDF-centric reasoning in the Java and .NET ecosystems

We compared reasoning results across different reasoners and explained why differences occur -- and what they tell us about the ontology's expressivity and the reasoner's capabilities.

We then examined reasoning from multiple complementary perspectives:

- **Mathematical perspective** -- model-theoretic semantics, subsumption, entailment, the tableau algorithm, decidability, and computational complexity
- **Historical perspective** -- from Aristotle's syllogism through Leibniz's dream of mechanical reasoning to modern OWL reasoners
- **Engineering perspective** -- reasoning as automated knowledge discovery, with the reasoner as a design assistant
- **Architectural perspective** -- reasoning as the bridge between knowledge ($K$) and triggers/execution ($\Theta$, $\Phi$) in the EKA framework
- **Industrial perspective** -- reasoning for problem-solving, decision-making, error detection, risk management, safety, and compliance

Within the EKA framework, **Stage 6** establishes the $R$ -- **Reasoning & Rules** layer. The knowledge graph is no longer static -- it now contains both asserted and inferred knowledge. The reasoner is the engine that drives this transformation.

The progression established across the first six stages of SKDL can therefore be summarized as:

| SKDL Stage | Primary Outcome |
|---|---|
| Stage 1 -- Conceptual Modeling | Identify concepts and organize them into a coherent semantic taxonomy |
| Stage 2 -- Semantic Description | Enrich concepts with formal semantic meaning |
| Stage 3 -- Knowledge Reuse | Transform validated semantic knowledge into reusable assets |
| Stage 4 -- Semantic Governance | Maintain semantic integrity through policies and standards |
| Stage 5 -- Semantic Validation | Verify correctness before deployment |
| Stage 6 -- Semantic Reasoning | Derive new knowledge through automated inference |

Together, these six stages establish both the semantic foundation and the intelligence required for trustworthy knowledge systems.

> **The reasoner is the engine of this intelligence.**

## 23.15 Looking Ahead -- Tower Executable Knowledge

By the end of this chapter, the `Pizza` ontology is not only validated but also intelligent. The reasoner has auto-classified `MargheritaPizza` and other pizzas under `CheesyPizza`. The inferred hierarchy now contains knowledge that was never explicitly asserted.

But intelligence is not the end. It is the beginning of **action**.

Now, with a validated and intelligent ontology, we can ask the question that has been waiting so long since the beginning of our journey:

> ***"How do we act on this knowledge?"***

This is the purpose of **Stage 7 -- Executable Knowledge** of the Semantic Knowledge Development Lifecycle (SKDL).

**Stage 7** addresses the operationalization of semantic knowledge -- connecting the ontology to **triggers ($\Theta$)**, **execution ($\Phi$)**, and the systems that act on knowledge. While reasoning asked *"What new knowledge can we derive?"*, execution asks *"What actions should we take based on that knowledge?"*

**Exercise 22** through **Exercise 36** introduce the building blocks of executable knowledge:

- universal restrictions,
- closure axioms,
- enumerated classes,
- `hasValue` restrictions,
- cardinality restrictions,
- data properties,
- DL queries,
- SWRL rules, and
- SQwRL queries.

The next chapter therefore explores how professional ontology engineers transform semantic knowledge into executable intelligence -- the final stage of the Semantic Knowledge Development Lifecycle.

This progression reflects another enduring engineering principle:

> **Reasoning is the bridge between trust and intelligence. Execution is the bridge between intelligence and action.**

The journey continues.

Last updated at 2026-09-16

## References

1. Original figure uploaded by Natasha Noy here https://www.researchgate.net/figure/Protege-architecture-Protege-is-built-using-a-tiered-architecture-The-Protege-knowledge_fig3_6151206 ↩
2. NNF - Negation normal form, https://en.wikipedia.org/wiki/Negation_normal_form ↩

# Chapter 24 - Stage 7 of SKDL -- Executable Knowledge (Part 1): Introduction & Boundary Control

**From Intelligent Knowledge to Actionable Systems**

The first six stages of the Semantic Knowledge Development Lifecycle (SKDL) transformed an abstract conceptual vocabulary into a **validated**, **intelligent**, **machine-reasonable** knowledge asset.

- Conceptual Modeling gave us structure.
- Semantic Description gave us meaning.
- Knowledge Reuse gave us composability.
- Semantic Governance gave us discipline.
- Semantic Validation gave us trust.
- Semantic Reasoning gave us intelligence.

Now, as knowledge begins to flow from ontology models into **production systems**, a new challenge emerges:

> ***"How do we act on this knowledge?"***

Executable Knowledge answers this question.

It is the discipline of encoding semantic knowledge with sufficient logical constraints to be interpreted deterministically and to drive action -- closing the loop between reasoning and execution.

Executable Knowledge transforms an ontology from a system that *knows* into a system that *acts*.

This chapter -- the first of three covering Stage 7 -- examines **Exercise 22** and **Exercise 23** from Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial* through the **lens of executability**. Together, these two exercises introduces the first two execution primitives: universal restrictions and closure axioms. Chapter 25 and 26 will continue the exploration with enumerated classes, `hasValue` restriction, and cardinality restrictions.

These constructs are the **execution primitives** of an ontology. They are what separate a descriptive model from an operational one.

Along the way, we will examine executability from multiple complementary perspectives: mathematical, engineering, architectural, and industrial -- revealing that executable knowledge is not a new invention, but the natural culmination of the entire SKDL arc.

**Table of Content**


- 24.1 Chapter Introduction -- From Intelligence to Action
  - 24.1.1 The Journey So Far
  - 24.1.2 The New Question
  - 24.1.3 The Central Insight
  - 24.1.4 How Stage 7 Is Organized Across Three Chapters
- 24.2 Learning Objectives
- 24.3 Positioning Within SKDL -- Stage 7 Highlighted
- 24.4 What is Executable Knowledge?
  - 24.4.1 A Working Definition
  - 24.4.2 The Three Layers of Semantic Knowledge
  - 24.4.3 Executable Knowledge vs. Reasoning vs. Validation
  - 24.4.4 The Open World Assumption and Executability
- 24.5 Exercise 22 -- Universal Restrictions and Deterministic Classification
  - 24.5.1 The Engineering Objective
  - 24.5.2 Implementation: Creating the Universal Restriction
  - 24.5.3 The Result: What the Reasoner Does Not Do
  - 24.5.4 Why the Universal Restriction Is Not Enough
  - 24.5.5 Engineering Perspective: Exercise 22 as a Case-Study
- 24.6 Exercise 23 -- Closure Axioms and the Open World Assumption
  - 24.6.1 The Engineering Objective

- 24.6.2 Implementation: Adding Closure Axioms to `MargheritaPizza` and `SohoPizza`
- 24.6.3 Extending the Demonstration: Closure at the Individual Level
- 24.6.4 From Description to Specification
- 24.6.5 Why Closure Axioms Matter for Execution
- 24.6.6 Engineering Perspective: Exercise 23 as a Case Study
- 24.6.7 The Design Pattern Perspective
- 24.7 Chapter Summary
- 24.8 Looking Ahead -- A Bridge to Chapter 25


# 24.1 Chapter Introduction -- From Intelligence to Action

## 24.1.1 The Journey So Far

The first six stages of the Semantic Knowledge Development Lifecycle established the foundations of ontology engineering.

**Stage 1 -- Conceptual Modeling** identified the concepts that define a knowledge domain and organized them into a coherent semantic taxonomy.

**Stage 2 -- Semantic Description** enriched those concepts with formal meaning using Description Logic, OWL restrictions, and class expressions.

**Stage 3 -- Knowledge Reuse** transformed validated semantic knowledge into reusable assets that could be shared, extended, and specialized systematically.

**Stage 4 -- Semantic Governance** established the policies, standards, and processes required to maintain semantic integrity across an evolving knowledge ecosystem.

**Stage 5 -- Semantic Validation** verified that the ontology is logically consistent, semantically complete, and fit for purpose before deployment.

**Stage 6 -- Semantic Reasoning** derived new knowledge through automated inference -- computing the logical consequences of the asserted axioms.

By the end of **Stage 6**, the `Pizza` ontology had become intelligent. The reasoner auto-classified `MargheritaPizza` and other cheese-topped pizzas as subclasses of `CheesyPizza`. The inferred hierarchy now contained knowledge that was never explicitly asserted.

The ontology had become **intelligent**.

But intelligent is **not action**.

An intelligent ontology that cannot act is like a brilliant advisor who never speaks. It contains valuable knowledge, but it does not use that knowledge to change the world.

It is passive, not active!

This is where **Stage 7 -- Executable Knowledge** enters the lifecycle.

## 24.1.2 The New Question

Reasoning asked: ***What else can this model tell us?***

Executable Knowledge asks: ***What should happen, given what we know?***

The distinction is fundamental.

Reasoning derives new knowledge from a validated ontology. Executable Knowledge uses that derived knowledge to **drive action**.

Without reasoning, execution has no foundation.

Without execution, reasoning has no consequence.

Together, they produce actionable intelligence.

## 24.1.3 The Central Insight

The central insight of this chapter is deceptively simple:

> **Every OWL axiom you write is a potential execution rule. The question is not whether your ontology can be executed -- it is whether you have written the axioms that make execution possible.**

The reasoner infers facts.

But inference alone does not trigger action.

Action requires:

- **Determinism** -- the ontology must produce a single, unambiguous result
- **Completeness** -- the ontology must guarantee that required data is present
- **Closure** -- the ontology must define boundaries, not leave them open

These are not automatic. They must be **engineered** through deliberate axioms.

**Exercise 22** and **Exercise 23** introduce the first two of these axioms. The remaining three are introduced in Chapter 25 and 26.

### 24.1.4 How Stage 7 Is Organized Across Three Chapters

Stage 7 is the most engineering-intensive stage of the SKDL. Unlike the earlier stages -- each of which introduced a single core capability -- Stage 7 introduces **five distinct execution primitives**, each solving a different aspect of the problem of acting on knowledge:

| Primitive | Role | Chapter |
|---|---|---|
| **Universal Restriction** | Defines what is allowed | Chapter 24 |
| **Closure Axiom** | Closes the world for a class or individual | Chapter 24 |
| **Enumerated Class** | Defines a finite value domain | Chapter 25 |
| `hasValue` **Restriction** | Binds a property to a specific value | Chapter 25 |
| **Cardinality Restriction** | Enforces data completeness | Chapter 26 |

Because these five primitives build on each other and are best understood together, Stage 7 is covered across three consecutive chapters rather than one:

- **Chapter 24 (this chapter) -- Introduction & Boundary Control.** Establishes the definition of executable knowledge, the OWA problem, and the first two execution primitives: universal restrictions (**Exercise 22**) and closure axioms (**Exercise 23**). By the end of this chapter, you will have built the boundary layer of an executable ontology -- the mechanism that tells the reasoner what is allowed and what is closed.
- **Chapter 25 -- Value Specification & Precise Binding.** Introduces enumerated classes (**Exercise 24**) and `hasValue` restrictions (**Exercise 25**). By the end of that chapter, you will have built the value layer of the executable ontology -- the mechanism that binds properties to specific values.
- **Chapter 26 -- Data Integrity & Comprehensive Architecture.** Introduces cardinality restrictions (**Exercise 26**), consolidates the five primitives into a single anatomy, and examines executability from mathematical, historical, industrial, and EKA perspectives. By the end of that chapter, you will have the complete picture of Stage 7.

Each chapter can be read independently, but they are designed to be read in sequence. Together, they establish the $\Theta$ **- Trigger** and $\Phi$ **- Execution** layers of the Executable Knowledge Architecture.

**Key Insight:**

> "Executable knowledge is not built in a single step. It is built in layers -- first the boundary, then the value, then the integrity. Each layer depends on the one before it."

# 24.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

**Knowledge:**

- Explain why Executable Knowledge is the seventh stage of the Semantic Knowledge Development Lifecycle (SKDL)
- Distinguish between descriptive knowledge and executable knowledge

- Understand the role of universal restrictions, closure axioms, enumerated classes, `hasValue` restrictions, and cardinality restrictions in enabling execution
- Explain how the Open World Assumption affects executability
- Understand how OWL constructs become execution primitives in the EKA framework

**Practical Skills (Stage 7):**

- Create universal restrictions ( `only` ) in Protégé
- Define closure axioms using combinations of `only` and cardinality restrictions
- Create enumerated classes using `EquivalentTo`
- Define `hasValue` restrictions to bind properties to specific individuals
- Define cardinality restrictions ( `min` , `max` , `exactly` ) to enforce data completeness
- Test executability by running the reasoner against test individuals

**Mathematical Perspective:**

- Understand how closure axioms convert open-world semantics to closed-world specifications
- Explain the mathematical relationships between `some` , `only` , `exactly` , and `value` restrictions
- Understand how enumeration defines a finite value domain
- Explain how cardinality restrictions constrain the extension of a class

**Historical Perspective:**

- Trace the concept of executability from early expert systems to modern rule engines
- Understand how the "knowledge is action" principle connects to cybernetics, control theory, and business process management

**Engineering Perspective:**

- Understand execution as the operationalization of semantic knowledge
- Explain why closure axioms are necessary for deterministic execution
- Map executable knowledge to the $\Theta$ (Trigger) and $\Phi$ (Execution) layers in the EKA framework
- Apply executable knowledge to industrial scenarios (manufacturing, compliance, safety)

**Note on scope.**

These learning objectives span the three chapters that covers Stage 7. Chapter 24 addresses the first two -- universal restrictions and closure axioms. Chapter 25 and 26 address the remaining three.

## 24.3 Positioning Within SKDL -- Stage 7 Highlighted

As introduced in **Chapter (17)**, ontology engineering progresses through seven stages of the Semantic Knowledge Development Lifecycle (SKDL), each stage contributing a distinct engineering objective toward the construction of **executable semantic knowledge systems.**

Having completed **Stage 6 -- Semantic Reasoning** in the previous chapter, this chapter advances to **Stage 7 -- Executable Knowledge.**

The objective of this stage is to operationalize semantic knowledge -- to encode the ontology with the constraints required for **deterministic**, **actionable** execution.

At the completion of **Stage 6**, the `Pizza` ontology contained an inferred hierarchy that included knowledge derived by the reasoner. However, the ontology was still **descriptive**. It could tell us that `MargheritaPizza` is a subclass of `CheesyPizza` . But it could not yet tell us what to ***do*** about it.

**Stage 7** addresses this gap by introducing:

- **Universal Restrictions** -- encoding exclusion and deterministic classification
- **Closure Axioms** -- converting open-world descriptions to closed-world specifications
- **Enumerated Classes** -- defining finite value domains
- `hasValue` **Restrictions** -- binding properties to specific values
- **Cardinality Restrictions** -- enforcing data completeness

> **Note**
>
> "This chapter covers the first two primitives -- universal restrictions and closure axioms. Chapter 25 and 26 will cover the remaining three."

Rather than introducing new semantic constructs, this stage establishes the engineering discipline required to make semantic knowledge actionable.

The primary deliverable of **Stage 7** is therefore:

> **not a new logical axiom, but an executable ontology -- one whose inferred knowledge can drive triggers, execute actions, and produce real-world effects.**

Within the EKA framework, **Stage 7** establishes the $\Theta$ **-- Trigger** and $\Phi$ **-- Execution** layers by providing the semantic conditions that fire triggers and the action bindings that execute them.

The progression across all seven stages can now be seen in full:

| SKDL Stage | Primary Outcome | EKA Contribution |
|---|---|---|
| Stage 1 -- Conceptual Modeling | Identify concepts and organize them into a coherent semantic taxonomy | $K$ (Knowledge Graph) (Structure) |
| Stage 2 -- Semantic Description | Enrich concepts with formal semantic meaning through Description Logic and OWL restrictions | $K$ (Meaning) + $R$ (Readiness) |
| Stage 3 -- Knowledge Reuse | Transform validated semantic knowledge into reusable assets | $K$ (Composable) + $\Gamma$ (Prepared) |
| Stage 4 -- Semantic Governance | Maintain semantic integrity through policies, standards, and lifecycle management | $\Gamma$ (Governance) |
| Stage 5 -- Semantic Validation | Verify correctness before deployment through consistency checking and constraint validation | $\Gamma$ + $K$ (Validation) |
| Stage 6 -- Semantic Reasoning | Derive new knowledge through automated inference | $R$ (Reasoning & Rules) |
| Stage 7 -- Executable Knowledge | Operationalize semantic knowledge for deterministic action | $\Theta$ (Trigger) + $\Phi$ (Execution) |

Each stage has built upon the previous one.

- **Stage 1** gave us concepts.
- **Stage 2** gave us meaning.
- **Stage 3** gave us reusability.
- **Stage 4** gave us governance.
- **Stage 5** gave us trust.
- **Stage 6** gave us intelligence.
- **Stage 7** now gives us **action.**

# 24.4 What is Executable Knowledge?

## 24.4.1 A Working Definition

**Executable Knowledge** is semantic knowledge that has been encoded with sufficient logical constraints to be interpreted deterministically and to drive action.

It answers the question:

> ***"What should happen, given what we know?"***

Not "what is true?" (that is reasoning).

Not "what is correct?" (that is validation).

But "what should be **done**?"

This is fundamentally an **operationalization** activity. It is the moment when the ontology becomes a decision system -- when its axioms do not merely describe the world but specify what to do about it.

**Key characteristics of executable knowledge:**

- **Deterministic** -- Given the same facts, the same action is triggered. No ambiguity.
- **Closed** -- The ontology specifies boundaries, not open-ended descriptions.
- **Complete** -- The ontology guarantees that required data is present before acting.
- **Actionable** -- The ontology binds semantic conditions to executable actions.

**Key Insight:**

> "Reasoning tells us what is true. Executable Knowledge tells us what to do about it. Without executability, the ontology is a library. With it, the ontology is a decision system."

## 24.4.2 The Three Layers of Semantic Knowledge

Semantic knowledge in an ontology operates at three distinct layers:

| Layer | Question | OWL Constructs | Stage |
|---|---|---|---|
| **Description** | What is true? | `SubClassOf` , `some` , `only` | Stage 1-2 |
| **Validation** | What is trustworthy? | Disjointness, domain/range, SHACL | Stage 4-5 |
| **Execution** | What should happen? | Closure axioms, cardinality, `hasValue` , enumerated classes | Stage 7 |

The **description layer** defines concepts. The **validation layer** ensures correctness. The **execution layer** enables action.

Without the execution layer, the ontology is correct but passive. With it, the ontology is actionable.

**Key Insight:**

> "Description without validation is unchecked. Validation without execution is theoretical. Execution without description and validation is dangerous. The three layers together form a complete semantic system."

## 24.4.3 Executable Knowledge vs. Reasoning vs. Validation

It is important to distinguish executability from reasoning and validation:

| Activity | Question | Timing | Output |
|---|---|---|---|
| **Validation** | "Is this model correct?" | Before reasoning | Pass/fail -- the ontology is consistent or not |
| **Reasoning** | "What else can this model tell us?" | After validation | Inferred facts -- new classification, relationships |
| **Executable Knowledge** | "What should happen?" | After reasoning | Actions -- triggered operations, workflow steps |

**Validation** ensures correctness. **Reasoning** derives new knowledge. **Executable Knowledge** acts on that knowledge.

The key distinction is this: **reasoning produces facts; executability produces actions.**

A reasoned ontology knows that a patient is `HighRiskPatient` , then an executable ontology sends an alert to the clinician.

**Key Insight:**

> "Reasoning is the brain. Execution is the hand. A brain without a hand is a philosopher. A hand without a brain is a hazard. Together, they produce intelligent action."

### 24.4.4 The Open World Assumption and Executability

The Open World Assumption (OWA) is foundational to OWL. It states that just because something is not asserted does not mean it is false. This is correct for reasoning -- but it creates a problem for execution.

Consider a pizza with `hasTopping some MozzarellaTopping`. Under OWA, the reasoner cannot conclude that the pizza has ***only*** mozzarella. There might be other toppings we simply don't know about.

For reasoning, this is correct.

For execution, it is problematic.

An execution system that recommends a pizza based on an incomplete description may recommend the wrong thing.

**Key Insight:**

> "The Open World Assumption is correct for reasoning and dangerous for execution. Execution requires the closed world. **Closure axioms** are the bridge."

This tension is the reason **Exercise 22 to 26** exist. They introduce the OWL constructs that allow us to ***close*** the world when execution requires it.

## 24.5 Exercise 22 -- Universal Restrictions and Deterministic Classification

Having established the principles of executable knowledge, we now turn to its practical implementation through **Exercise 22** in Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial*.

This exercise represents the first hands-on activity of **Stage 7 -- Executable Knowledge** within the Semantic Knowledge Development Lifecycle (SKDL).

Unlike previous exercises, which introduced individual OWL constructs or semantic modeling patterns, **Exercise 22** focuses on the construct that makes deterministic classification possible: **the universal restriction**.

### 24.5.1 The Engineering Objective

**Exercise 22** introduces the **universal restriction**, represented in OWL Manchester Syntax by the keyword `only`.

The universal restriction specifies that **all** values of a property must satisfy a specified condition. It does not require that any values exist -- only that, if they exist, they must satisfy the condition.

The definition for a vegetarian pizza, for example, might be:

```
Class: VegetarianPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping only (CheeseTopping or VegetableTopping)
```

This means: "Every topping on a vegetarian pizza must be either a cheese topping or a vegetable topping."

The engineering objective of **Exercise 22** is to introduce the `only` construct and demonstrate how it enables deterministic classification -- the ability to ***execute*** as well as include.

**Key Insight:**

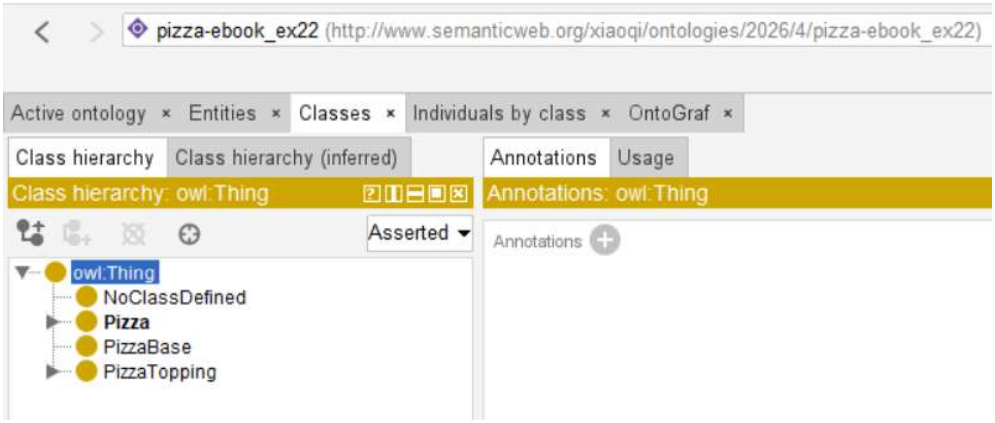
> - Existential restrictions tell the reasoner what must be present.
> - Universal restrictions tell the reasoner what must not be present.
>
> **Execution needs both.**

### 24.5.2 Implementation: Creating the Universal Restriction

Let's walk through the implementation step by step. As you'll see, the exercise **deliberately** includes a moment where the expected result does ***not*** appear -- and that moment is the most important part of the lesson.

**Step 1: Open the `Pizza` ontology**

The RDF file for this exercise is named as `pizza-ebook_ex22.rdf`. You may find it in the ebook's `\rdf` folder. Open it in Protégé. Navigate to the Classes tab. (Tip: you may open `pizza-ebook_ex21.rdf` as the starting ontology snapshot.)

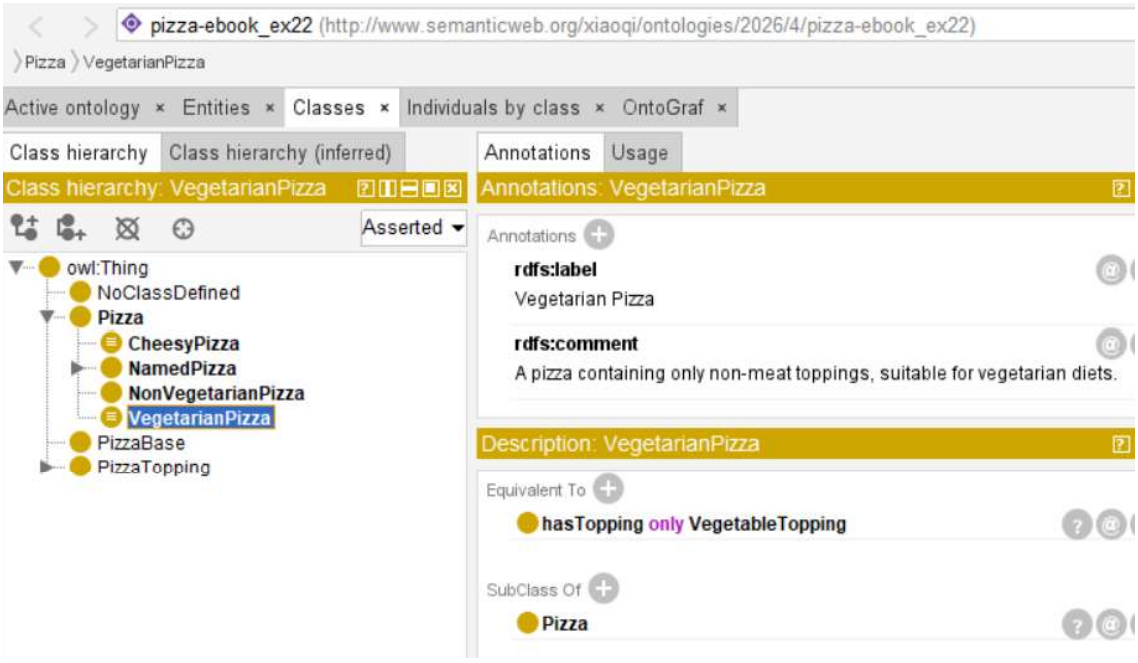



**Step 2: Select `VegetarianPizza`**

Select `VegetarianPizza` in the class hierarchy. In a previous exercise, we have already defined this class as

```
Class: VegetarianPizza
  SubClassOf: Pizza
  SubClassOf: hasBase some PizzaBase
  EquivalentTo: hasTopping only VegetableTopping
```

We will update it now to match **Exercise 22.**




**Step 3: Add the universal restriction**

1. In the Description view, locate the **SubClassOf** section
2. Click the Add icon (+)
3. In the Class Expression Editor, enter `hasTopping only (CheeseTopping or VegetableTopping)`
4. Click OK

After this step, `VegetarianPizza` has:

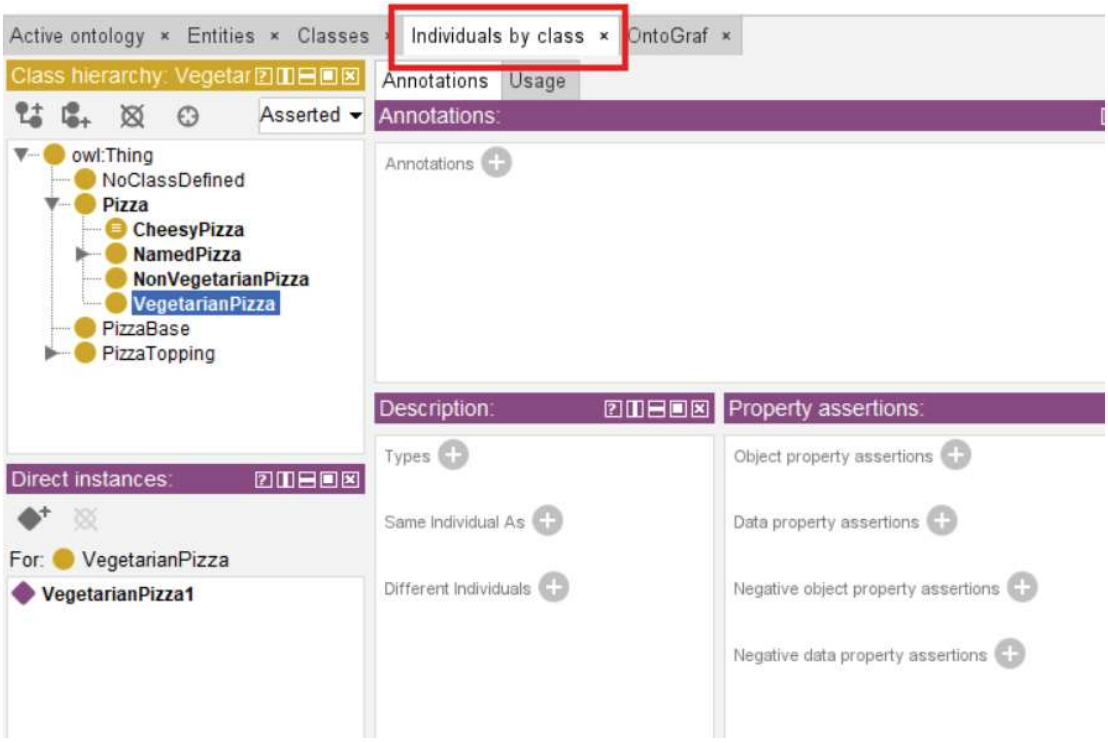
```
Class: VegetarianPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping only (CheeseTopping or VegetableTopping)
```

The ontology after this step is as below:

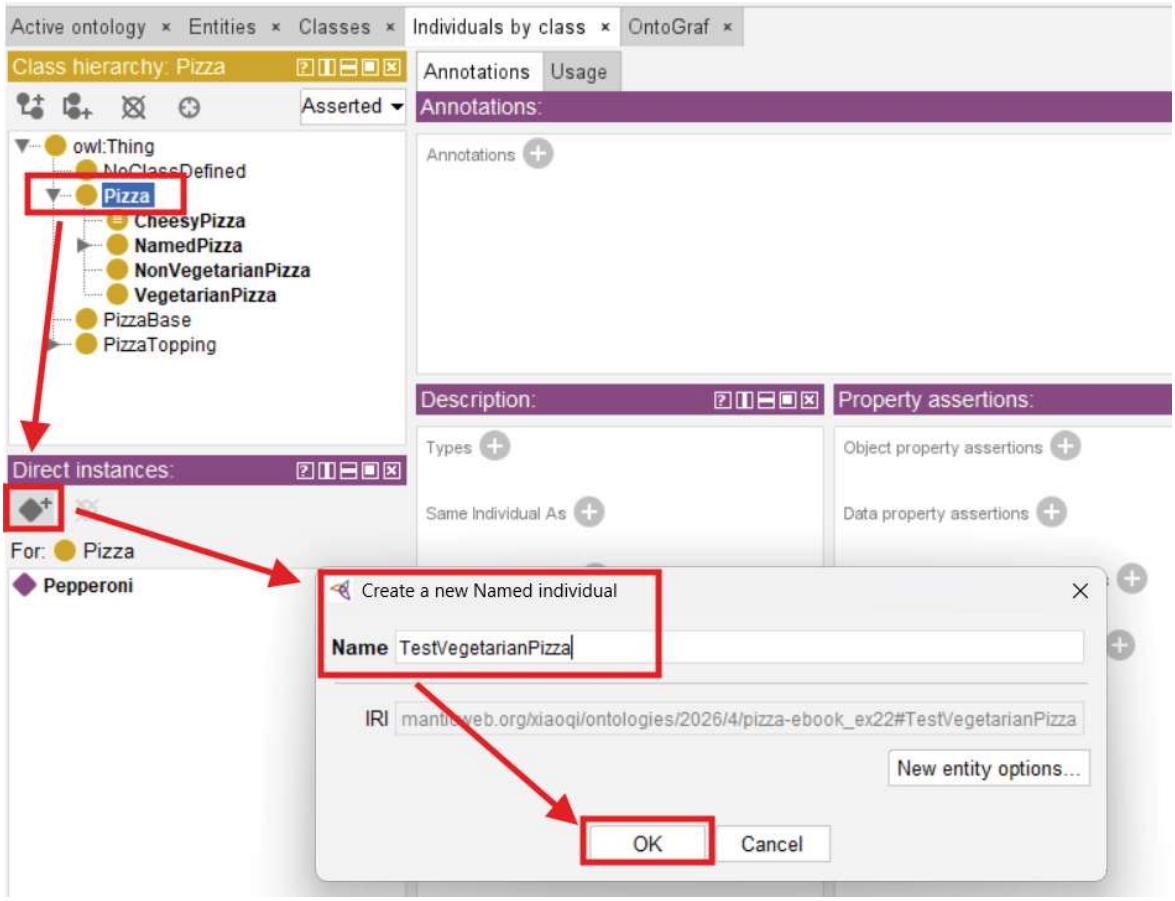
**Step 4: Add a test individual**

To observe the effect of the universal restriction, let's add a test individual.

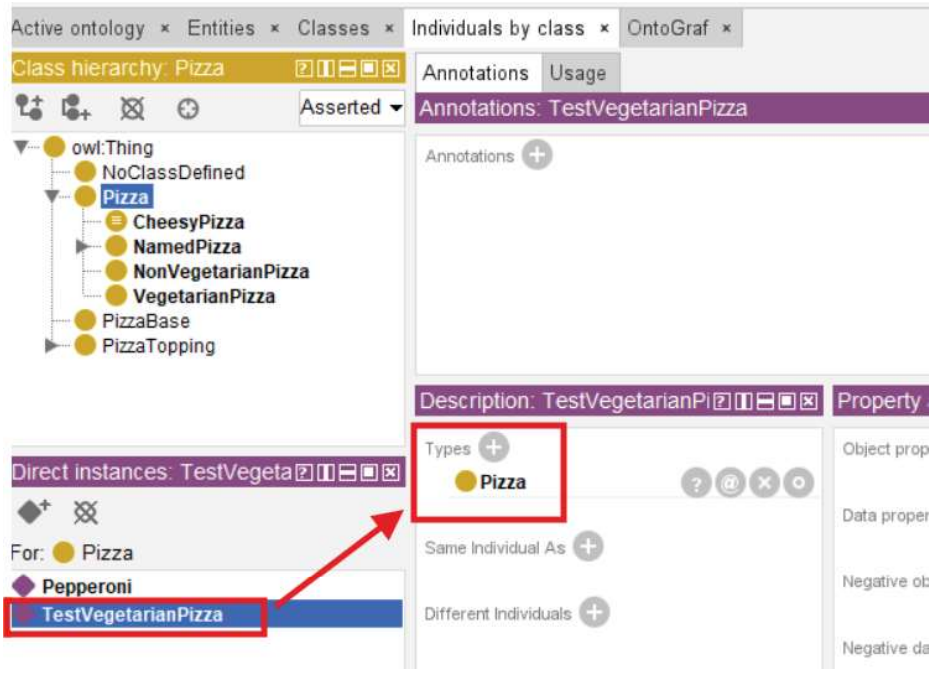
1. Navigate to the `Individuals by class` tab




2. Keep class `Pizza` selected in the class hierarchy, create a new individual named `TestVegetarianPizza`

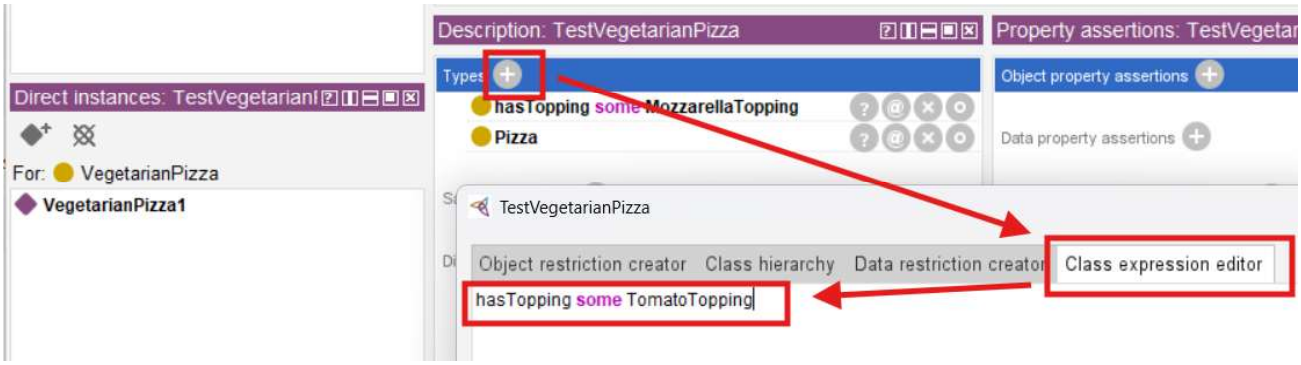



3. After click `OK` , notice the type of individual is `Pizza` now

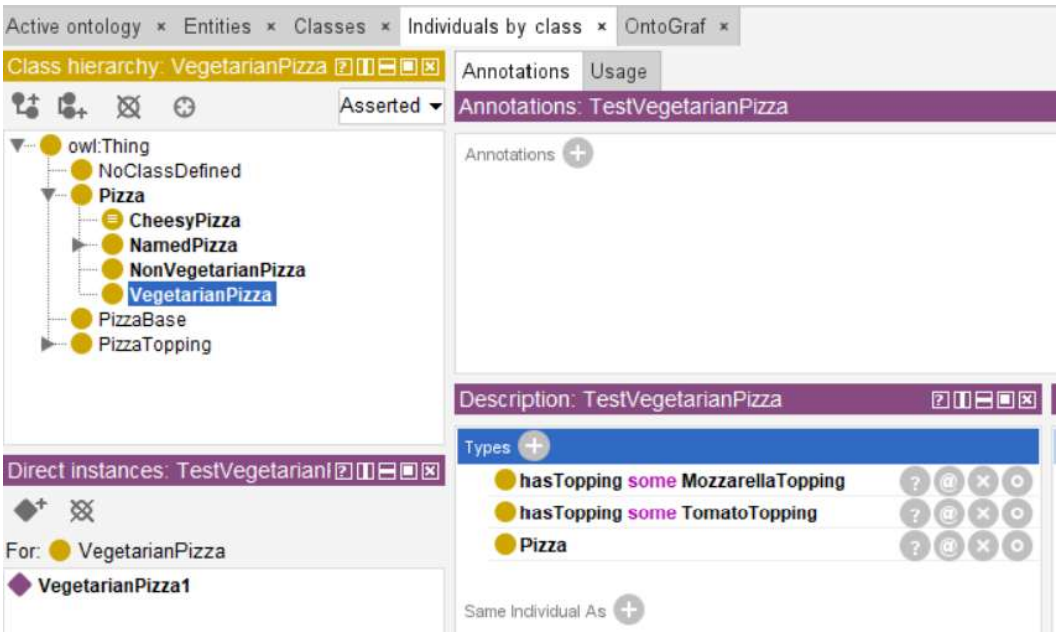



4. Add the property assertion `hasTopping some MozzarellaTopping`

5. Add the property assertion `hasTopping some TomatoTopping`



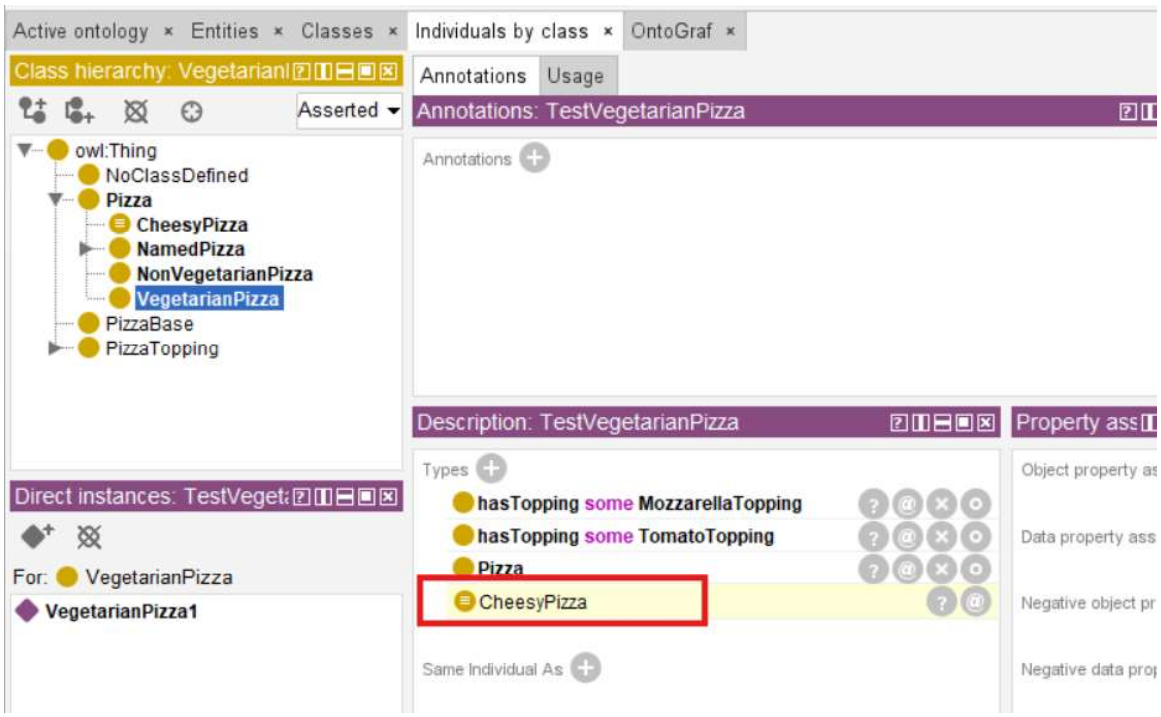

6. Now ensure your ontology is looked like this




**Step 5: Synchronize the reasoner**

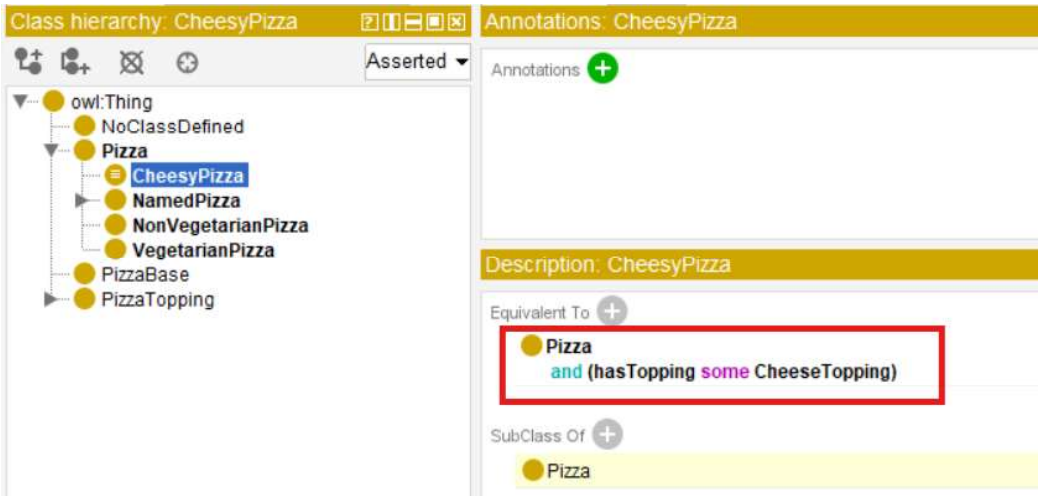
Select `Reasoner > Synchronize reasoner` (Ctrl+R).

Observe the result.




With the ontology designed so far, the reasoner classifies `TestVegetarianPizza` as a `CheesyPizza`, because we have defined `CheesyPizza` earlier using `EquivalentTo` (defined class) and `TestVegetarianPizza` has toppings `MozzarellaTopping` (a `CheeseTopping`):

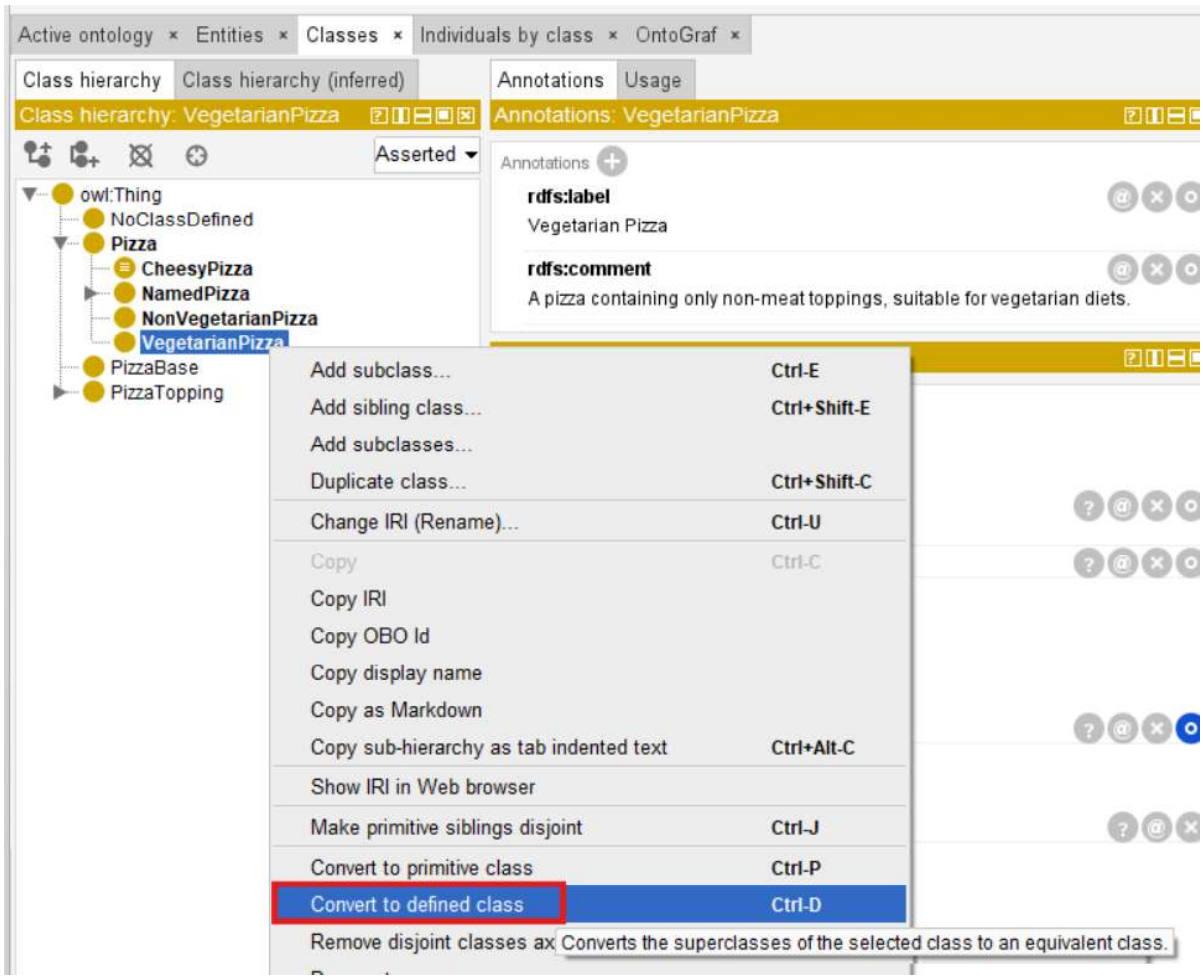
```
Class: CheesyPizza
  Equivalent To: Pizza and (hasTopping some CheeseTopping)
  SubClass Of: Pizza
```




But until now the `VegetarianPizza` is still under only `SubClassOf` (primitive class), so based on Open World Assumption (OWA), the reasoner cannot infer `TestVegetarianPizza` as a type of `VegetarianPizza`.

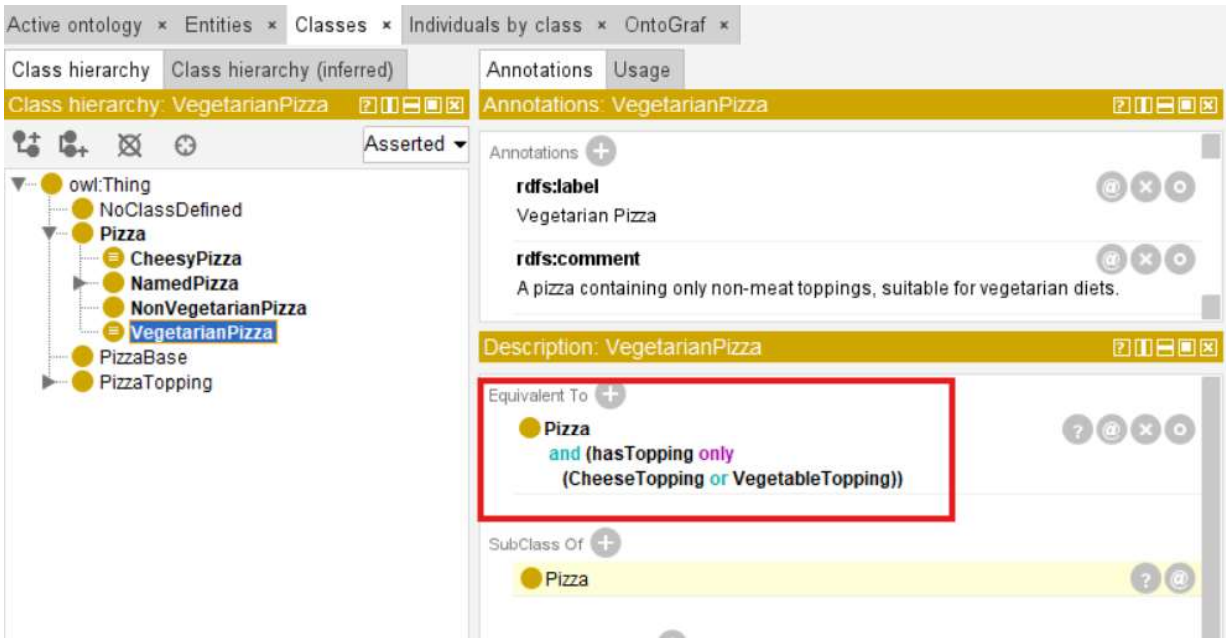
**Step 6: Convert `VegetarianPizza` to a defined class**

This is the crucial step. Select `VegetarianPizza` in the Classes tab, then select `Edit > Convert to defined class` (or from right-click menu).




Observe what happens in the Description view. The `SubClassOf` axiom for the universal restriction is moved to `EquivalentTo`:

```
Class: VegetarianPizza
  Equivalent To: Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
  SubClass Of: Pizza
```




**Note**

The `VegetarianPizza` is now shown with three horizontal stripes in the class icon -- the visual indicator that it is a defined class.

**Tip**

**Following the Tutorial**

The conversion you just performed -- `Edit > Convert to defined class` on `VegetarianPizza` -- is exactly **Step 4** of **Exercise 22** in the original `Pizza` Tutorial. In the tutorial, this step occurs *after* adding the universal restriction and *before* running the reasoner.

In this book, we have re-ordered the steps slightly: we added `TestVegetarianPizza` *before* converting to a defined class, so that you can observe the OWA problem as it happens -- first without the conversion (Step 5), and then with it (Step 7-8). Both orderings produce the same final ontology state. The tutorial's minimal path is: add restriction → convert to defined class → synchronize. Our path adds the test individuals in between, to make the OWA lesson visible.

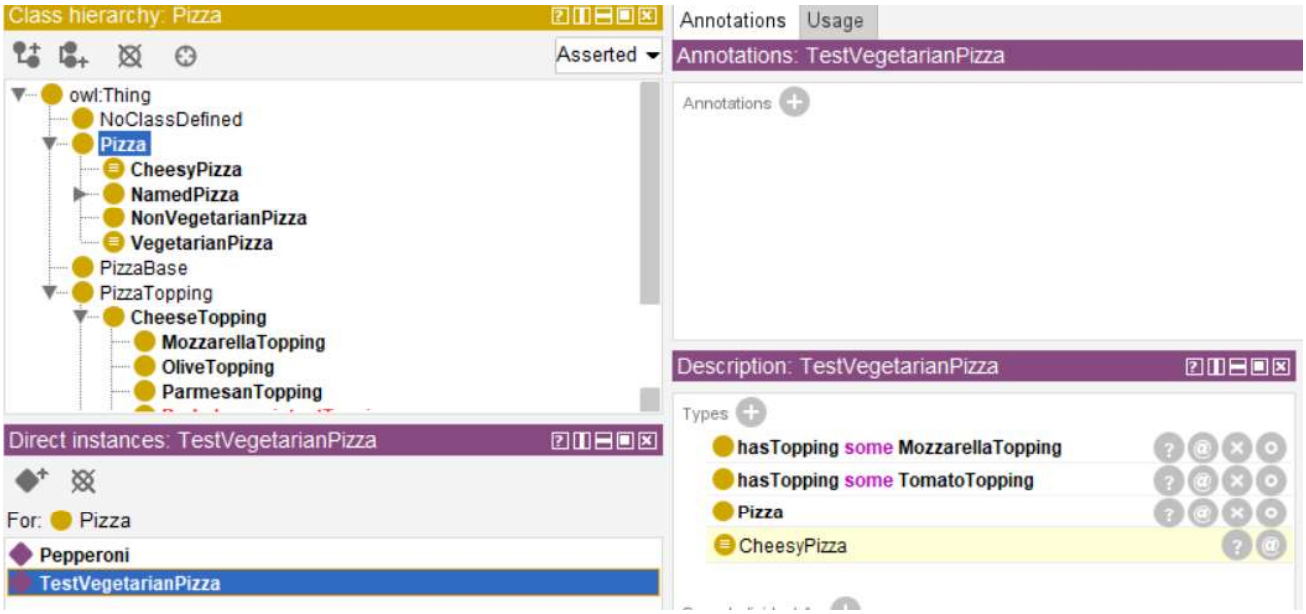
If you are following along with Michael DeBellis' tutorial in parallel, you can safely perform the conversion now as Step 4 of the tutorial, and then return to observe the OWA behavior when you reach Step 8 in our book.

**Step 7: Synchronize the reasoner again**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)

**Step 8 -- Observe the result -- and be surprised!**

Look at `TestVegetarianPizza` in the **Individual by class** tab.




You will see that the reasoner has kept classifying `TestVegetarianPizza` as:

- ✅ `CheesyPizza` (because it has at least one cheese topping -- mozzarella)

But it has **NOT** classified `TestVegetarianPizza` as:

- ❌ `VegetarianPizza`

This is surprising. `TestVegetarianPizza` has only mozzarella and tomato toppings. Both are vegetarian. **Why is it not a `VegetarianPizza` ?**

**Step 9 -- Pause and understand**

This result is not a bug. It is still the **Open World Assumption (OWA)** in action, although we have converted `VegetarianPizza` from primitive class to defined class, and it is one of the most important lessons in ontology engineering.

When we define `VegetarianPizza` as:

```
Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
```

we are saying: **"All toppings must be `CheeseTopping` or `VegetableTopping` .**

But `TestVegetarianPizza` only has two *asserted* toppings. Under the Open World Assumption, the reasoner cannot conclude that these are the *only* toppings -- there might be other toppings that exist in the world (somewhere, anywhere) not been added to the ontology yet.

The reasoner can only classify an individual into a defined class when it has enough information to prove that the individual satisfies the definition. And it does not have enough information here.

**Why `CheesyPizza` works but `VegetarianPizza` doesn't**:

| Class | Definition | Reasoner behavior |
|---|---|---|
| `CheesyPizza` | `Pizza and (hasTopping some CheeseTopping)` | Existential restriction.<br>The reasoner only needs to find **one** cheese topping -- which exists ( `MozzarellaTopping` ).<br>✅ Classified. |
| `VegetarianPizza` | `Pizza and (hasTopping only (CheeseTopping or VegetableTopping))` | Universal restriction.<br>The reasoner must verify that **all** toppings satisfy the constraint -- but OWA prevents it from knowing that no |

| Class | Definition | Reasoner behavior |
|---|---|---|
| | | other toppings exist.<br>❌ Not classified. |

> Something unexpected has happened. `TestVegetarianPizza` has only mozzarella topping (a `CheeseTopping`) and tomato topping (a `VegetableTopping`). We defined `VegetarianPizza` as a defined class using a universal restriction. And yet the reasoner has classified `TestVegetarianPizza` as a `CheesyPizza` but **not** as a `VegetarianPizza`.
>
> This is not a bug in Protégé, and it is even not an error in our ontology. It is a fundamental property of OWL that every ontology engineer must understand. The reason for this behavior -- and the construct that resolves it -- is the subject of the next exercise.

## 24.5.3 The Result: What the Reasoner Does Not Do

When the reasoner is synchronized, something important -- or rather, something important does **not** happen.

**What to observe:** (let's recap)

- `CheesyPizza` is correctly inferred for `TestVegetarianPizza` (highlighted in yellow, indicating a reasoner inference).
- `TestVegetarianPizza` is **not** classified as a `VegetarianPizza`.

The reasoner has all the information it needs to conclude that `TestVegetarianPizza` is a `VegetarianPizza` -- it has mozzarella (a `CheeseTopping`) and tomato (a `VegetableTopping`), both of which are permitted by the universal restriction on `VegetarianPizza`. **And yet it does not make that inference.**

This is unexpected. We defined `VegetarianPizza` as a defined class with the universal restriction `hasTopping only (CheeseTopping or VegetableTopping)`. We created `TestVegetarianPizza` with exactly the kinds of toppings that restriction permits. Intuitively, from human being's perspective, the reasoner should classify it as a `VegetarianPizza`.

**But it does not.**

**Key Insight:**

> "The universal restriction tells the reasoner what is allowed. It does not tell the reasoner what is *exhaustively* present. This distinction is the difference between a description and a specification.

Why this happens -- and what construct resolves it -- is the subject of **Exercise 23.**

## 24.5.4 Why the Universal Restriction Is Not Enough

To understand why the reasoner does not classify `TestVegetarianPizza` as a `VegetarianPizza`, we need to examine how universal restrictions behave under the **Open World Assumption (OWA).**

The universal restriction:

```
VegetarianPizza
  EquivalentTo
    Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
```

means: "If something is a `VegetarianPizza`, then every one of its toppings must be either a `CheeseTopping` or a `VegetableTopping`."

For the reasoner to conclude the `TestVegetarianPizza` **is** a `VegetarianPizza`, it must verify that the universal restriction holds -- that *all* of `TestVegetarianPizza`'s toppings are cheese or vegetable. But under the Open World Assumption, the reasoner cannot assume that the two toppings it knows about are the *only* toppings. There might be other toppings that exist but have not been asserted.

So the reasoner cannot prove that `TestVegetarianPizza` satisfies the universal restriction, and therefore cannot classify it as a `VegetarianPizza`.

This is not a flaw in the ontology. It is a faithful consequence of how OWL is designed. The Open World Assumption is what makes OWL suitable for open, federated domains -- but it is also what makes universal restrictions insufficient for classification on their own.

**Key Insight:**

> "The universal restriction tells the reasoner: 'If you find a `VegetarianPizza`, check that all its toppings are either cheese or vegetable.' It does not tell the reasoner: 'If you find a pizza whose known toppings are all cheese or vegetable, it is a `VegetarianPizza`.' The second statement requires more information than the universal restriction alone can supply."

### 24.5.5 Engineering Perspective: Exercise 22 as a Case-Study

From the perspective of Protégé, **Exercise 22** involves adding a single restriction and creating a test individual. (Note: the addition of the test individual is a book-level extension to the tutorial for triggering more discussion.) It appears to be a small modeling activity.

From the perspective of ontology engineering, however, something significant has occurred. The exercise has introduced the first execution primitive -- the universal restriction -- *and* surfaced the first significant engineering obstacle:

> the OWA between what a universal restriction describes and what execution requires.

The engineering contribution of **Exercise 22** can be summarized as follows:

| Engineering Perspective | Contribution from Exercise 22 |
|---|---|
| **SKDL Stage** | Stage 7 -- Executable Knowledge |
| **Primary Objective** | Introduce the universal restriction as an execution primitive |
| **Modeling Activity** | Add `hasTopping only (VegetableTopping or CheeseTopping)` to `VegetarianPizza`; convert to defined class; add `TestVegetarianPizza` individual |
| **Engineering Outcome** | Demonstrate the universal restriction's expressive power -- and its limitation under OWA |
| **Preparation for Later Sections** | Set up the OWL problem that **Exercise 23** will resolve through closure axioms |

**Key Insight:**

> "**Exercise 22** is a lesson in what the universal restriction can and cannot do. It can **exclude**. It cannot, on its own, **prove** exhaustiveness. The task belongs to the closure axiom."

## 24.6 Exercise 23 -- Closure Axioms and the Open World Assumption

In **Exercise 22**, we saw a surprising result: even though `VegetarianPizza` was correctly defined with a universal restriction, the reasoner did not classify `TestVegetarianPizza` as a `VegetarianPizza`. The reason was the Open World Assumption (OWA) -- the reasoner could not conclude that `TestVegetarianPizza` has *only* mozzarella and tomato toppings, because it could not rule out the existence of other toppings that had not been asserted.

This is the same phenomenon that Michael DeBellis demonstrates in Section 4.13 of the `Pizza` Tutorial, where he shows that `MargheritaPizza` and `SohoPizza` -- despite having only cheese and vegetable toppings -- are not automatically classified as subclasses of `VegetarianPizza` by the reasoner.

The construct that resolves this problem is called the **closure axiom**.

This exercise follows the tutorial's **Exercise 23**, and then extends it slightly to tie back to the `TestVegetarianPizza` individual from the previous exercise.

### 24.6.1 The Engineering Objective

**Exercise 23** introduces the **closure axiom** -- a universal restriction that explicitly specifies the complete set of values a property may take for a class or an individual.

A closure axiom for `MargheritaPizza` would be:

```
Class: MargheritaPizza
  SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

This says: "Every topping on a `MargheritaPizza` must be either `MozzarellaTopping` or `TomatoTopping`, and no other toppings exist."

By adding this axiom, we close the world for `MargheritaPizza`. The reasoner now knows that the two toppings we asserted for `MargheritaPizza` are the *only* toppings it has. This is the information it needs to classify `MargheritaPizza` as a subclass of `VegetarianPizza`.

**Key Insight:**

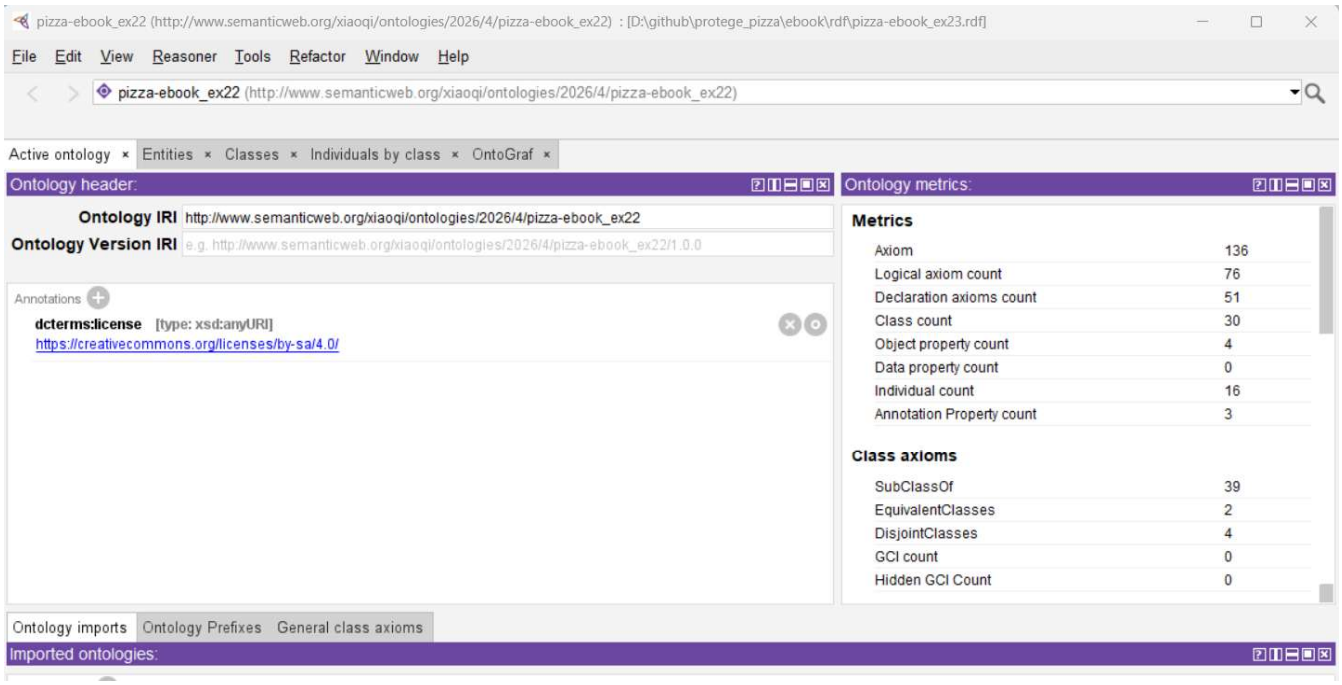
> "A closure axiom is the OWL equivalent of saying 'this is the complete picture.'
>
> - Without closure, the reasoner remains cautious.
> - With closure, the reasoner can be decisive."

## 24.6.2 Implementation: Adding Closure Axioms to `MargheritaPizza` and `SohoPizza`

We begin with the tutorial's **Exercise 23**, including the discussion in tutorial's Section 4.13 (before the exercise), which applies closure axioms at the class level.

### Step 1: Open the `Pizza` ontology

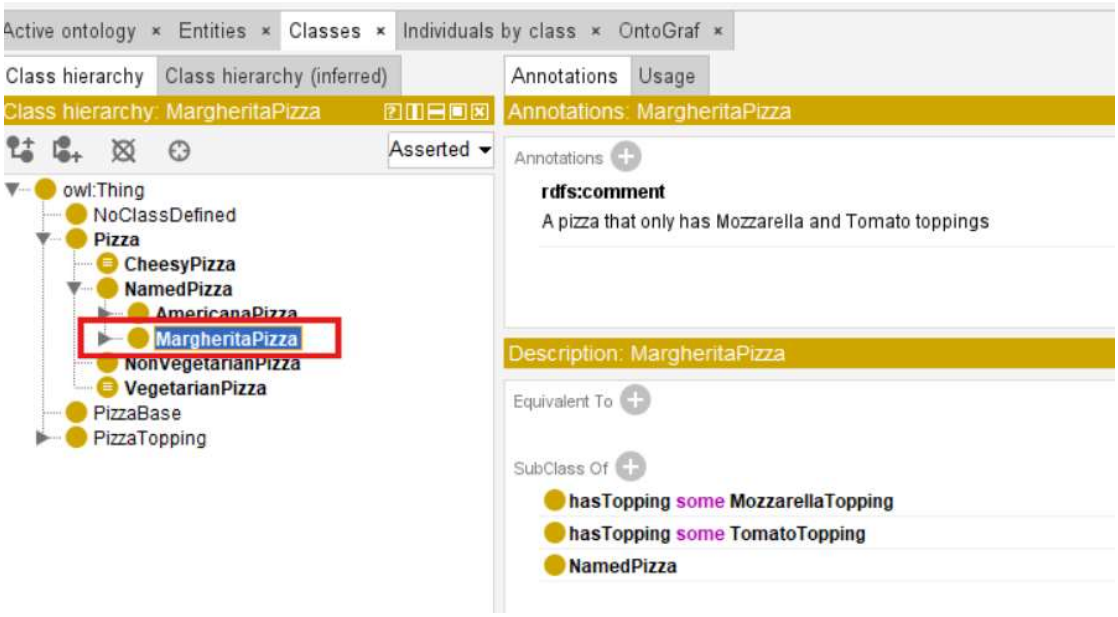
Open `pizza-ebook_ex22.rdf` in Protégé -- the file from the previous exercise, which already contains `TestVegetarianPizza` and the inferred-but-incomplete classification we observed.




> **Note**
>
> The finished state of **Exercise 23** will be saved as `pizza-ebook_ex23.rdf`.

### Step 2: Select `MargheritaPizza`

In the Classes tab, select `MargheritaPizza` in the class hierarchy.

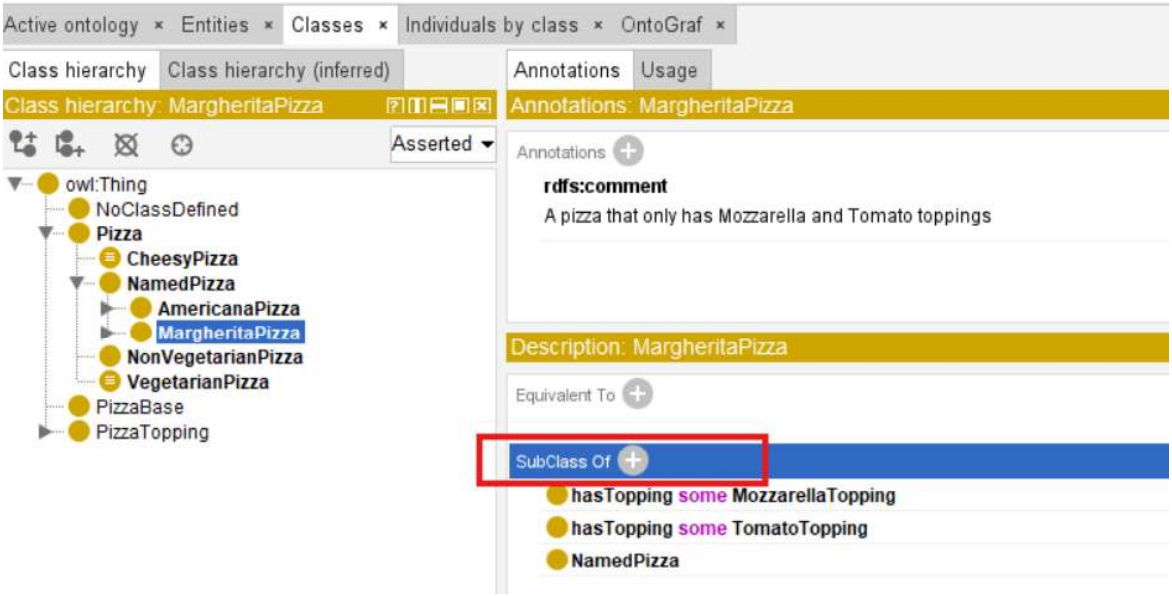



So far, the `MargheritaPizza` is defined as:

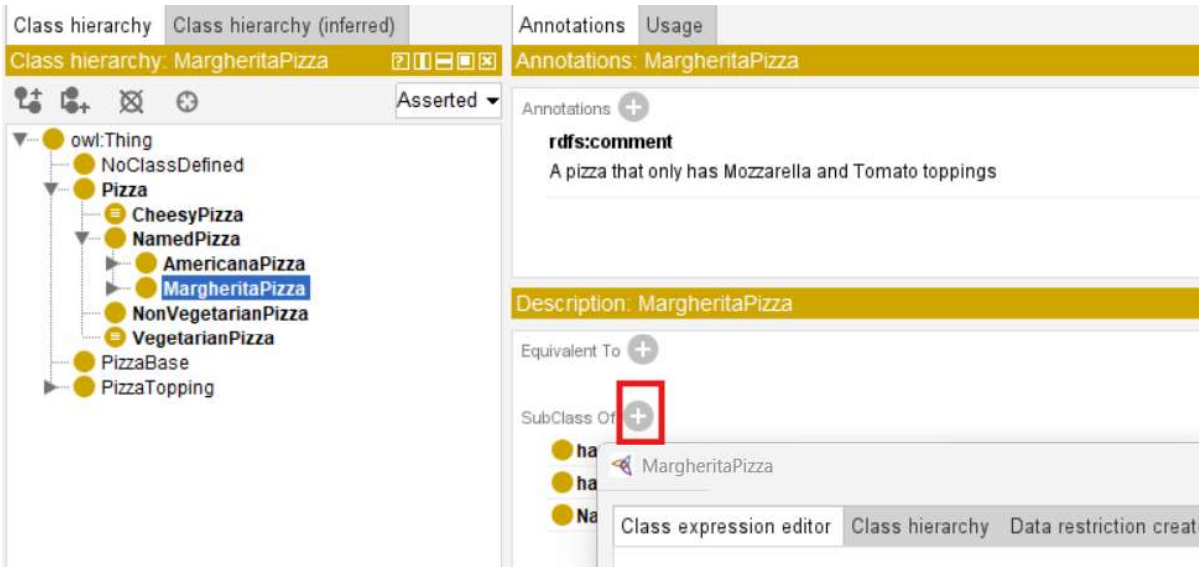
```
Class: MargheritaPizza
  SubClassOf: NamedPizza
  SubClassOf: hasTopping some MozzarellaTopping
  SubClassOf: hasTopping some TomatoTopping
```

**Step 3: Add the closure axiom to `MargheritaPizza`**

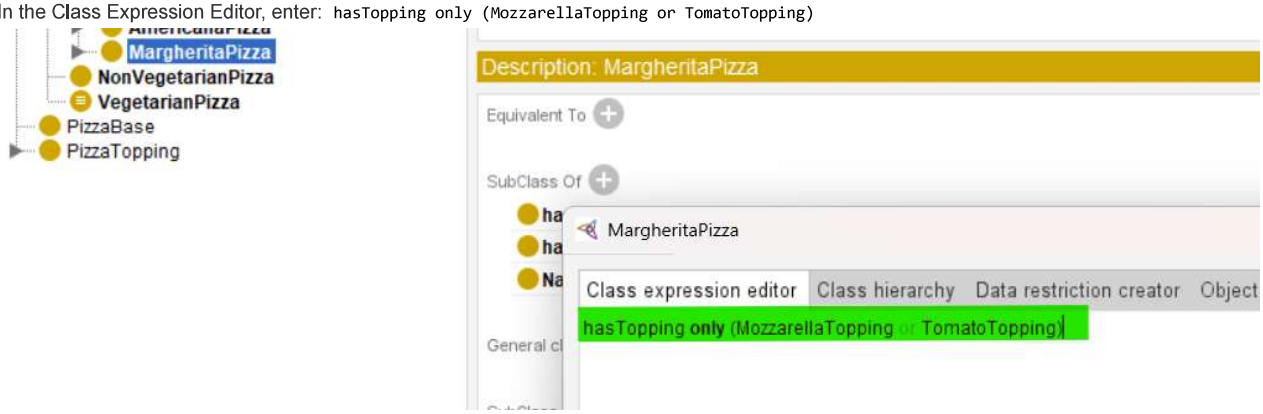
1. In the Description view, locate the **SubClass Of** section




2. Click the Add icon (+)



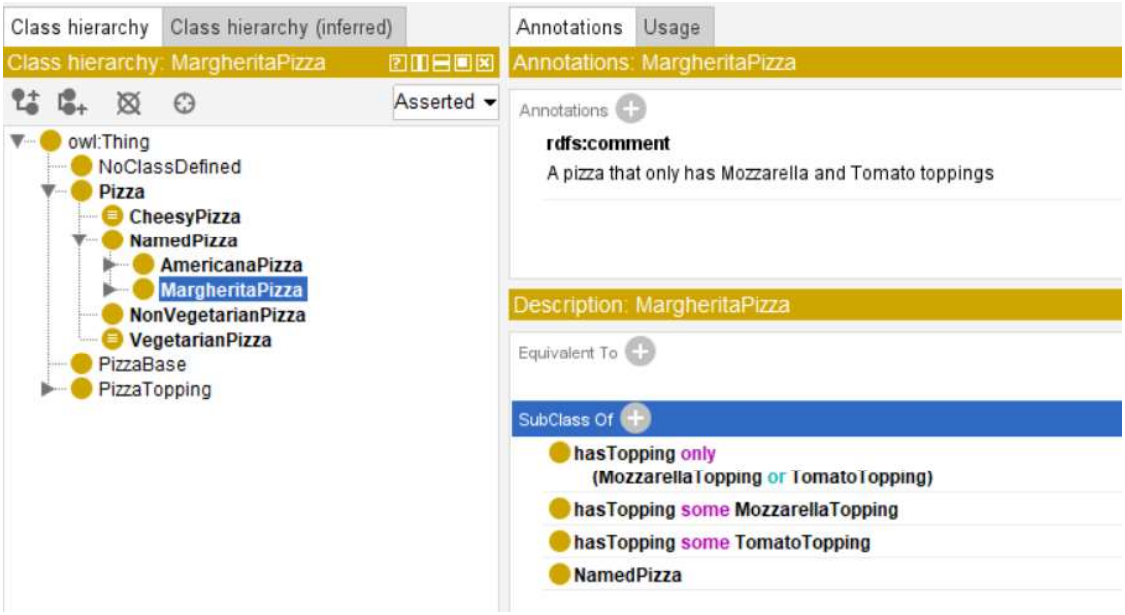

3. In the Class Expression Editor, enter: `hasTopping only (MozzarellaTopping or TomatoTopping)`




4. Click `OK`.

After this step, `MargheritaPizza` has the following description:

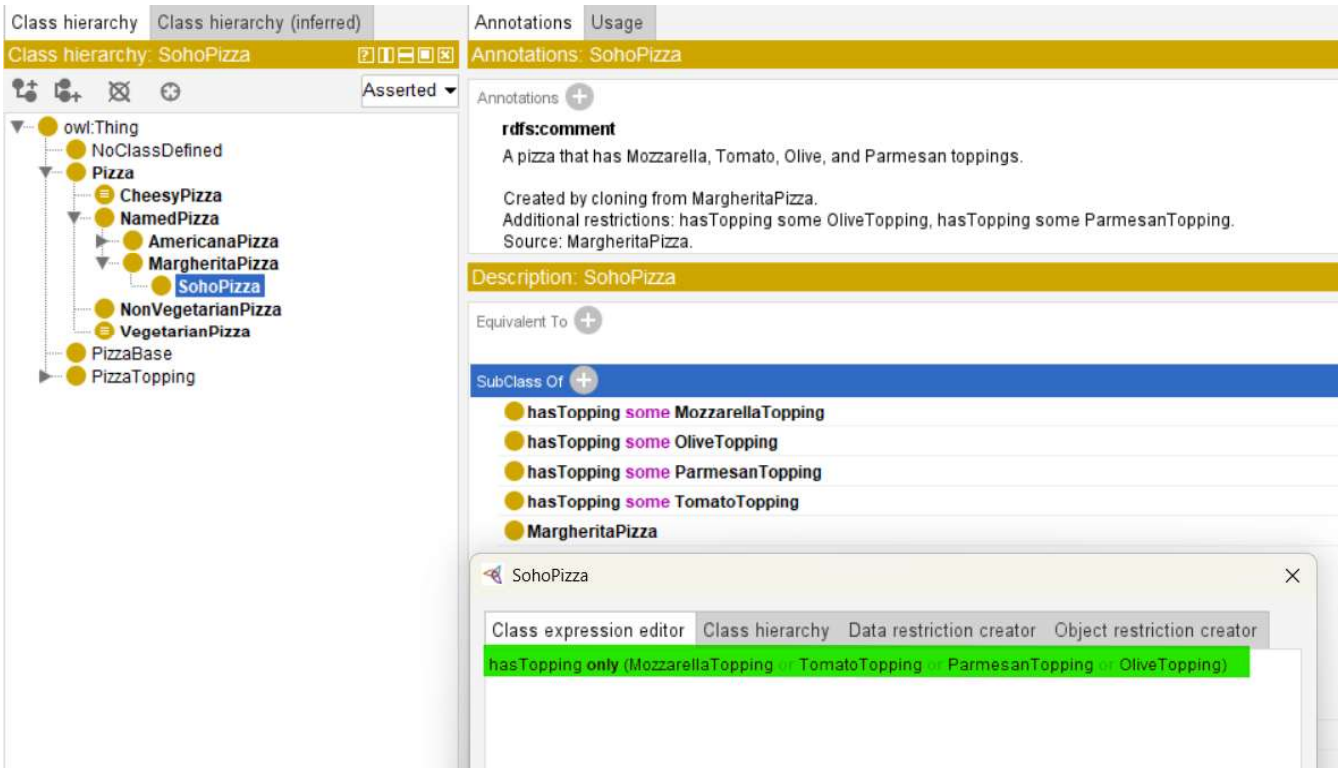
```
Class: MargheritaPizza
  SubClassOf: NamedPizza
  SubClassOf: hasTopping some MozzarellaTopping
  SubClassOf: hasTopping some TomatoTopping
  SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```



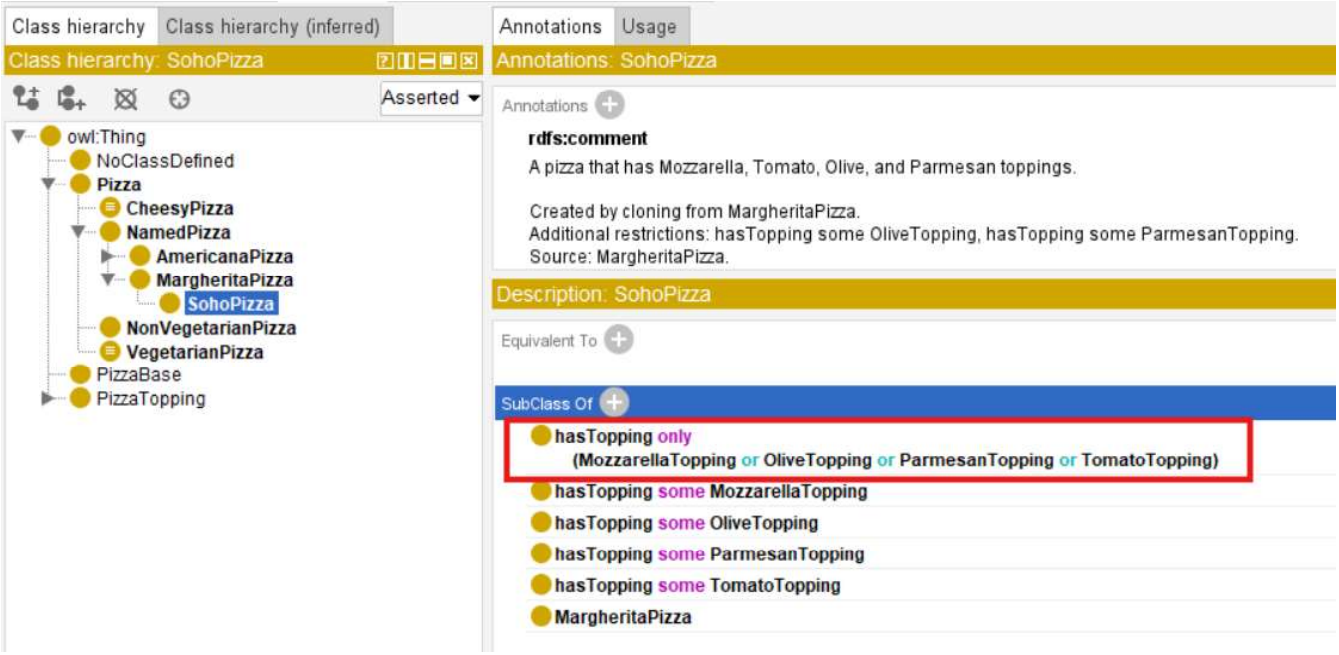

**Step 4: Add the closure axiom to `SohoPizza`**

Repeat Step 2-3, but this time select `SohoPizza` and enter the axiom:

```
hasTopping only (MozzarellaTopping or TomatoTopping or ParmesanTopping or OliveTopping)
```




After this step, `SohoPizza` has the closure axiom that enumerates its full set of allowed toppings.



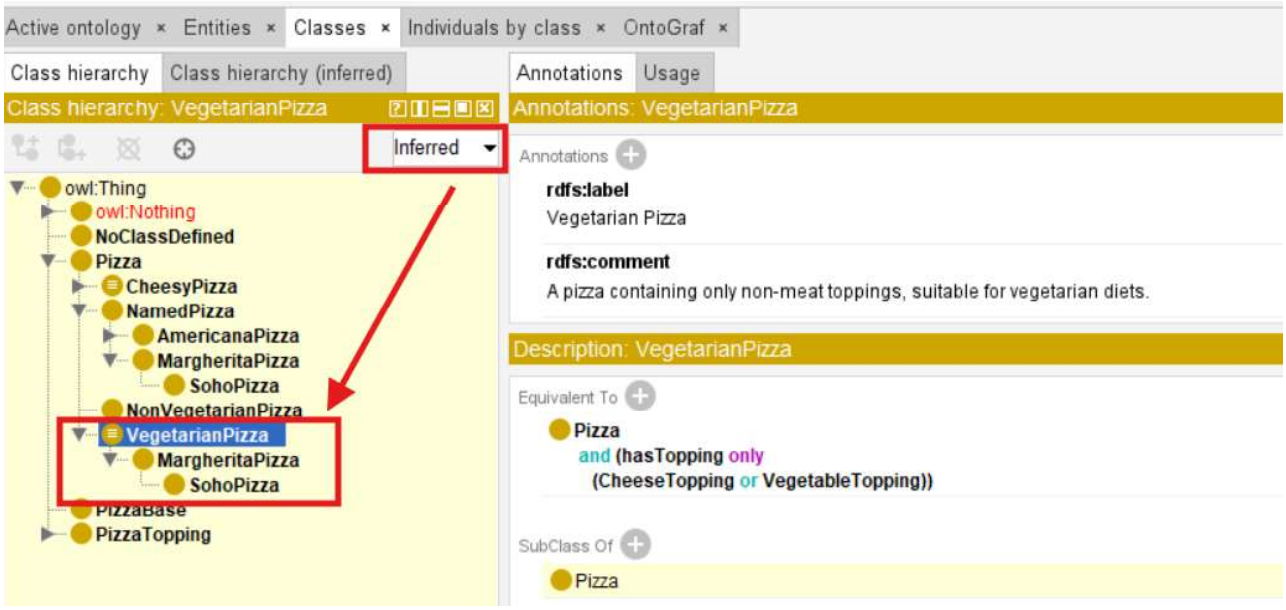

**Step 5: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)

**Step 6: Observe the result**

Look at the **Class hierarchy (inferred)** tab.

The reasoner now classifies both `MargheritaPizza` and `SohoPizza` (note the `SohoPizza` is still a subclass of `MargheritaPizza`) as subclasses of `VegetarianPizza`. **This is the result we were looking for:**




Why does it work now? Because the closure axiom supplies the missing information (knowledge) -- it tells the reasoner that *only* the enumerated toppings exist for these classes. Without that information, the reasoner could not verify that the universal restriction on `VegetarianPizza` held for `MargheritaPizza` and `SohoPizza`.

**Key Insight:**

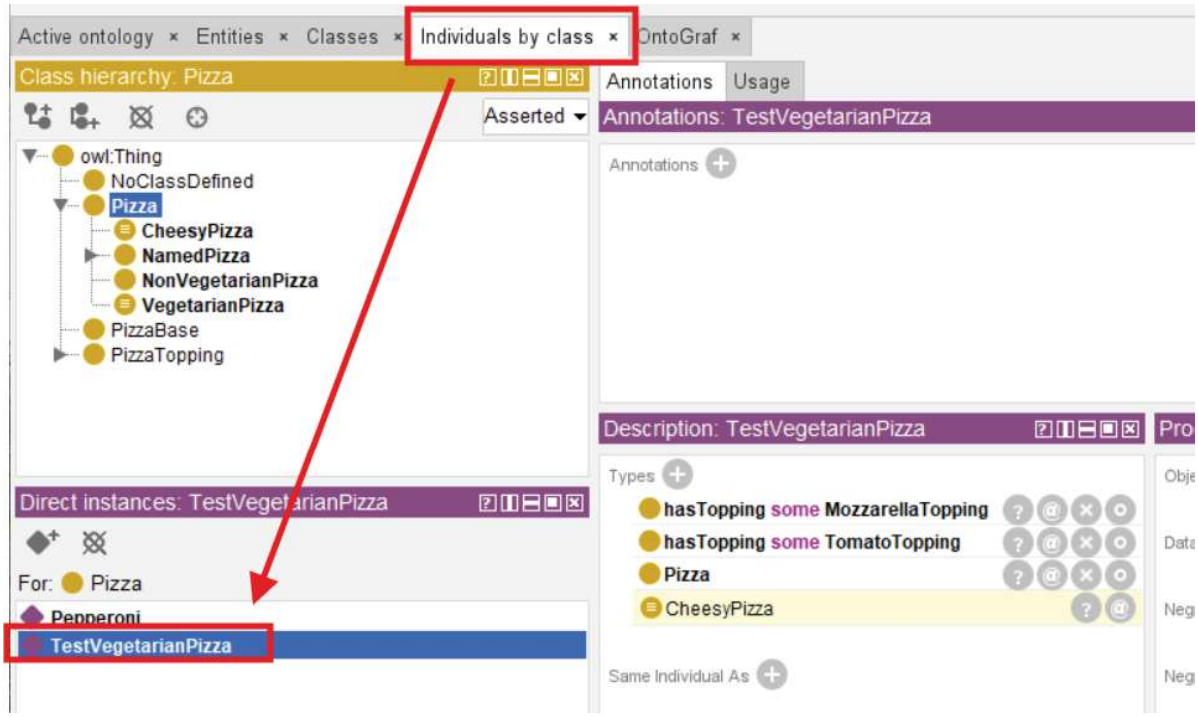
> "The reasoner did not change. The definition of `VegetarianPizza` did not change. Only the ***world*** changed -- from **open** to **closed**. Once the world is closed, the reasoner can make the inference."

## 24.6.3 Extending the Demonstration: Closure at the Individual Level

We have now seen closure axioms applied at the **class** level, following the tutorial. But our `TestVegetarianPizza` individual from **Exercise 22** is still unclassified. Let's resolve that too, using the same principle.

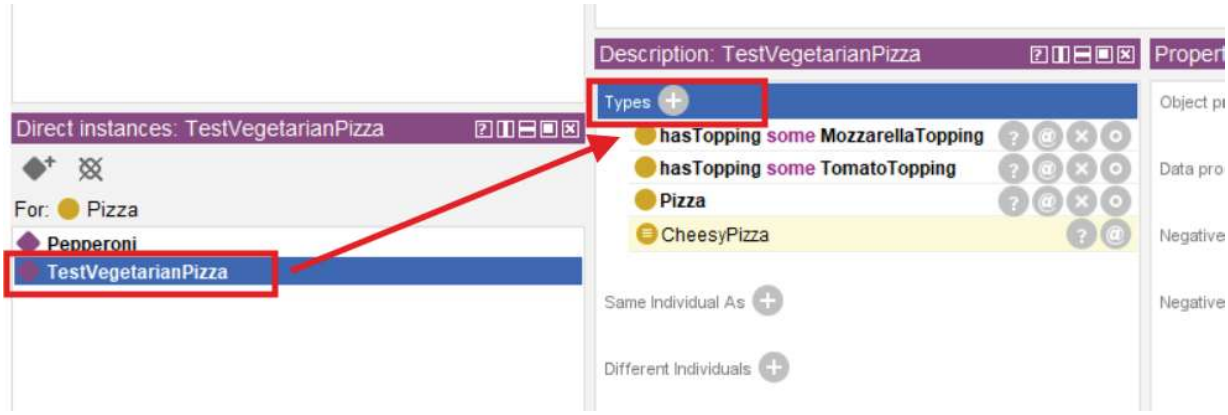
**Step 1: Select `TestVegetarianPizza` individual**

Navigate to the **Individual by class** tab and select `TestVegetarianPizza`.



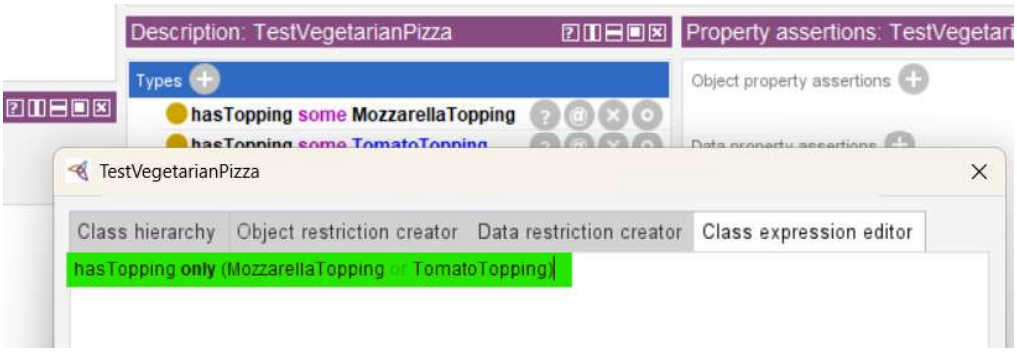

**Step 2: Add the closure axiom**

1. In the Description view, locate the **Types** section

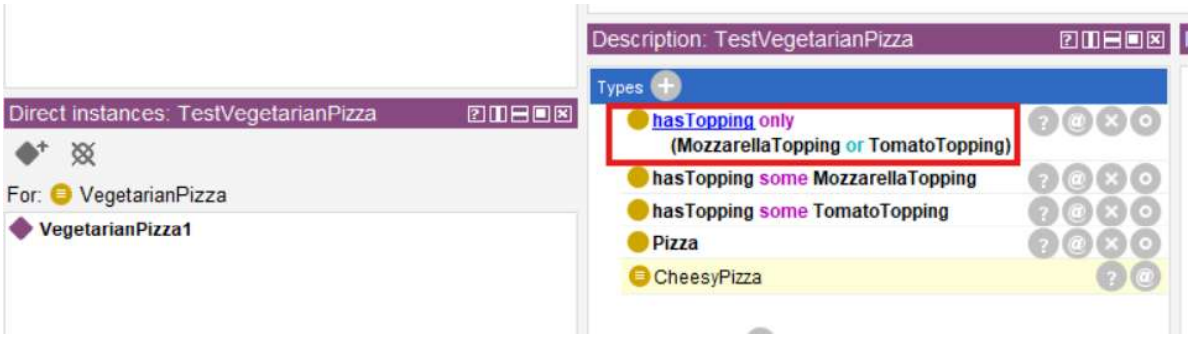



2. Click the Add icon (+)

3. In the Class Expression Editor, enter: `hasTopping only (MozzarellaTopping or TomatoTopping)`



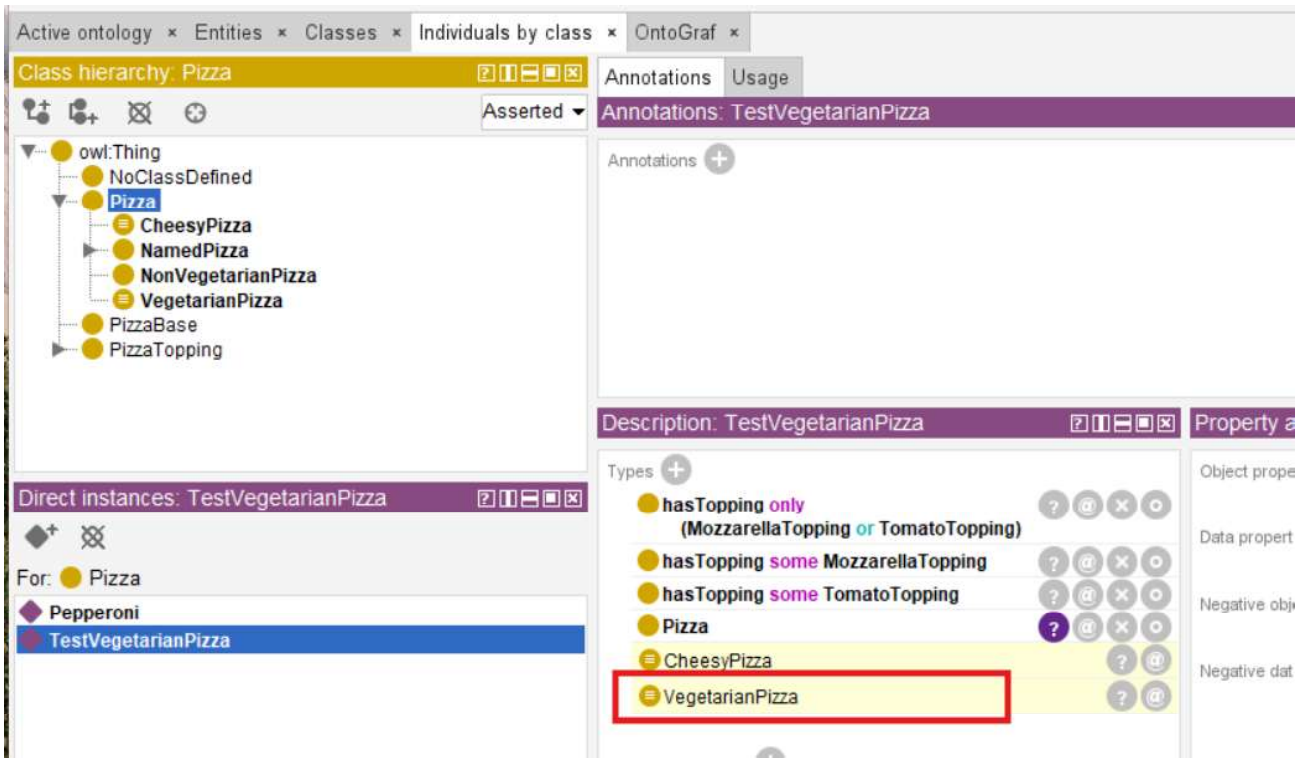

4. Click `OK`




**Step 3: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)

**Step 4: Observe the result**




`TestVegetarianPizza` is now classified as a `VegetarianPizza`, along with `CheesyPizza`. The closure axiom has resolved the same OWA problem at the individual level that it resolved at the class level.

This confirms that closure axioms are effective at both levels. In production ontologies, they are usually applied at the class level (as we did for `MargheritaPizza` and `SohoPizza`) so that every instance of the class inherits the closure. But the same principle applies to individual-level closure when a one-off case requires it.

**Key Insight:**

> "Closure axioms are a general mechanism. They close the world for whatever they are attached to -- a class or an individual. The reasoning principle is the same."

## 24.6.4 From Description to Specification

Both the class-level and individual-level closure axioms have converted descriptions into specifications.

Before the closure axiom, the reasoner could say: "If you find a `MargheritaPizza`, it has mozzarella and tomato." It could not say: "If you find a pizza with mozzarella and tomato, it is a `MargheritaPizza`."

After the closure axiom, the reasoner can say both. The ontology is now deterministic for these classes and individuals.

**Key Insight:**

> "The closure axiom is the bridge from description to specification. Before it, the ontology describes. After it, the ontology specifies. Specification is what enables execution."

## 24.6.5 Why Closure Axioms Matter for Execution

Closure axioms are essential for execution because they solve the central tension between OWA and executability:

| Aspect | Without Closure Axiom | With Closure Axiom |
|---|---|---|
| **Toppings allowed** | Any topping not explicitly forbidden | Only the enumerated toppings |
| **Reasoner behavior** | Cannot exclude possibilities | Can exclude possibilities |

| Aspect | Without Closure Axiom | With Closure Axiom |
|---|---|---|
| **Execution readiness** | Not executable | Executable |
| **Semantic strength** | Description | Specification |

The closure axiom is the OWL construct that most directly enables execution.

**Key Insight:**

> "Every closed-world requirement in an execution system corresponds to a closure axiom in the ontology. Closure is the semantic foundation of executability."

## 24.6.6 Engineering Perspective: Exercise 23 as a Case Study

From the perspective of Protégé, **Exercise 23** involves adding a small number of axioms. It appears to be a minor modeling activity.

From the perspective of ontology engineering, however, something profound has occurred. The ontology has crossed the boundary from description to specification. It is no longer just telling us what is true -- it is telling us what is allowed.

The engineering contribution of **Exercise 23** can be summarized as follows:

| Engineering Perspective | Contribution from Exercise 23 |
|---|---|
| **SKDL Stage** | Stage 7 -- Executable Knowledge |
| **Primary Objective** | Introduce the closure axiom as the bridge from description to specification |
| **Modeling Activity** | Add closure axioms to `MargheritaPizza`, `SohoPizza`, and `TestVegetarianPizza` |
| **Engineering Outcome** | Convert descriptions into closed-world specifications; the reasoner now correctly classifies `MargheritaPizza`, `SohoPizza`, and `TestVegetarianPizza` as subclasses or instances of `VegetarianPizza` |
| **Preparation for Later Chapters** | Establish the closure foundation for enumerated classes, `hasValue`, and cardinality restrictions |

**Key Insight:**

> "**Exercise 23** is the moment the ontology becomes a specification. It is the most important step toward executability."

## 24.6.7 The Design Pattern Perspective

The closure axiom is more than a one-off modeling trick for `MargheritaPizza` and `SohoPizza`. It is a **reusable design pattern** -- a named, repeatable shape that solves a recurring engineering problem: *how to convert an open-world description into a closed-world specification.*

The pattern has a fixed structure:

> **For a class or individual $X$ whose property $p$ is known to take values from a finite set $v1, v2, ..., v_n$, add $p$ only $(v_1 \vee v_2 \vee \cdots \vee v_n)$ to $X$.**

The pattern is reusable because the problem it solves is universal. Whenever the reasoner must decide whether an individual *belongs* to a defined class -- not merely what it *implies* -- the world must be closed first. Any class whose definition depends on a universal restriction will need a closure axiom somewhere upstream. This is not specific to pizza; it recurs in manufacturing (a bill of materials with a bounded component set), compliance (a permit with an enumerated set of authorized activities), and safety (a procedure with a closed set of permitted steps).

The design patterns of Stage 7 now begin to accumulate:

| Pattern | Purpose | Introduced |
|---|---|---|
| **Probe Class** | Detects membership through a discriminating marker | Chapter 22 |
| **Closure Axiom** | Closes the world around a boundary | This chapter |

| Pattern | Purpose | Introduced |
|---|---|---|
| **Enumerated Class** | Closes the value domain of a property | Chapter 25 |
| `hasValue` **Restriction** | Binds a property to a specific value | Chapter 25 |
| **Cardinality Restriction** | Enforces data completeness | Chapter 26 |

This naming matters. In Volume 5, we will treat the OWL design pattern landscape as a subject in its own right -- but the patterns themselves are not new inventions. They are the recurring shapes that experienced ontology engineers have been using all along, and Stage 7 is where they become visible as a family.

## 24.7 Chapter Summary

This chapter introduced the first part of **Stage 7 -- Executable Knowledge** of the Semantic Knowledge Development Lifecycle (SKDL), and took the reader through the first two of the five execution primitives.

We began with the question that motivates the entire stage: **how do we act on this knowledge?** Validation (Stage 5) told us the ontology is correct. Reasoning (Stage 6) told us what new knowledge the ontology can derive. Executable Knowledge (Stage 7) now tells us what to ***do*** about it.

The chapter established the working definition of executable knowledge -- semantic knowledge that has been encoded with sufficient logical constraints to be interpreted deterministically and to drive action -- and distinguished it from validation and reasoning:

| Activity | Question | Output |
|---|---|---|
| Validation | "Is this model correct?" | Pass/Fail |
| Reasoning | "What else can this model tell us?" | Inferred facts |
| Executable Knowledge | "What should happen?" | Actions |

We then identified the central obstacle to executability: the **Open World Assumption (OWA).** OWA is correct for reasoning -- it allows federated domains to grow incrementally -- but it is problematic for execution. An execution system that fires a trigger on incomplete information may fire on the wrong condition, or fail to fire when it should. To bridge this gap, we introduced the first two execution primitives:

**Exercise 22 -- Universal Restrictions.** We created `VegetarianPizza` as a defined class with the universal restriction `hasTopping only (CheeseTopping or VegetableTopping)`. We then added a test individual `TestVegetarianPizza` with exactly the toppings the restriction permits, and observed something surprising: the reasoner classified it as a `CheesyPizza`, but **not** as a `VegetarianPizza`. The reason was OWA -- the reasoner could not rule out the existence of additional toppings that had not been asserted. This demonstrated that universal restrictions alone are insufficient for classification.

**Exercise 23 -- Closure Axioms.** We then introduced the closure axiom -- a universal restriction that explicitly enumerates the complete set of allowed values for a property. Adding closure axioms to `MargheritaPizza` and `SohoPizza` (the tutorial's own examples) caused the reasoner to classify both as subclasses of `VegetarianPizza`. We then extended the demonstration to the individual level by adding a closure axiom directly to `TestVegetarianPizza`, which caused it to be classified as a `VegetarianPizza` -- resolving the OWA problem that **Exercise 22** had surfaced.

The lesson across the two exercises is this:

> **Universal restrictions establish a boundary.**
> **Closure axioms close the world around that boundary.**
> **Together, they form the first layer of an executable ontology.**

We also paused at §24.6.7 to name a concept that has been present throughout: the **design pattern.** The Probe Class (Chapter 22), the closure axiom (this chapter), the enumerated class and `hasValue` restriction (Chapter 25), and the cardinality restriction (Chapter 26) are all reusable shapes -- the design patterns of Stage 7. This naming will become important in Volume 5 when we cover the OWL design pattern landscape in depth.

By the end of this chapter, you have:

- Understood why executability requires a departure from the pure OWA stance of reasoning
- Created and applied universal restrictions to express boundaries
- Observed the OWA failure mode directly

- Created and applied closure axioms to close the world
- Classified both classes and individuals using the closure mechanism
- Recognized the design-pattern nature of the constructs introduced

These capabilities prepare you for Chapter 25, which introduces the value layer of executability: enumerated classes and `hasValue` restrictions.

## 24.8 Looking Ahead -- A Bridge to Chapter 25

The boundary layer of the executable ontology is now in place. We have learned:

- **How to express what is allowed** using universal restrictions ( `hasTopping only ...` )
- **Why universal restrictions alone are not enough** for classification under OWA
- **How to close the world** around a boundary using closure axioms
- **How the same principle applies** at both the class and the individual level

But boundaries alone do not execute.

A boundary tells the system what is *permitted*; it does not tell the system what to *do*.

To drive action, the ontology must also be able to express **what specific value** a property may take -- and to bind those values into trigger conditions.

This is the subject of Chapter 25.

Chapter 25 introduces two further execution primitives:

- **Enumerated Classes (Exercise 24).** The enumerated class is the construct that closes the *value domain* of a property. Using the `Spiciness` case from the `Pizza` tutorial, we define `Spiciness` as `{Hot, Medium, Mild}` -- a closed set of exactly three values. Once enumerated, the value domain is exhaustive: no other spiciness value is possible.
- `hasValue` **Restrictions (Exercise 25).** The `hasValue` restriction is the construct that binds a property to a *specific value* from that domain. We attach `hasSpiciness value Hot` to `JalapenoPepperTopping` , and then define `SpicyPizza` as `hasTopping some (hasSpiciness value Hot)` -- a restriction inside a restriction. The reasoner then classifies `AmericanaHotPizza` as a `SpicyPizza` automatically, without the engineer ever asserting that it is one.

Together, the enumerated class and the `hasValue` restriction form the **value layer** of executability. The boundary layer told us *what is allowed*; the value layer tells us *what specifically to act on*.

The chapter that follows will continue the arc. By the end of Chapter 25, the reader will have:

- Created a closed value domain with an enumerated class
- Bound a property to a specific individual with a `hasValue` restriction
- Defined a class using a nested restriction
- Observed the reasoner deriving a classification through composition

And Chapter 26 will complete the picture: cardinality restrictions for data completeness, the anatomy of an executable ontology, and the mathematical, historical, industrial, and EKA perspectives that make the whole architecture coherent.

The journey from boundary to value to integrity is the arc of Stage 7. This chapter has established the first leg. Chapter 25 continues the journey.

**Key Insight:**

> "A boundary tells the system what is permitted. A value tells the system what to act on. Together, they turn a knowledge model into a decision system."

Last updated at 2026-10-02

# Chapter 25 - Stage 7 of SKDL -- Executable Knowledge (Part 2): Value Specification & Precise Binding

**Table of Content**


- 25.1 Chapter Introduction
  - 25.1.1 How Chapter 25 Continues Chapter 24
- 25.2 Learning Objectives
- 25.3 A Brief Note on OWL Design Patterns
- 25.4 Exercise 24 -- Enumerated Classes and Deterministic Value Sets
  - 25.4.1 The Engineering Objective
  - 25.4.2 Implementation: Creating the Enumerated Class
  - 25.4.3 The Result: A Deterministic Value Domain
  - 25.4.4 Why Enumerated Classes Matter for Execution
  - 25.4.5 Engineering Perspective: Exercise 24 as a Case Study
- 25.5 Exercise 25 -- `hasValue` Restrictions and Specific-Value Binding
  - 25.5.1 The Engineering Objective
  - 25.5.2 Implementation: Creating the `hasValue` Restriction
  - 25.5.3 The Result: Value-Based Execution Logic
  - 25.5.4 Why `hasValue` Restrictions Matter for Execution
  - 25.5.5 Engineering Perspective: Exercise 25 as a Case Study
  - 25.5.6 The Design Pattern Perspective
- 25.6 Chapter Summary
- 25.7 Looking Ahead -- A Bridge to Chapter 26


## 25.1 Chapter Introduction

In **Chapter 24**, we built the first layer of an executable ontology: the **boundary layer**. Universal restrictions told the reasoner what was allowed; closure axioms closed the world around those boundaries. By the end of that chapter, the ontology could classify `MargheritaPizza` and `SohoPizza` as `VegetarianPizza` -- deterministically and without ambiguity.

But a boundary tells the system what is *permitted*; it does not tell the system what to *do*.

Consider a pizza that is `Hot`. Or an order whose status is `Pending`. Or a customer whose tier is `Gold`. Each of these is a specific value -- not a boundary, not a range, but one point in a closed set. Execution often fires on exactly these points. "If the spiciness is `Hot`, alert the kitchen. If the order is `Pending`, send a reminder. If the tier is `Gold`, apply the discount."

Chapter 25 builds the layer that makes these triggers possible: the **value layer**.

The value layer has two halves:

- An **enumerated class** closes the *value domain* -- it declares the complete, finite set of values a property may take. `Spiciness = {Hot, Medium, Mild}`. No other value is possible.
- A **`hasValue` restriction** binds a property to *one specific value* from that domain. `JalapenoPepperTopping hasSpiciness value Hot`. Not a class of values -- the value itself.

Together, they allow the ontology to express and execute rules that depend on *specific values* rather than *categories*. This is the semantic foundation of the trigger.

This chapter follows **Exercise 24** and **Exercise 25** in Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial*, read through the lens of executability. It is the second of three chapters on **Stage 7 -- Executable Knowledge**.

**Key Insight:**

> "A boundary tells the system what is allowed. A value tells the system what to act on. The value layer is where execution becomes specific."

### 25.1.1 How Chapter 25 Continues Chapter 24

Chapter 24 built the boundary layer from two primitives: the universal restriction (**Exercise 22**) and the closure axiom (**Exercise 23**). Chapter 25 builds the value layer from two more: the enumerated class (**Exercise 24**) and the `hasValue` restriction (**Exercise 25**).

| Layer | Primitive | Exercise | What It Closes |
|---|---|---|---|
| **Boundary** | Universal Restriction ( `only` ) | Ex 22 | What is permitted |
| **Boundary** | Closure Axiom | Ex 23 | What exhaustively exists |
| **Value** | Enumerated Class | Ex 24 | The set of possible values |
| **Value** | `hasValue` Restriction | Ex 25 | The specific value to act on |
| **Integrity** | Cardinality Restriction | Ex 26 (Ch 26) | The number of values required |

Each layer depends on the one before it. The value layer is meaningless without the boundary layer -- binding a value to a property whose permitted range is not closed would leave the binding ambiguous. Conversely, the boundary layer is incomplete without the value layer -- a boundary that cannot fire on specific values cannot drive action.

**Key Insight:**

> "The boundary layer says what is closed. The value layer says what to fire on. Execution needs both."

## 25.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

**Knowledge:**

- Explain why the value layer is the second layer of executable knowledge
- Distinguish between a *closed value domain* and a *specific value binding*
- Understand how enumerated classes differ from asserted class membership
- Understand how `hasValue` restrictions differ from `some` and `only` restrictions
- Explain why both primitives are required for value-based triggers

**Practical Skills (Stage 7, Chapter 25):**

- Create an enumerated class using `EquivalentTo {...}` in Protégé
- Create the individuals that populate an enumerated class
- Create an object property with a defined domain and range
- Add a `hasValue` restriction to a class
- Define a class using a nested restriction ( `some` containing `value` )
- Convert a class to a defined class
- Test value-based classification by synchronizing the reasoner

**Mathematical Perspective:**

- Understand enumeration as the finite specification of a class extension: $C \equiv \{a_1, \ldots, a_n\}$
- Understand `hasValue` as a restriction whose filler is an individual, not a class: $p \text{ value } a$
- Explain the composition of `some` and `value` as a nested restriction: $\exists p.(\exists q.\{a\})$
- Understand the relationship between enumeration, closure, and exhaustiveness

**Engineering Perspective:**

- Understand the value layer as the semantic foundation of triggers
- Map the enumerated class to the concept of an `enum` in programming
- Map the `hasValue` restriction to the concept of a state-machine transition condition
- Apply the two primitives to industrial scenarios (alerting, workflow routing, decision tables)

**Note on scope.**

These learning objectives are scoped to **Chapter 25**. Chapter 26 will add cardinality restrictions and the consolidated Stage 7 architecture. Chapter 24 (already covered) introduced the boundary layer.

## 25.3 A Brief Note on OWL Design Patterns

Before we build our next execution primitive, it is worth pausing to name a concept that has quietly been present throughout this book: **the design pattern**.

The idea of a design pattern comes originally from architecture -- Christopher Alexander's *A Pattern Language* (1977) -- and was brought into software engineering by the "Gang of Four" in their 1994 book *Design Patterns*.

> **Alexander's insight** was that good design does not begin from nothing; it draws on a shared vocabulary of proven solutions to recurring problems. Each solution can be adapted to a specific context without being reinvented.

In OWL, the same idea applies. A **design pattern** is a reusable combination of OWL constructs that solves a recurring modeling problem. It sits at a higher level of abstraction than a single class or axiom -- it is a **shape**, not a thing!

By this point in the book, you have already used several OWL design patterns without necessarily calling them that:

| Pattern | Introduced | Purpose |
|---|---|---|
| **Probe Class** | Chapter 22 (Exercise 19) | A deliberately inconsistent class used to test whether validation infrastructure works |
| **Closure Axiom** | Chapter 24 (Exercise 23) | A universal restriction that closes the world for a class or individual |
| **Enumerated Class** | This chapter (Exercise 24) | A class whose extension is defined by listing all of its instances |
| `hasValue` **Restriction** | This chapter (Exercise 25) | A restriction that binds a property to a specific individual |
| **Cardinality Restriction** | Chapter 26 (Exercise 26, upcoming) | A restriction on the number of values a property may have |

Each of these is a **reusable shape**, not a one-off modeling decision. You can apply the same shape to any class in any domain -- pizza, manufacturing, financing, healthcare -- and it will work the same way.

That reusability is what makes design patterns valuable for governance and execution:

- **Reuse**: A pattern written once can be instantiated many times.
- **Consistency**: Different engineers applying the same pattern produce structurally similar (or even same) models.
- **Governance**: Patterns can be named, documented, and governed like any other asset.
- **Execution**: The execution primitives in this chapter (closure, enumeration, `hasValue`, cardinality) are effectively the design patterns of Stage 7.

The full OWL design pattern landscape -- the ODP catalog, logical vs. content patterns, pattern composition, and pattern selection strategies -- is a topic of its own. It belongs to the era of **Knowledge Graph Engineering**, when you are building systems that span multiple ontologies and reuse libraries of proven patterns.

> **Note**
>
> **About the ODP catalog.** The Ontology Design Patterns (ODP) catalog is maintained by the Ontology Design Patterns Association (ODPA) at `ontologydesignpatterns.org`. At the time of writing, the site is served over HTTP and must be accessed via its full wiki path — the bare host may time out depending on the network.
>
> If you want to access the catalog, here is the direct URL: http://www.ontologydesignpatterns.org/index.php/Ontology_Design_Patterns_._org_(ODP) -- available at the date of this note.
>
> *-- Note at 2026-10-03*

We will return to it in **Volume 5**.

For now, the key point is this:

> **When you build an execution primitive in this chapter, you are not just writing an axiom -- you are instantiating a design pattern.**

The patterns will grow with you as your ontology grows.

# 25.4 Exercise 24 -- Enumerated Classes and Deterministic Value Sets

Having established closure axioms in **Exercise 23**, we now turn to the first execution primitive of the **value layer**: the enumerated class.

As we saw in **§24.6.7**, the enumerated class is one of the design patterns of Stage 7 — here we instantiate it.

## 25.4.1 The Engineering Objective

**Exercise 24** introduces the **enumerated class** -- a class whose extension is defined by explicitly listing all of its instances.

An enumerated class for spiciness, for example, would be:

```
Class: Spiciness
  EquivalentTo: {Hot, Medium, Mild}
```

This means: "The class `Spiciness` has exactly three instances: `Hot`, `Medium`, and `Mild`."

The engineering objective for **Exercise 24** is to demonstrate how enumeration provides a **complete, deterministic value domain** -- a prerequisite for execution.

Why does execution need enumeration?

Consider an execution rule that fires when a pizza is "spicy". For the rule to be reliable, the system must know the complete set of possible spiciness values. If the ontology only says "there exists a `Spiciness` value" but does not enumerate what that value could be, the rule cannot be exhaustively evaluated.

Enumeration closes the value domain. It says: *these are all the possible values -- nothing more, nothing less.*

This is the value-domain analogue of the closure axiom. The closure axiom closed the set of permitted *toppings* for a class; the enumerated class closes the set of permitted *values* for a property. Both replace an open-world description with a closed-world specification -- one at the property-range level, one at the value level. Together, they are the two halves of boundary closure.

**Key Insight:**

> "An enumerated class is a closed-world specification of a value domain. It says not just what values exist, but that **no other values exist**."

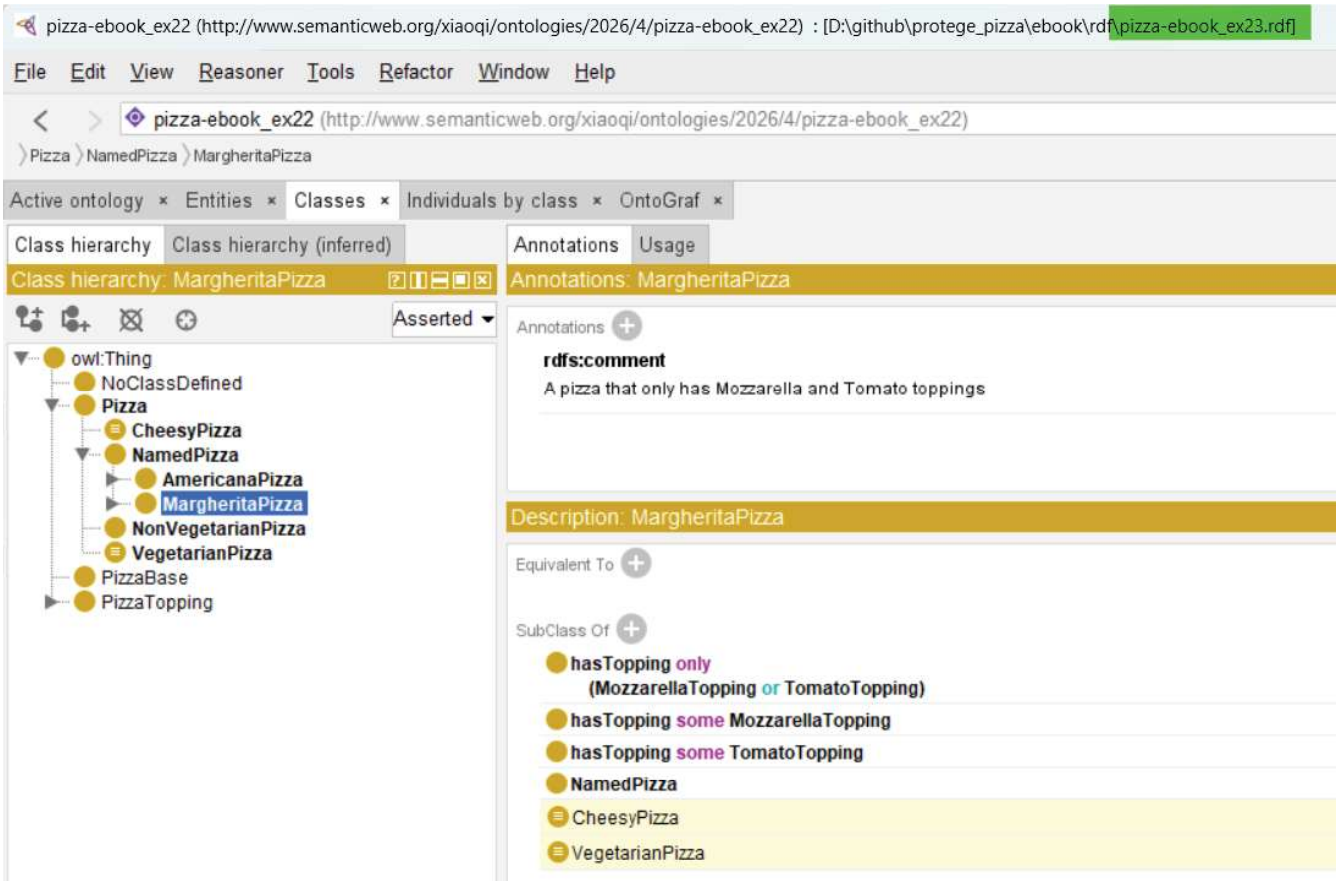
## 25.4.2 Implementation: Creating the Enumerated Class

Let's walk through the implementation step by step.

As you'll see, the workflow is slightly different from **Exercises 22 and 23** -- enumerated classes are created by combining three OWL constructs: a class, its three instances, and an `EquivalentTo` axiom that lists them.

**Step 1: Open the `Pizza` ontology**

Open `pizza-ebook_ex23.rdf` in Protégé - the file from the previous exercise, which now contains closure axioms on `MargheritaPizza` and `SohoPizza`, and the individual-level closure on `TestVegetarianPizza`.

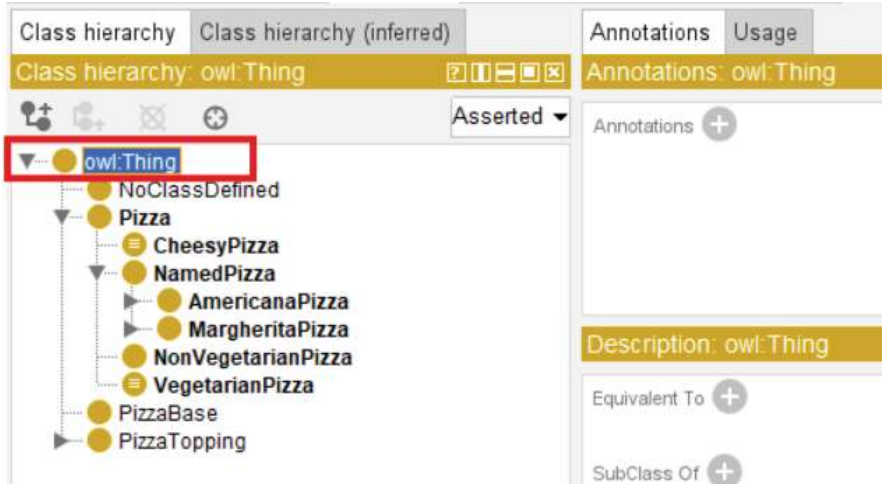



**Note**

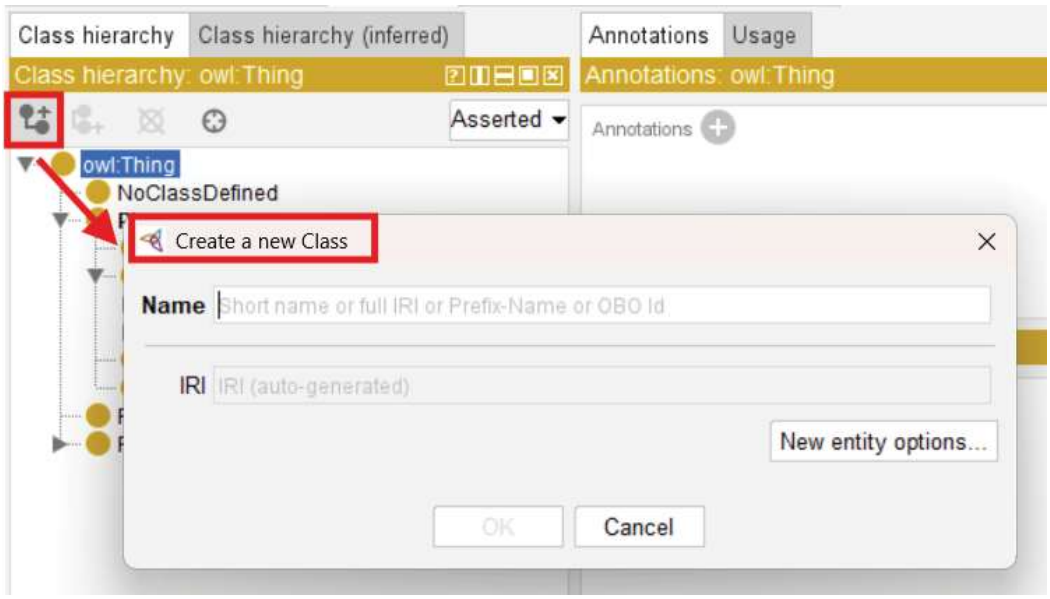
The finished state of **Exercise 24** is saved as `pizza-ebook_ex24.rdf`.

### Step 2: Create the `Spiciness` class

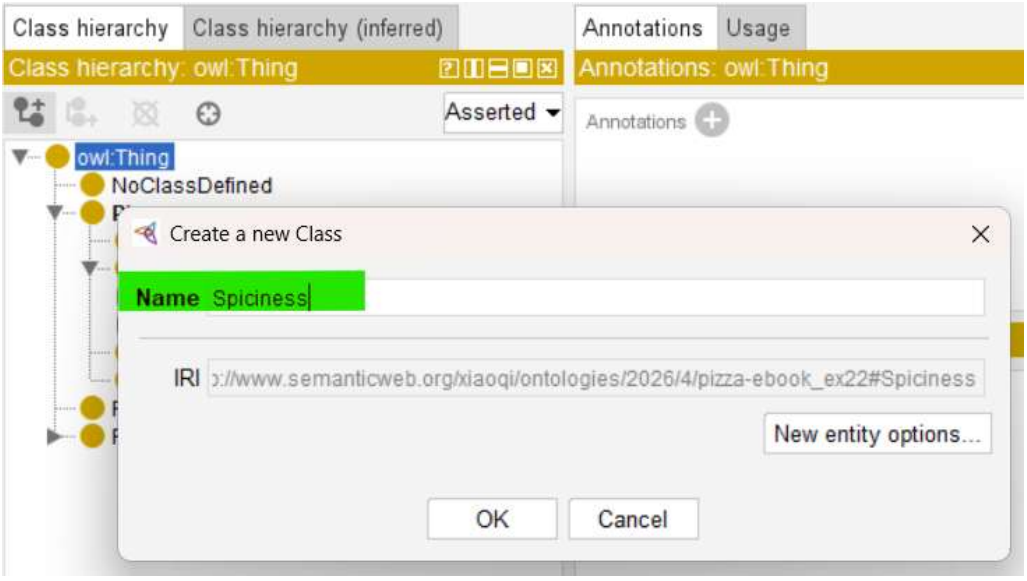
1. Navigate to the Classes tab. (You may already at this tab)
2. Select `owl:Thing` in the class hierarchy



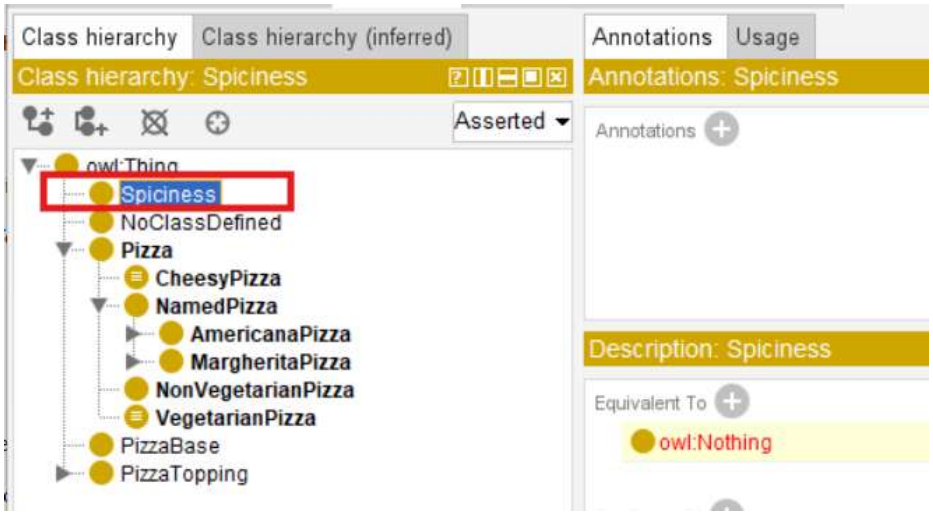

3. Click the **Add subclass** icon.




4. Name the new class `Spiciness`



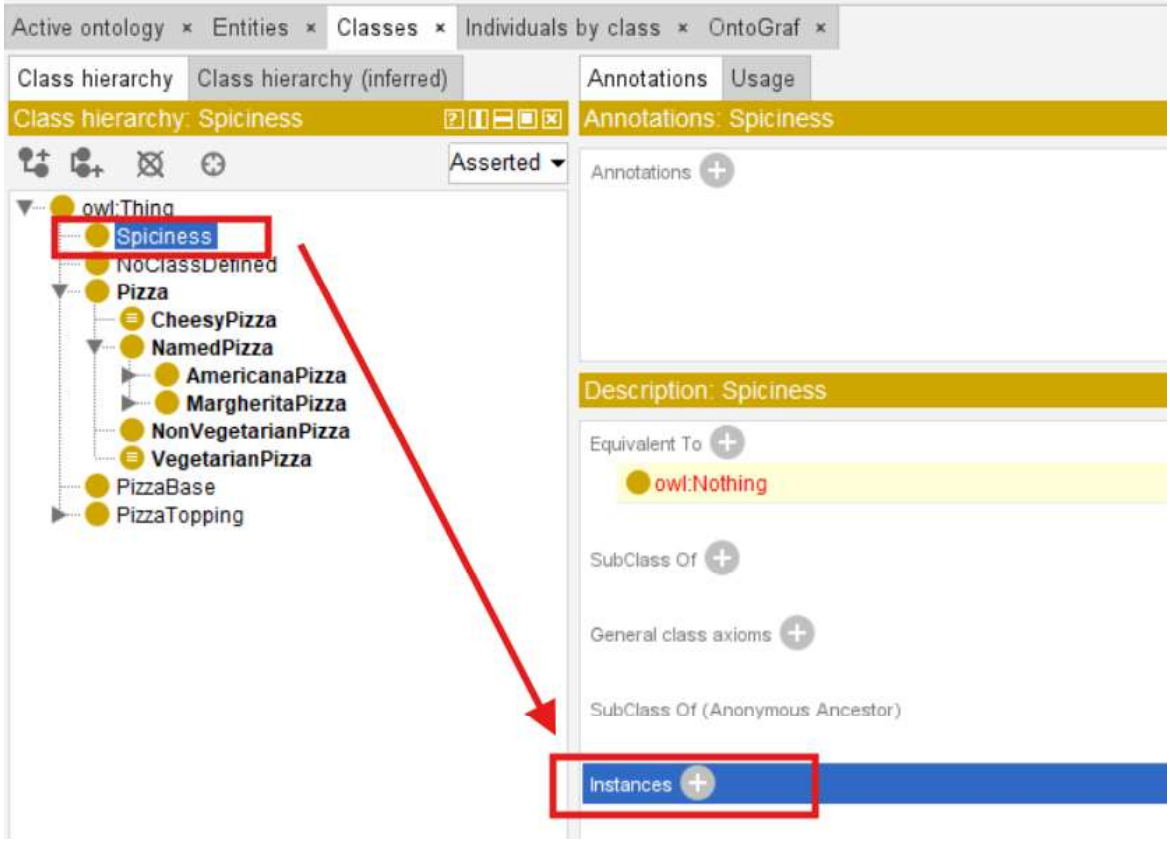

5. Click `OK`



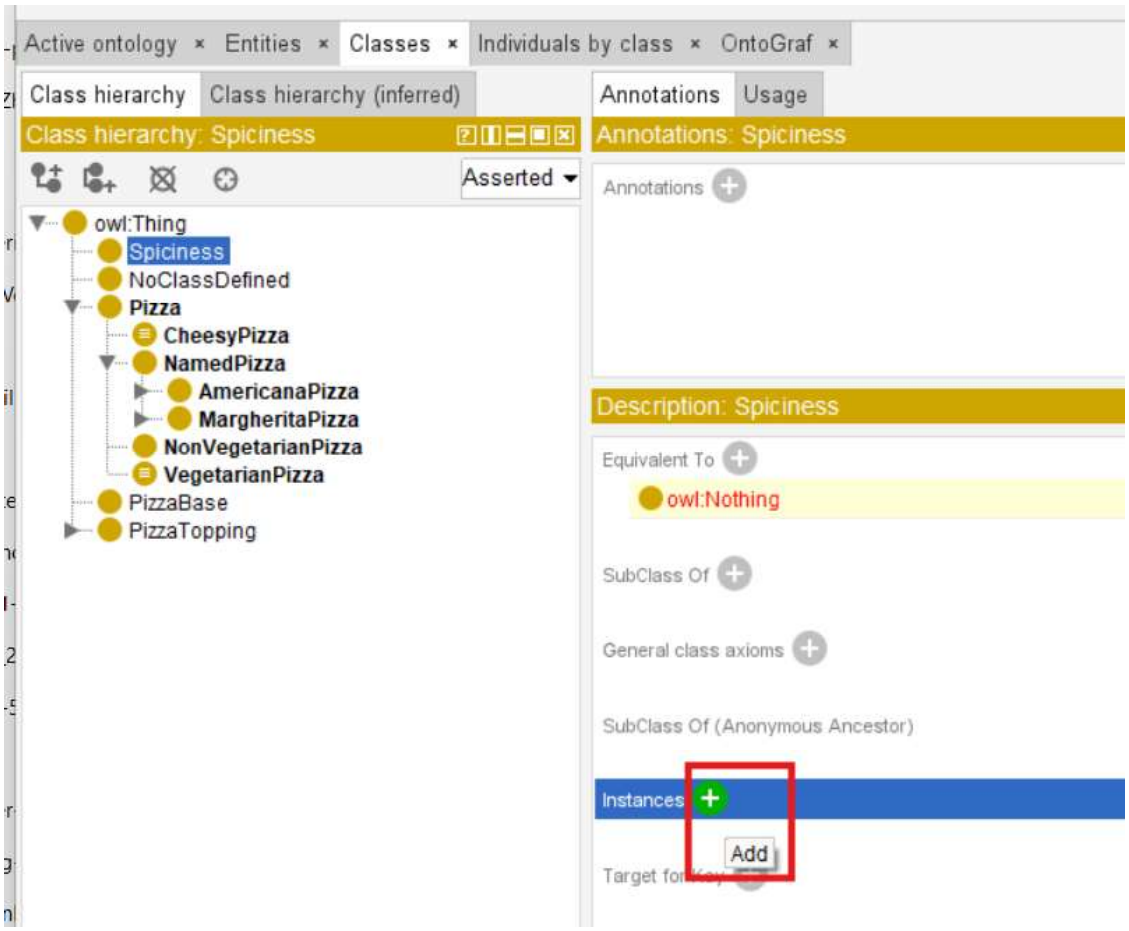
At this point, `Spiciness` is an empty class -- a class with no instances and no definition. We will now give it both.

**Step 3: Create the individuals for the enumerated class**

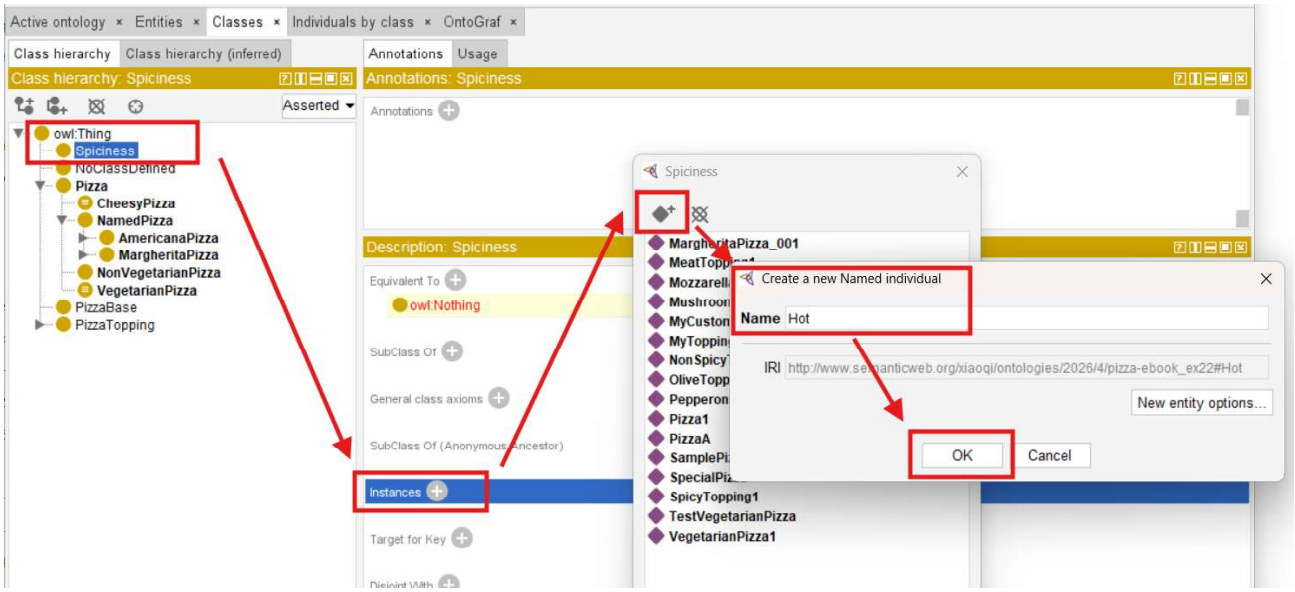
1. In the Classes tab, make sure `Spiciness` is selected. (After above Step2.5, the cursor should be by default on `Spiciness` .)
2. In the Description view, locate the `Instances` section near the bottom. (You may also switch to **Individuals by class** tab)



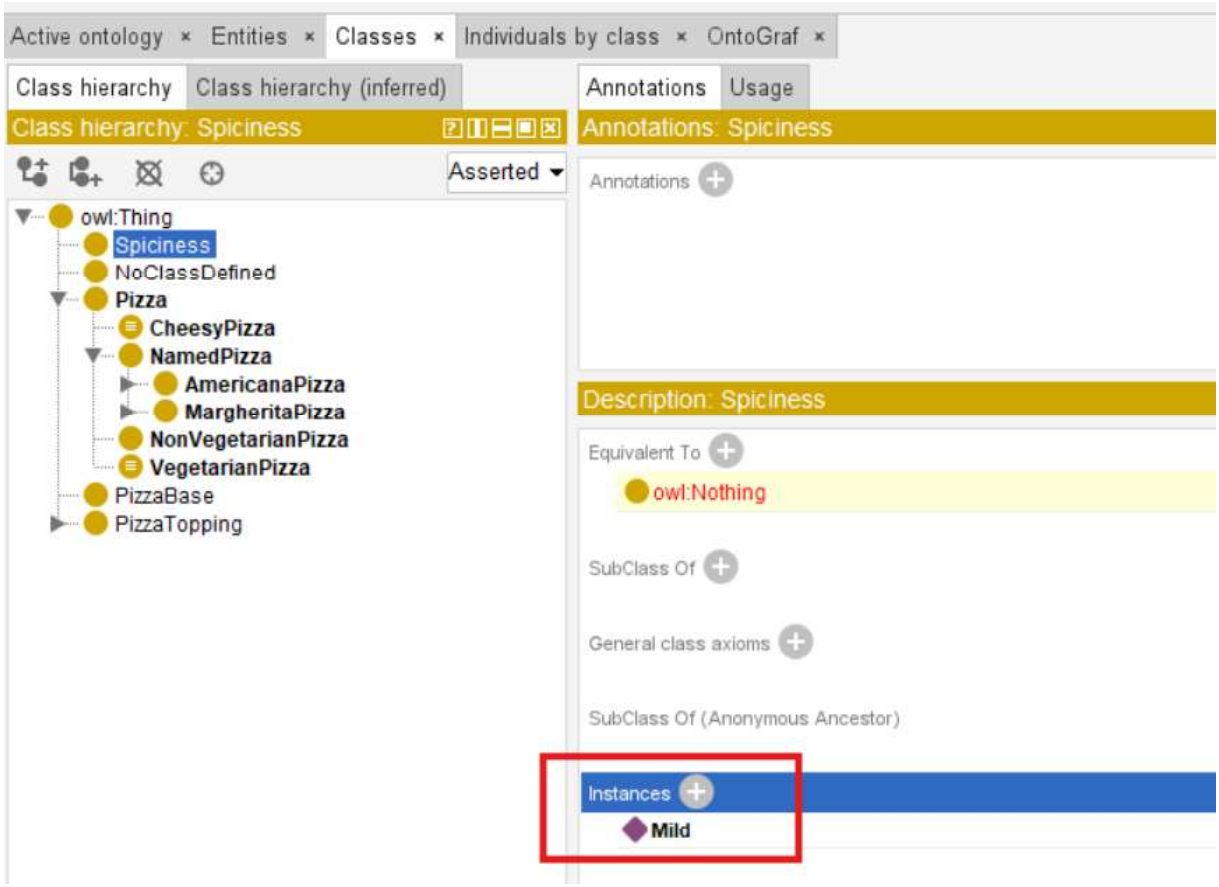

3. Click the Add icon (+) next to **Instances**




4. A dialog will appear with a diamond icon (create new individual) and a delete icon. Use the diamond icon to create three individuals in sequence: `Hot`, `Medium`, and `Mild`.

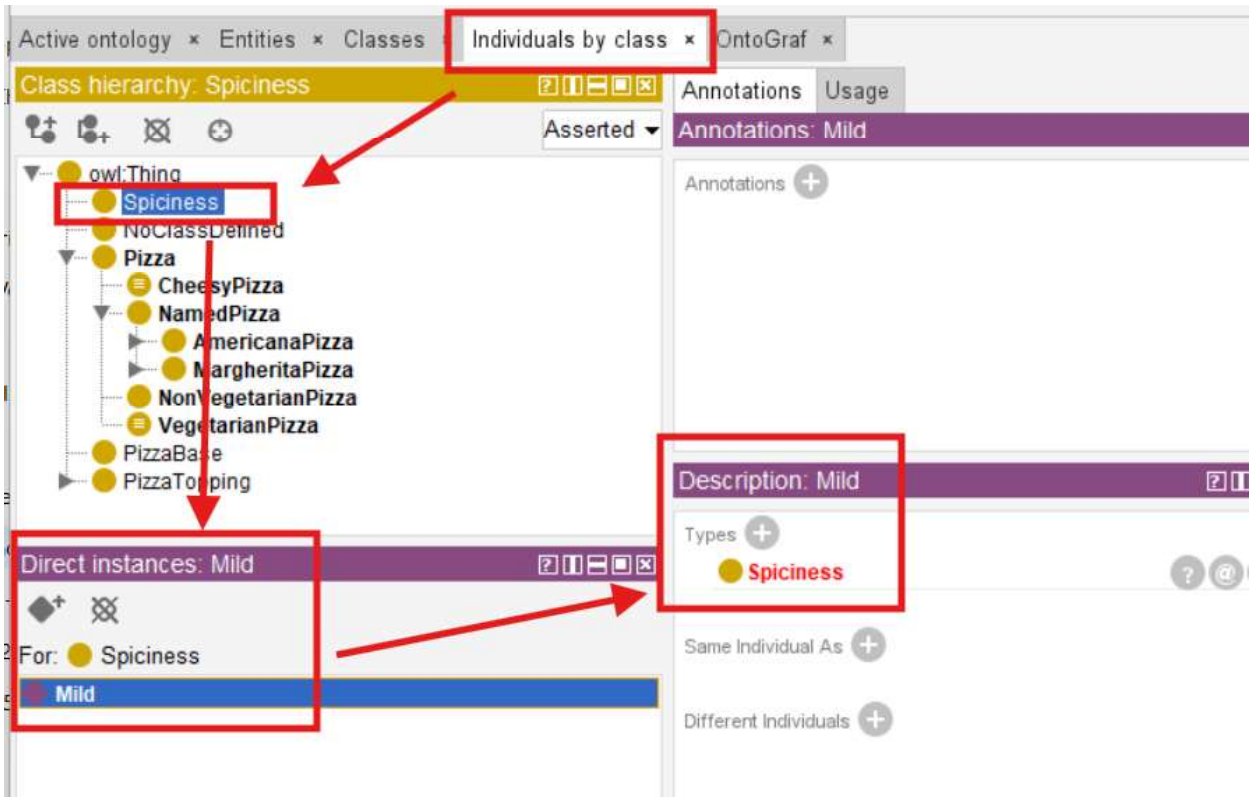



5. After creating, your `Spiciness` class' individuals should be like below:




**Tip**

You may also use **Individuals by class** tab to create those individuals, below is the screen of the three individuals after creating:



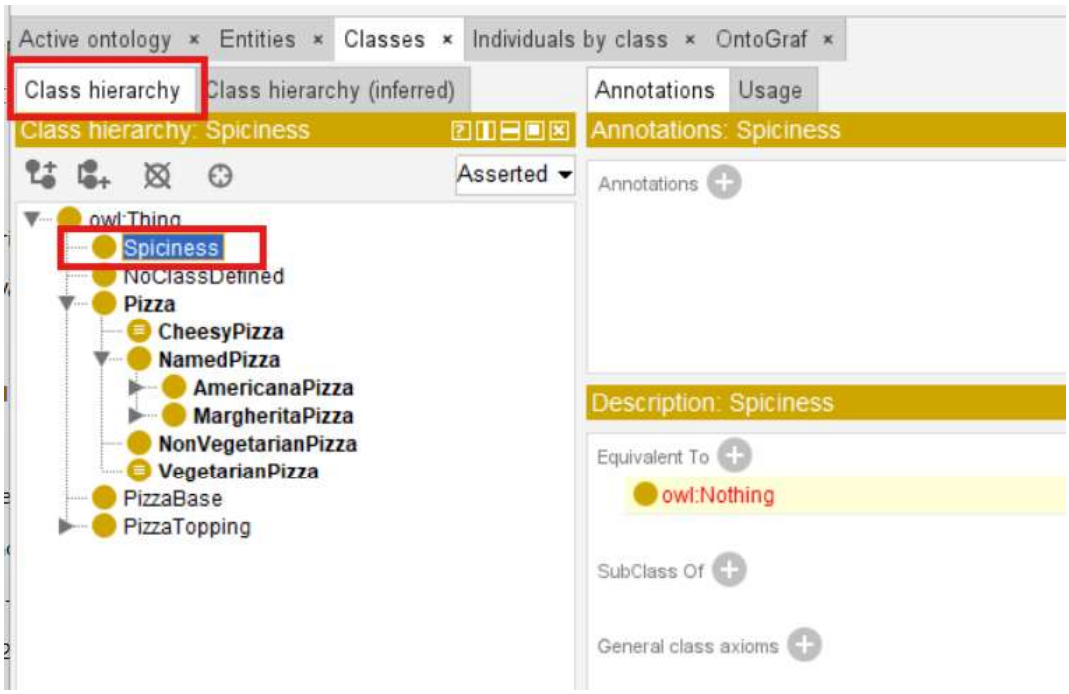

Choose either way which you are preferred.

### Note

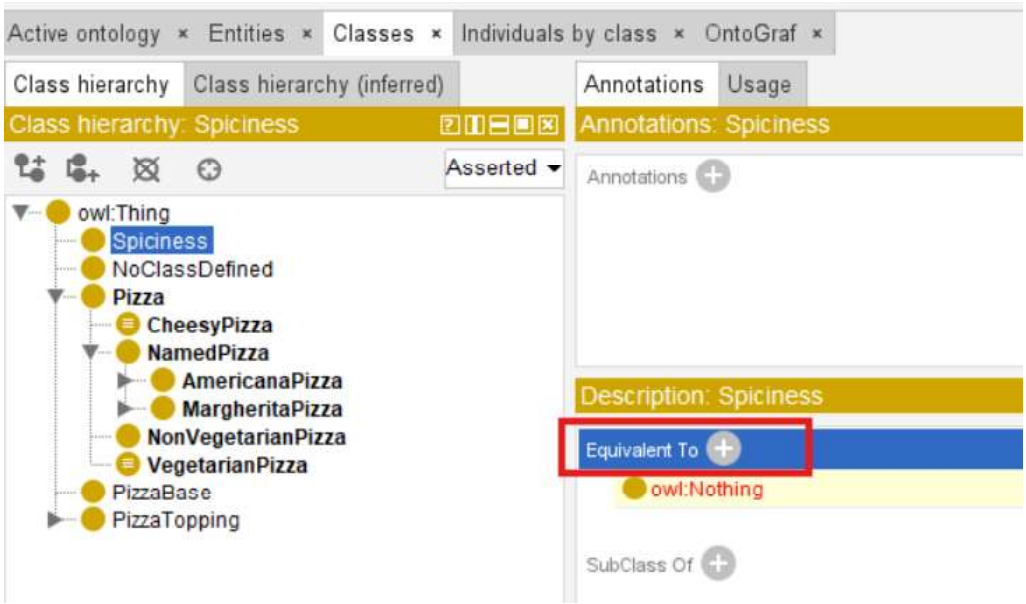
You may notice that only one of the three new individuals is actually created as an instance of `Spiciness` at this point. That is expected. The next step will supply the reasoner with enough information to recognize all three as instances of the class.

#### Step 4: Define `Spiciness` as an enumerated class

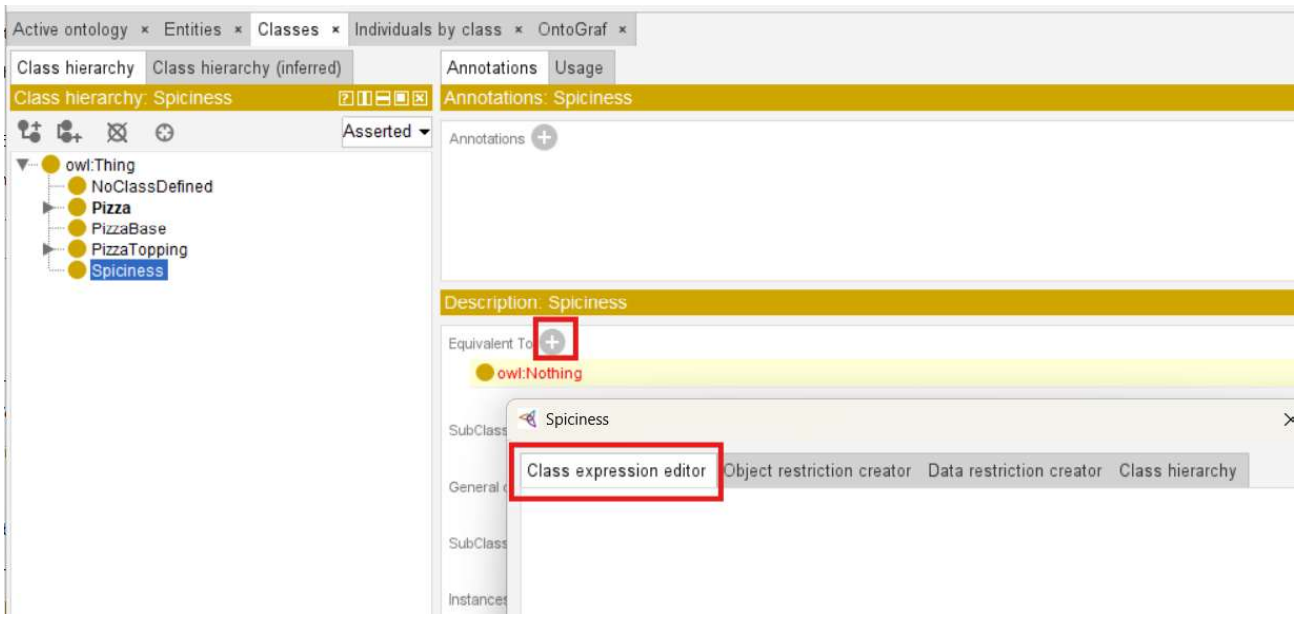
1. Make sure `Spiciness` is still selected



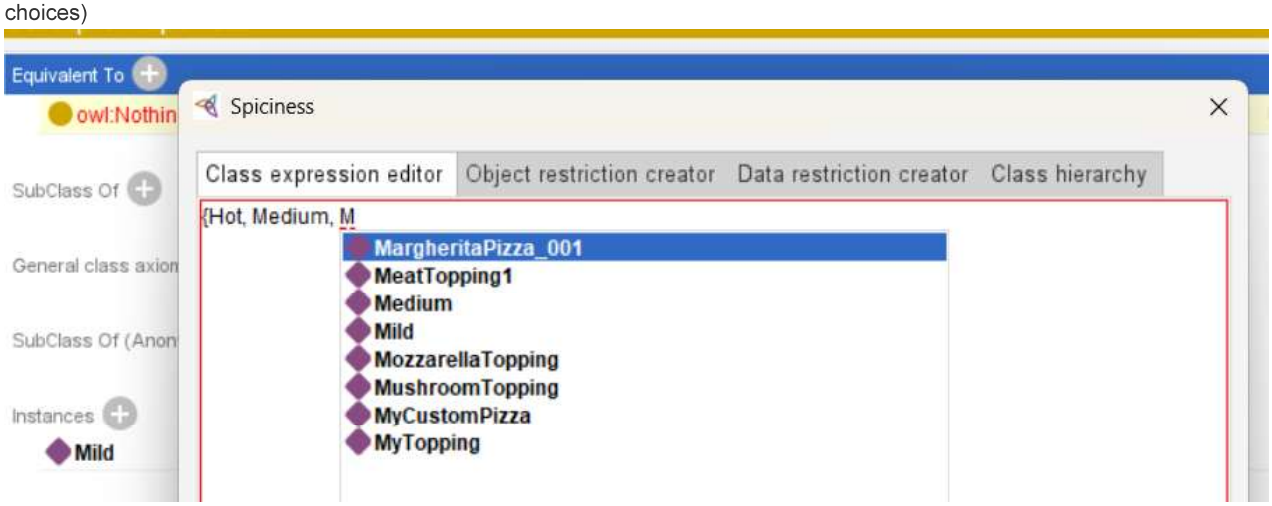

2. In the Description view, locate the `Equivalent To` section.



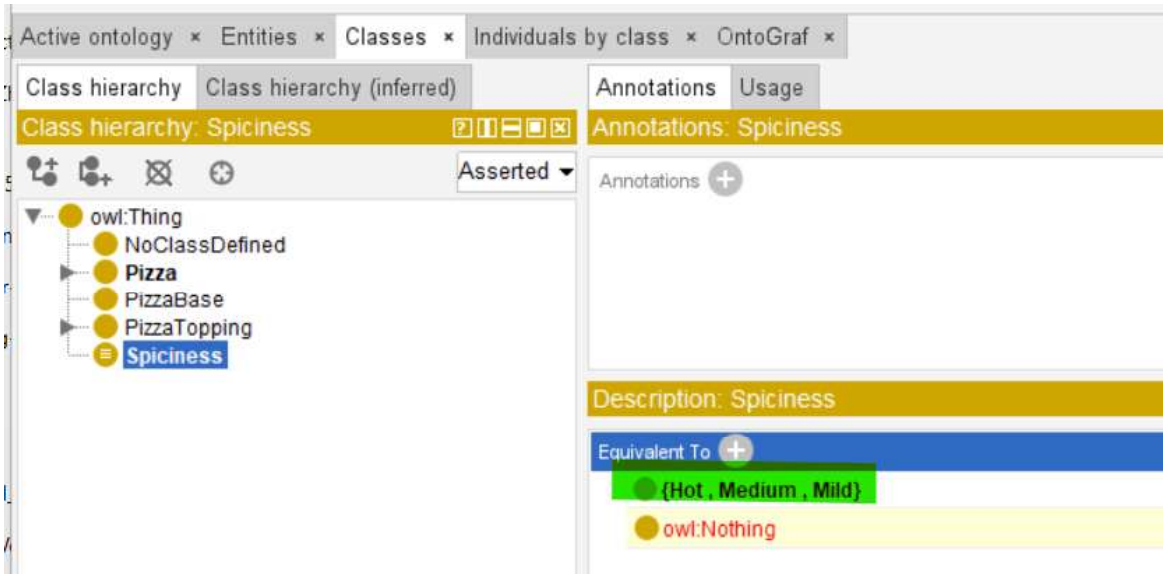

3. Click the Add icon (+)




4. In the Class Expression Editor, enter: `{Hot, Medium, Mild}`. (Tip: remember you may click Tab while typing to get possible choices)




5. Click `OK`.

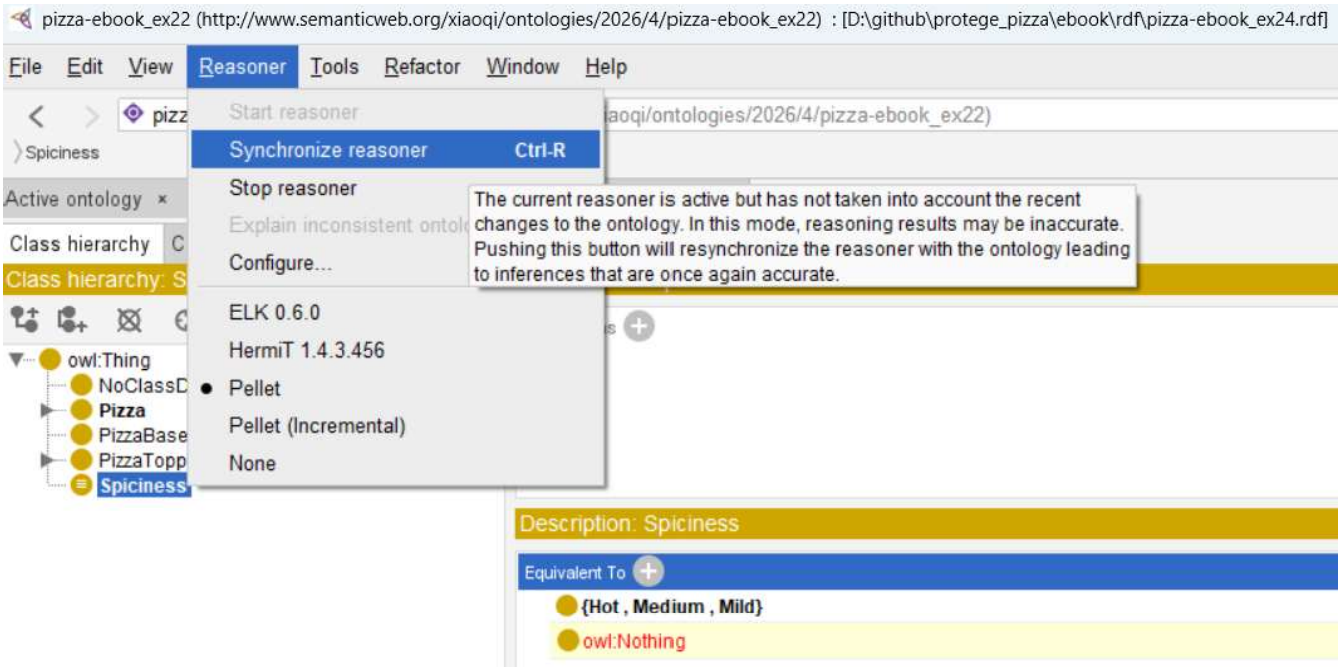


After this step, `Spiciness` class has the following description:

```
Class: Spiciness
  EquivalentTo: {Hot, Medium, Mild}
```

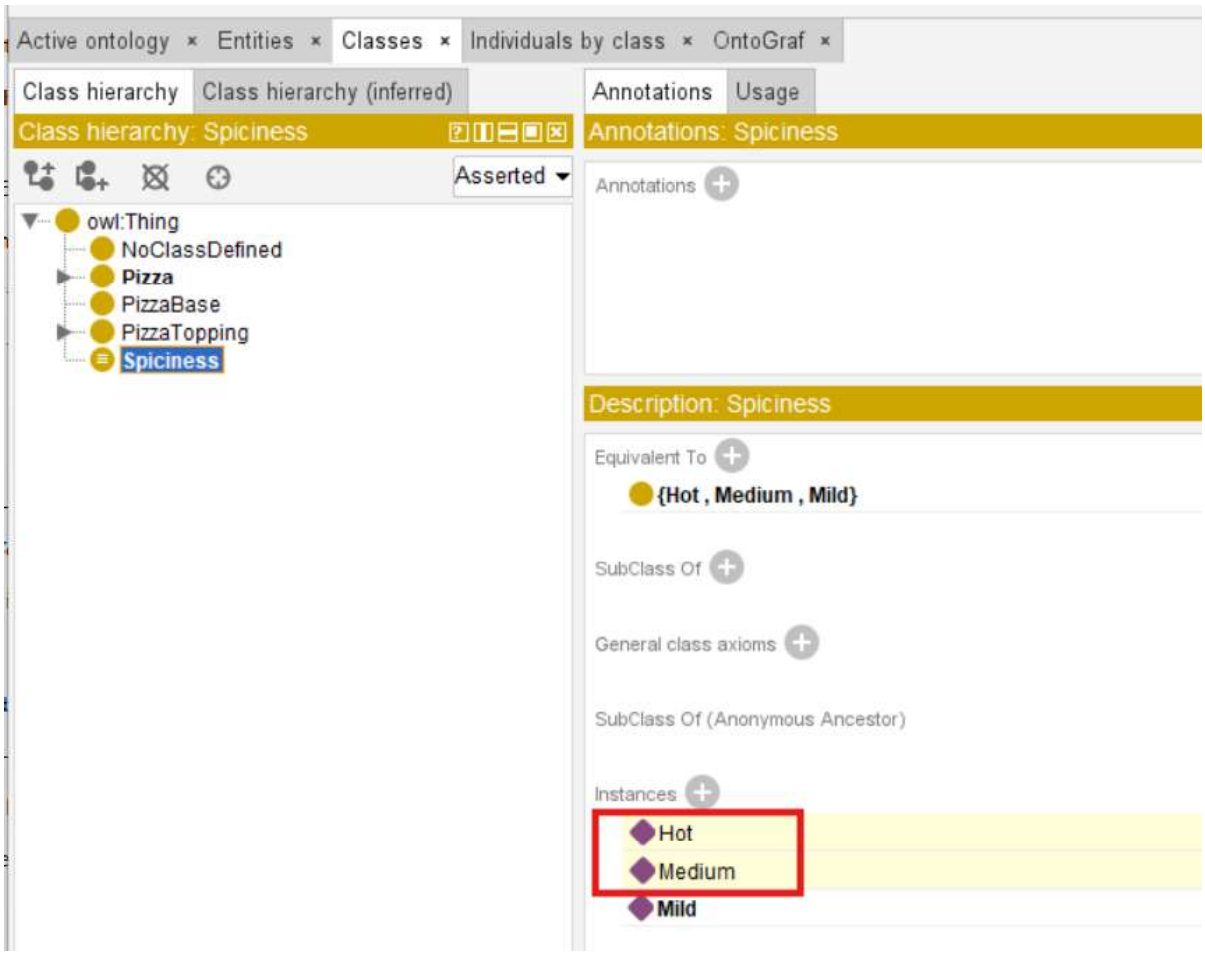
**Step 5: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)




**Step 6: Observe the result**

Look at `Spiciness` in the Classes tab.




The reasoner now recognizes `Spiciness` as a **defined class** (three horizontal stripes in the class icon), with exactly three instances: `Hot`, `Medium`, and `Mild`. No other individual can be an instance of `Spiciness`, because the class definition enumerates the complete set of allowed values.

The reasoner has done two things:

1. **Classified the three individuals as instances of `Spiciness`** -- even though we asserted only one of them directly, the enumeration definition now covers all three.
2. **Closed the value domain** -- the class `Spiciness` now has exactly three instances, and no more are possible.

This is the first time in the chapter that we have closed the world for *values* rather than for *properties*. The effect is immediate: what was previously an open-ended class with unknown membership is now a fully specified enumeration.

**Key Insight:**

> "Before the `EquivalentTo` axiom, `Spiciness` was an empty class -- a name without an extension. After the axiom, `Spiciness` has a complete, closed-world specification: exactly three values, and no others."

## 25.4.3 The Result: A Deterministic Value Domain

The enumerated class has provided a **deterministic value domain** for `Spiciness`.

For execution, this is essential. When an execution system encounters a `Spiciness` value, it knows the value must be one of `Hot`, `Medium`, or `Mild`. It can handle all possible cases exhaustively.

Contrast this with the alternative. If we had only declared `Spiciness` as a class (with `Hot`, `Medium`, and `Mild` as asserted instances), the reasoner would still recognize those three individuals as instances of `Spiciness`. But it would not know that they are the *only* instances. Under the Open World Assumption, there might be other spiciness values not yet asserted.

Enumeration removes this uncertainty. The `EquivalentTo {Hot, Medium, Mild}` axiom does two things simultaneously:

- It says that `Hot`, `Medium`, and `Mild` **are** instances of `Spiciness` (sufficient condition).
- It also says that anything else is **not** an instance of `Spiciness` (necessary condition).

This is what makes enumeration a closed-world specification.

**Key Insight:**

> "Enumerated classes are the OWL equivalent of an `enum` in a programming language. They define the complete set of possible values -- nothing more, nothing less."

## 25.4.4 Why Enumerated Classes Matter for Execution

Enumerated classes are essential for execution because:

1. **They define finite value domains.** Execution systems need to know the complete range of possible values to handle every case.
2. **They enable exhaustiveness checking.** If an execution system handles `Hot`, `Medium`, and `Mild`, it has handled all cases -- because the ontology guarantees there are no others.
3. **They convert open-world value domains into closed-world specifications.** Without enumeration, a property might have unknown or additional values. With enumeration, all values are known.
4. **They compose with other execution primitives.** Enumeration works with `hasValue` restrictions (**Exercise 25**) and cardinality restrictions (**Exercise 26**) to build richer execution logic. For example, an execution rule can fire on `hasSpiciness value Hot`, confident that `Hot` is a fixed, well-known value from a closed enumeration.

The table below contrasts the behavior of a property with and without an enumerated value domain:

| Aspect | Without Enumeration | With Enumeration |
|---|---|---|
| **Value domain** | Open -- any value is possible in principle | Closed -- only the listed values are possible |
| **Reasoner behavior** | Cannot exhaustively check value conditions | Can exhaustively check value conditions |
| **Semantic strength** | Description of a class | Specification of a value domain |
| **Execution readiness** | Not executable for value-based triggers | Executable for value-based triggers |

**Key Insight:**

> "Execution cannot handle unknown values. Enumeration is how the ontology guarantees that all values are known."

## 25.4.5 Engineering Perspective: Exercise 24 as a Case Study

From the perspective of Protégé, **Exercise 24** involves creating a class, creating three individuals, and adding a single `EquivalentTo` axiom. It appears to be a small modeling activity.

From the perspective of ontology engineering, however, something significant has occurred. The ontology now contains a **completely specified value domain** -- a closed-world set of possible values.

> What was previously an open class has become a fully enumerated one.

The engineering contribution of **Exercise 24** can be summarized as follows:

| Engineering Perspective | Contribution from Exercise 24 |
|---|---|
| **SKDL Stage** | Stage 7 -- Executable Knowledge |
| **Primary Objective** | Introduce the enumerated class as a value-domain specification |
| **Modeling Activity** | Define `Spiciness` as `{Hot, Medium, Mild}` |
| **Engineering Outcome** | Provide a finite, deterministic value domain for execution; the reasoner now treats `Spiciness` as a closed-world enumeration |
| **Preparation for Later Sections** | Establish the enumerated value foundation for the `hasValue` restrictions in **Exercise 25** and the cardinality-based logic in **Exercise 26** |

**Key Insight:**

> "**Exercise 24** is the moment the ontology specifies a complete value domain. This is essential for exhaustive execution logic."

**Tip**

Following the Tutorial

**Exercise 24** in the original `Pizza` Tutorial introduces the enumerated class `Spiciness` with the individuals `Hot`, `Medium`, and `Mild`, and defines `Spiciness` as `EquivalentTo {Hot, Medium, Mild}`. The tutorial then demonstrates the reasoner recognizing all three individuals as instances of the class.

In this book, we have followed the tutorial exactly -- same class, same individuals, same axiom. The only difference is presentational: we have added the observation that this constitutes a *closed-world specification of a value domain*, which is what makes enumeration as execution primitive for **Stage 7.** The tutorial introduces the construct; the book frames it in terms of executability.

# 25.5 Exercise 25 -- `hasValue` Restrictions and Specific-Value Binding

Having established the enumerated value domain in **Exercise 24**, we now turn to **Exercise 25**, which introduces the `hasValue` **restriction** -- the construct that binds a property to a specific individual.

This exercise completes the value-binding layer of Stage 7. In **Exercise 24**, we closed the world for a *value domain* ( `Spiciness = {Hot, Medium, Mild}` ). In **Exercise 25**, we close the world for a *specific value* -- binding a property to one particular individual from that domain.

## 25.5.1 The Engineering Objective

**Exercise 25** introduces the `hasValue` **restriction**, which specifies that a property must have exactly one specific value.

A `hasValue` restriction for a spicy pizza, for example, would be:

```
Class: SpicyPizza
  EquivalentTo: Pizza and (hasTopping some (hasSpiciness value Hot))
```

This means:

> "A pizza is a `SpicyPizza` if and only if ( `iff` ) it has at least one topping whose spiciness is `Hot`."

The engineering objective of **Exercise 25** is to demonstrate how `hasValue` restrictions enable **value-based execution logic** -- the ability to fire a trigger on a specific value rather than a general class of values.

Why does execution need `hasValue` ?

Consider an execution rule that fires when a pizza's spiciness is `Hot` . For the rule to be reliable, the system must be able to detect the *specific value* `Hot` -- not just any value from the `Spiciness` class. The `hasValue` restriction is the OWL construct that expresses this specific-value condition.

Where **Exercise 24** gave us the *closed value domain*, **Exercise 25** gives us the *specific value binding*.

Together, they form the value layer of executability.

**Key Insight:**

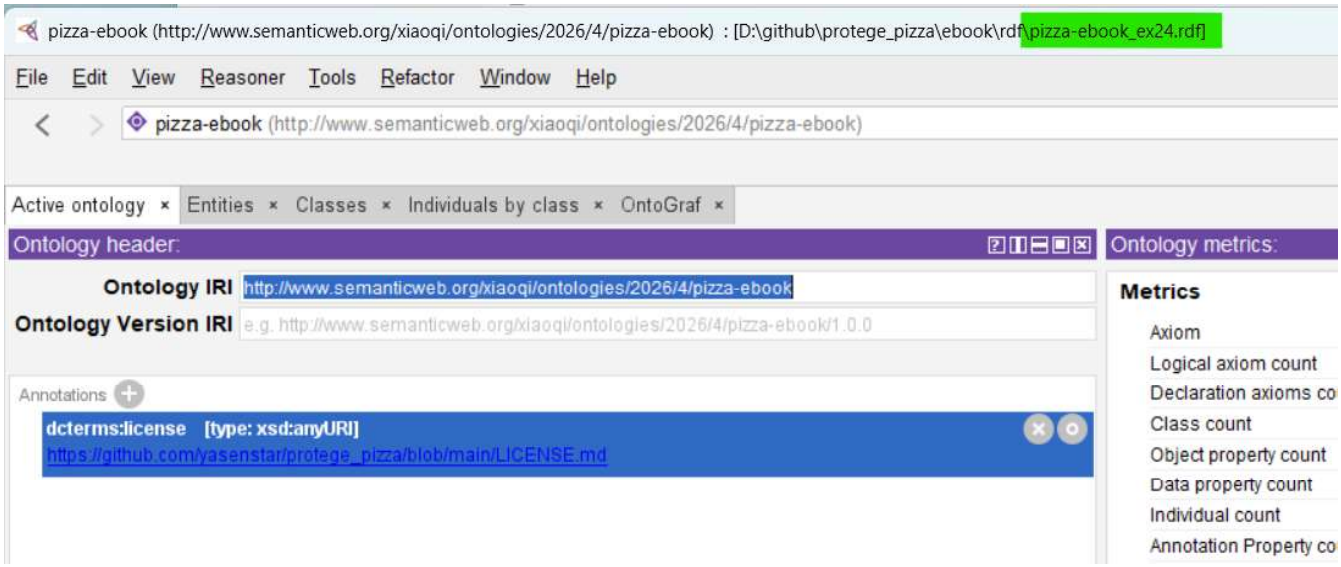
> "The `hasValue` restriction is the semantic equivalent of a conditional statement. It says: if the value is $X$, then the condition is true."

## 25.5.2 Implementation: Creating the `hasValue` Restriction

Let's walk through the implementation step by step. The workflow introduces a new property, then uses it to bind a specific value in two places: on a topping class, and in a defined pizza class.

**Step 1: Open the `Pizza` ontology**

Open `pizza-ebook_ex24.rdf` in Protégé -- the file from the previous exercise, which now contains the enumerated class `Spiciness` with its three instances `Hot` , `Medium` , and `Mild` . It also contains closure axioms from **Exercise 23**.




**Note**

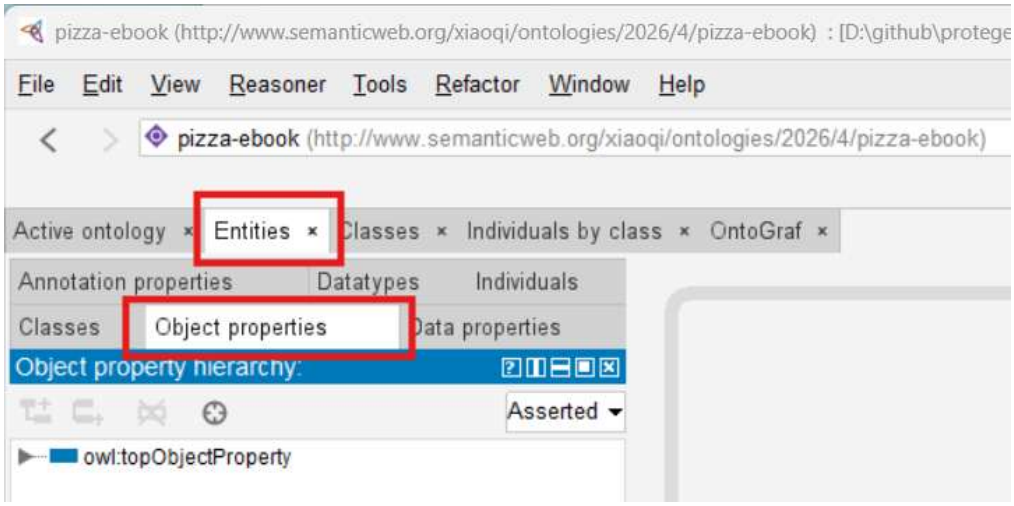
The finished state of **Exercise 25** will be saved as `pizza-ebook_ex25.rdf` .
Two changes in the RDF for your attention:

1. Ontology IRI remove the exercise number context, keep as `http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook` so that you don't need to change PREFIX when you create SPARQL queries.

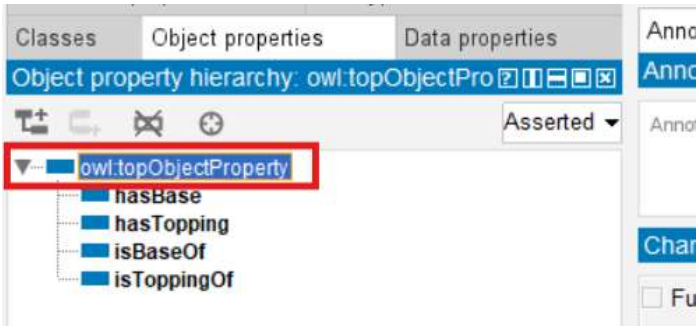
2. License link is updated as https://github.com/yasenstar/protege_pizza/blob/main/LICENSE.md

**Step 2: Create the `hasSpiciness` object property**

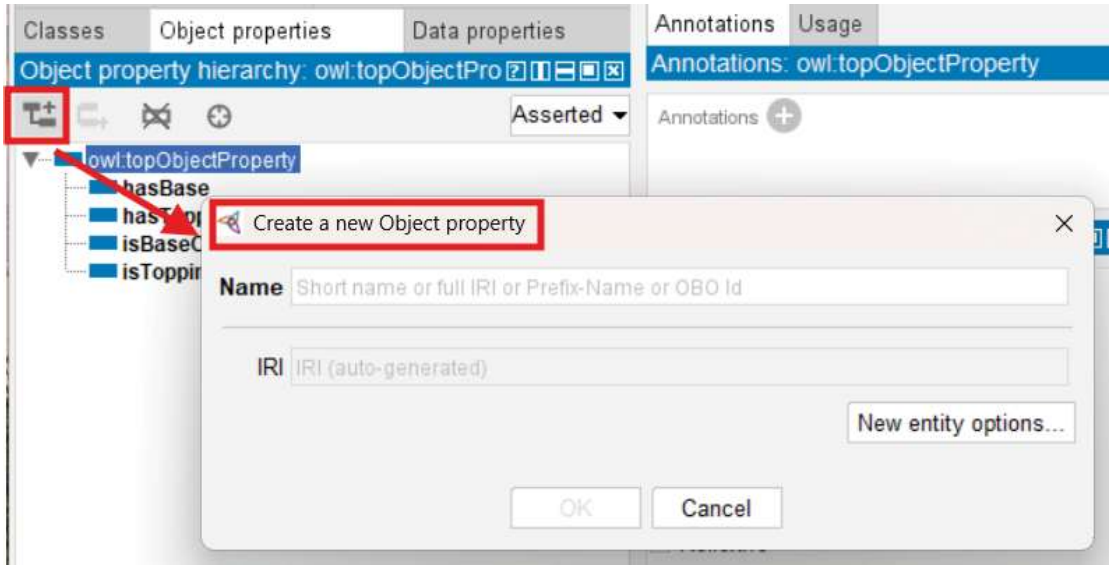
1. Navigate to the **Object properties** tab. (Under **Entities** tab)



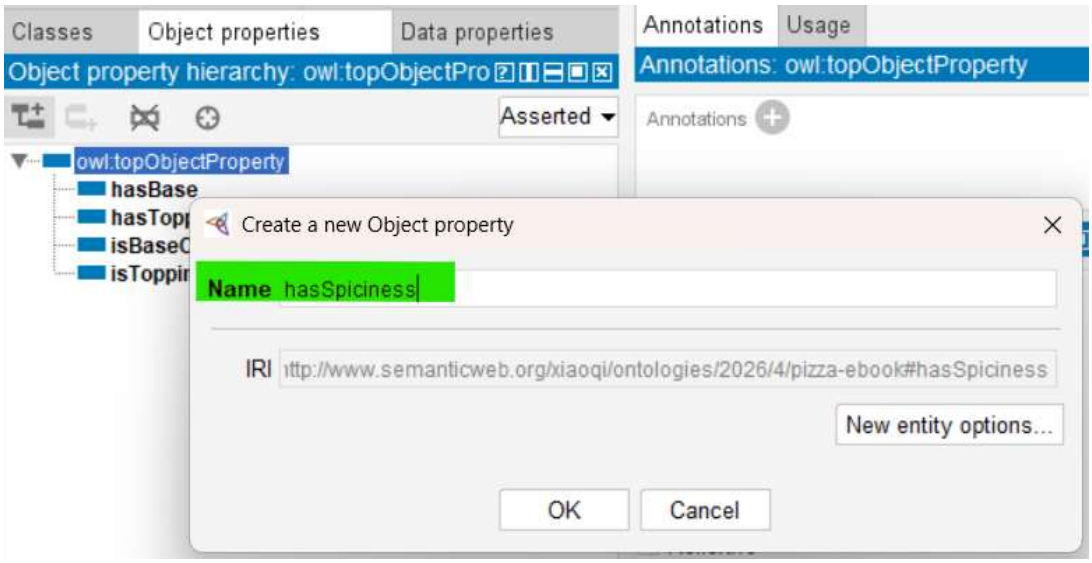

2. Select `owl:topObjectProperty` in the property hierarchy



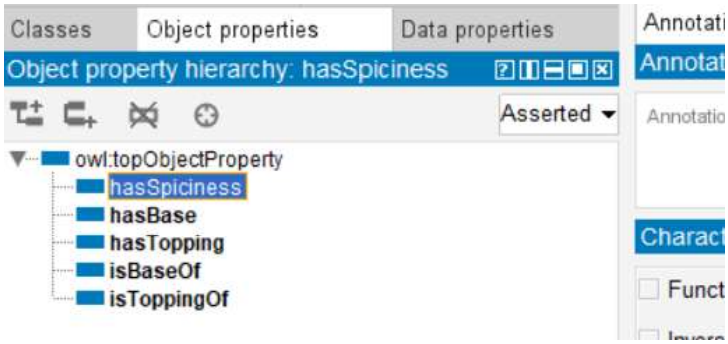

3. Click the **Add sub property** icon




4. Name the new property `hasSpiciness`

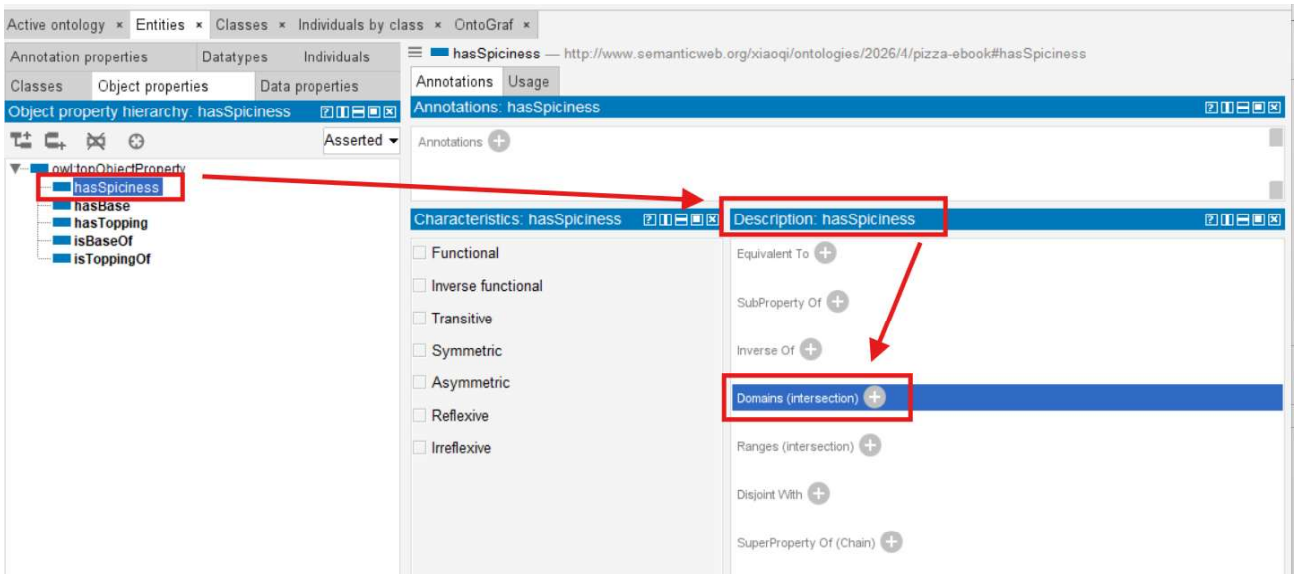



5. Click `OK`.



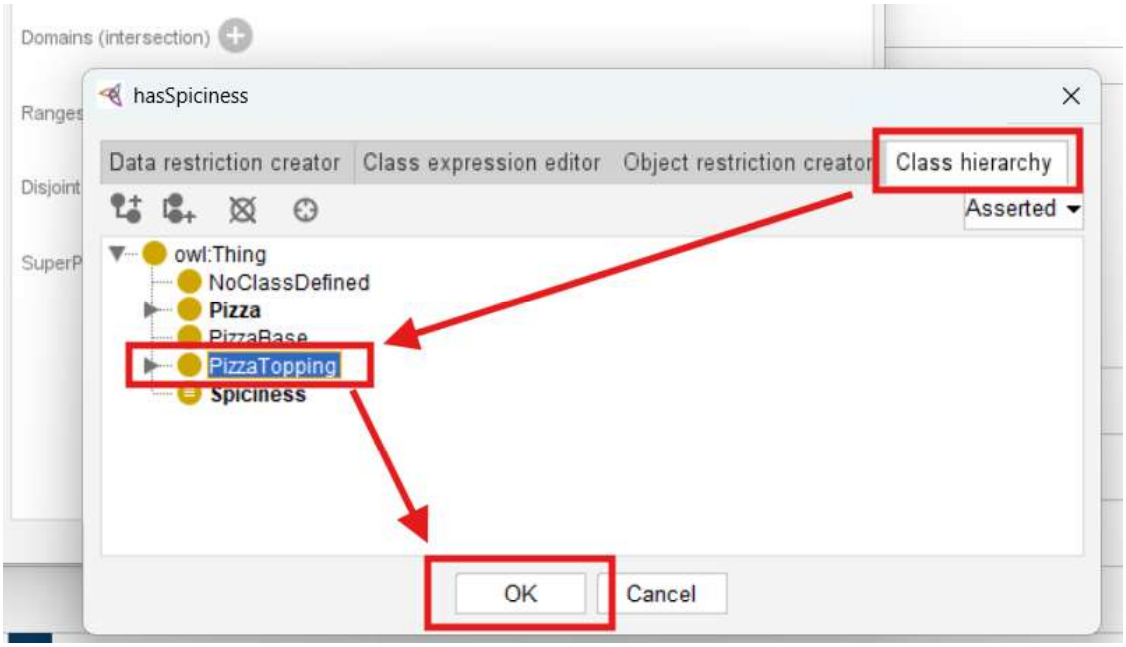

**Step 3: Define the domain and range of `hasSpiciness`**

1. Select `hasSpiciness` in the property hierarchy
2. In the Description view, locate the **Domains (intersection)** section

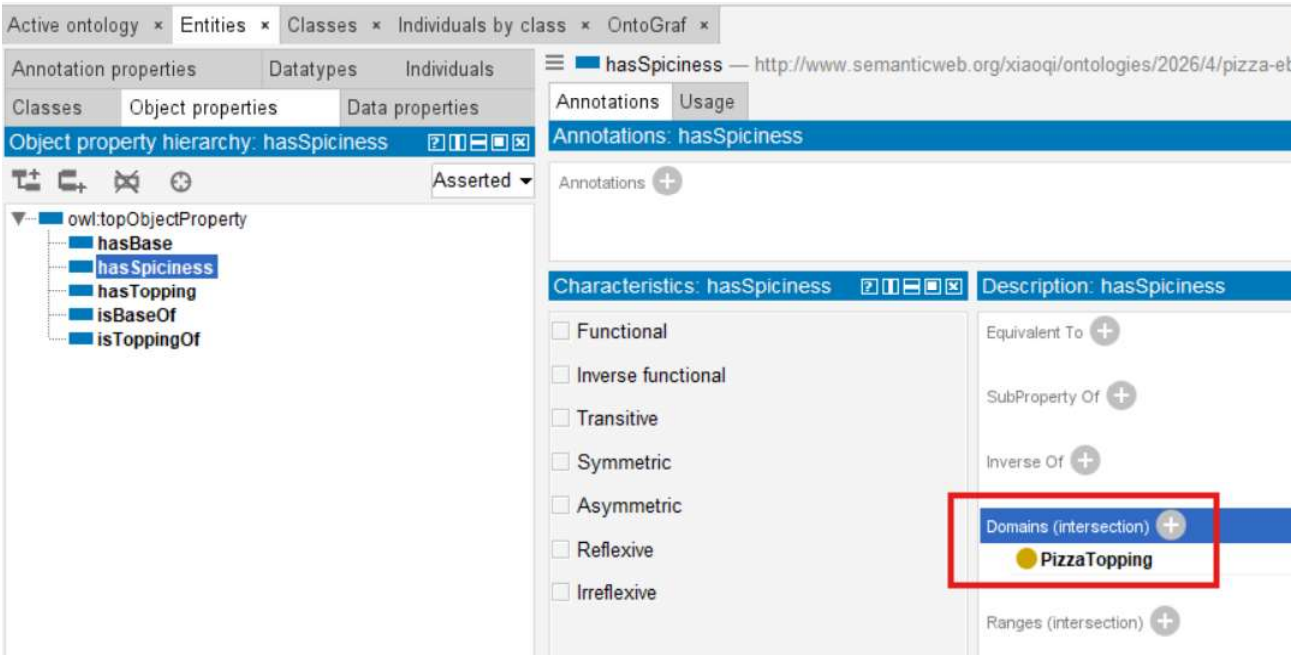



3. Click the Add icon (+), it may keep showing your previous tab, like this `Class expression editor`

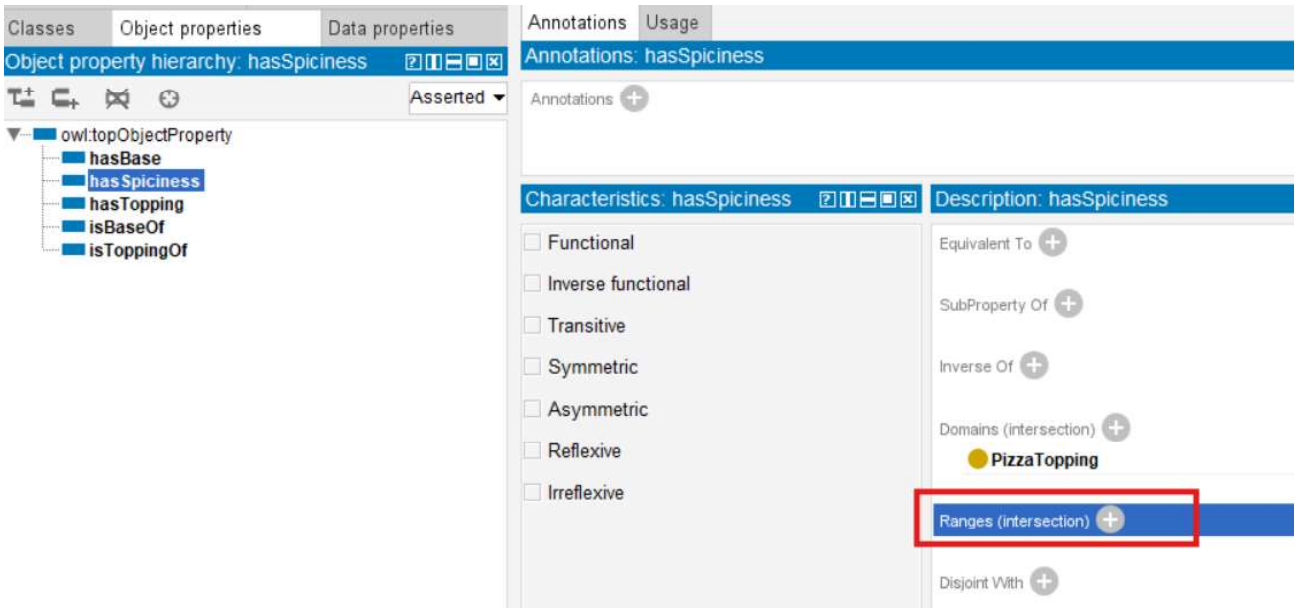
4. In the dialog, use the **Class hierarchy** tab to select `PizzaTopping`



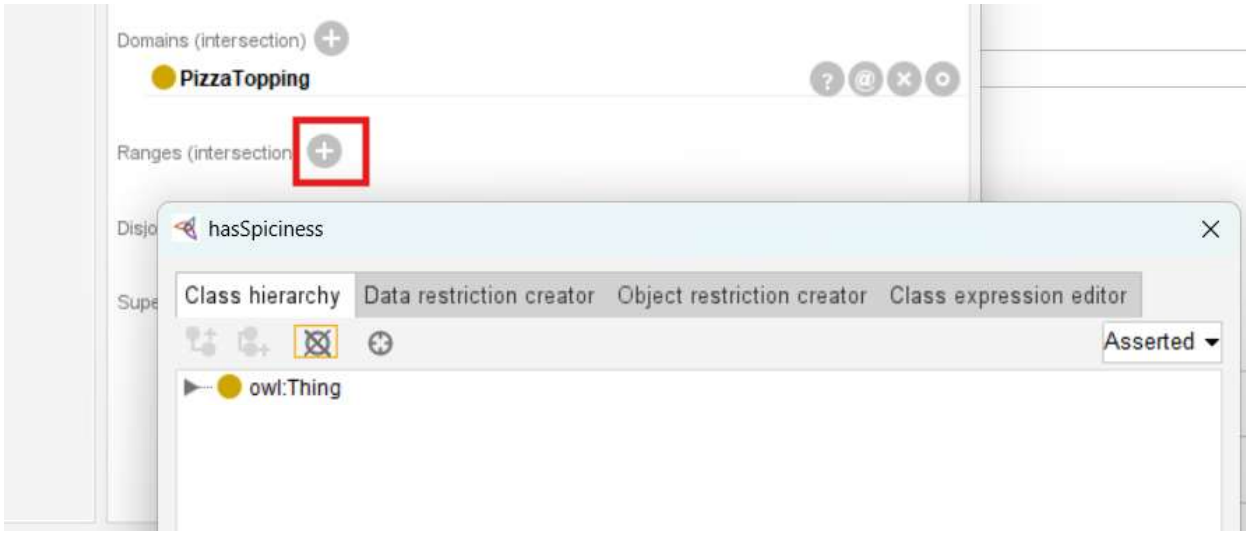

5. Click `OK`



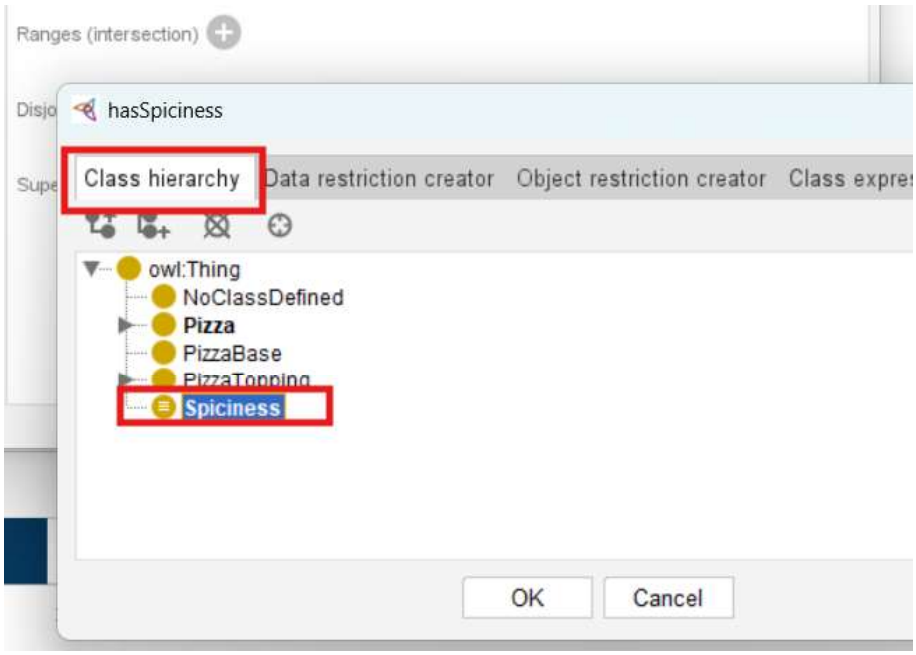

6. In the Description view, locate the **Range (intersection)** section



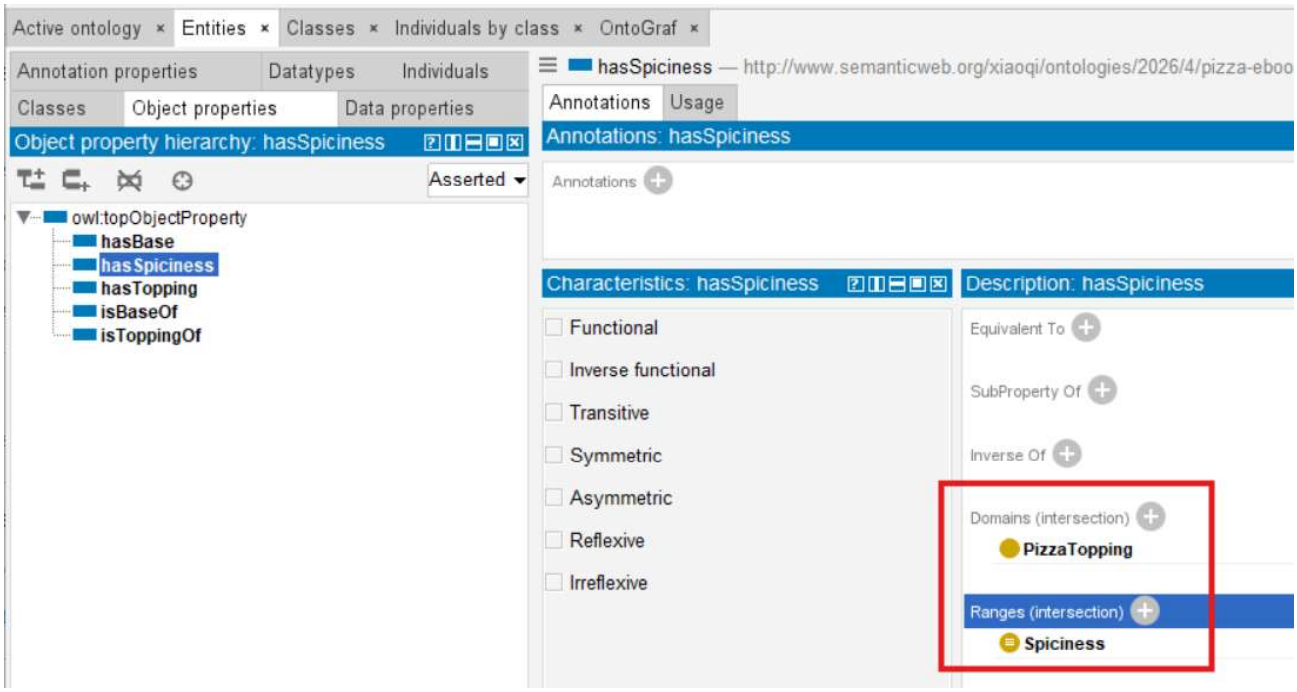

7. Click the Add icon (+)




8. In the dialog, use the **Class hierarchy** tab to select `Spiciness`




9. Click `OK`




After this step, `hasSpiciness` has the following description:

```
Object Property: hasSpiciness
   Domain: PizzaTopping
   Range: Spiciness
```

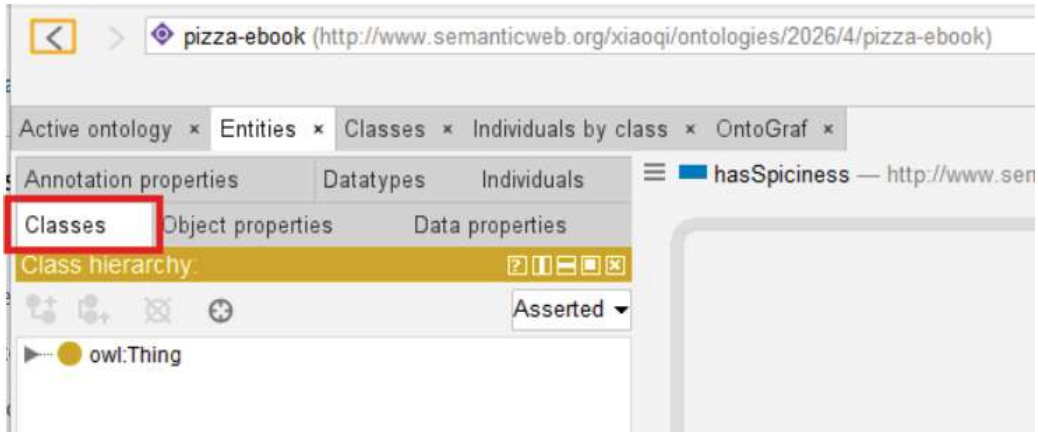
**Step 4: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R).

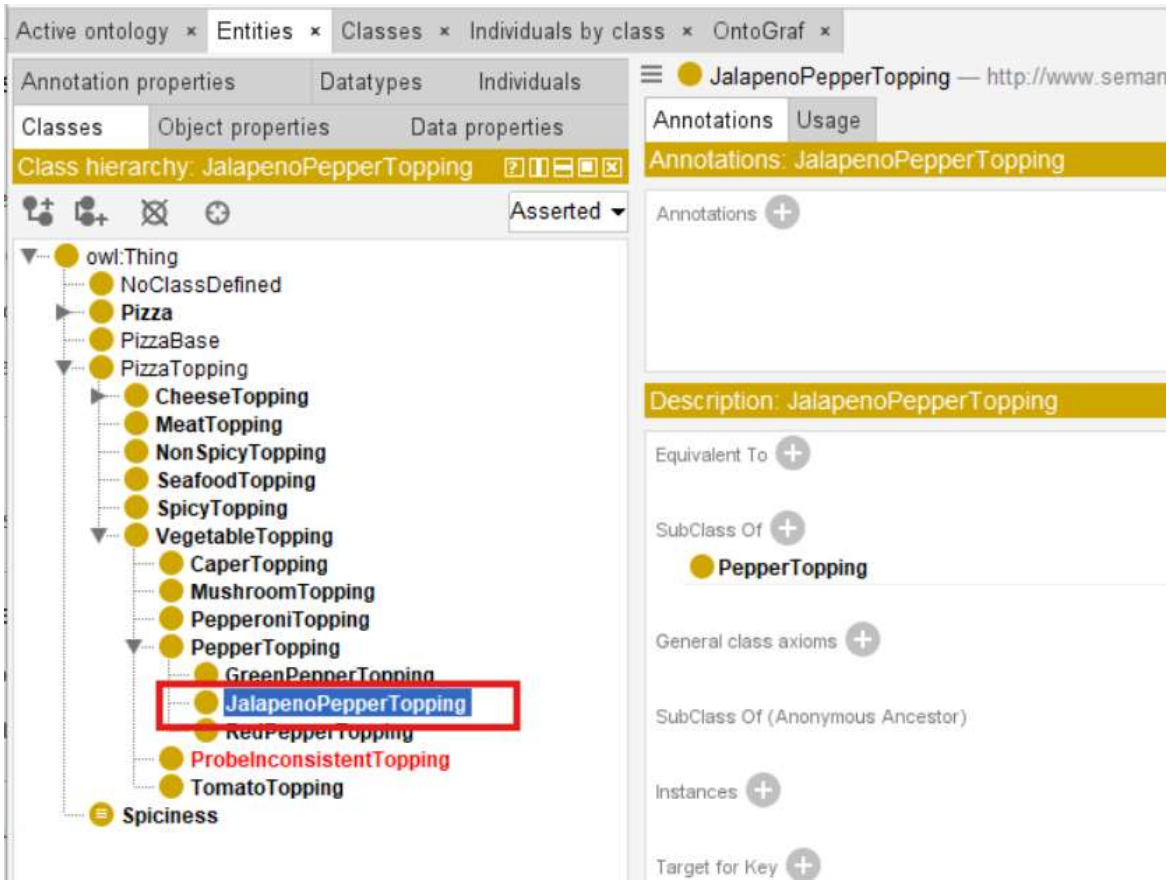
The reasoner now knows about the new property and its domain/range constraints. This is important before we use the property in restrictions -- the reasoner will validate that the restrictions we add are consistent with the property's signature.

**Step 5: Add the `hasValue` restriction to `JalapenoPepperTopping`**

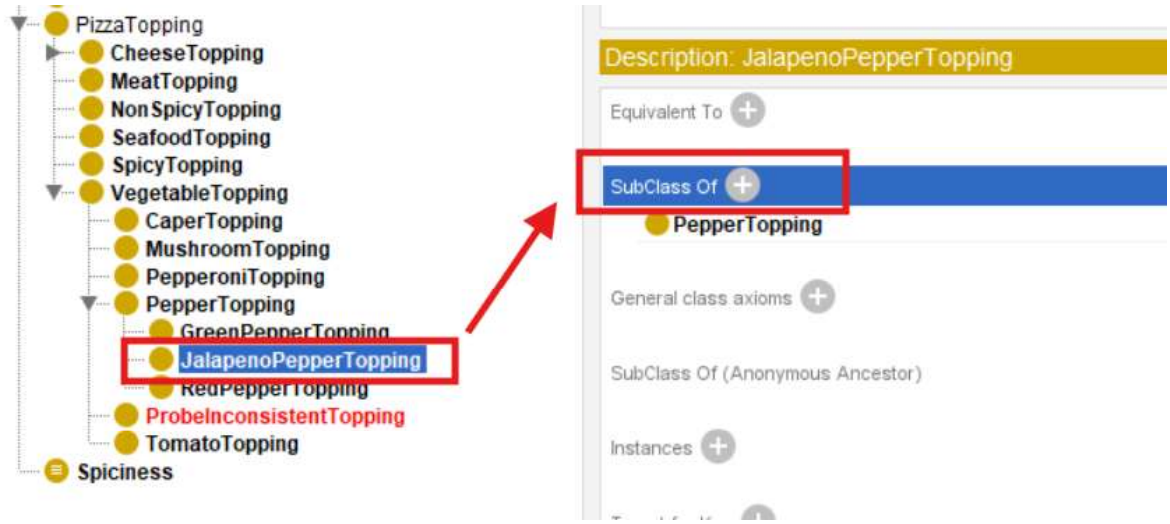
1. Navigate to the **Classes** tab



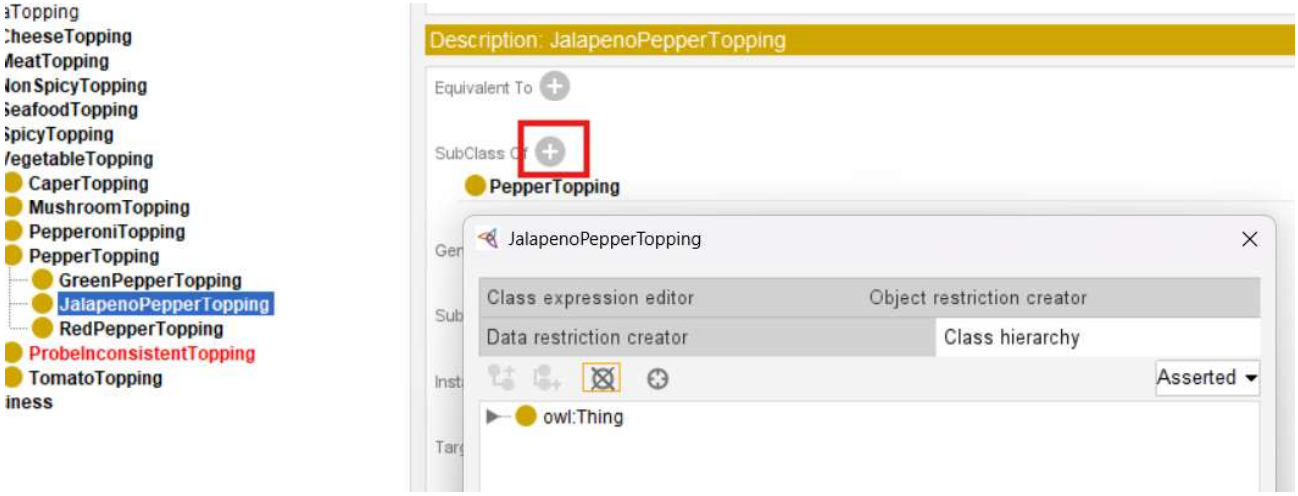

2. Select `JalapenoPepperTopping` in the class hierarchy



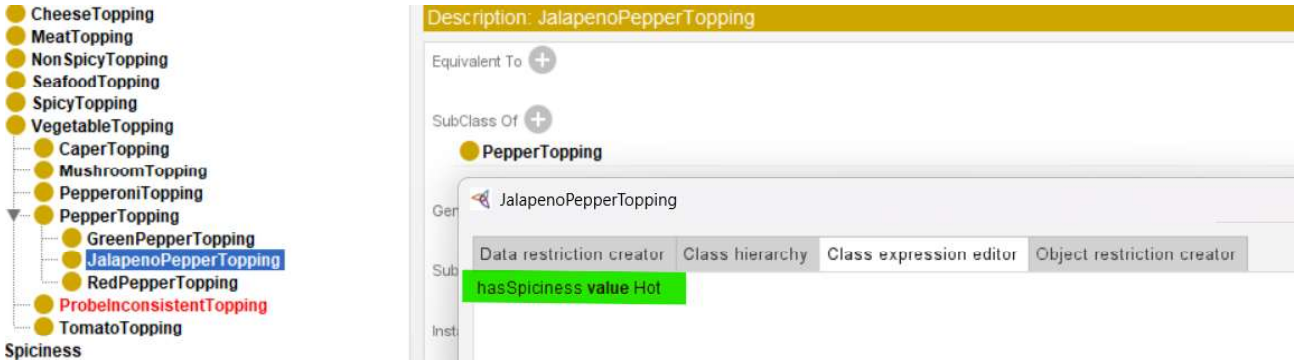

3. In the **Description** view, locate the **SubClass Of** section



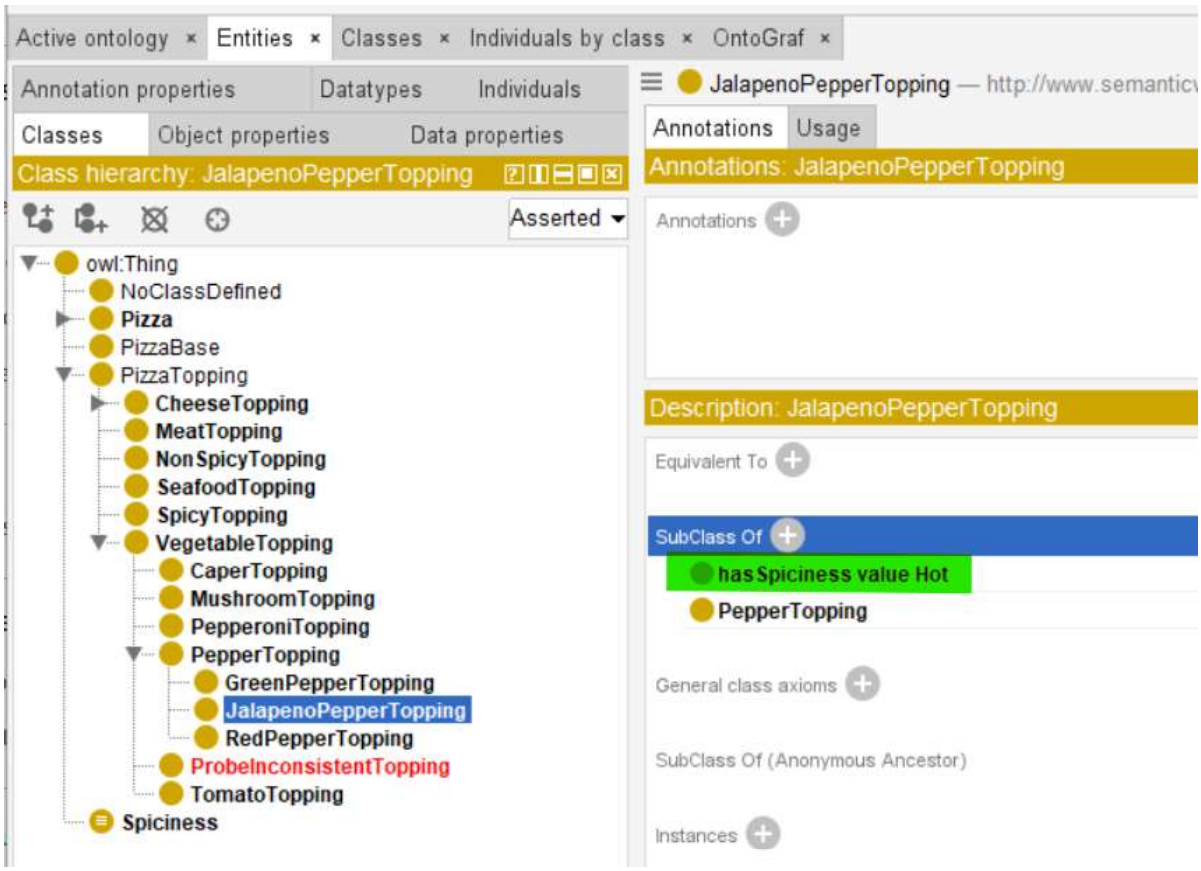

4. Click the Add icon (+)




5. In the Class Expression Editor, enter: `hasSpiciness value Hot`




6. Click OK




After this step, `JalapenoPepperTopping` has:

```
Class: JalapenoPepperTopping
   SubClassOf: PepperTopping
   SubClassOf: hasSpiciness value Hot
```

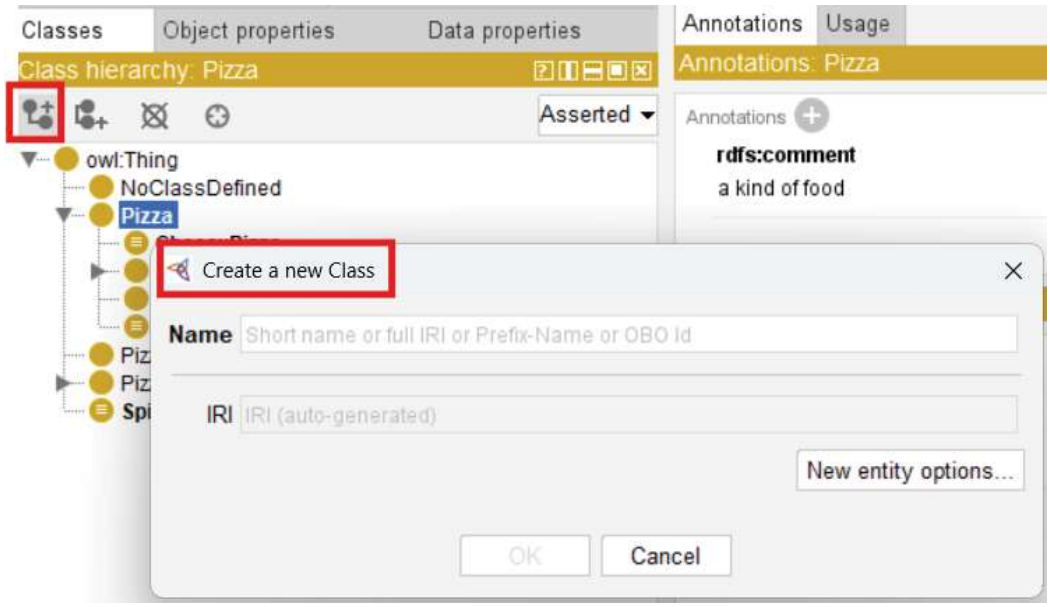
**Note**

This is a different kind of restriction from the ones we have ever used before. In `some` and `only` restrictions, the filler was a *class* -- `hasTopping some CheeseTopping` says "at least one cheese topping," but does not specify *which* cheese topping. In a `value` restriction, the filler is a *specific individual* -- `hasSpiciness value Hot` says the spiciness must be exactly the individual `Hot`. This is why the keyword is `value` rather than `some` or `only`.

### Step 6: Create the `SpicyPizza` class

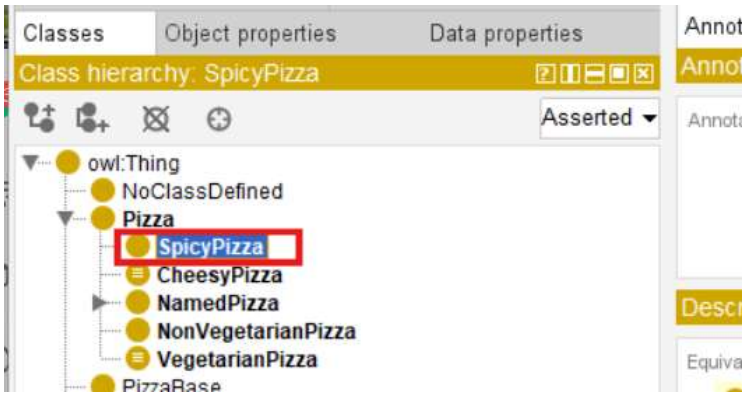
1. Make sure `Pizza` is selected in the class hierarchy

2. Click the **Add subclass** icon.




3. Name the new class `SpicyPizza`

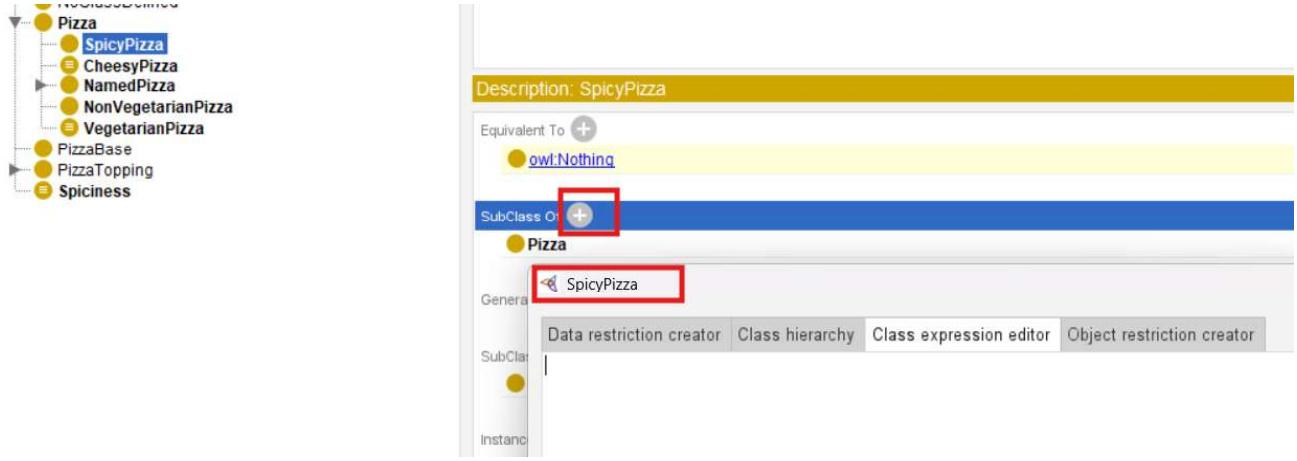
4. Click `OK`



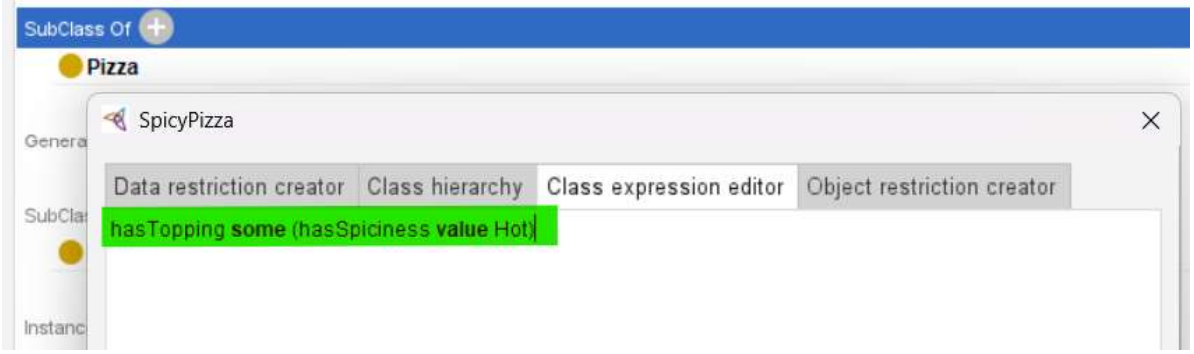

**Step 7: Add the nested `hasValue` restriction to `SpicyPizza`**

1. Select `SpicyPizza`
2. In the **Description** view, locate the **SubClass Of** section

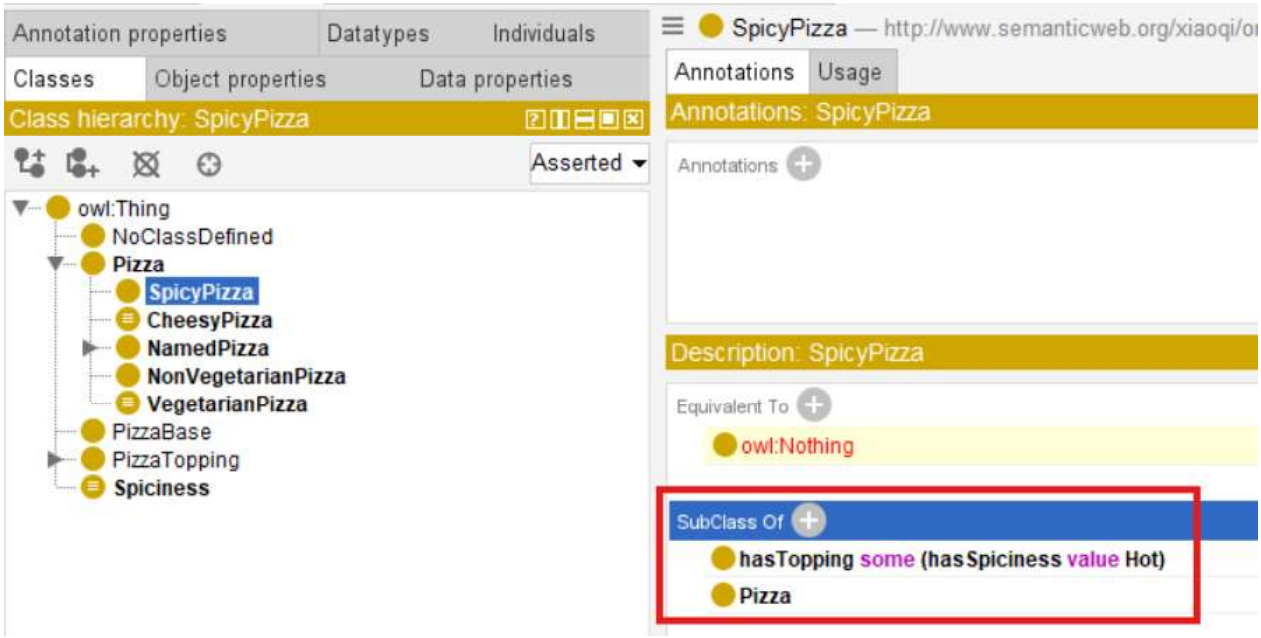
3. Click the Add icon (+)




4. In the Class Expression Editor, enter: `hasTopping some (hasSpiciness value Hot)`




5. Click `OK`



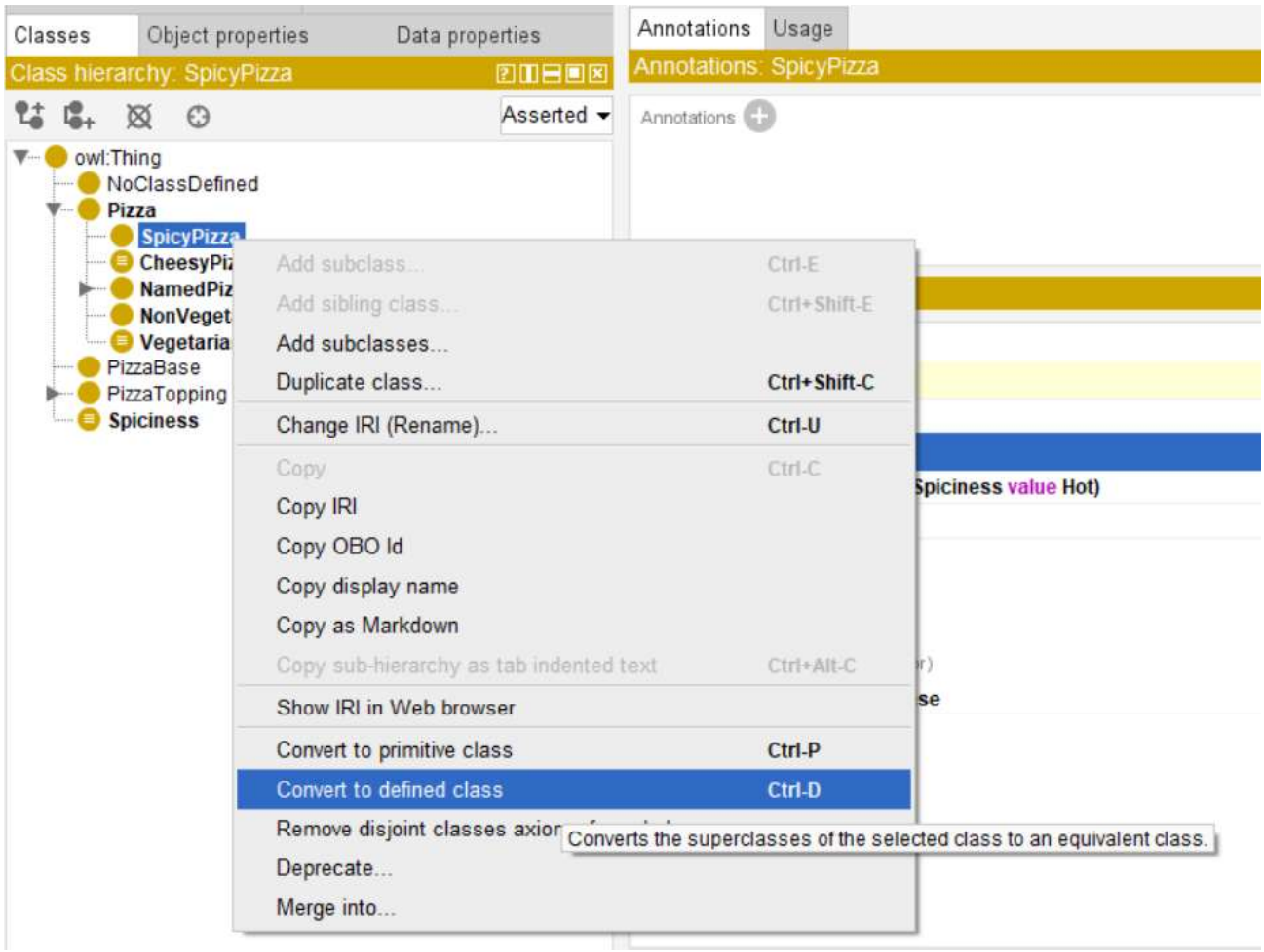

After this step, `SpicyPizza` has:

```
Class: SpicyPizza
   SubClassOf: Pizza
   SubClassOf: hasTopping some (hasSpiciness value Hot)
```

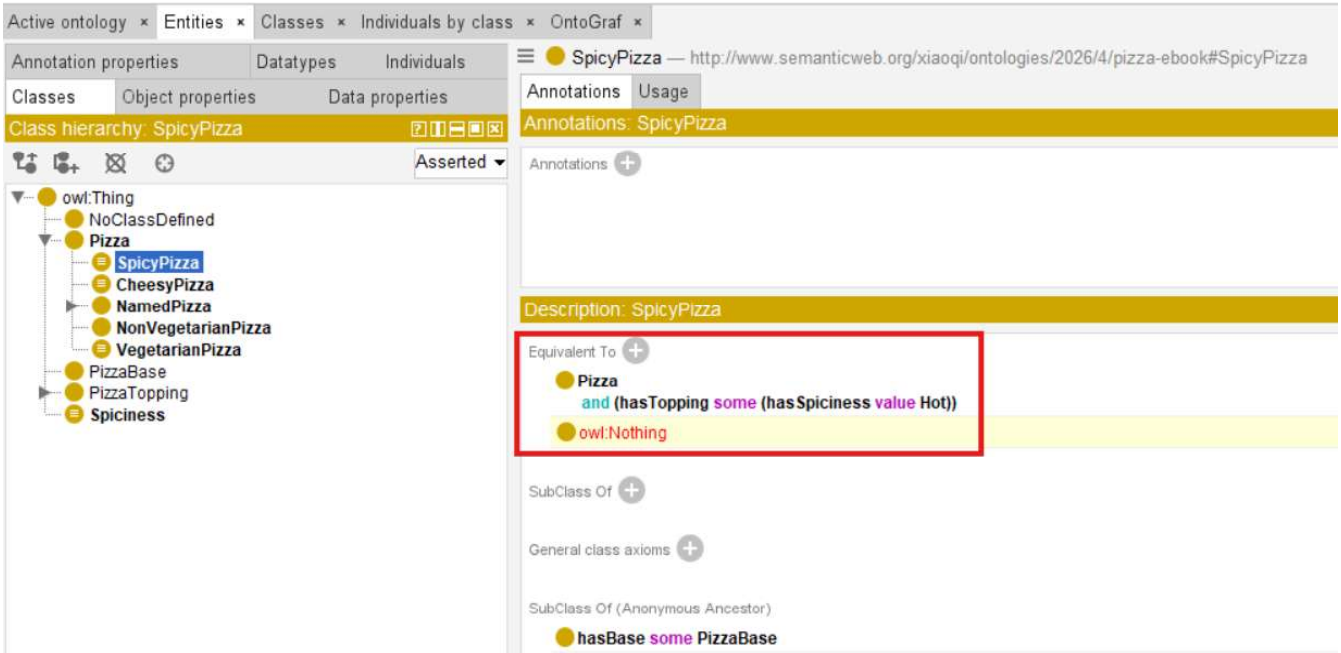
This says: "A `SpicyPizza` must have at least one topping whose `hasSpiciness` property value is `Hot`." Note the *nesting*: `hasTopping some (...)` is an existential restriction whose filler is itself a `hasValue` restriction. This composition is what makes the pattern powerful -- a restriction inside a restriction.

**Step 8: Convert `SpicyPizza` to a defined class**

1. Make sure `SpicyPizza` is still selected
2. Select `Edit > Convert to defined class` (or from right-click menu)




Observe what happens in the Description view. The `SubClassOf` axiom for the existential restriction is moved to `EquivalentTo`:



```
Class: SpicyPizza
    EquivalentTo: Pizza and (hasTopping some (hasSpiciness value Hot))
    SubClassOf: Pizza
```

**Note**

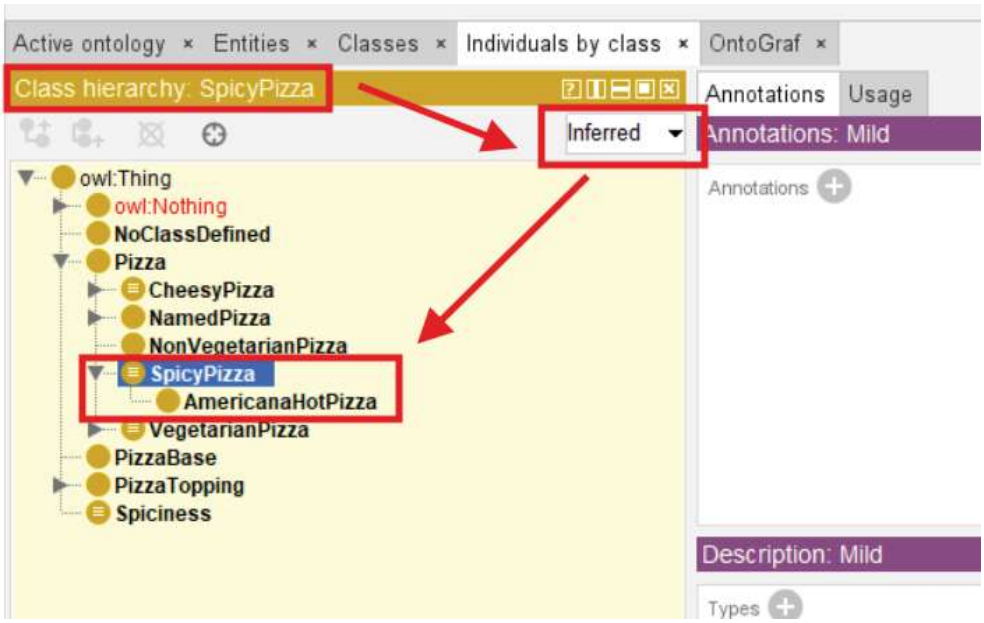
The `SpicyPizza` class is now shown with three horizontal stripes in the class icon -- the visual indicator that it is a defined class.

**Step 9: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)

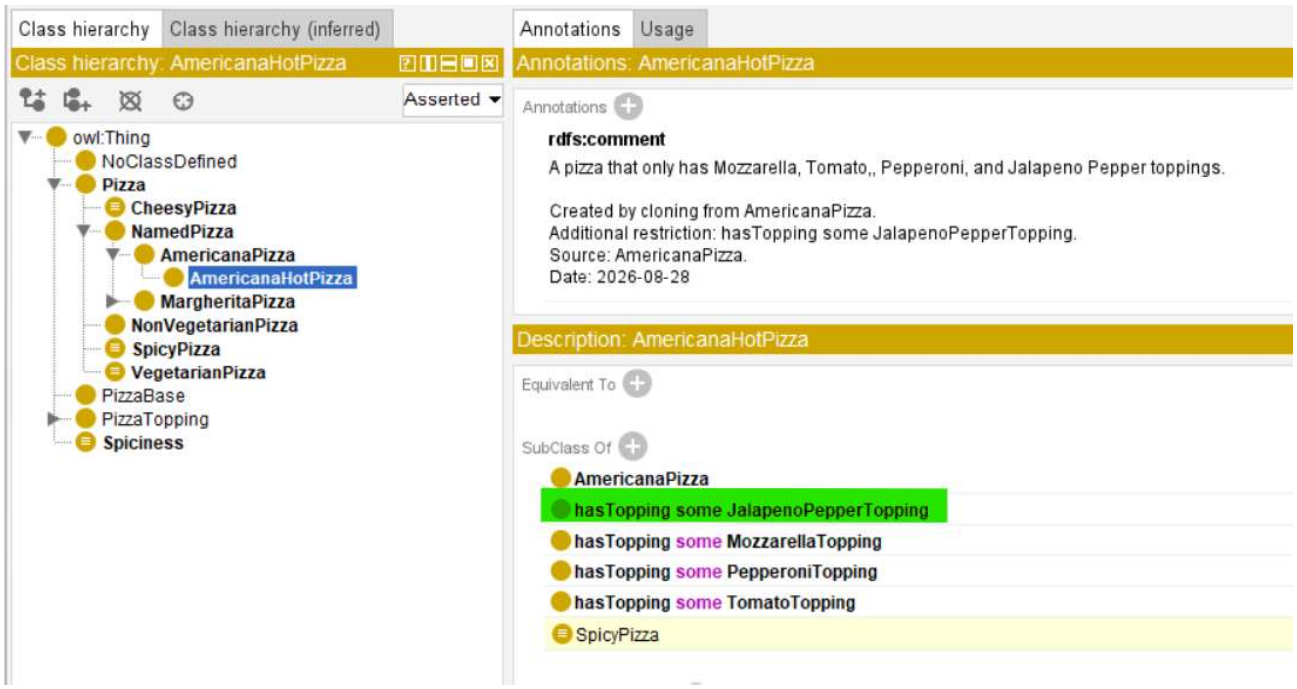
**Step 10: Observe the result**

Look at the **Class hierarchy (inferred)** tab:

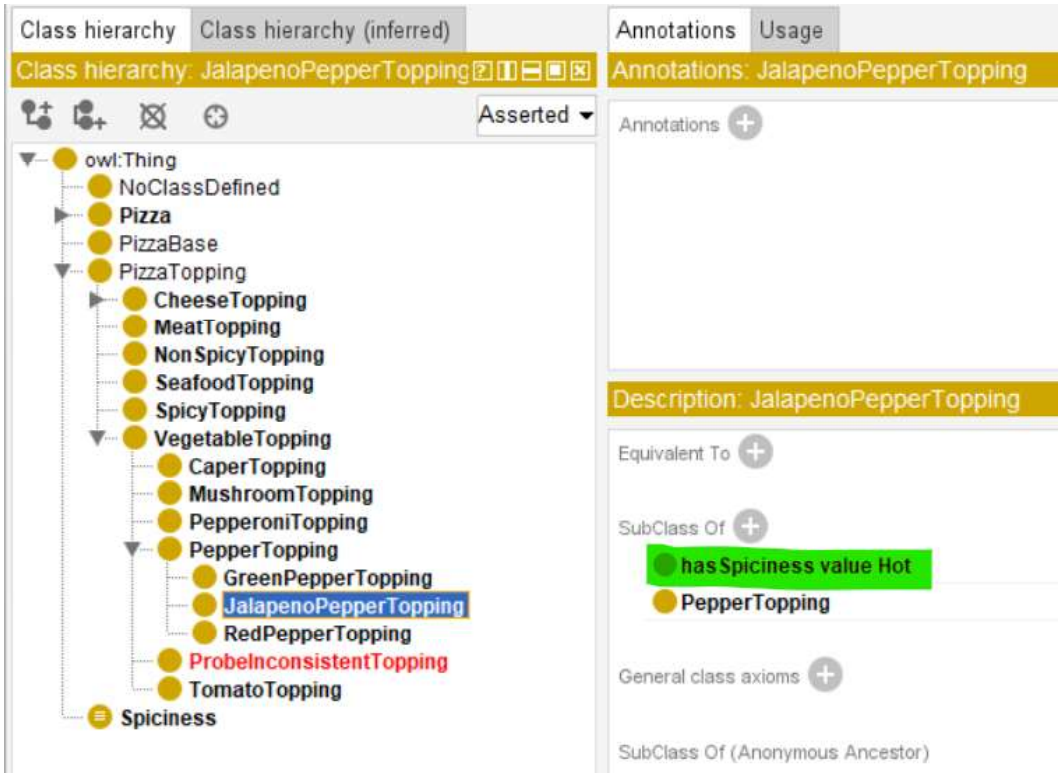



The reasoner now classifies `AmericanaHotPizza` as a **subclass of** `SpicyPizza` . This is because:

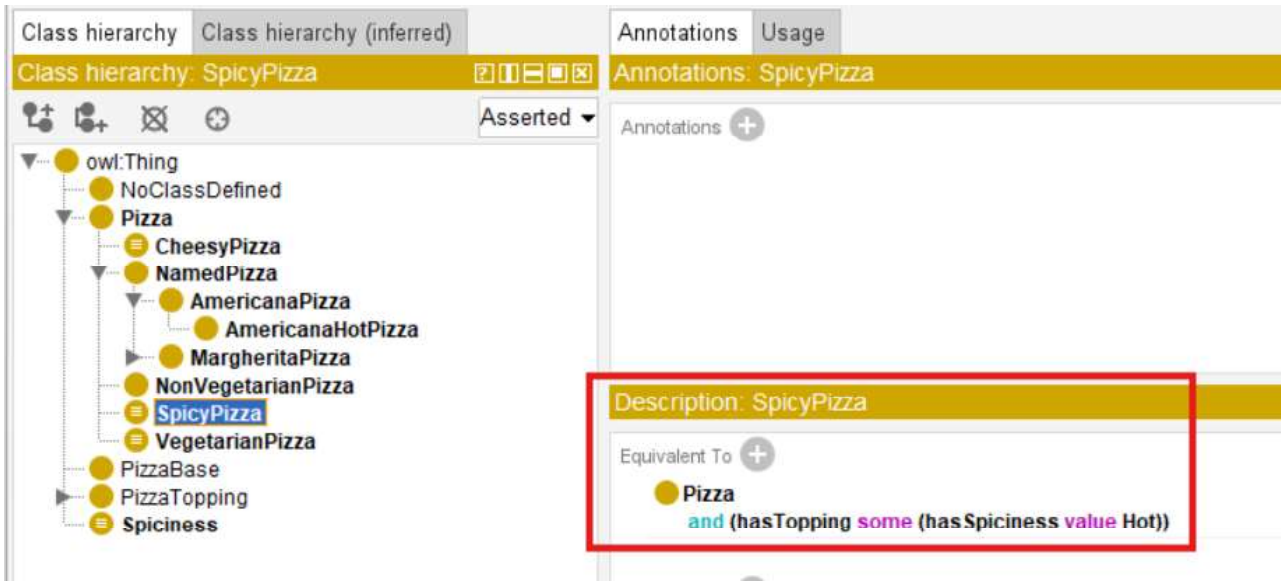
1. `AmericanaHotPizza` has a topping `JalapenoPepperTopping` (inherited from `AmericanaPizza` )



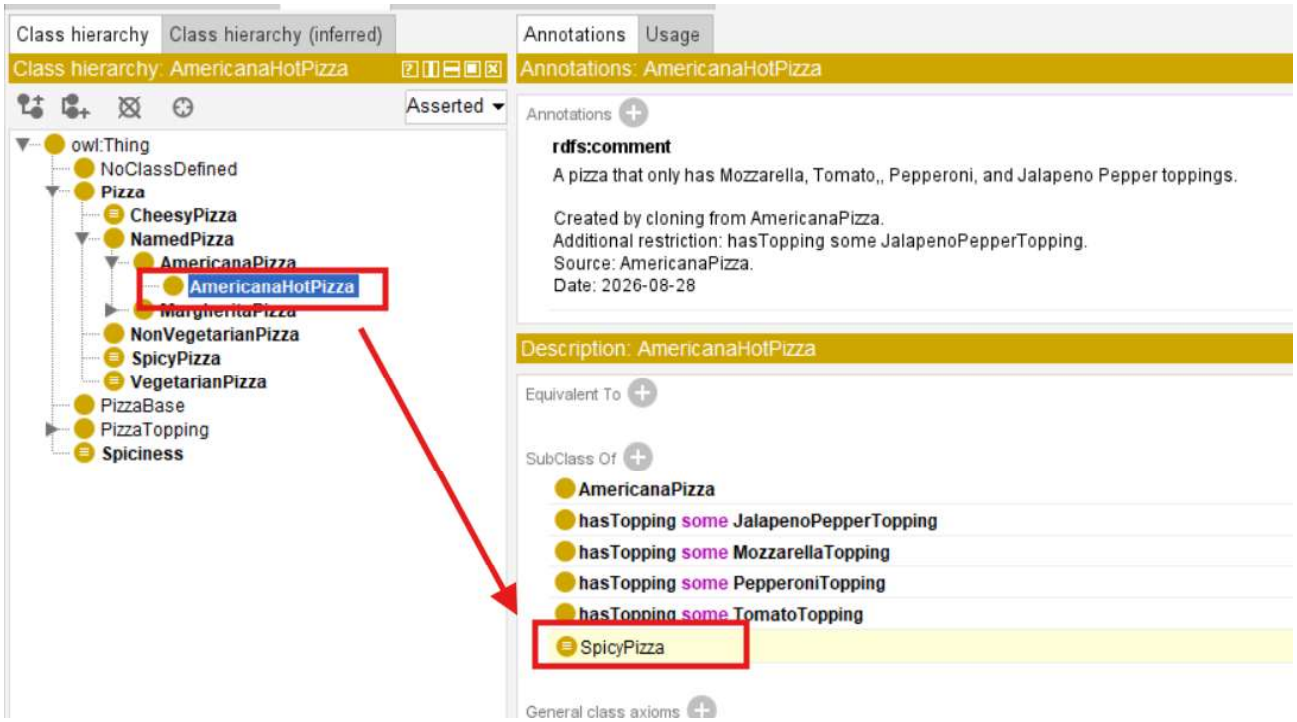

2. `JalapenoPepperTopping` has `hasSpiciness value Hot`.




3. `SpicyPizza` is defined as a `Pizza` that has at least one topping whose `hasSpiciness` value is `Hot`




4. Therefore, `AmericanaHotPizza` satisfies the definition of `SpicyPizza`




The reasoner has done something the human ontology engineer did not explicitly state. We never asserted that `AmericanaHotPizza` is a `SpicyPizza`. We asserted that it has `JalapenoPepperTopping`, and that `JalapenoPepperTopping` has spiciness `Hot`. The reasoner combined these facts with the definition of `SpicyPizza` and derived the classification automatically.

**Key Insight:**

> "The reasoner did not change. The definition of `SpicyPizza` did not change. Only the chain of facts changed -- from '`JalapenoPepperTopping` is a topping of `AmericanaHotPizza`' and '`JalapenoPepperTopping` has spiciness `Hot`' to '`AmericanaHotPizza` is a `SpicyPizza`'. Once the chain is complete, the reasoner can make the inference."

## 25.5.3 The Result: Value-Based Execution Logic

The `hasValue` restriction has enabled **value-based execution logic**.

For execution, this is essential. Many execution rules depend on **specific values**, not general classes of values.

For example:

- "If the temperature reading is `Critical`, then trigger an alert."
- "If the order status is `Pending`, then send a reminder."
- "If the customer tier is `Gold`, then apply a discount."

Each of these rules fires on a *specific value* from a closed value domain. The `hasValue` restriction is the OWL construct that expresses this condition.

Contrast this with the alternative. In **Exercise 22**, we used to universal restriction ( `hasTopping only ...` ) to describe *what is allowed*. That restriction expressed a boundary, not a specific trigger. In **Exercise 23**, we used closure axioms to *close the world* for those boundaries. Now, in **Exercise 25**, we use `hasValue` to *bind a trigger* to a specific value.

The four constructs work together:

| Construct | Role | Exercise |
|---|---|---|
| Universal restriction ( `only` ) | Defines what is allowed | Ex 22 |
| Closure axiom | Closes the world for a class or individual | Ex 23 |
| Enumerated class | Defines a closed value domain | Ex 24 |
| **`hasValue` restriction** | **Binds a property to a specific value** | **Ex 25** |

Each construct builds on the previous one. The universal restriction establishes a boundary. The closure axiom makes the boundary usable for classification. The enumerated class narrows the value domain to a finite set. And the `hasValue` restriction binds a trigger to one specific value within that domain.

**Key Insight:**

> " `hasValue` is the most direct connection between ontology and execution logic. It is the semantic equivalent of a state-machine transition -- `when the value is $X$, do $Y$.'"

## 25.5.4 Why `hasValue` Restrictions Matter for Execution

`hasValue` restrictions are essential for execution because:

1. **They bind properties to specific values.** Execution rules often depend on specific values, not general categories. The `hasValue` restriction is how OWL expresses this specificity.
2. **They enable precise triggers.** A trigger can be defined as "fire if the property has this specific value." Without `hasValue`, a trigger on `Spiciness` could fire on any spiciness -- `Hot`, `Medium`, or `Mild`. With `hasValue value Hot`, the trigger fires only on `Hot`.
3. **They compose with other constructs.** `hasValue` can be nested inside existential or universal restrictions, enabling complex conditions. The `SpicyPizza` definition -- `hasTopping some (hasSpiciness value Hot)` -- is a restriction inside a restriction. This composition is what makes the pattern expressive enough for real execution logic.
4. **They connect closed value domains to specific triggers.** In **Exercise 24**, we closed the value domain ("world") for `Spiciness`. In **Exercise 25**, we use that closed domain to bind specific triggers. The two constructs work as a team.

The table below contrasts the behavior of a property with and without `hasValue`:

| Aspect | Without `hasValue` | With `hasValue` |
|---|---|---|
| **Trigger condition** | Fires on any value of the property | Fires on one specific value |
| **Reasoner behavior** | Cannot distinguish between values | Can distinguish specific values |
| **Semantic strength** | Description of a class of conditions | Specification of a specific condition |
| **Execution readiness** | Not executable for value-specific triggers | Executable for value-specific triggers |

**Key Insight:**

> "Every semantic trigger is ultimately a `hasValue` restriction somewhere. The trigger fires when the property has the specific value that the trigger was designed to detect."

To see how `hasValue` becomes an execution rule, consider a simple trigger:

```
WHEN hasSpiciness value Hot
THEN trigger AlertSpicy
```

In the ontology, this trigger is expressed as a defined class or a query: "find all individuals whose `hasSpiciness` value is `Hot`." The reasoner computes the answer; the execution system reads it and fires the alert. The `hasValue` restriction is the semantic side; the alert is the operational side. Together, they form a **$\Theta$-Trigger**.

This is why Chapter 24 called Stage 7 the layer of $\Theta$ (Trigger) and $\Phi$ (Execution). The ontology supplies the trigger condition; the runtime supplies the action.

## 25.5.5 Engineering Perspective: Exercise 25 as a Case Study

From the perspective of Protégé, **Exercise 25** involves creating a property, adding a restriction to a topping class, and defining a pizza class with a nested restriction. It appears to be moderate modeling activity.

From the perspective of ontology engineering, however, something important has occurred. The ontology now contains **value-based conditions** -- the semantic equivalent of conditional logic. The reasoner can fire on a specific value, not just a class of values.

The engineering contribution of **Exercise 25** can be summarized as follows:

| Engineering Perspective | Contribution from Exercise 25 |
|---|---|
| **SKDL Stage** | Stage 7 -- Executable Knowledge |
| **Primary Objective** | Introduce the `hasValue` restriction as a value-binding primitive |
| **Modeling Activity** | 1. Create `hasSpiciness` property;<br>2. Add `hasSpiciness value Hot` to `JalapenoPepperTopping`;<br>3. Define `SpicyPizza` using `hasTopping some (hasSpiciness value Hot)` |
| **Engineering Outcome** | Enable value-based execution logic; the reasoner now classifies `AmericanaHotPizza` as a subclass of `SpicyPizza` by composing the value binding with the topping relationship |
| **Preparation for Later Sections** | Establish the value-binding foundation for the data-integrity logic in **Exercise 26** and the stage-level synthesis in **Chapter 26** |

**Key Insight:**

> "**Exercise 25** is the moment the ontology gains the ability to express value-based conditions. This is the semantic foundation of triggers."

**Following the Tutorial**

**Exercise 25** in the original `Pizza` Tutorial creates the `hasSpiciness` property with domain `PizzaTopping` and range `Spiciness`, adds the `hasSpiciness value Hot` restriction to `JalapenoPepperTopping`, and defines `SpicyPizza` as `hasTopping some (hasSpiciness value Hot)`, then converts it to a defined class and synchronizes the reasoner.

In this book, we have followed the tutorial's construction -- same property, same restriction, same defined class. The only difference is **presentational**: we have added the observation that this constitutes a *value-binding execution primitive*, and we have explicitly noted that the `value` keyword differs from `some` and `only` in binding a specific individual rather than a class of individuals.

The tutorial introduces the construct; the book frames it in terms of executability.

## 25.5.6 The Design Pattern Perspective

**Exercise 25** completes the value layer of executability. Two design patterns are now in place:

- The **Enumerated Class** pattern -- closes a value domain by listing its complete extension.
- The **`hasValue` Restriction** pattern -- binds a property to one specific value from a closed domain.

These two patterns are designed to be used **together**, in a fixed order:

> **First close the domain, then bind the value.**
>
> For a property $p$ whose permitted values are the finite set $V = \{v_1, v_2, \ldots, v_n\}$:
>
> 1. Define the value class as $V$ (enumerated class).
> 2. Bind specific values with $p$ value $v_i$ ( `hasValue` restriction).

The pattern is reusable because the problem it solves is universal. Any execution rule that fires on a specific value depends on both the *domain* being closed (so the rule knows all possible cases) and the *value* being bound (so the rule knows which case to fire on). Neither pattern alone is sufficient; the domain closure tells the rule what it must handle, and the value binding tells it what to fire on.

The Stage 7 pattern family now has four members:

| Pattern | Purpose | Introduced |
|---|---|---|
| **Probe Class** | Detects membership through a discriminating marker | Chapter 22 |
| **Closure Axiom** | Closes the world around a boundary | Chapter 24 |
| **Enumerated Class** | Closes the value domain of a property | This chapter |
| **`hasValue` Restriction** | Binds a property to a specific value | This chapter |

One more will join them in **Chapter 26** -- the **Cardinality Restriction** -- which closes the *number* of values a property may take, completing the boundary-value-integrity triad of Stage 7.

# 25.6 Chapter Summary

This chapter introduced the second part of **Stage 7 -- Executable Knowledge**: the **value layer**. Where Chapter 24 built the boundary layer (universal restrictions and closure axioms), Chapter 25 built the layer that binds specific values to properties and triggers.

The chapter established the value layer from two execution primitives:

**Exercise 24 -- Enumerated Classes.** We defined `Spiciness` as `EquivalentTo {Hot, Medium, Mild}`. This closed the *value domain* of the spiciness property: the reasoner now knows that `Hot`, `Medium`, and `Mild` are the *only* possible spiciness values. No other individuals can be spiciness values, because the enumeration is exhaustive.

**Exercise 25 -- `hasValue` Restrictions.** We created the `hasSpiciness` object property, bound `JalapenoPepperTopping` to `hasSpiciness value Hot`, and defined `SpicyPizza` as `hasTopping some (hasSpiciness value Hot)`. The reasoner then classified `AmericanaHotPizza` as a `SpicyPizza` -- automatically, without any explicit assertion that it is one. The `hasValue` restriction is the semantic foundation of the trigger: it binds a property to a specific value, and a specific value is what a trigger fires on.

The two primitives work as a pair. Enumeration closes the domain; `hasValue` binds the value. Neither alone is sufficient. Together, they enable **value-based execution logic** -- the ability to fire on *specific values* rather than *categories*.

We also extended the design-pattern thread. §25.3 named the OWL design-pattern concept and placed Stage 7's primitives in a family; §25.4.5, §25.5.5, and §25.5.6 recorded the two new patterns, enumerated class and `hasValue` restriction, alongside the closure axiom from Chapter 24 and the Probe Class from Chapter 22. The Stage 7 pattern family now has four members, and one more -- the cardinality restriction -- will join them in Chapter 26.

By the end of this chapter, you have:

- Understood why value specification is the second layer of executable knowledge
- Created a closed value domain with an enumerated class
- Bound a property to a specific individual with a `hasValue` restriction
- Defined a class using a nested restriction ( `some` containing `value` )
- Observed the reasoner deriving a classification through composition
- Recognized the two new design patterns of the value layer

These capabilities prepare you for Chapter 26, which completes the executable ontology with cardinality restrictions for data integrity -- and consolidates the five primitives into a single anatomy of Stage 7.

**Key Insight:**

> "Enumeration closes the domain. `hasValue` binds the value. Together, they turn a class of possible values into a specific trigger."

# 25.7 Looking Ahead -- A Bridge to Chapter 26

The value layer of the executable ontology is now in place. We have learned:

- **How to close a value domain** using an enumerated class ( `Spiciness = {Hot, Medium, Mild}` )
- **How to bind a property to a specific value** using a `hasValue` restriction
- **How to compose restrictions** to build value-based conditions (nested `some` / `value` )
- **How the reasoner derives classifications** through value composition

But the value layer, like the boundary layer before it, is only half the picture. A trigger that fires on a specific value is not yet a *complete* specification. To execute safely, the ontology must also guarantee that the required values are *present* -- that a property has at least one value, or at most one, or exactly one.

This is the subject of **Chapter 26**.

Chapter 26 introduces the final execution primitive:

> **Cardinality Restrictions (Exercise 26).**
> Cardinality restrictions constrain the *number* of values a property may take: `min`, `max`, and `exactly`. They complete the executable ontology by enforcing data completeness and integrity. Where the boundary layer closed *what* is permitted and the value layer bound *which value* fires, the cardinality layer guarantees *that the data required to fire is present*.

Chapter 26 also does more than introduce the last primitives. It consolidates the five primitives of Stage 7 into a single **anatomy of an executable ontology**, and examines executability from four complementary perspectives:

- **Mathematical** -- the logical foundations of closure, enumeration, binding, and cardinality
- **Historical** -- how the concept of executability evolved from expert systems to modern rule engines
- **Industrial** -- how executable ontologies are applied in manufacturing, compliance, and safety
- **EKA** -- how the five primitives map to the $\Theta$ (Trigger) and $\Phi$ (Execution) layers of the Executable Knowledge Architecture, and how that architecture bridges to the multi-layer model developed in **Chapter 27**.

The journey from boundary to value to integrity is the arc of Stage 7. Chapter 25 has established the second leg. Chapter 26 completes the journey -- and opens the door to the architectural synthesis that follows.

**Key Insight:**

> "A boundary tells the system what is permitted. A value tells the system what to act on. Cardinality tells the system when it has enough to act. Only all three together make execution safe."

Last updated at 2026-10-03

# Chapter 26 - Stage 7 of SKDL -- Executable Knowledge (Part 3): Data Integrity & Comprehensive Architecture

**Completing the Executable Ontology**

Chapter 24 built the boundary layer of an executable ontology. Chapter 25 built the value layer. This chapter builds the last layer -- **data integrity** -- and then steps back to view the whole of Stage 7 as a single architecture.

By the end of this chapter, we will have consolidated **five execution primitives** into a single anatomy, examined executability from four complementary perspectives (mathematical, historical, industrial, and EKA), and closed the design-pattern family that began in Chapter 22.

This is the final chapter of Stage 7. It completes the SKDL lifecycle and prepares the synthesis that follows in Chapter 27


- 26.1 Chapter Introduction
- 26.2 Learning Objectives
- 26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness
  - 26.3.1 The Engineering Objectives
  - 26.3.2 Implementation: Creating the Cardinality Restriction
  - 26.3.3 Why the Reasoner Did Not Classify the Three Pizzas
  - 26.3.4 How to Make the Cardinality Fire
    - Attempt 1 -- Add a closure axiom (the pattern from Chapter 24)
    - Attempt 2 -- Correct the first Stage 1 modeling error, then re-test
      - Attempt 2.1 -- Diagnose the remaining two pizzas
      - Attempt 2.2 -- Apply the remaining corrections and re-test
      - Attempt 2.3 -- Investigate the remaining two classes
      - Attempt 2.4 -- Investigate the apparent non-classification
      - The lesson: verify inference in the right view
    - Approach 2 -- Assert the cardinality directly (an alternative)
  - 26.3.5 Understanding Protégé UI Behavior: Why the Description Panel Can Mislead
    - 26.3.5.1 The Mechanics of Conversion: Why "SubClass Of" Becomes Empty
    - 26.3.5.2 The Inferred-Ancestor Behavior: Why a Subclass May Not Show Its Inferred Parent
    - 26.3.5.3 The Underlying Principle: Local View vs. Global View
    - 26.3.5.4 A Verification Recipe
    - 26.3.5.5 Applying the Lesson
- 26.4 The Anatomy of an Executable Ontology
  - 26.4.1 The Five Primitives
  - 26.4.2 The Three Layers
  - 26.4.3 The Anatomy in the EKA Framework
  - 26.4.4 A Minimal Worked Example
  - 26.4.5 What the Anatomy Enables
  - 26.4.6 From Anatomy to Perspective
- 26.5 Mathematical Perspective
  - 26.5.2 Closure as Set Restriction
  - 26.5.3 Enumeration as Finite Extension
  - 26.5.4 `hasValue` as Singleton Binding
  - 26.5.5 Cardinality as Bounded Extension
  - 26.5.6 The Mathematical Arc of Stage 7
  - 26.5.7 Distinctness -- The Underlying Condition
  - 26.5.8 What the Mathematics Teaches
  - 26.5.9 The Mathematical Arc Across Stage 7
- 26.6 Historical Perspective
- 26.7 Industrial Perspective
- 26.8 EKA Perspective -- Stage 7 Establishes $Theta$ + $Phi$
- 26.9 Engineering Guidelines for Executable Knowledge
- 26.10 Key Concepts
- 26.11 Chapter Summary
- 26.12 Looking Ahead -- Toward SKDL Synthesis


## 26.1 Chapter Introduction

We have now built two layers of an executable ontology.

**The boundary layer** (Chapter 24). Universal restrictions told the reasoner what was *allowed*; closure axioms closed the world around those boundaries. The ontology could classify `MargheritaPizza` and `SohoPizza` as `VegetarianPizza`, deterministically and without ambiguity.

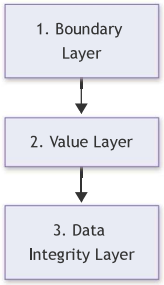
**The value layer** (Chapter 25). Enumerated classes closed the *set of possible values* for a property ( `Spiciness = {Hot, Medium, Mild}` ). `hasValue` restrictions bound a property to a *specific value* from that set ( `JalapenoPepperTopping hasSpiciness value Hot` ). The ontology could fire a trigger on a specific value rather than a category.

But a boundary and a value together are still not enough to execute safely.

Consider what happens when the ontology classifies a pizza as `SpicyPizza` because it has at least one `Hot` topping. What if the pizza has *no* topping? What if it has *two* `hasBase` assertions? What if the property that the rule depends on is simply *missing*?

The reasoner under the Open World Assumption will not complain. It will simply *not* fire the rule, and the execution system will silently do nothing. That is the quiet failure mode of an ontology that has not been engineered for **integrity**.

The third layer of executability solves this. It is the **data integrity layer** -- the layer that guarantees the *presence and cardinality* of the data that triggers depend on.




This chapter introduces the last execution primitive -- the **cardinality restriction** (**Exercise 26**) -- and then consolidates all five primitives into a single architecture.

**key Insight:**

> "A boundary tells the system what is permitted. A value tells the system what to act on. Cardinality tells the system *when it has enough to act*. Only all three together make execution safe."

## 26.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

**Knowledge:**

- Explain why data integrity is the third layer of an executable ontology
- Distinguish the three cardinality restrictions: `min` , `max` , and `exactly`
- Understand how cardinality differs from the closure and value layers
- Explain how the five execution primitives of Stage 7 form a single anatomy
- Trace the two historical lineages of executability: AI reasoning and industrial process
- Apply executable knowledge to industrial scenarios (manufacturing, compliance, process)

**Practical Skills:**

- Create cardinality restrictions ( `min` , `max` , `exactly` ) in Protégé
- Use `hasTopping min 3 PizzaTopping` and other cardinality patterns
- Convert a class using cardinality to a defined class
- Test cardinality-based classification by synchronizing the reasoner
- Verify that reasoner-inferred classifications match the intended integrity constraints

**Mathematical Perspective:**

- Understand cardinality as the bounded-extension restriction on a property
- Explain the relationship between cardinality, functional properties, and `hasValue`
- Understand how cardinality restrictions close the "number of values" dimension

**Engineering Perspective:**

- Understand data integrity as the *last* layer, not the first
- May the five Stage 7 primitives to the $\Theta/\Phi$ architecture
- Apply executable ontologies in industrial contexts

## 26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness

Having established closure in **Exercise 23** and value binding in **Exercise 24 and 25**, we not turn to the final primitive of Stage 7: the **cardinality restriction.**.

This exercise follows the tutorial's **Exercise 26**, which introduces `InterestingPizza` -- a pizza with three or more toppings.

## 26.3.1 The Engineering Objectives

**Exercise 26** introduces the **Cardinality Restriction** -- a restriction on the *number* of values a property may have.

OWL provides three forms:

| Form | Meaning | Syntax |
|---|---|---|
| **Minimum** | At least *n* values | `hasTopping min 3 PizzaTopping` |
| **Maximum** | At most *n* values | `hasTopping max 5 PizzaTopping` |
| **Exactly** | Exactly *n* values | `hasTopping exactly 2 PizzaTopping` |

A cardinality restriction for an "interesting pizza", for example, would be:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

This means: "An `InterestingPizza` is a `Pizza` with at least three toppings."

The engineering objective of **Exercise 26** is to demonstrate how cardinality restrictions **guarantee data completeness** -- the third and final requirement for **execution**.

Why does execution need cardinality?

Consider an execution rule that fires only when a pizza has enough toppings to qualify as "interesting". Without a cardinality restriction, the ontology cannot determine *how many* toppings the pizza has -- it can only say that *some* exist. A rule depends on count cannot fire reliably.

Cardinality closes the *number* dimension. Where the closure axiom closes the *set* of permitted toppings and `hasValue` closed the *specific value*, cardinality closes *how many* values the property may take.

**Key Insight:**

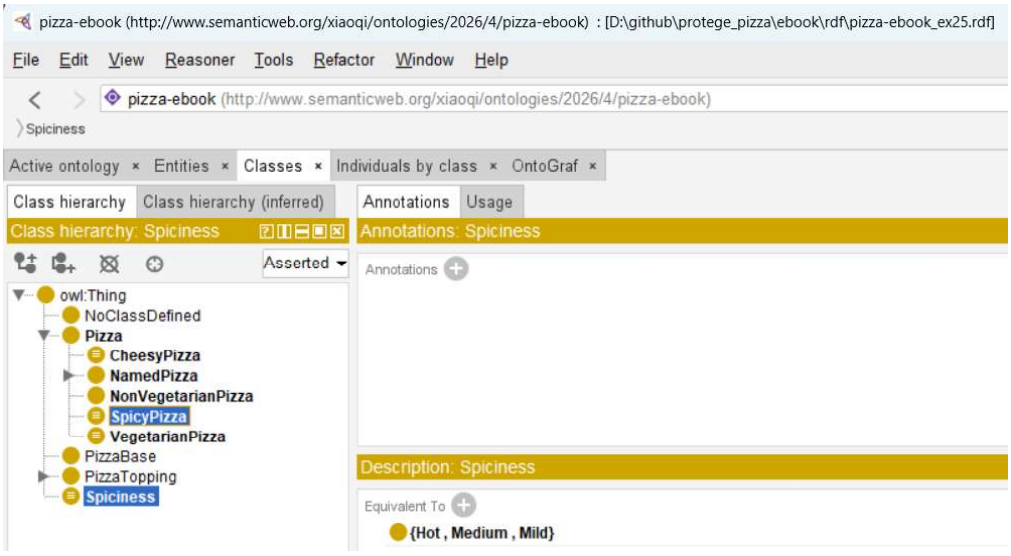
> "Cardinality is the last mile of executability. A rule that fires on 'at least three toppings' requires the ontology to know that count is enforced."

## 26.3.2 Implementation: Creating the Cardinality Restriction

Let's walk through the implementation step by step.

**Step 1: Open the `Pizza` ontology**

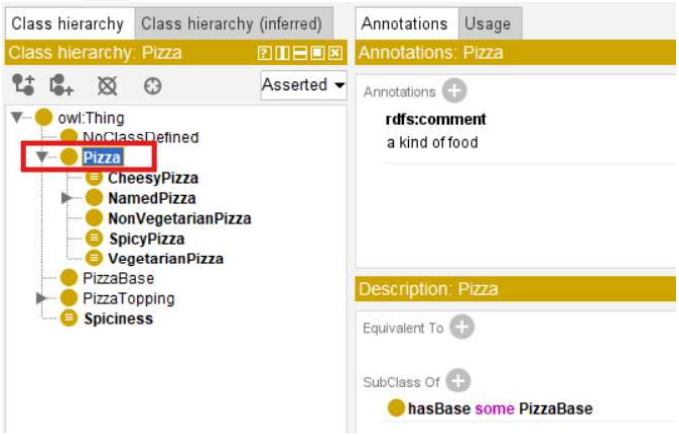
Open `pizza-ebook_ex25.rdf` in Protégé -- the file from the previous exercise, which now contains the enumerated class `Spiciness`, the `hasSpiciness` property, and the defined class `SpicyPizza`.



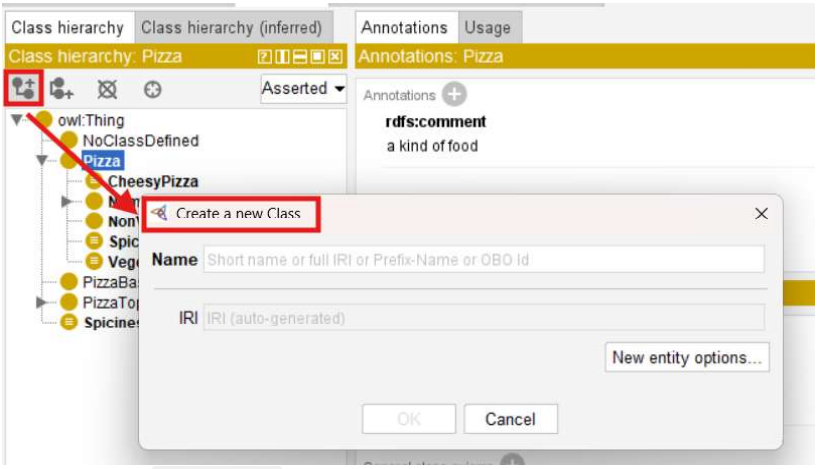

> **Note**
>
> The finished state of **Exercise 26** will be saved as `pizza-ebook_ex26.rdf`.

**Step 2: Create the `InterestingPizza` class**

1. Make sure `Pizza` is selected in the class hierarchy

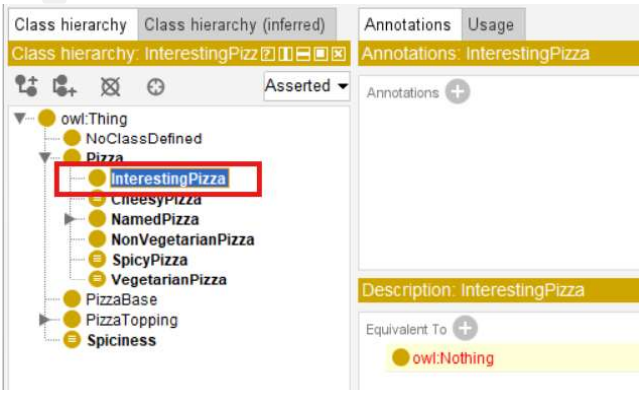



2. Click the **Add subclass** icon



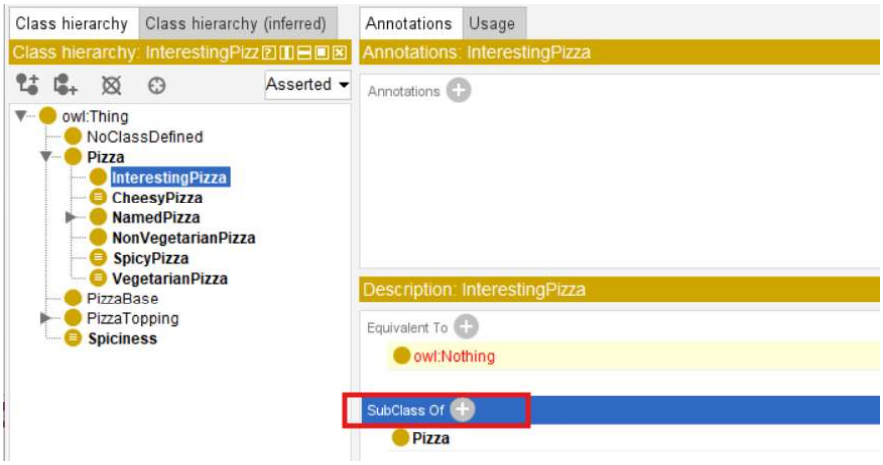

3. Name the new class `InterestingPizza`
4.

5. Click `OK`

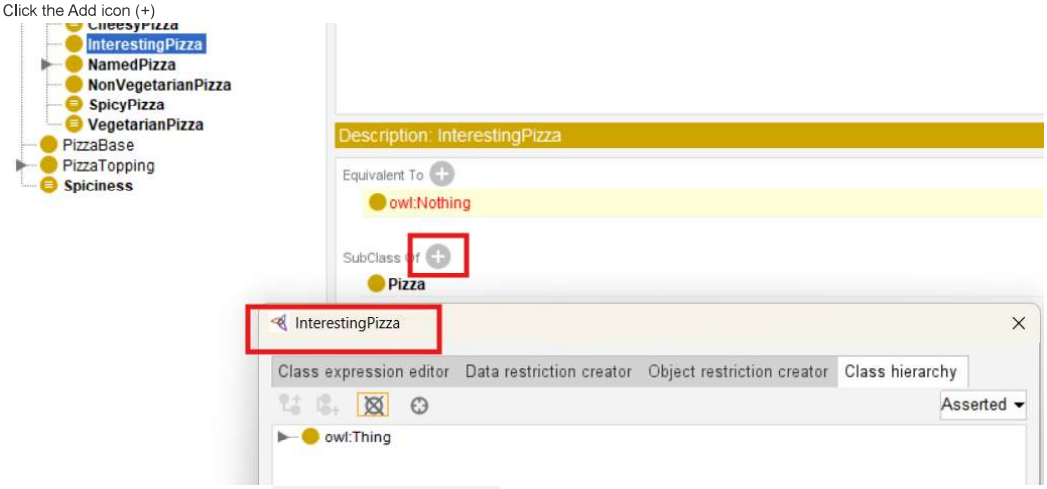



**Step 3: Add the cardinality restriction**

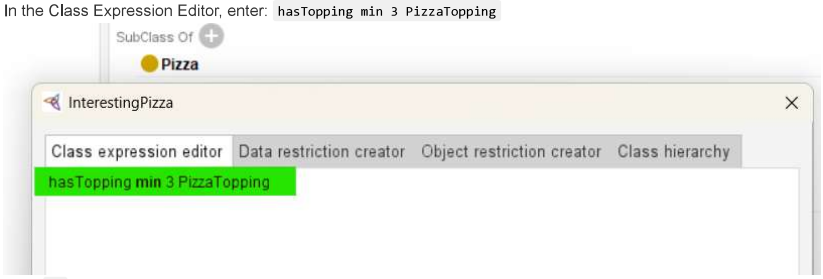
1. Select `InterestingPizza` (should already in focus after previous step)
2. In the Description view, locate the **SubClass Of** section



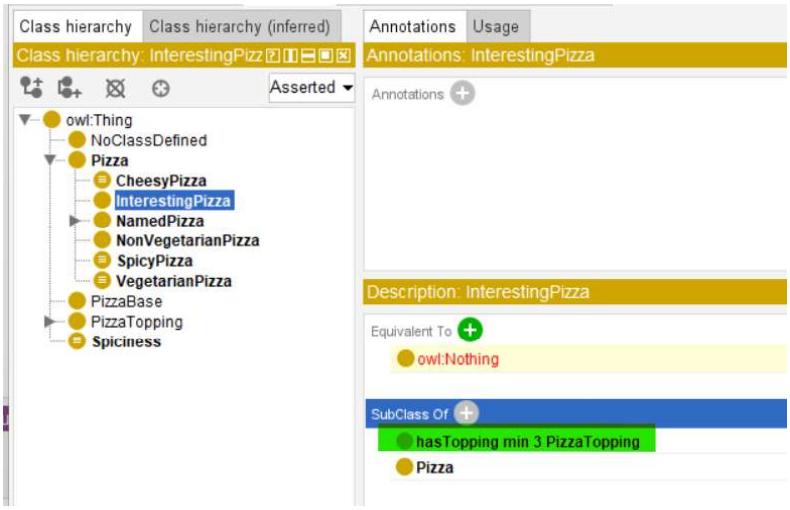

3. Click the Add icon (+)




4. In the Class Expression Editor, enter: `hasTopping min 3 PizzaTopping`




5. Click `OK`



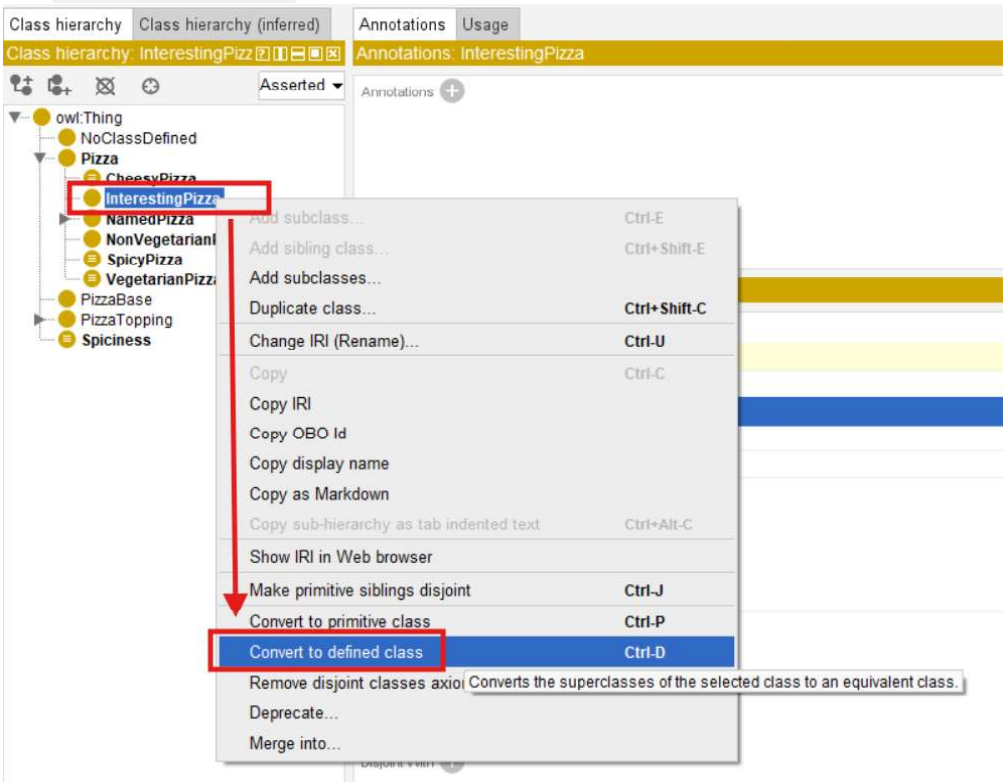

After this step, `InterestingPizza` has:

```
Class: InterestingPizza
    SubClassOf: Pizza
    SubClassOf: hasTopping min 3 PizzaTopping
```

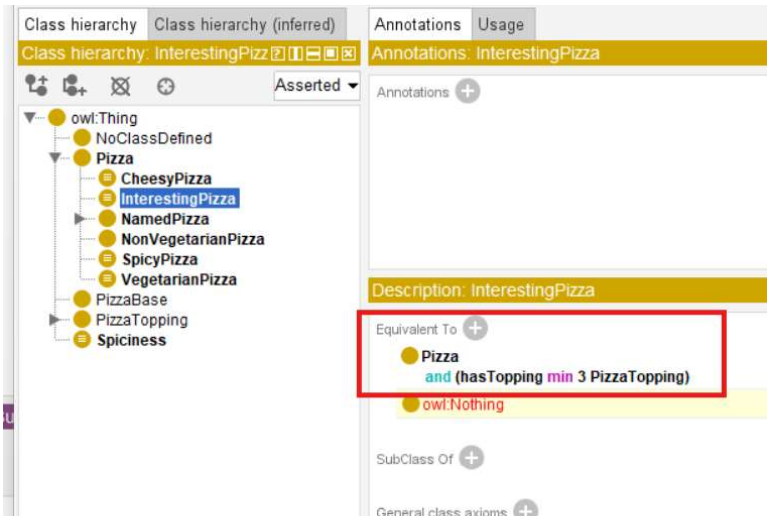
This says: "An `InterestingPizza` is a pizza and must have at least three toppings."

**Step 4: Convert `InterestingPizza` to a defined class**

1. Make sure `InterestingPizza` is still selected
2. Select `Edit > Convert to defined class`




Observe what happens in the Description view. The `SubClassOf` axiom for the cardinality restriction is moved to `EquivalentTo`:

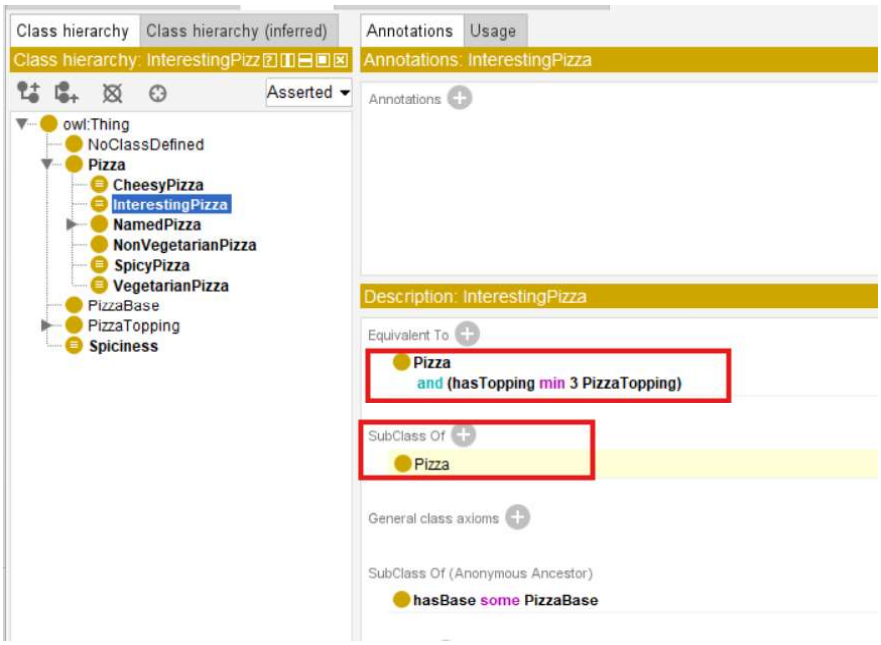



```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
    EquivalentTo: owl:Nothing
    SubClassOf: <Empty>
```

> **Note**
>
> You may notice the **SubClass Of** section is empty, to learn why, check later §26.3.5 for some clarification.

**Step 5: Synchronize the reasoner**

Select `Reasoner > Synchronize reasoner` (Ctrl+R)



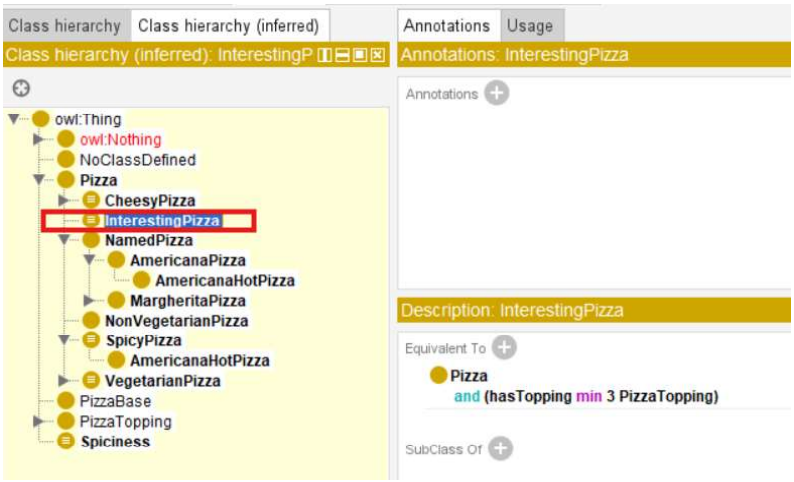

After reasoner synchronized, the **SubClass Of** section contains `Pizza` again.

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
    SubClassOf: Pizza
```

**Step 6: Observe the result**

We would assume that the reasoner classifies pizzas with three or more toppings as subclasses of `InterestingPizza`. In the tutorial's data, these include `AmericanaHotPizza`, `AmericanaPizza`, and `SohoPizza`.

However, look at the **Class hierarchy (inferred)** tab:




You will see something surprising: **Non of `AmericanaHotPizza`, `AmericanaPizza`, or `SohoPizza` are classified as subclasses of `InterestingPizza`.**

This is not as tutorial mentioned and unexpected. All three pizzas have three ot more toppings assertions:

| Pizza | Toppings |
|---|---|
| `AmericanaPizza` | Mozzarella, Pepperoni, Tomato (3) |
| `AmericanaHotPizza` | Mozzarella, Pepperoni, Tomato, Jalapeno (4) |
| `SohoPizza` | Mozzarella, Tomato, Olive, Parmesan (4) |

And yet the reasoner does not classify any of them under `InterestingPizza`.

No worry! This is not a bug in Protégé. It is a subtle consequence of how cardinality restrictions interact with existential restrictions under the Open World Assumption. It deserves its own explanation -- see **§26.3.3**.

## 26.3.3 Why the Reasoner Did Not Classify the Three Pizzas

The behavior in above Step 6 is one of the most instructive moments in this chapter. It looks like a failure, but it is a faithful reflection of OWL's semantics -- and understanding it is essential to writing executable ontologies that work in production.

**The apparent contradiction.** Each of the three pizzas has three or four existential toppings restrictions, e.g. `AmericanaPizza`:

```
Class: AmericanaPizza
    SubClassOf: hasTopping some MozzarellaTopping
    SubClassOf: hasTopping some PepperoniTopping
    SubClassOf: hasTopping some TomatoTopping
```

A human reads this and concludes: "This pizza has three toppings." The `InterestingPizza` definition asks for `hasTopping min 3 PizzaTopping`. The count matches. So why doesn't the reasoner agree?

**The OWL semantics.** Existential restrictions do **not** commit to the number of distinct fillers. `hasTopping some X` means "there exists at least one value of `hasTopping` that is of type `X`." It does not say "there is exactly one" or "this is the only one of its kind." Under the Open World Assumption, the reasoner is forbidden from assuming the three existentials refer to three distinct individuals unless it can prove it from the axioms.

Even when the filler classers are declared disjoint -- as `MozzarellaTopping`, `PepperoniTopping`, and `TomatoTopping` are -- disjointness establishes that *if* three disjoint individuals exist, they are of different types. It does not establish that three individuals exist. The three existential restrictions could, in the reasoner's view, all be satisfied by the same single individual if that individual happened to be a member of all three classes. (In practice, disjointness rules that out here, but the reasoner cannot bridge from "there exist some X, some Y, some Z" to "there exist at least three individuals" without additional axioms.)

The `min 3 PizzaTopping` restriction, therefore, has nothing to count. The reasoner sees three *existential restrictions*, not three *toppings*, and it cannot satisfy the min-3 cardinality.

**Why this is important?** This is the single most common source of confusion when engineers first write cardinality restriction in OWL. It looks like the ontology should be able to count; semantically, it cannot, without more information. The tutorial's expectation that these pizzas would be inferred as `InterestingPizza` reflects human intuition about the data, not the reasoner's semantics.

This is not a flaw in OWL. It is a *design choice*: OWL refuses to commit to the number of values a property has unless the ontology explicitly closes the world. The consequence for execution is direct: **if a trigger depends on a count, the ontology must be engineered to that the count is provable.**

**Key Insight:**

> "The reasoner counts what the ontology proves, **not** what the engineer intends. Existential restrictions prove existence, not count. Only closure or explicit cardinality makes the count provable."

## 26.3.4 How to Make the Cardinality Fire

The fix is not to remove the cardinality restriction. The fix is to give the reasoner enough information to evaluate it. But as we are about to see, the "obvious" fix does not work -- and reason it does not work is one of the most important lessons in this chapter.

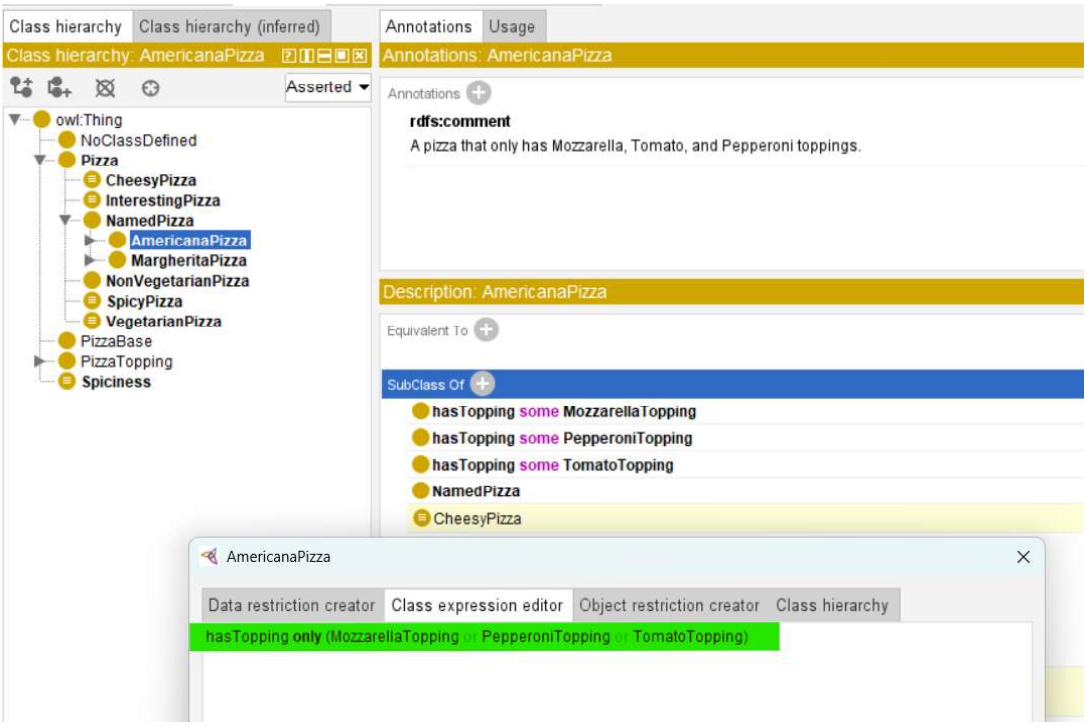
We will walk through the fix in stages. Each stage tries something, observes the result, and diagnoses why it did or did not work. This is what semantic engineering looks like in practice: **Not one big correct answer, but a disciplined sequence of hypotheses, tests, and corrections.**

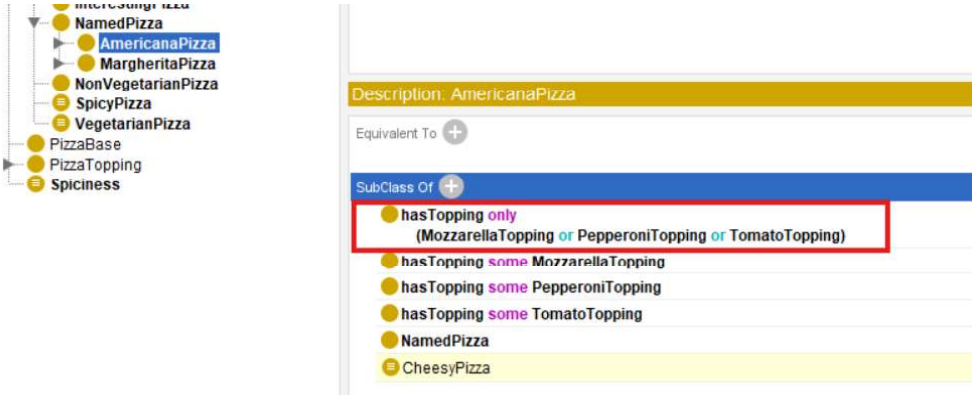
### Attempt 1 -- Add a closure axiom (the pattern from Chapter 24)

> **Keep** `pizza-ebook_ex26.rdf` **as one snapshot. From now on, the fix for Attempt 1 is saved in** `pizza-ebook_ex26-approach1.rdf`

The natural first move is to apply the closure pattern from Chapter 24. If the reasoner cannot prove the count because it does not know the toppings are exhaustively the listed ones, then close the world for each pizza class using a universal restriction:

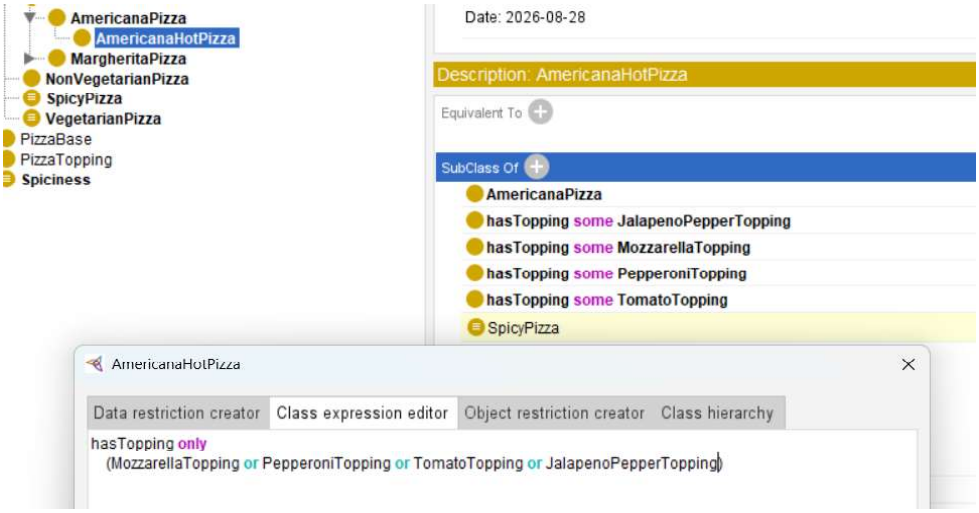
```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```



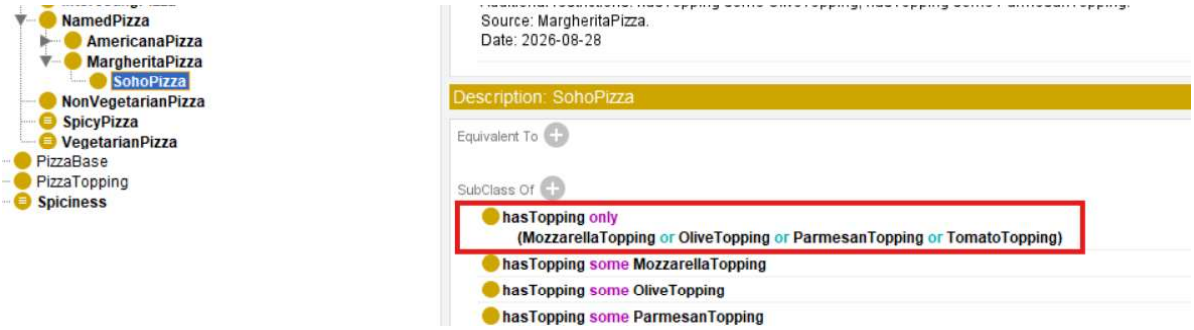



Apply the same closure to `AmericanaHotPizza` (Mozzarella, Pepperoni, Tomato, JalapenoPepper) and `SohoPizza` (Mozzarella, Tomato, Olive, Parmesan). This is exactly the pattern that worked in **Chapter 24** for `MargheritaPizza` and `SohoPizza`. It should work here too.

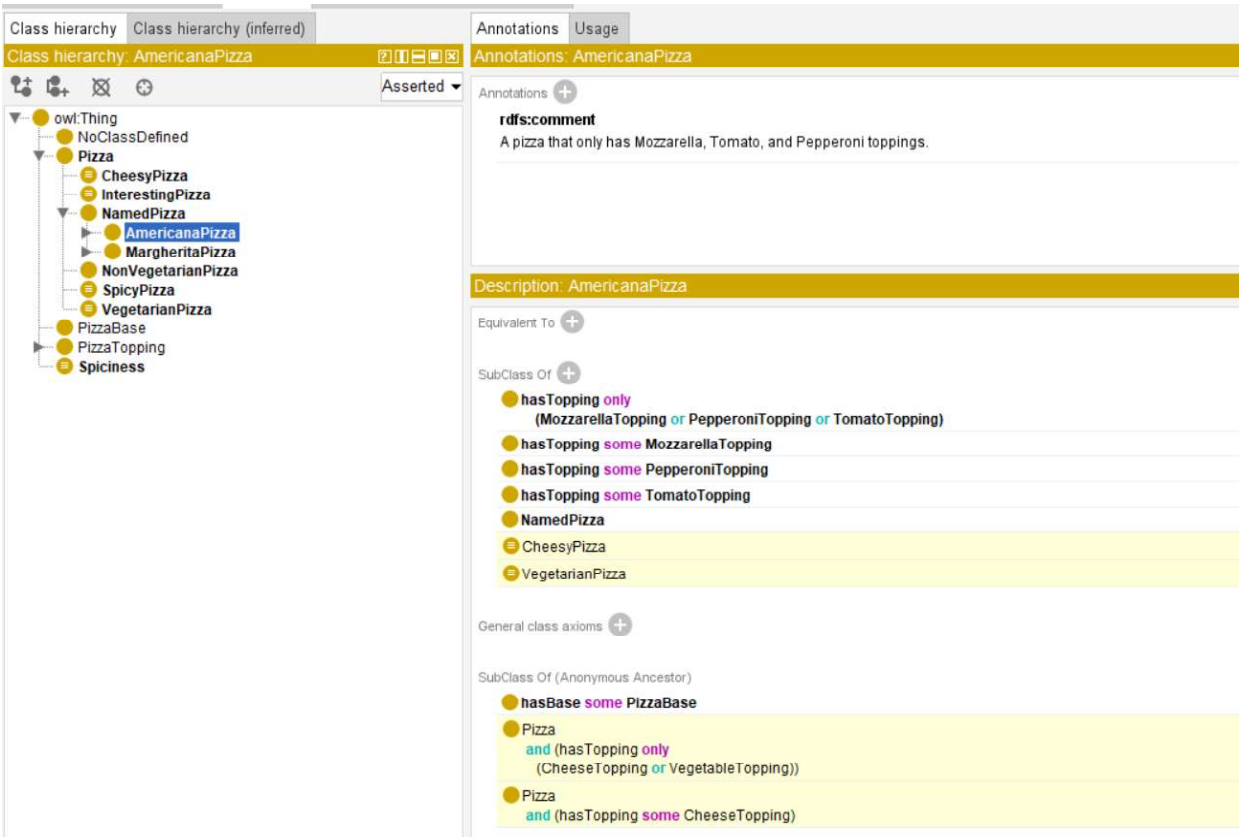
```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoPizza or JalapenoPepperTopping)
Class: SohoPizza
    SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping or OliveTopping or ParmesanTopping)
```




You may find `SohoPizza` has already had the needed universal restriction in the working ontology.




**Synchronize the reasoner and observe.** For `AmericanaPizza`, the result is **not** what we expect:



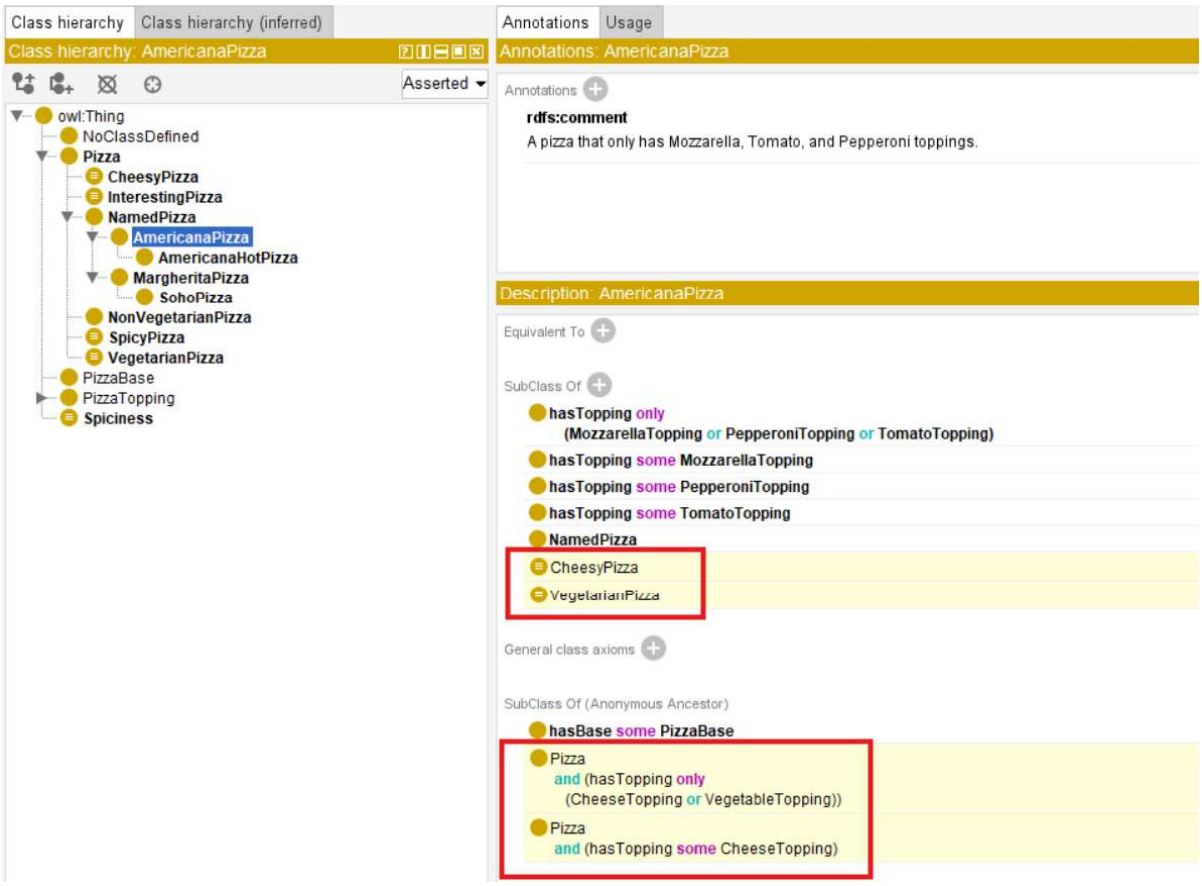

None of the three pizzas are classified as `InterestingPizza`. The closure axiom is in place -- we verified in the Protégé UI (the `hasTopping only (...)` restriction is visible in the `SubClass Of` section of `AmericanaPizza`) -- and yet the min-3 cardinality still does not fire.

**Attempt 1 has failed.** This is worth pausing on, because the failure is not a bug. It is a ***signal***.

**Why Attempt 1 failed -- and what the failure reveals?**

The closure axiom restricts the *range* of `hasTopping`: every topping must be in the listed union. This is necessary for the reasoner to bound the possible toppings. But for the min-3 cardinality to fire, the reasoner must still prove that **three distinct topping individuals exist**.

Even with the closure in place, the reasoner can prove at most **two** distinct toppings for `AmericanaPizza`, see from it's `SubClass Of` section below:




Here is why.

The reasoner's ability to count distinct individuals depends on being able to prove that two values cannot be the same individual. It proves this from **disjointness axioms**. If two classes `A` and `B` are declared disjoint, then no individual can be both an `A` and a `B`, so any individual of type `A` is distinct from any individual of type `B`.

For `AmericanaPizza`'s three toppings -- `MozzarellaTopping`, `PepperoniTopping`, `TomatoTopping` -- the reasoner needs to know that all three are pairwise disjoint. It checks the ontology:

- `MozzarellaTopping` is a subclass of `CheeseTopping`.
- `PepperoniTopping` is declared (in the working ontology) as a subclass of `VegetableTopping`.
- `TomatoTopping` is a subclass of `VegetableTopping`.

The ontology's disjointness declarations establish that `CheeseTopping`, `MeatTopping`, and `VegetableTopping` are pairwise disjoint. So:

- `MozzarellaTopping` (Cheese) is provably distinct from `PepperoniTopping` (Vegetable). ✅
- `MozzarellaTopping` (Cheese) is provably distinct from `TomatoTopping` (Vegetable). ✅
- `PepperoniTopping` (Vegetable) and `TomatoTopping` (Vegetable) are **not** declared disjoint from each other. ❌

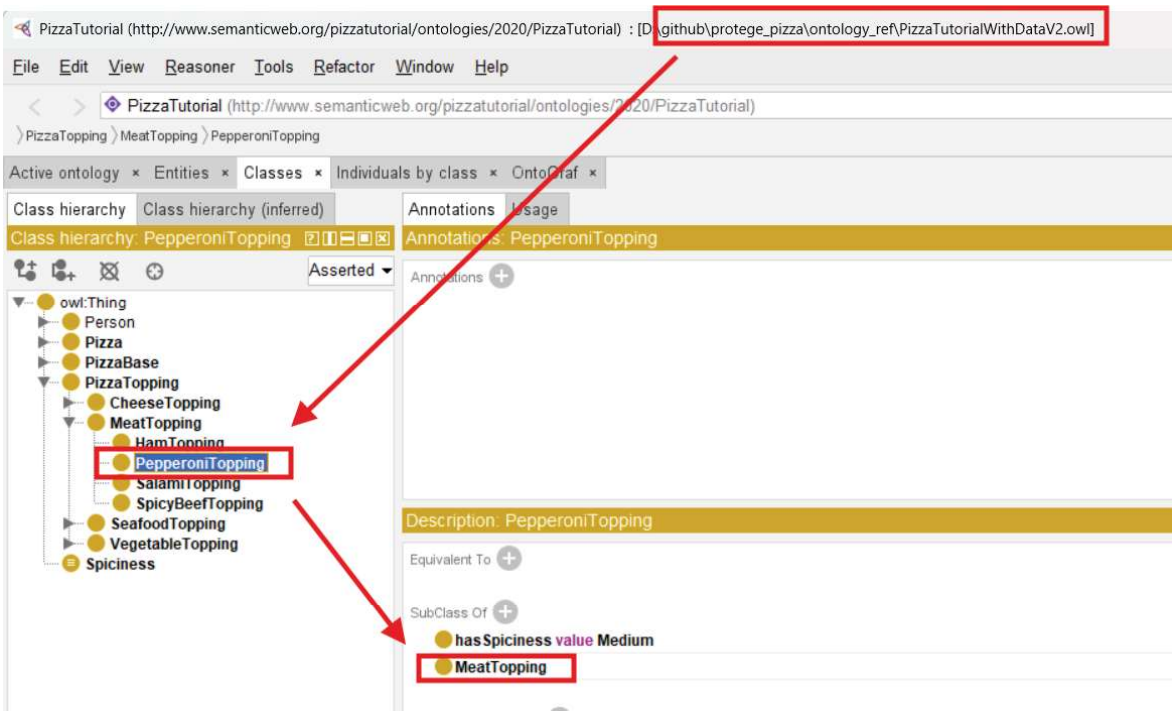
The reasoner can prove at most **two** distinct toppings for `AmericanaPizza` -- Mozzarella is distinct from the others, but Pepperoni and Tomato might be the same individual as far as the reasoner can tell. The min-3 cardinality cannot be satisfied.

**But, wait, there is a deeper problem here.** The reason `PepperoniTopping` and `TomatoTopping` are not disjoint is not an accident of the ontology's design. It is a **modeling error**.

**Pepperoni is a meat product.** It should be subclass of `MeatTopping`, not `VegetableTopping`. The working ontology got this wrong. This is the same kind of error the SKDL was designed to catch -- but it has slipped through to Stage 7.

Let's verify the correct taxonomy. Michael DeBellis' reference ontology ( `PizzaTutorialWithDataV2.owl` ) declares:

```
<owl:Class rdf:about="...#PepperoniTopping">
    <rdfs:subClassOf rdf:resource="...#MeatTopping"/>
    ...
</owl:Class>
```




The reference is unambiguous: `PepperoniTopping` is a subclass of `MeatTopping`. The working ontology has it under `VegetableTopping`. **This is a Stage 1 modeling error.**

**Stage 7 cannot fix a Stage 1 error.**

This is the moment where the SKDL's discipline shows its value.

The SKDL is a **lifecycle**, not a checklist. Stage 7 (Executable Knowledge) depends on Stage 6 (Reasoning), which depends on Stage 5 (Validation), which depends on Stage 4 (Governance), which depends on Stage 3 (Reuse), which depends on Stage 2 (Description), which depends on Stage 1 (Conceptual Modeling).

If a modeling error exists at Stage 1, **no amount of engineering at Stage 7 will fix it**. The reasoner will faithfully compute the logical consequences of the axioms it is given -- including the axioms that encode the wrong taxonomy. Attempt 1 failed because the closure axiom's effect was undermined by a Stage 1 classification error that the closure could not compensate for.

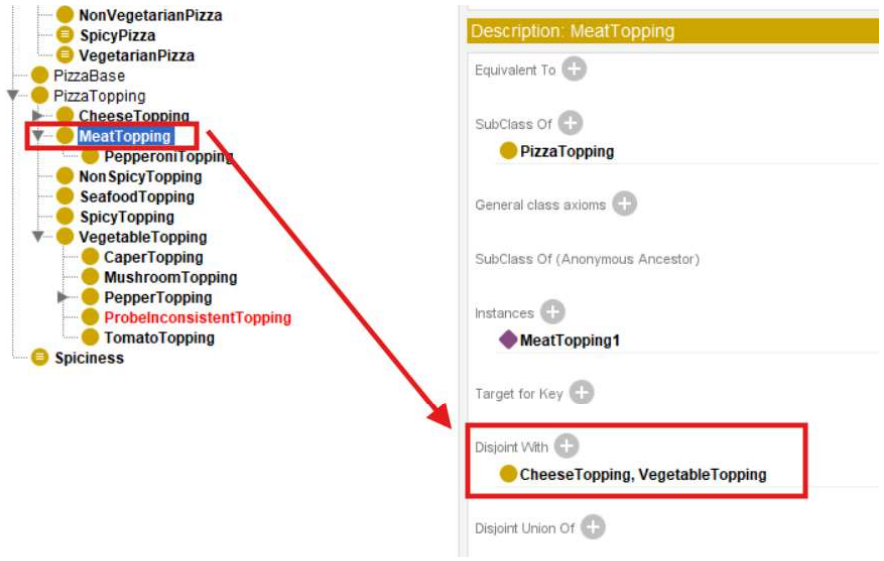
**The honest engineering response is to go back to Stage 1, correct the model, and then re-run the Stage 7 work.**

### Attempt 2 -- Correct the first Stage 1 modeling error, then re-test

The fix is a one-line change in the class hierarchy:

```
Class: PepperoniTopping
    SubClassOf: MeatTopping     # was: VegetableTopping
```

This moves `PepperoniTopping` out from under `VegetableTopping` and into its correct position under `MeatTopping`. Now `PepperoniTopping` is a subclass of `MeatTopping`, and `MeatTopping` is disjoint from `CheeseTopping` and `VegetableTopping` (per the ontology's `AllDisjointClasses` axioms).

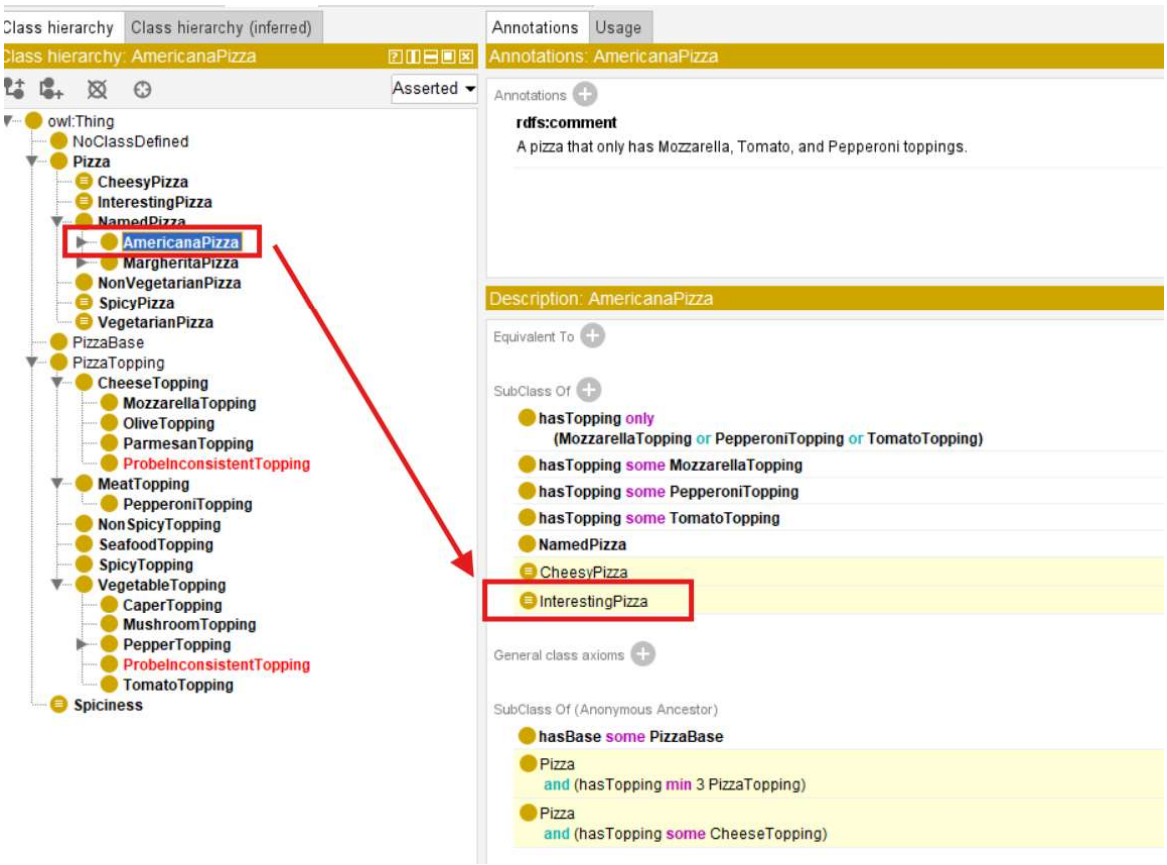



With this correction:

- `MozzarellaTopping` (Cheese) is provably distinct from `PepperoniTopping` (Meat). ✅
- `MozzarellaTopping` (Cheese) is provably distinct from `TomatoTopping` (Vegetable). ✅
- `PepperoniTopping` (Meat) is provably distinct from `TomatoTopping` (Vegetable). ✅

All three of `AmericanaPizza`'s toppings are now provably pairwise distinct. The reasoner can count them. Combined with the closure axiom from Attempt 1, the min-3 cardinality should now fire for `AmericanaPizza`.

**Synchronize the reasoner and observe.** This time, the result of `AmericanaPizza` is what we expect:



`AmericanaPizza` is now classified as subclass of `InterestingPizza`. **Good progress!** Its three toppings {Mozzarella, Pepperoni, Tomato} are pairwise provably distinct (Cheese ⊥ Meat ⊥ Vegetable), and the closure from Attempt 1 supplies the exhaustive count.

**However, `AmericanaHotPizza` and `SohoPizza` are still not classified.** Correcting one modeling error has fixed one pizza, but not the other two. The journey continues.

**Attempt 2.1 -- Diagnose the remaining two pizzas**

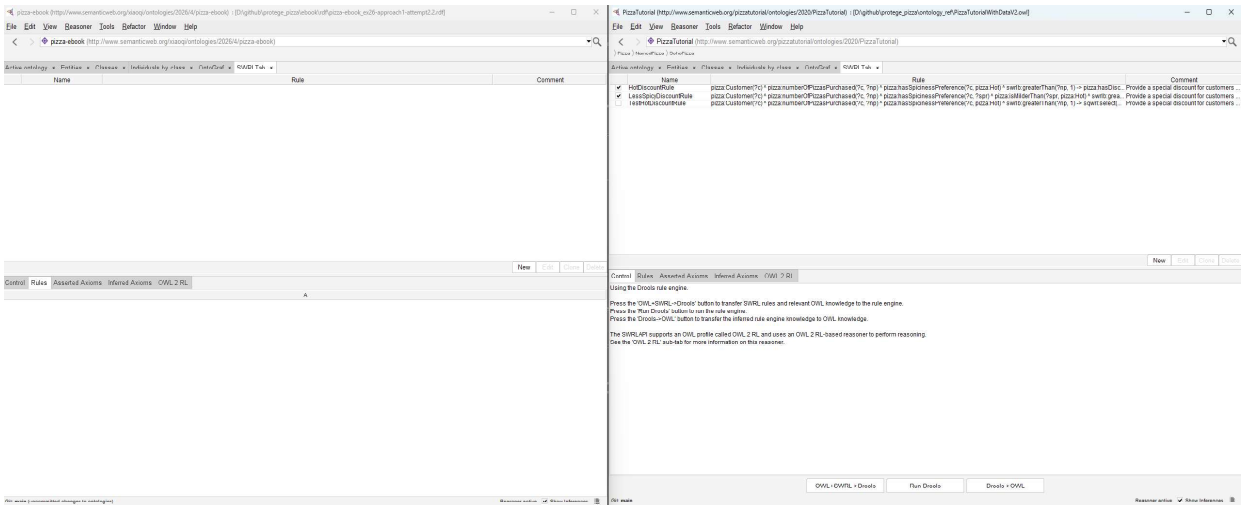
This is a **diagnostic** step. No new corrections are applied here. We investigate *why* the other two pizzas still fail.

**A preliminary tooling issue**

Before we trust the reasoner's output, there is one thing to check. The Protégé log shows:

```
ERROR An error occurred during reasoning: A SWRL rule uses a built-in atom,
but built-in atoms are not supported yet.
java.lang.IllegalArgumentException: A SWRL rule uses a built-in atom,
but built-in atoms are not supported yet.
```

The active reasoner is **HermiT**, which does not support SWRL built-ins. The reference ontology contains two SWRL rules ( `HotDiscountRule` , `LessSpicyCiscountRule` ), both using `swrlb:greaterThan` . HermiT fails to load the ontology. Protégé falls backs to Pellet, but the reasoner state is unreliable.



After comparing from above screenshot, the problem is not in the working ontology. The working ontology has no SWRL rules so far. The `A SWRL rule uses a built-in atom` error appeared only when the **reference ontology ( `PizzaTutorialWithDataV2.owl` )** was open in a separate Protégé window during earlier work. In that session, HermiT refused to load the reference ontology, but the error is unrelated to the working ontology

The working ontology has no SWRL rules and no reasoner configuration issue. Pellet or HermiT both work.

> **Note 1: Not all reasoners support all features**
>
> HermiT supports OWL 2 DL but not SWRL built-ins. Pellet supports both. When the ontology uses SWRL, select Pellet. Reading the reasoner log -- rather than guessing -- is a discipline that pays off repeatedly in ontology engineering.

> **Note 2: A note on multi-window Protégé sessions.**
>
> When the working ontology and the reference ontology are open in separate Protégé windows, errors from one window can appear in the log of the other, causing confusion. The SWRL error above was from the reference ontology, not the working ontology. As a general practice, work with one ontology per Protégé session when diagnosing reasoner behavior. This avoids cross-contamination of logs and reasoner state.

**Why `AmericanaHotPizza` still fails**

`AmericanaHotPizza` has four toppings: Mozzarella, Pepperoni, Tomato, JalapenoPepper.

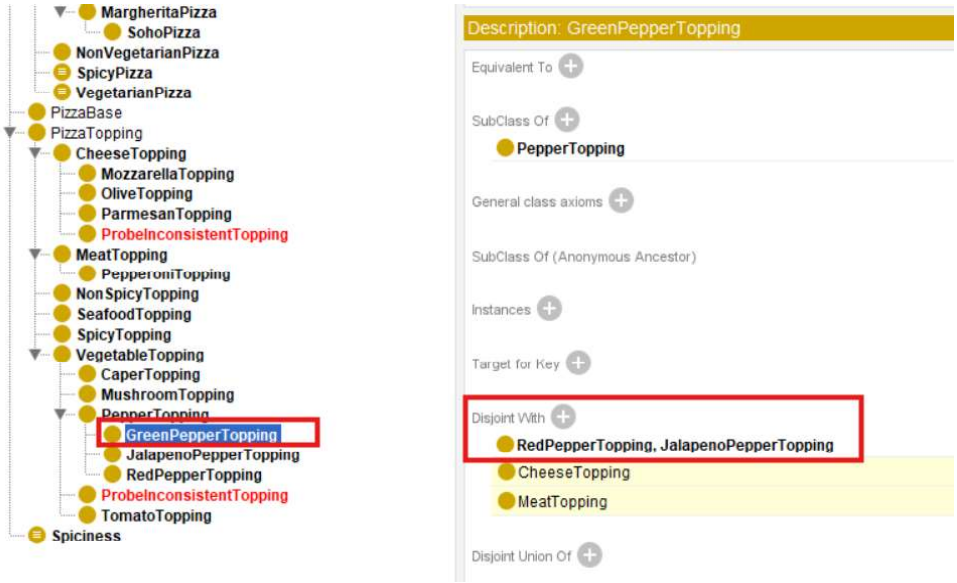
> **Not include `Topping` postfix for simplified displaying purpose only.**

After the Pepperoni fix (Correction 1), the disjointness checks are:

- Mozzarella (Cheese) vs. Pepperoni (Meat) ✅
- Mozzarella (Cheese) vs. Tomato (Vegetable) ✅
- Mozzarella (Cheese) vs. JalapenoPepper (Vegetable > Pepper) ✅

- Pepperoni (Meat) vs. Tomato (Vegetable) ✅
- Pepperoni (Meat) vs. JalapenoPepper (Vegetable > Pepper) ✅
- **Tomato (Vegetable) vs. JalapenoPepper (Vegetable > Pepper)** ❌

Both `TomatoTopping` and `JalapenoPepperTopping` sit under `VegetableTopping`. The working ontology declares only the three pepper subclasses (`GreenPepper`, `JalapenoPepper`, and `RedPepper`) as pairwise disjoint.



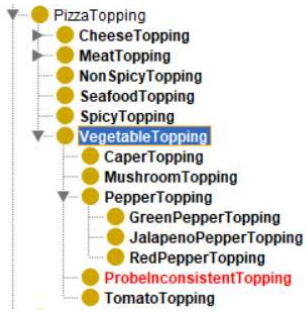
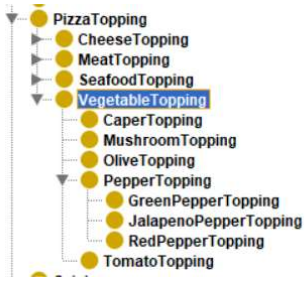

It does **not** declare the broader vegetable subclasses (`Caper`, `Mushroom`, `Pepper`, `Tomato`) as disjoint.

> ✎ **You may still see that `ProbeInconsistentTopping` as subclass of `VegetableTopping`, leave it as that's not impact this fixing result.**

The reference ontology has this broader declaration for `VegetableTopping`:

```
<owl:members rdf:parseType="Collection">
    <rdf:Description rdf:about="...#CaperTopping"/>
    <rdf:Description rdf:about="...#MushroomTopping"/>
    <rdf:Description rdf:about="...#OliveTopping"/>
    <rdf:Description rdf:about="...#PepperTopping"/>
    <rdf:Description rdf:about="...#TomatoTopping"/>
</owl:members>
```

> ✎ **The working ontology doesn't have `OliveTopping` as a subclass of `VegetableTopping` so far, which is mentioned below as another modeling difference.**

| Working Ontology | Reference Ontology |
|---|---|
|  |  |



The working ontology is **missing this broader disjointness declaration**, also `OliveTopping` is wrongly modeled as a subclass of `CheeseTopping` while should be a subclass of `VegetableTopping`. Without the correct modeling and proper broader disjointness declaration, the reasoner cannot prove that Tomato and JalapenoPepper are disjoint.

> ✎ **You may still see `ProbeInconsistentTopping` as a subclass of `VegetableTopping`. Leave it -- it does not impact this fixing result.**

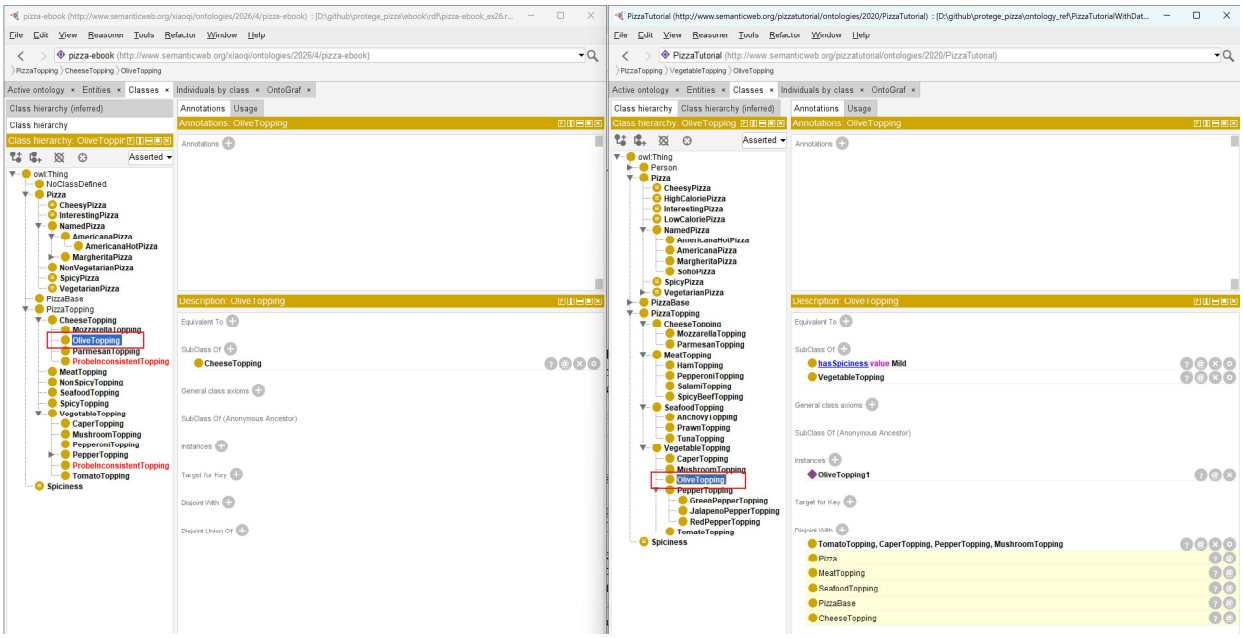
**Why `SohoPizza` still fails**

`SohoPizza` also has four toppings: Mozzarella, Olive, Parmesan, Tomato. Two problems, of different kinds.

**Problem 1 -- a modeling error in the topping taxonomy.** In the working ontology, as found in above `AmericanaHotPizza` analysis, `OliveTopping` is declared as a subclass of `CheeseTopping`:

```
<owl:Class rdf:about="...#OliveTopping">
    <rdfs:subClassOf rdf:resource="...#CheeseTopping"/>
</owl:class>
```

But Olive is not a cheese. In the reference ontology, `OliveTopping` is a subclass of `VegetableTopping` (see table above). **This is a second Stage 1 modeling error.**



With Olive misplaced under Cheese, `SohoPizza`'s toppings become: Mozzarella (Cheese), Tomato (Vegetable), Olive (Cheese), Parmesan (Cheese). The reasoner checks:

- Mozzarella (Cheese) vs. Tomato (Vegetable) ✅
- Mozzarella (Cheese) vs. Olive (Cheese) -- both cheese, no disjointness declared ❌
- Mozzarella (Cheese) vs. Parmesan (Cheese) -- both cheese, no disjointness declared ❌
- Olive (Cheese) vs. Parmesan (Cheese) ❌
- Tomato (Vegetable) vs. all cheeses ✅

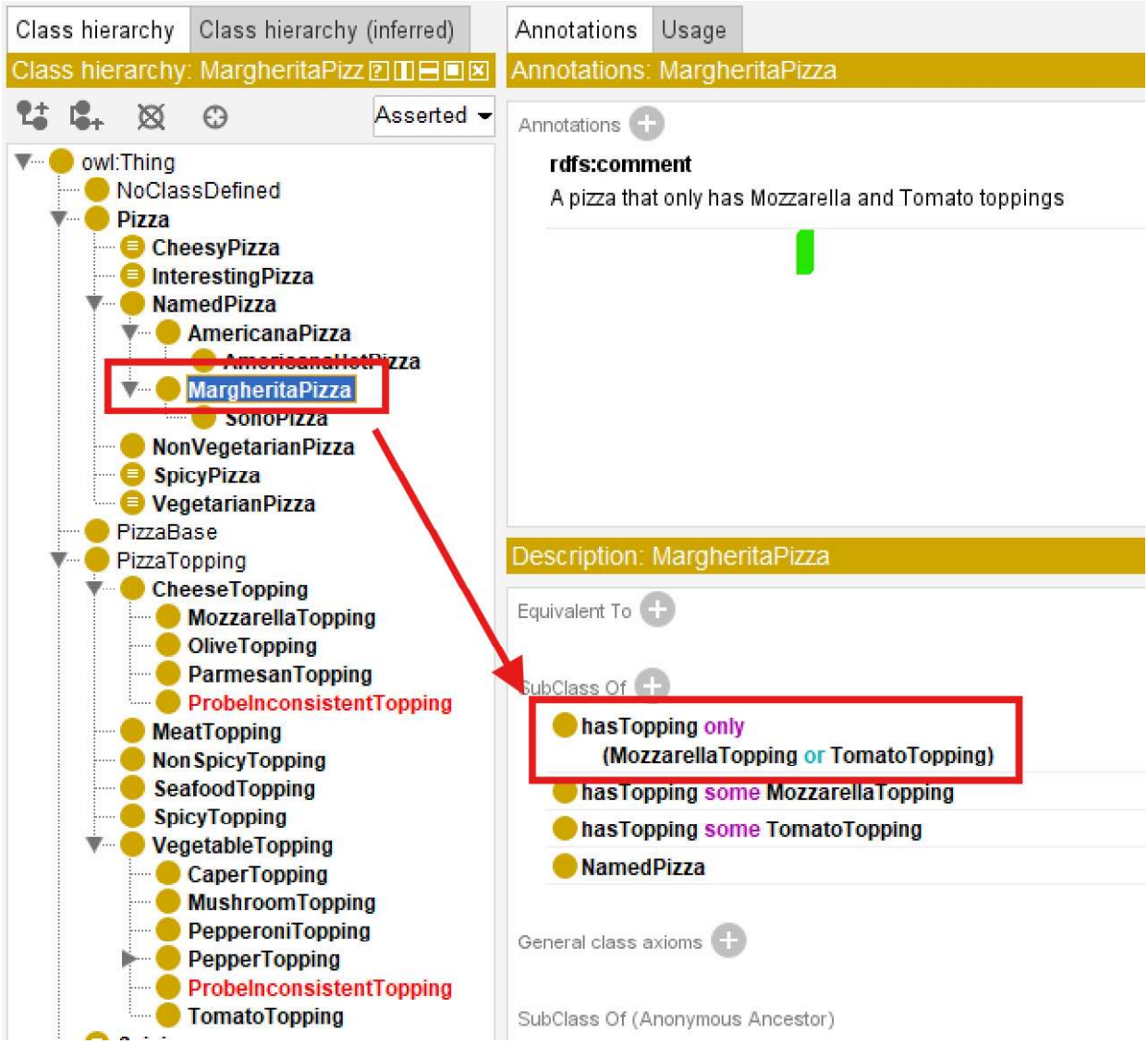
The reasoner can prove at most **two** distinct toppings for `SohoPizza`. The min-3 fails.

**Problem 2 -- a closure-inheritance conflict.** `SohoPizza` is declared as s subclass of `MargheritaPizza`:

```
<owl:Class rdf:about:"...#SohoPizza">
    <rdfs:subClassOf rdf:resource="...#MargheritaPizza"/>
    ...
</owl:Class>
```

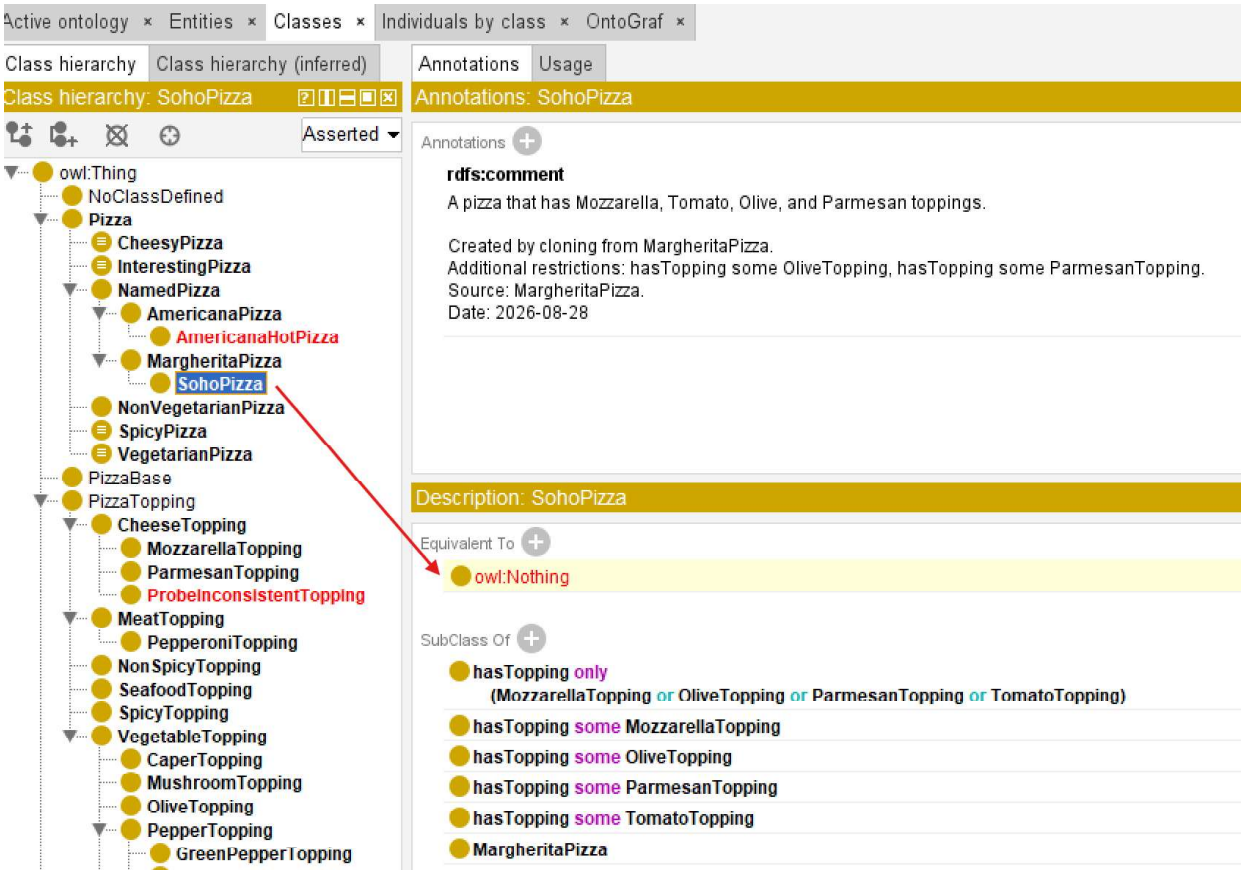
And, in working ontology, `MargheritaPizza` carries a closure axiom from Chapter 24:

```
Class: MargheritaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

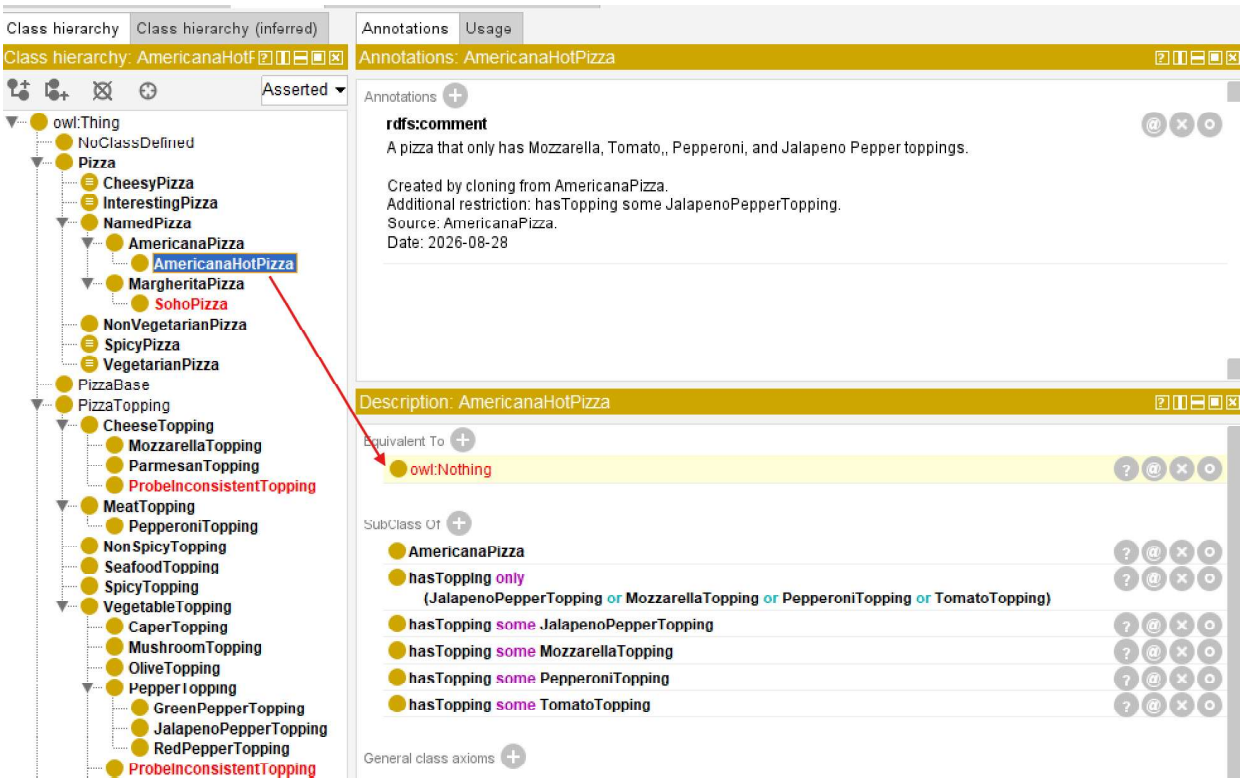



This closure, due to `SohoPizza` ⊑ `MargheritaPizza`, restricts every `MargheritaPizza`'s toppings to {Mozzarella, Tomato}. `SohoPizza` inherits this. But `SohoPizza` also has `hasTopping some OliveTopping` and `hasTopping some ParmesanTopping`, requiring at least one of each. Neither Olive nor Parmesan is in the inherited union.

The two axioms cannot both be satisfied. The reasoner correctly infers `SohoPizza` to be `owl:Nothing` -- an unsatisfiable class.




**The same closure-inheritance conflict affects `AmericanaHotPizza`.** `AmericanaHotPizza` is declared as a subclass of `AmericanaPIzza`, which carries a closure to {Mozzarella, Pepperoni, Tomato}. `AmericanaHotPizza` adds `hasTopping some JalapenoPepperTopping`. Jalapeno is not in the inherited union. `AmericanaHotPizza` is also inferred to be `owl:Nothing`.




**Summary of Attempt 2.1**

We now know:

- `AmericanaPizza` was fixed in Attempt 2 (Correction 1).
- `AmericanaHotPizza` needs the broad vegetable disjointness (Correction 3) **and** a resolution of the closure-inheritance conflict (Corrections 4-6).
- `SohoPizza` needs the Olive reclassification (Correction 2), the broad vegetable disjointness (Correction 3), **and** a resolution of the closure-inheritance conflict (Corrections 4-6).
- The reasoner must be switched to use Pellet.

No fix is applied in this step. Attempt 2.2 applies them.

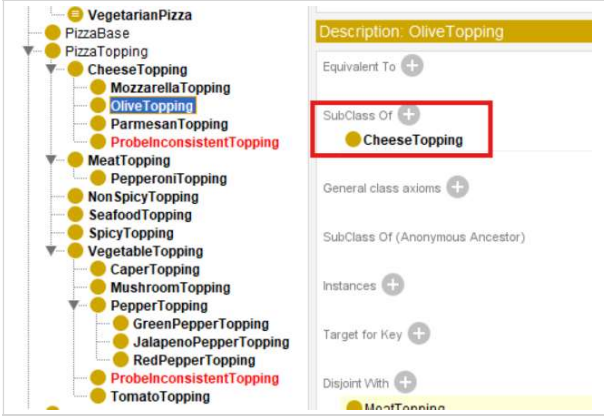
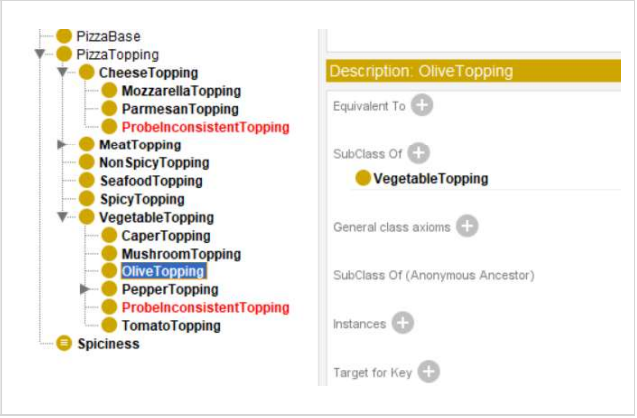
**Attempt 2.2 -- Apply the remaining corrections and re-test**

The complete set of corrections, with the order in which they will be applied:

| # | Correction | Type | Applied in |
|---|---|---|---|
| 1 | `PepperoniTopping` $\sqsubseteq$ `MeatTopping` (was `VegetableTopping`) | Taxonomy | Attempt 2 |
| 2 | `OliveTopping` $\sqsubseteq$ `VegetableTopping` (was `CheeseTopping`) | Taxonomy | Attempt 2.2 |
| 3 | `DisjointClasses: Caper, Mushroom, Olive, Pepper, Tomato` | Disjointness | Attempt 2.2 |
| 4 | `SohoPizza` $\sqsubseteq$ `NamedPizza` (was `MargheritaPizza`) | Hierarchy | Attempt 2.2 |
| 5 | Remove closure from `AmericanaPizza` | Closure | Attempt 2.2 |
| 6 | Introduce `PlainAmericanaPizza` with closure {M, P, T} | Structure | Attempt 2.2 |

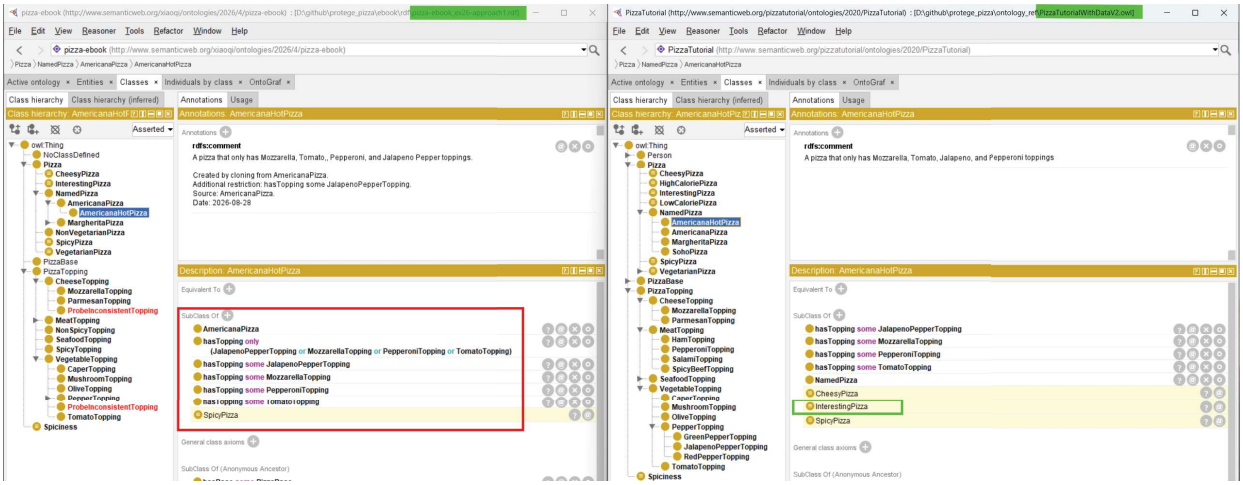
**Correction 2 -- Move `OliveTopping` to `VegetableTopping`**

```
Class: OliveTopping
    SubClassOf: VegetableTopping    # was: CheeseTopping
```

| Before Fixing | After Fixing |
| --- | --- |
|  |  |



After this correction, synchronizing the reasoner. `AmericanaHotPizza` and `SohoPizza` **still do not classify**:

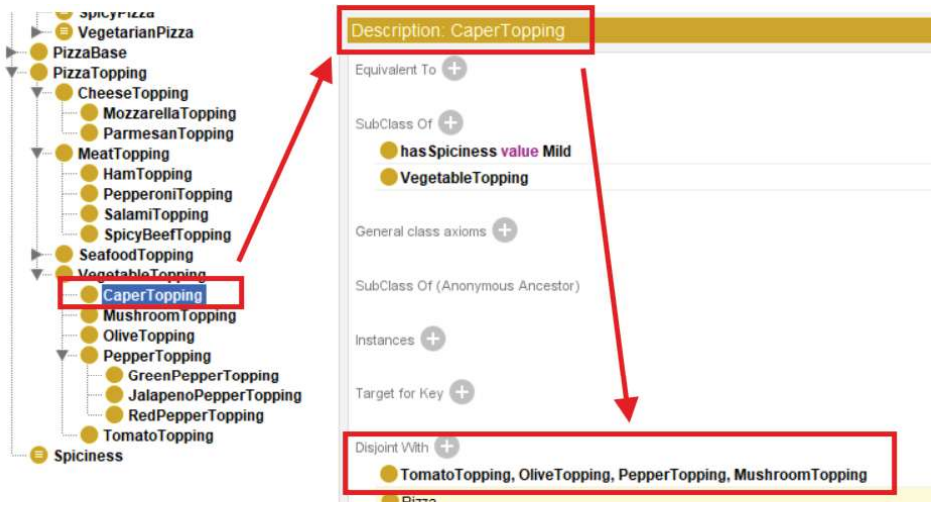


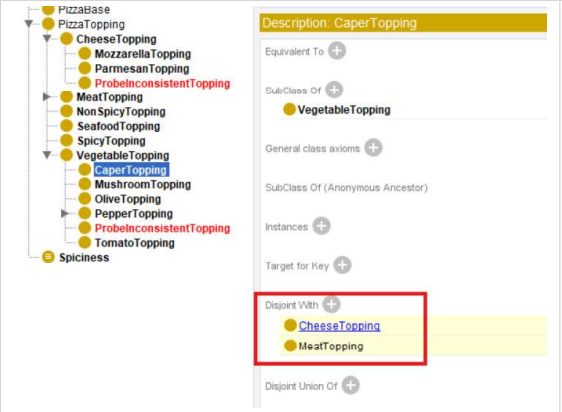
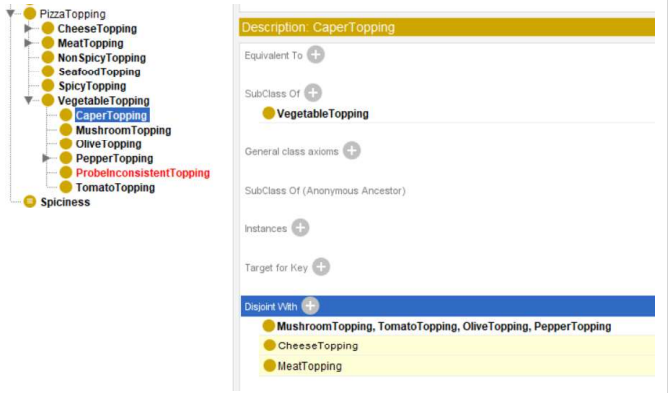
The fixing is not completed. This is expected -- the closure-inheritance conflict remains, and the broader disjointness is still missing.

**Correction 3: Add the broad vegetable disjointness declaration**

```
DisjointClasses: CaperTopping, MushroomTopping, OliveTopping, PepperTopping, TomatoTopping
```

The reference ontology shows this kind of broad vegetable disjointness:



| Before adding broad Disjointness | After adding broad Disjointness |
| --- | --- |
|  |  |



After this correction, synchronize the reasoner again. Still, `AmericanaHotPizza` and `SohoPizza` do not classify. **The disjointness was not the only blocker.** The remaining blocker is the closure-inheritance conflict.

**Diagnosing the closure-inheritance conflict**

Let us name the pattern we have discovered, because it is general:

> **A closure axiom on a parent class propagates to every subclass. If any subclass extends the parent's topping set, the inherited closure contradicts the subclass' own restrictions, and the subclass becomes unsatisfiable ( `owl:nothing` ).**

Both branches of our pizza hierarchy exhibit this conflict:

- **Americana branch**: `AmericanaPizza` has closure {M, P, T}; `AmericanaHotPizza` extends it with Jalapeno. **Conflict.**
- **Margherita branch**: `MargheritaPizza` has closure {M, T} (added in Chapter 24); `SohoPizza` extends it with Olive and Parmesan. **Conflict.**

The reference ontology avoids this by **flattening the structure** -- all four named pizzas are siblings under `NamedPizza`. Our working ontology uses a richer, hierarchical structure, which is a legitimate modeling choice -- an Americana Hot Pizza *is a kind of* Americana Pizza with jalapeño added. But the hierarchy forces us to resolve the closure-inheritance conflict correctly.

There are four candidate fixes.

**Option A -- Flatten the structure**

Move `AmericanaHotPizza` and `SohoPizza` to be direct subclasses of `NamedPizza`, matching the reference ontology.

- *Pros*: Simple, known to work, no new classes.
- *Cons*: Loses the "is a kind of" semantic relationship.

**Option B -- Widen the parent's closure to include the subclass's toppings**

Change `AmericanaPizza`'s closure to include Jalapeno:

```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping or JalapenoPepperTopping)
```

- *Pros*: Keeps the hierarchy. Small textual change.
- *Cons*: **This is semantically wrong.** A closure axiom is a universal restriction -- `hasTopping only X` says *for every value v of hasTopping, $v \in X$*. Widening the parent's closure to include Jalapeno says "every Americana Pizza may have a Jalapeno topping." But a plain Americana Pizza does not have jalapeño. We have leaked the subclass's extension into the parent's definition. The parent's closure no longer describes the parent; it describes the union of all descendants. The axiom loses its semantic precision.

**We reject Option B.** It resolves the reasoner's complaint by weakening the semantics of the parent.

**Option C -- Introduce "plain" leaf classes that carry the closures**

Keep the hierarchy, but remove the closure from the parent. Introduce a leaf class representing "plain Americana Pizzas with exactly the standard three toppings":

```
Class: PlainAmericanaPizza
    SubClassOf: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```

Closures now live only on the leaves. **No inheritance conflict.**

- *Pros*: Semantically clean. This is the well-known **two-tier** or **definition/closure separation** pattern.
- *Cons*: Introduces new classes. Existing instances would need to be reclassified.

**Option D -- Hybrid: keep inheritance where it does not conflict; revert to flat where it does**

Observe that the two branches have different historical constraints. `MargheritaPizza`'s closure was placed in Chapter 24 for the `VegetarianPizza` classification -- it is used elsewhere in the book. `AmericanaPizza`'s closure was placed in Attempt 1 of this exercise.

The pragmatic hybrid:

- **Americana branch:** keep the hierarchy, introduce `PlainAmericanaPizza` for the closure. This is Option C applied to one branch.
- **Margherita branch:** match the reference. Move `SohoPizza` to be a direct subclass of `NamedPizza`. This preserves Chapter 24's `MargheritaPizza` closure and its `VegetarianPIzza` classification.

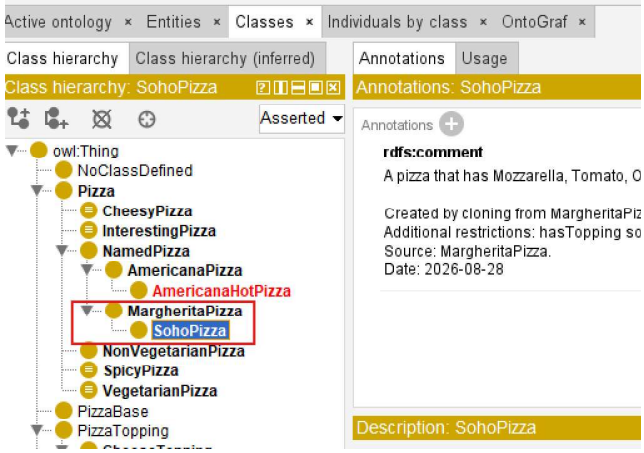
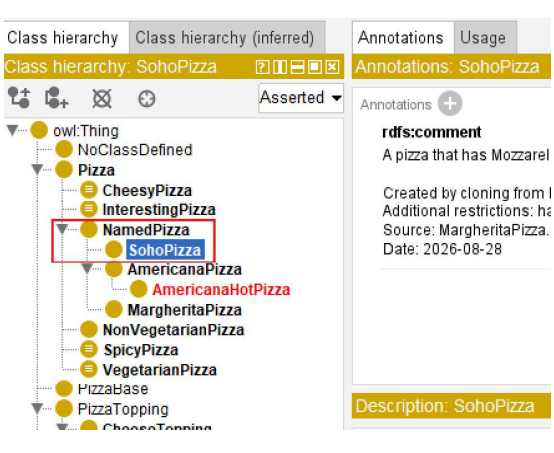
The result is asymmetric -- and that asymmetry is honest. It reflects the specific interactions between closures, inheritance, and the chapters already written.

**Applying the hybrid fix (Option D)**

> In `rdf` folder, I've saved previous `approach1` ontology to a new file named as `pizza-ebook_ex26-approach1-attempt2.2.rdf`, which you can use for practicing below corrections.

**Correction 4 -- Move `SohoPizza` to `NamedPizza`**

```
Class: SohoPizza
    SubClassOf: NamedPizza      # was: MargheritaPizza
```

| Before | After |
| --- | --- |
|  |  |

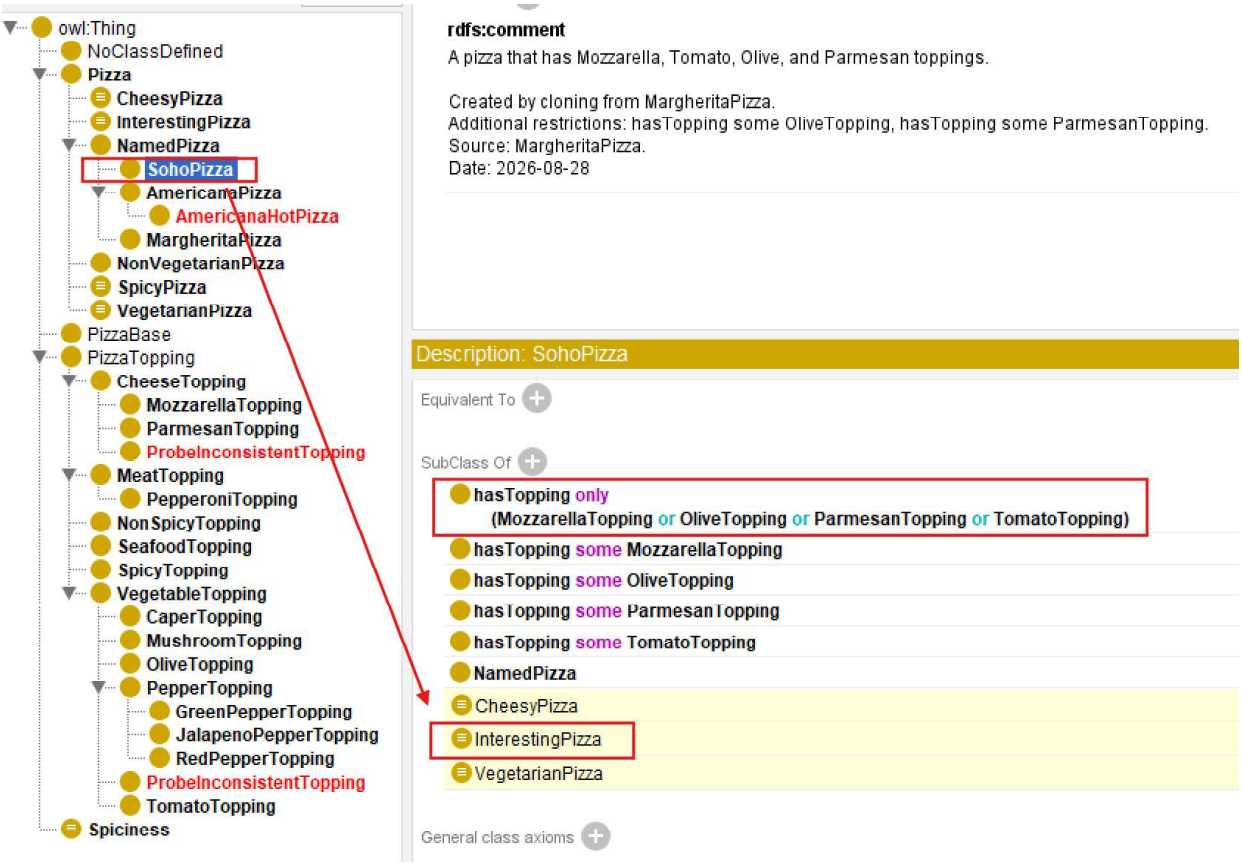




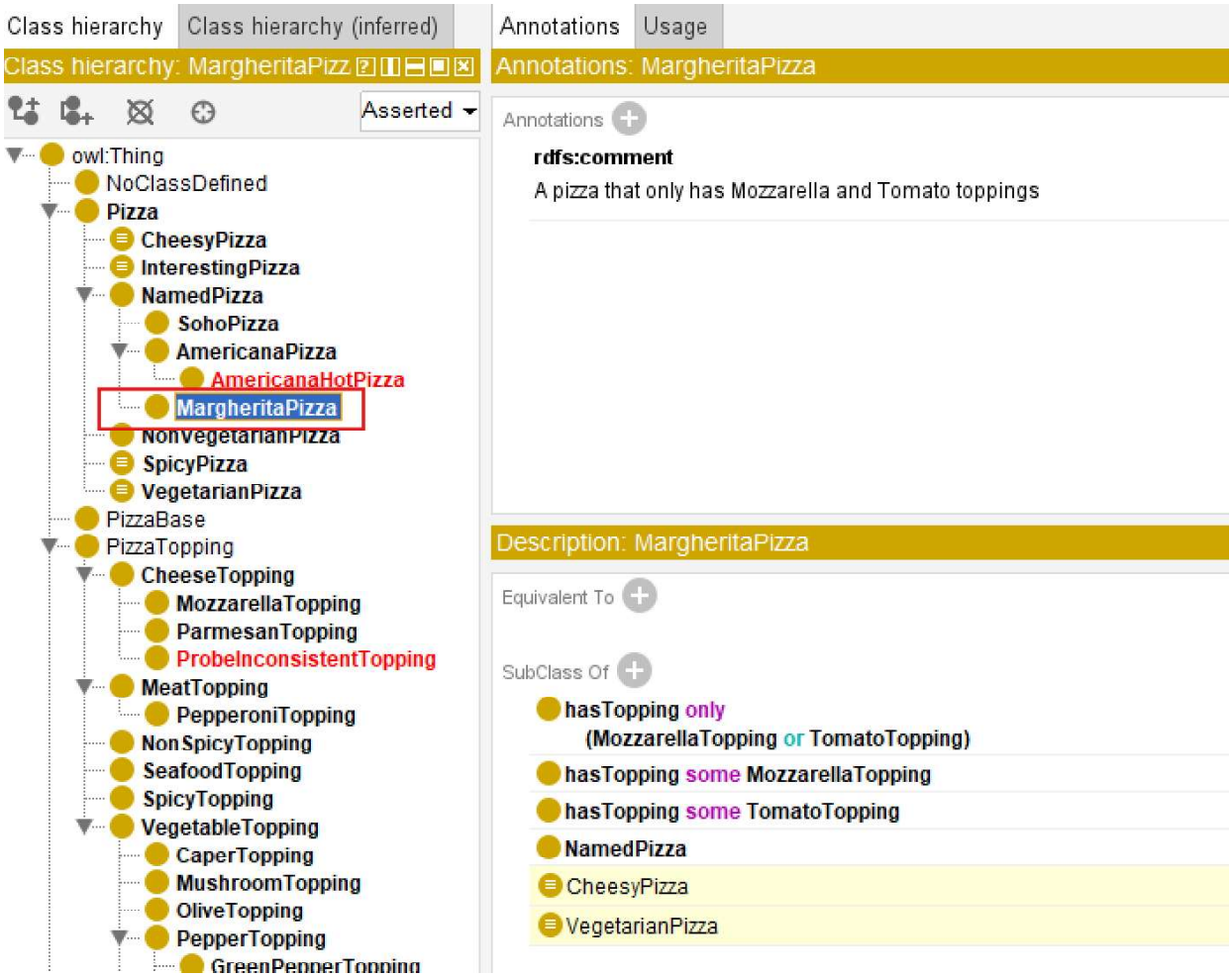
`SohoPizza` no longer inherits `MargheritaPizza`'s closure. Its own closure to {Mozzarella, Tomato, Olive, Parmesan} is the only closure that applies to it, and it is sound. `MargheritaPizza` retains its Chapter 24 closure -- no conflict, because it now has no subclass that extends its topping set.

**Synchronize the reasoner and observe**

`SohoPizza` (M, T + O + Pa): three provably distinct (M, T, O via the vegetable disjointness). ✅



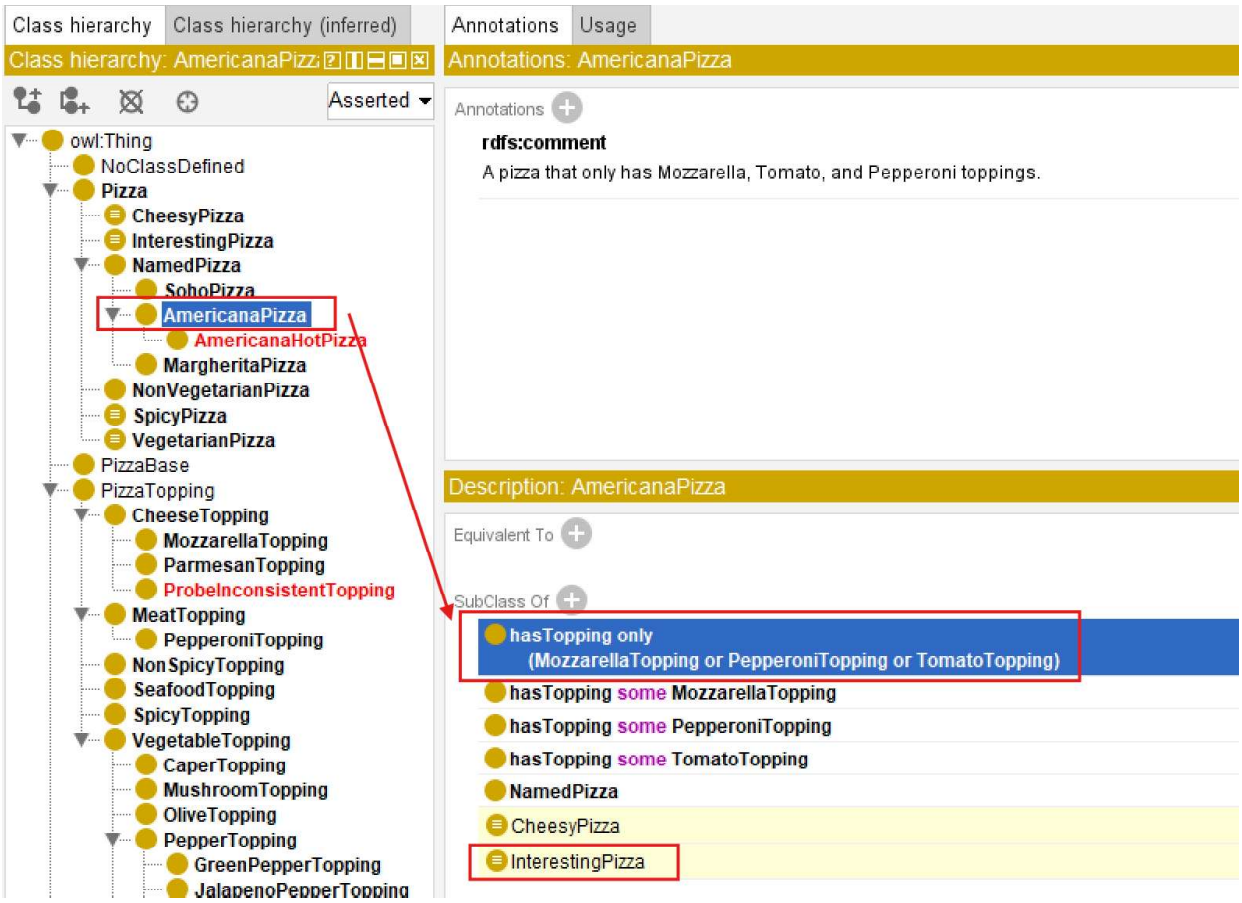

`MargheritaPizza` (M, T): two toppings -- correctly **not** classified as `InterestingPizza`.



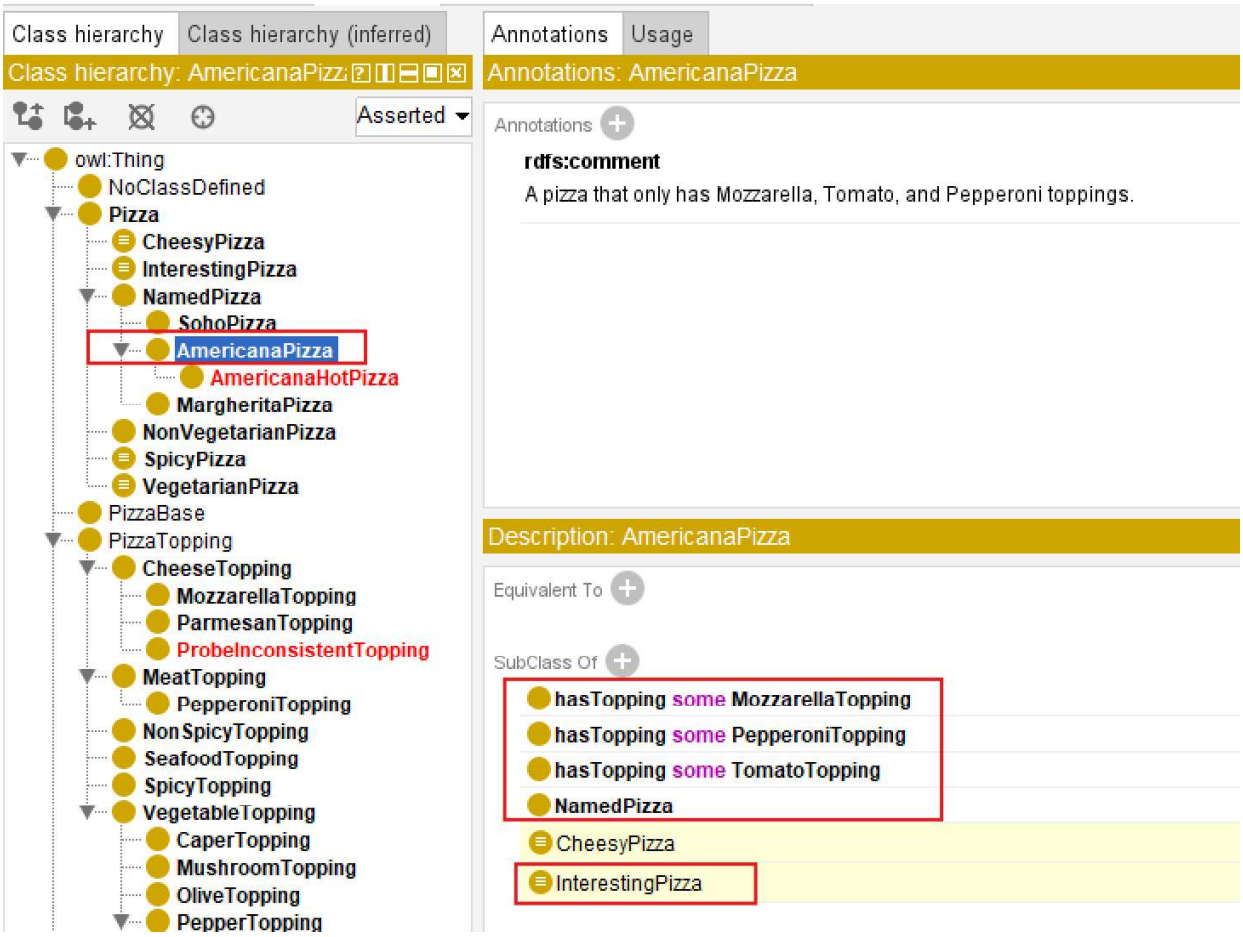

**Correction 5 -- Remove the closure from `AmericanaPizza`**

```
Class: AmericanaPizza
    SubClassOf: NamedPizza
    SubClassOf: hasTopping some MozzarellaTopping
    SubClassOf: hasTopping some PepperoniTopping
    SubClassOf: hasTopping some TomatoTopping
    # closure removed
```

Before Correction 5:

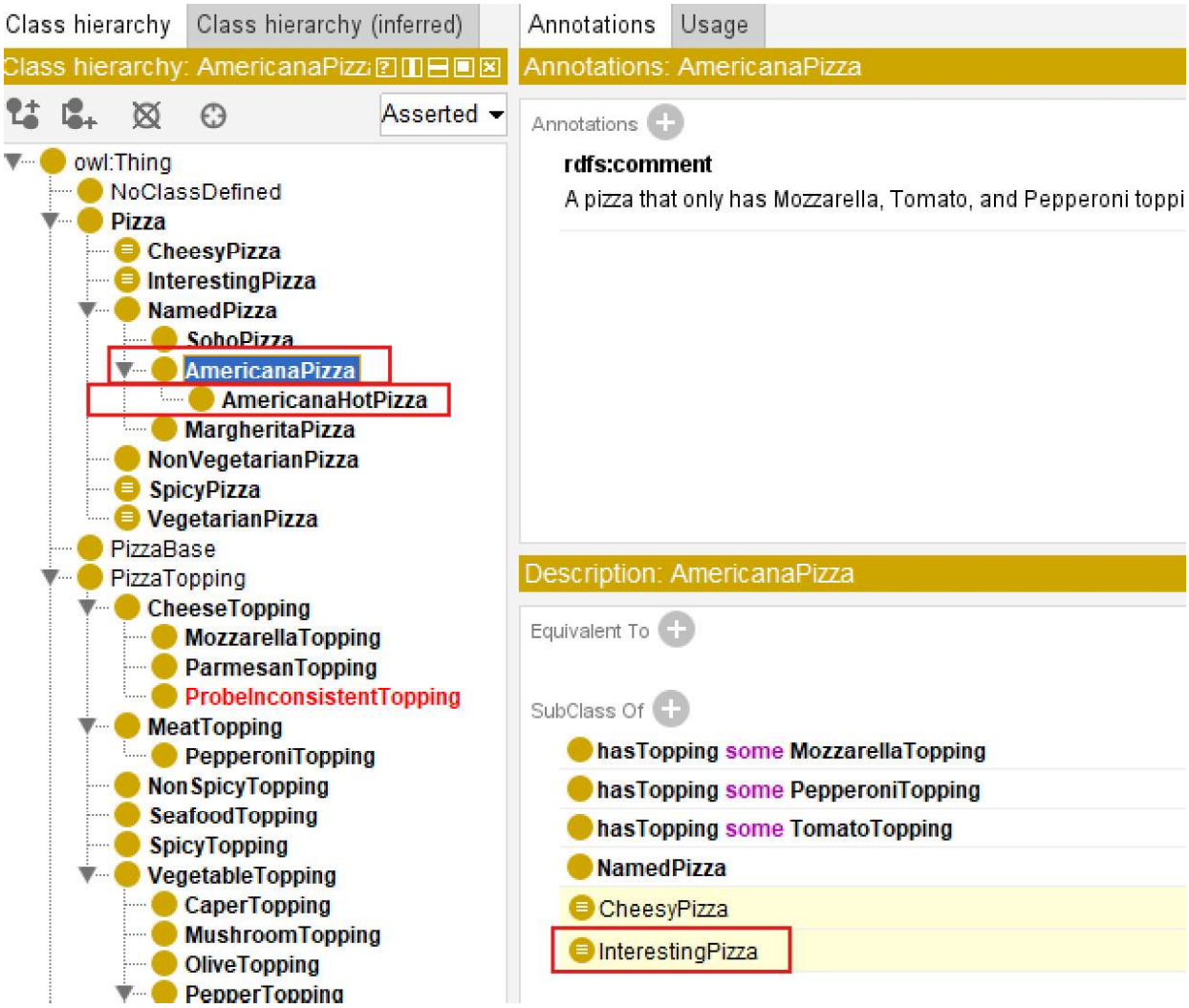



After Correction 5:

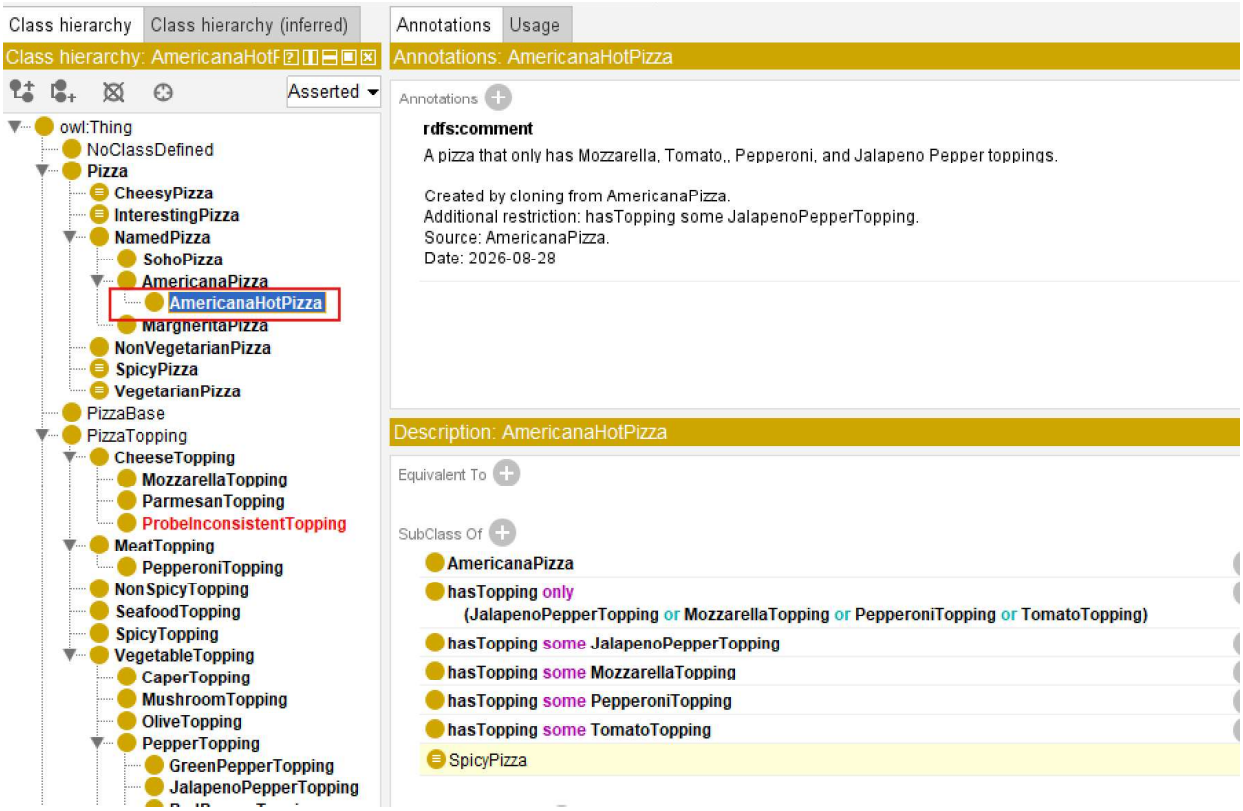



**Synchronize the reasoner and observe**

`AmericanaPizza` is still classified as subclass of `InterestingPizza`, since it fulfills the 3 distinct toppings criteria; and you may notice the inconsistency (red color) on `AmericanaHotPizza` has disappeared, means there's no axiom inheritance conflicts after this correction.

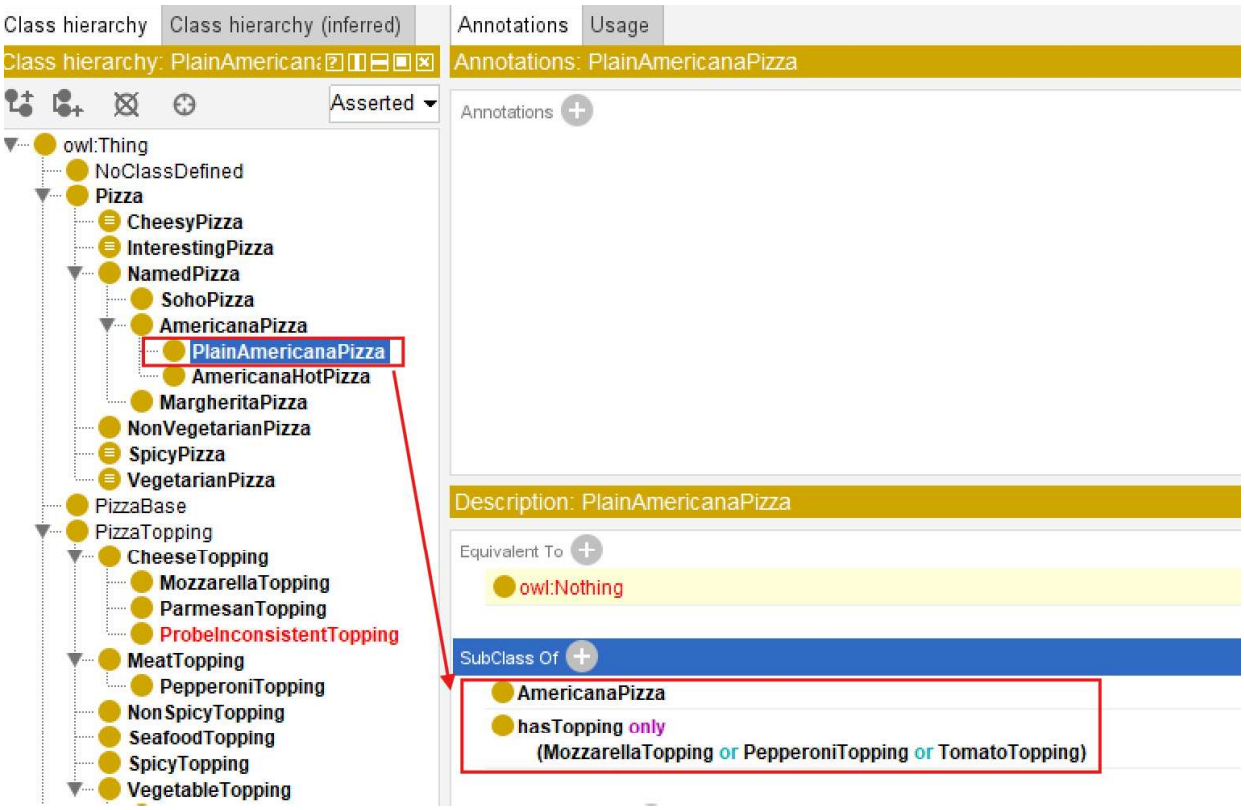



However, `AmericanaHotPizza` is still not classified as subclass of `InterestingPizza`:




**Correction 6 -- Introduce `PlainAmericanaPizza` with the closure**

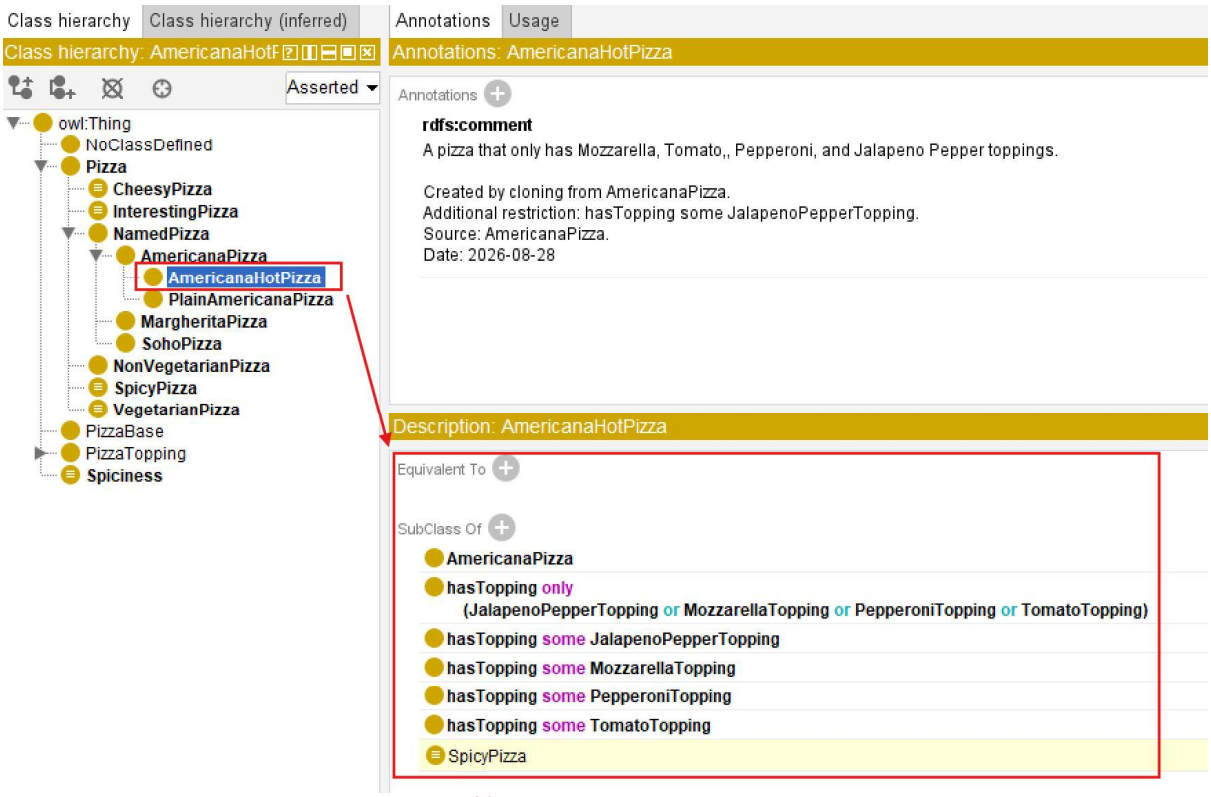
```
Class: PlainAmericanaPizza
    SubClassOf: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```



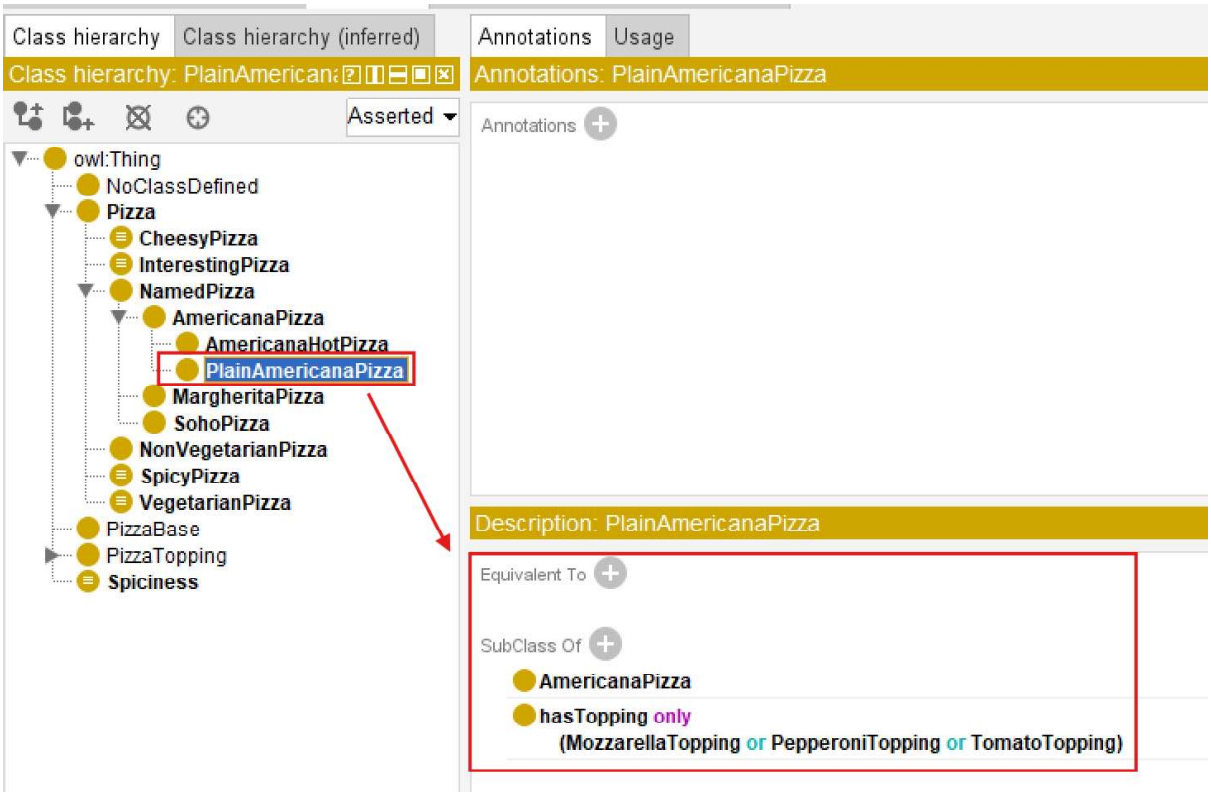

The closure now lives on the leaf. `AmericanaHotPizza` keeps its own closure to {M, P, T, J}. Neither conflicts with the other.

**Synchronize the reasoner and observe**

- `AmericanaPizza`, no closure: classified as subclass of `InterestingPizza`. ✅
- `PlainAmericanaPizza (M, P, T): three provably distinct toppings. ❌




- `AmericanaHotPizza (M, P, T + J): four provably distinct toppings. ❌



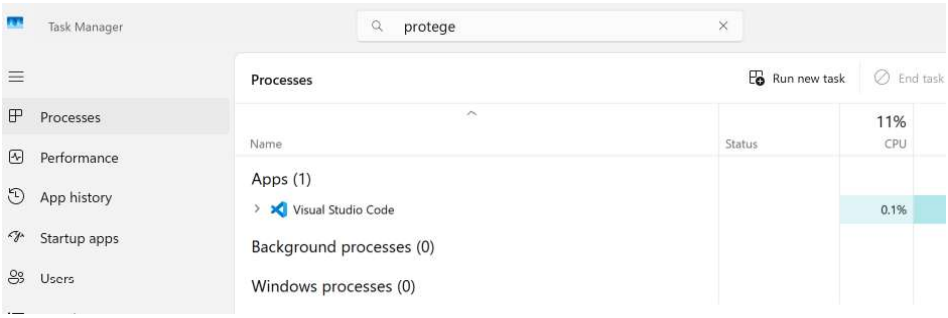

After applying all six corrections, `AmericanaPizza` and `SohoPizza` classify correctly. However, `PlainAmericanaPizza` and `AmericanaHotPizza` do not, despite having logically correct axioms.

**Attempt 2.3 -- Investigate the remaining two classes**

Two steps:

1. **Workspace hygiene.** Confirm that only the working ontology is open. Confirm no SWRL rules exist. Confirm the reasoner is Pellet or HermiT (both work for this ontology).
2. **Restart and re-synchronize.** Clear the reasoner cache by restarting Protégé.

Below Task Manager in Windows shows no `Protégé` process (except VS Code is used to editing this book) exists:

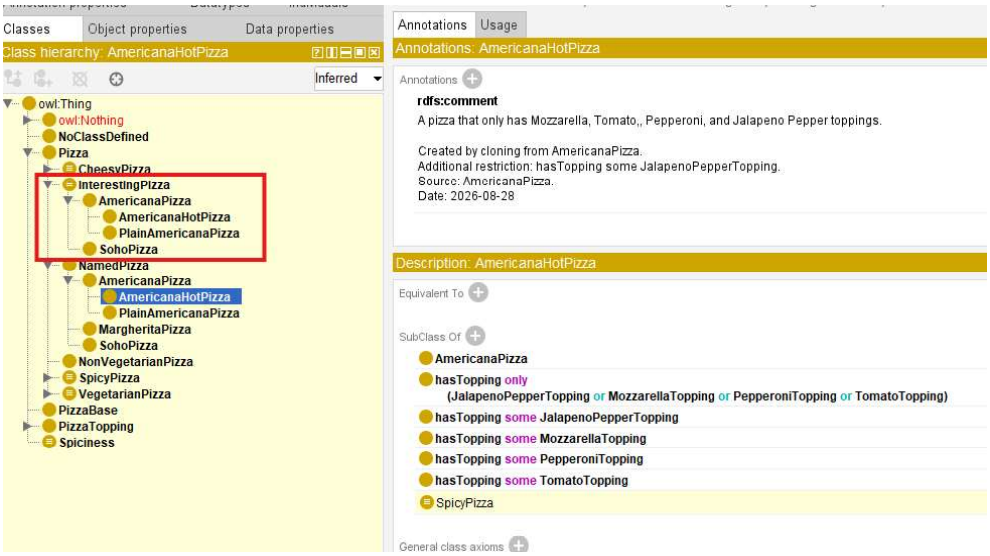



However, after the clean restart both reasoners agree the classes still do not classify.

**Attempt 2.4 -- Investigate the apparent non-classification**

After applying Corrections 1-6, `AmericanaPizza` and `SohoPizza` appear to classify correctly -- their inferred position in the tree is under `InterestingPizza`. `AmericanaHotPizza` and `PlainAmericanaPizza` appear **not** to classify, because their Description panels do not show `InterestingPizza` in the `SubClass Of` field.

Before assuming a reasoner failure, verify by switching to the **Class hierarchy (inferred)** tab and inspecting the class hierarchy around `InterestingPizza`.




The inferred tree shows `InterestingPizza` with **all four of the target pizzas nested under it**: `AmericanaPizza`, `AmericanaHotPizza`, `PlainAmericanaPizza`, and `SohoPizza`. The reasoner **has** inferred the classification.

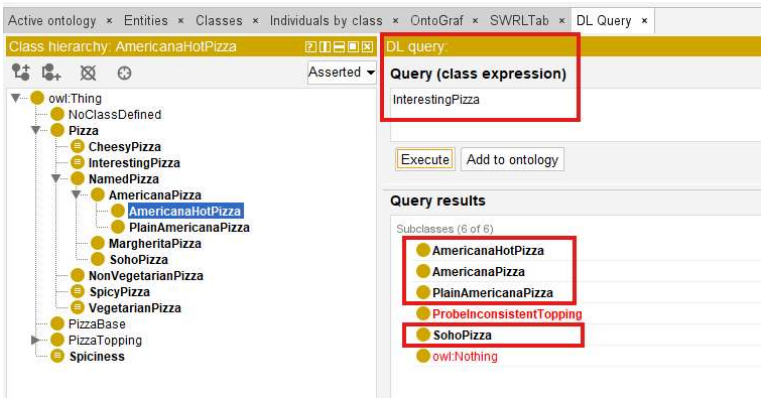
**The apparent failure was a Protégé UI display artifact.** When the reasoner infers $\mathrm{A} \sqsubseteq \mathrm{C}$ transitively (through $\mathrm{A} \sqsubseteq \mathrm{B} \sqsubseteq \mathrm{C}$), Protégé's **inferred class hierarchy tree** shows `A` under `C`, but the **Description panel** for `A` may not show `C` in the `SubClass Of` field. This is because the Description panel displays asserted axioms and directly inferred axioms, not all transitive entailments.

To verify an inferred ancestor, use one of:

- The **inferred class hierarchy tree** (authoritative global view).
- The **DL Query** tab -- query for the candidate superclass, check "Subclasses".
- Select the **candidate superclass** in the tree and inspect its inferred children.

Here, from the 2nd verification method:

1. Use the `DL Query` tab
2. Enter the query: `InterestingPizza`
3. Check the `Instances` and `Subclasses` boxes.
4. The result should list the four pizzas.




**The full correction set worked.** All four pizzas with three or more toppings classify as `InterestingPizza`. `MargheritaPizza` with 2 toppings correctly does not.

**The lesson: verify inference in the right view**

When working with reasoner-inferred classifications in Protégé, always verify in the **inferred class hierarchy tree**, not in the Description panel. The Description panel tells you what the class *has been asserted to be*. The inferred tree tells you what the reasoner has *derived*. These are different views, and Protégé does not always surface every inference in the Description panel.

> **A Protégé UI behavior**
>
> The Description panel shows the asserted axioms of the selected class and its directly inferred axioms. When a superclass is inferred **transitively** (through another superclass), it may not appear in the Description panel.

**Key Insight:**

> "The reasoner's output is in the inferred hierarchy tree. The Description panel shows you what you said about a class; the tree shows you what the reasoner derived. When they seem to disagree, trust the tree -- or better, verify in a third view (DL Query) to be sure."

### Approach 2 -- Assert the cardinality directly (an alternative)

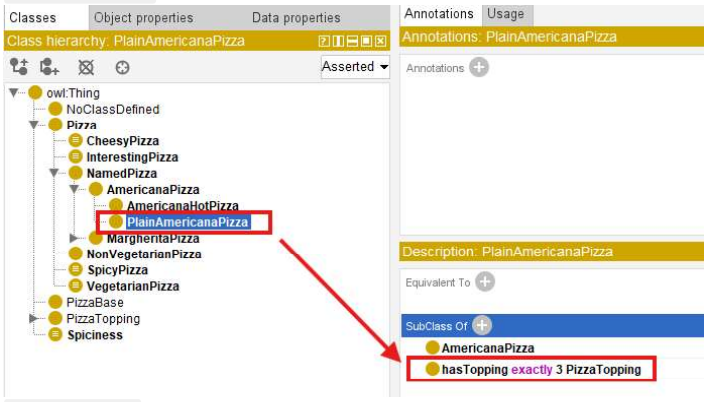
In some cases, the reasoner does not derive the intended classification, even when the axioms are correct. When this happens, one pragmatic alternative is to assert the cardinality directly on the leaf classes:

```
Class: PlainAmericanaPizza
    SubClassOf: hasTopping exactly 3 PizzaTopping
Class: AmericanaHotPizza
    SubClassOf: hasTopping exactly 4 PizzaTopping
```

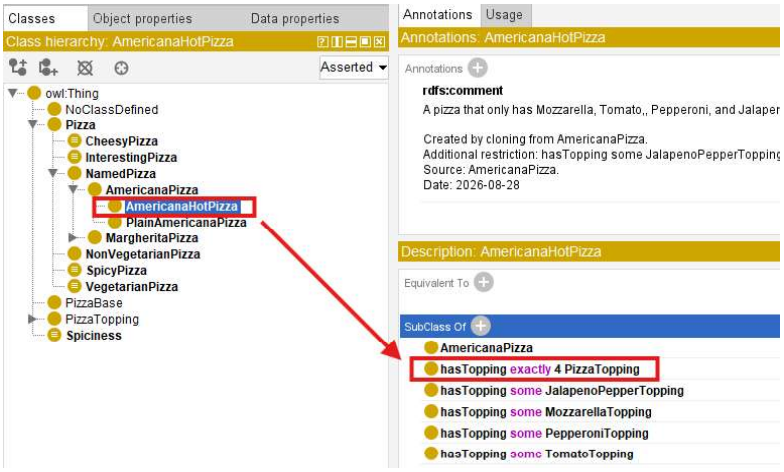
> Start from `pizza-ebook_ex26.rdf` in folder `rdf`, save it to `pizza-ebook_ex26-approach2.rdf` for this approach.

New create `PlainAmericanaPizza` and set as above direct cardinality:

- `PlainAmericanaPizza`




- `AmericanaHotPizza`

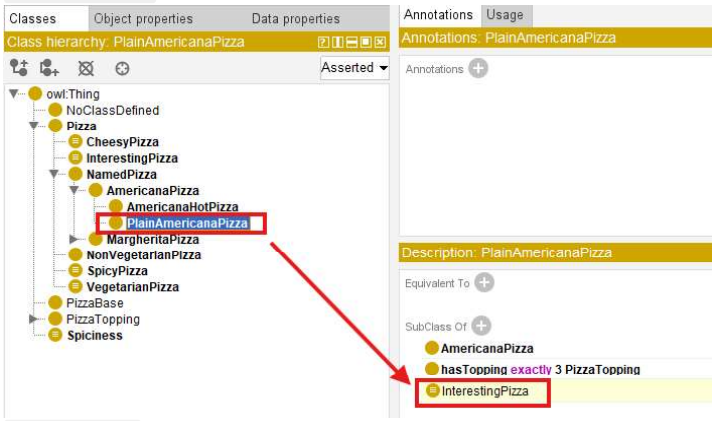



This tells the reasoner the count without requiring it to derive the count from existentials and disjointness. It is more direct but less flexible – if the pizza's topping set changes, the cardinality axiom must be updated manually.

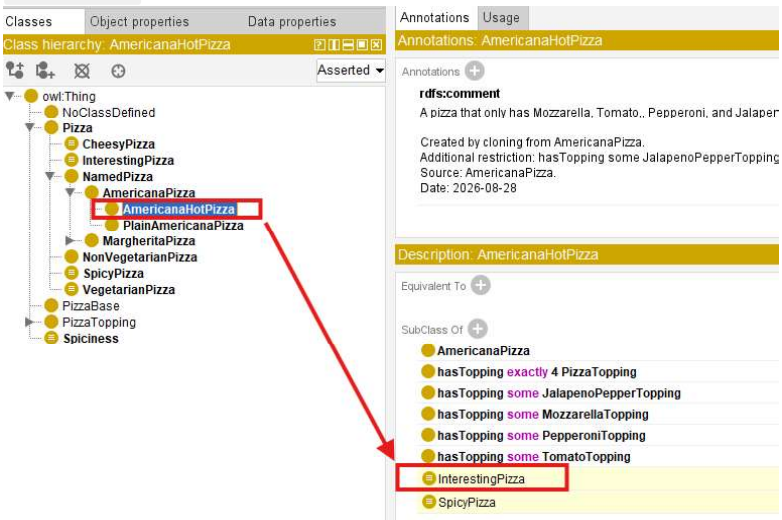
**Synchronize reasoner and observe**

Both `PlainAmericanaPizza` and `AmericanaHotPizza` are classified directly as subclass of `InterestingPizza`:

- `PlainAmericanaPizza`




- `AmericanaHotPizza`




**Which to use.** In this exercise, the closure-based approach (Approach 1) succeeded. But in production ontologies, when the reasoner does not fire the intended inference, Approach 2 is a legitimate fallback. Both approaches are used in practice; the choice depends on whether the engineer values derivability (Approach 1) or robustness (Approach 2).

**Key Insight:**

> "**When the reasoner refuses to derive what the axioms entail**, asserting the intended fact directly is a legitimate engineering fallback. The primitives are the correct pattern; the fallback is the pragmatic response."

## 26.3.5 Understanding Protégé UI Behavior: Why the Description Panel Can Mislead

Protégé is a powerful environment, but its UI displays information from multiple views that do not always agree. Two behaviors in particular confuse new users and can lead to wrong conclusions about what the reasoner has actually derived. We encountered both during Exercise 26. This section explains them together, because they share a common root.

### 26.3.5.1 The Mechanics of Conversion: Why "SubClass Of" Becomes Empty

When you select **Edit > Convert to defined class**, a subtle but crucial change occurs in the Protégé interface:

- **What Protégé does.** Converting a class to a defined class automatically *moves* the restrictions or superclasses from the **SubClass Of** section up into the **Equivalent To** section. Consequently, the **SubClass Of** box for that class will often appear completely empty immediately after conversion.
- **Why this happens logically.** A *primitive* class only states necessary conditions (it is a subclass of something). A *defined* class states necessary *and* sufficient conditions (`EquivalentTo`). By shifting `hasTopping min 3 PizzaTopping` into `EquivalentTo`, the reasoner now knows it can both recognize pizzas that fit this rule automatically and ensure full logical equivalence.
- **How to verify.** If you notice your axioms have "disappeared" from *SubClass Of*, check the **Equivalent To** field to confirm they safely relocated there. Once you start the reasoner, any class instance meeting this criteria will correctly and dynamically inherit its classification under the target class.

This behavior is expected and correct. The axioms are not lost; they are reorganized into their logically stronger form.

### 26.3.5.2 The Inferred-Ancestor Behavior: Why a Subclass May Not Show Its Inferred Parent

A second, more subtle behavior emerges when the reasoner infers a superclass **transitively** — that is, through an intermediate class.

Consider a class `A` with $A \sqsubseteq B$ and $B \sqsubseteq C$. The reasoner derives $A \sqsubseteq C$. In Protégé:

- The **inferred class hierarchy tree** correctly shows `A` nested under `C` (as well as under `B`). This is the authoritative global view of the inferred hierarchy.
- The **Description panel** for `A` may show only $A \sqsubseteq B$, not $A \sqsubseteq C$. The Description panel displays the asserted axioms of the selected class and some inferred axioms — but not necessarily all transitive entailments. Whether $A \sqsubseteq C$ appears depends on the Protégé version and settings.

**This behavior can cause a false alarm.** During Exercise 26, we initially believed `AmericanaHotPizza` and `PlainAmericanaPizza` were not classified as `InterestingPizza`, because their Description panels did not show `InterestingPizza` in the `SubClass Of` field. In fact, the reasoner had correctly inferred the relationship — the inferred class hierarchy tree showed all four pizzas nested under `InterestingPizza`. The apparent failure was a UI display artifact, not a reasoning failure.

### 26.3.5.3 The Underlying Principle: Local View vs. Global View

The two behaviors share a root: **Protégé's Description panel is a *local* view; the inferred class hierarchy tree and the DL Query tab are *global* views.**

| View | What it shows | When to use it |
|---|---|---|
| **Description panel** | Asserted axioms of the selected class, plus some inferred axioms (version-dependent) | Inspecting what *you* wrote about a class; checking the definition's form |
| **Inferred class hierarchy tree** | The reasoner's full subsumption hierarchy — every inferred parent and child relationship | Verifying classifications; inspecting where the reasoner placed a class |
| **DL Query tab** | Direct answers to queries about subclasses, superclasses, instances | Confirming a specific inference; getting a clean list of inferred subclasses |

When these views seem to disagree, the principle is: **trust the global view.** The inferred class hierarchy tree and the DL Query tab are the reasoner's output. The Description panel is a projection of that output onto the selected class.

### 26.3.5.4 A Verification Recipe

When you are unsure whether a class has been correctly classified by the reasoner, use this sequence:

1. **Switch to the inferred class hierarchy tree.** Locate the candidate parent class (e.g., `InterestingPizza`) and expand it. The class should appear as a child if it was inferred.
2. **If you need a clean list, use the DL Query tab.** Enter the class expression (e.g., `InterestingPizza`), check the **Subclasses** box, and click **Execute**. The result lists all inferred subclasses.
3. **If you want to see the class's own inferred ancestors, select the *candidate parent* and inspect its inferred children —** or use the DL Query tab with the child class as the query and check **Superclasses**.

> **A Protégé UI behavior**
>
> The Description panel shows a local view of a class. When a superclass is inferred transitively (through another superclass), it may not appear in the Description panel, even though the reasoner has derived the relationship. To see the reasoner's full output for a class, use the inferred class hierarchy tree or the DL Query tab.

**Key Insight:**

> "Protégé shows you many views of the same ontology. The Description panel tells you what you said about a class. The inferred class hierarchy tree tells you what the reasoner derived. When they seem to disagree, trust the tree — or better, verify in a third view (the DL Query tab). The reasoner's output is in the tree; the local view can mislead."

#### 26.3.5.5 Applying the Lesson

The lesson is not specific to Protégé. Any ontology development environment has this split between local views (what the engineer asserted) and global views (what the reasoner derived). Knowing which view to consult — and knowing when the two can legitimately disagree — is part of the semantic engineering discipline.

In **Exercise 26**, the two views temporarily disagreed. Investigating the discrepancy revealed a real behavior of the reasoner that had been correct all along. Understanding the discrepancy is what closed the exercise — not a change to the ontology, but a change to how we read it.

## 26.4 The Anatomy of an Executable Ontology

We have now introduced all five execution primitive of Stage 7. In this section we step back and view them as a single architecture -- the **anatomy** of an executable ontology.

The purpose of the anatomy is not to introduce new constructs. It is to show how the five primitives we have built up across **Chapter 24, 25, and 26** fit together, and to name the layers they form.

### 26.4.1 The Five Primitives

The five execution primitives, and the exercises that introduced them:

| # | Primitive | Exercise | Role |
|---|---|---|---|
| 1 | **Universal Restriction** ( `only` ) | Ex 22 | Defines the boundary: what values are permitted |
| 2 | **Closure Axiom** | Ex 23 | Closes the boundary: what values exhaustively exist |
| 3 | **Enumerated Class** | Ex 24 | Closes the value domain: the finite set of possible values |
| 4 | `hasValue` **Restriction** | Ex 25 | Binds the value: the specific value to act on |
| 5 | **Cardinality Restriction** | Ex 26 | Enforces completeness: how many values are required |

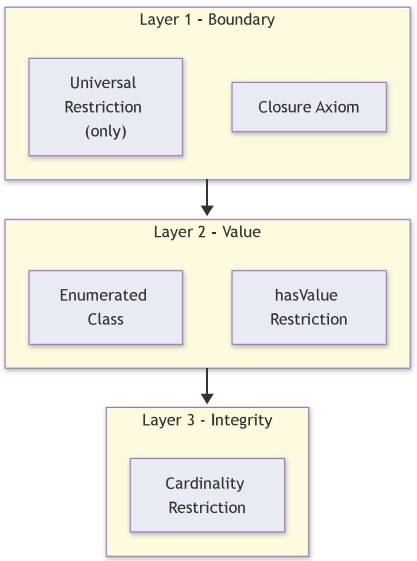
Read down the list, and Stage 7 reads as a single sentence:

> *First define what is permitted. Then close the boundary. Then close the value domain. Then bind the specific value. Then guarantee the count.*

That sentence is the anatomy of an executable ontology.

### 26.4.2 The Three Layers

The five primitives organize into three layers, each building on the one before it:




**Layer 1 -- Boundary** (Chapter 24). The universal restriction and the closure axiom define *what* values are permitted and *what* values exhaustively exist. This layer answers the question: *what is the world?*

**Layer 2 -- Value** (Chapter 25). The enumerated class and the `hasValue` restriction close the *set of possible values* and bind a *specific value* to be acted on. This layer answers: *what value fires the trigger?*

**Layer 3 -- Integrity** (Chapter 26, this Chapter). The cardinality restriction guarantees that the required *number* of values is present. This layer answers: *how much data do I need before I can act?*

Each layer depends on the one before it:

- **Without the boundary layer,** the value layer has no closed domain to draw from. A `hasValue` restriction that binds a property to a specific value is only meaningful if the value's type is exhaustively known.
- **Without the value layer,** the integrity layer has no specific value to count. A cardinality restriction that guarantees "at least three toppings" is only meaningful if the *kinds* of toppings that count are already closed.
- **Without the integrity layer,** the boundary and value layers are correct but incomplete: the ontology knows what is permitted and which value to act on, but cannot guarantee the data required to act is present.

The layering is not arbitrary. It reflects the logical dependency between the primitives. Boundary closure enables value binding. Value binding enables count. Count completes the executable specification.

**Key Insight:**

> "Executable knowledge is not a single axiom. It is three layers, built in order, each one making the next possible.
>
> - The boundary layer says what is closed.
> - The value layer says what to act on.
> - The integrity layer says when to act."

## 26.4.3 The Anatomy in the EKA Framework

In the EKA framework (introduced in earlier chapters), Stage 7 of Semantic Knowledge Development Lifecycle (SKDL) establishes the $\Theta$ **(Trigger)** and $\Phi$ **(Execution)** layers. The anatomy of the five primitives maps directly onto these layers.

$\Theta$ **(Trigger) -- what fires.** The $\Theta$ layer is the set of semantic conditions that fire a trigger. It is built from:

- Universal restrictions (what is permitted)
- Closure axioms (what exhaustively exists)
- Enumerated classes (what value domain is bounded)
- `hasValue` restrictions (what specific value fires)

The $\Theta$ layer is not a single axiom. It is the *conjunction* of boundary and value axioms that together define a class whose instances fire a trigger. When `PlainAmericanaPizza` is classified as an `InterestingPizza`, the $\Theta$ layer has fired.

$\Phi$ **(Execution) -- what executes.** The $\Phi$ layer is the mechanism that, once the trigger has fired, executes the corresponding action. In the ontology itself, this appears as the cardinality restriction, which guarantees that the data required for execution is complete. Beyond the ontology, $\Phi$ is the runtime system that reads the classified instance and takes the corresponding action.

The integrity layer's contribution to $\Phi$ is subtle but essential. Without it, the trigger might fire on incomplete data. With it, the trigger fires only when the data is sufficient. The cardinality restriction is what *binds* the trigger to the execution: it says "this condition fires only when the data is complete enough to act on."

**Key Insight:**

> "The boundary and value layers are the semantic side of the trigger. The integrity layer is the operational side. Together, they establish the $\Theta$-Trigger and $\Phi$-Execution layers that close the loop between reasoning and action."

## 26.4.4 A Minimal Worked Example

Let us see the anatomy in action, using the `Pizza` ontology.

**Boundary** (Exercise 22, Exercise 23).

- `VegetarianPizza` $\equiv$ `Pizza and (hasTopping only (CheeseTopping or VegetableTopping))`. The universal restriction says "every topping must be cheese or vegetable."
- The closure axiom on `MargheritaPizza` $\equiv$ `hasTopping only (MozzarellaTopping or TomatoTopping)` says "exactly mozzarella and tomato are the toppings."

Together, they establish that anything called a `VegetarianPizza` has only vegetarian toppings -- and that `MargheritaPizza` and `SohoPizza` are provably vegetarian.

**Value** (Exercise 24, Exercise 25).

- `Spiciness` $\equiv$ `{Hot, Medium, Mild}` closes the spiciness domain.
- `JalapenoPepperTopping` $\sqsubseteq$ `hasSpiciness value Hot` binds a specific topping to a specific value.

`SpicyPizza` $\equiv$ `Pizza and (hasTopping some (hasSpiciness value Hot))` fires when a pizza has at least one hot topping.

**Integrity** (Exercise 26).

- `InterestingPizza` $\equiv$ `Pizza and (hasTopping min 3 PizzaTopping)` fires when a pizza has at least three toppings.

The cardinality restriction guarantees that the count is provable -- once the boundary and value layers have made distinctness provable.

**Key Insight:**

> "Compose the three layers, and the ontology can answer: *'Is this pizza both spicy and interesting?'* That is an actionable query. The answer is derivable. It is not intuition -- it is a decision."

### 26.4.5 What the Anatomy Enables

A **descriptive ontology** tells us what is true. An **executable ontology** what to *do*.

The anatomy enables a small but profound shift:

- **Before Stage 7**, the ontology could classify `AmericanaHotPizza` as `SpicyPizza`. But that classification did not trigger anything -- it was a static fact in a library.
- **After Stage 7**, the ontology can classify `AmericanaHotPizza` as `SpicyPizza` *and* as `InterestingPizza`, *and* guarantee that the count and value conditions are met. The classification is now a dynamic *decision*, not a fact.

The anatomy is what makes this shift possible. Without the boundary layer, classification is open-ended. Without the value layer, classification has no specific trigger. Without the integrity layer, the trigger fires on incomplete data. All three together make the ontology **executable**.

**Key Insight:**

> "A descriptive ontology knows. An executable ontology decides. The anatomy of the five primitives is what turns knowledge into decision."

### 26.4.6 From Anatomy to Perspective

The anatomy gives us the *structure* of an executable ontology. But it does not, by itself, explain *why* this structure works. That requires four complementary perspectives:

1. **Mathematical** (§26.5) -- the logical foundations of closure, enumeration, binding, and cardinality.
2. **Historical** (§26.6) -- how executability evolved from expert systems to modern rule engines.
3. **Industrial** (§26.7) -- how executable ontologies are applied in manufacturing, compliance, and process automation.
4. **EKA** (§26.8) -- how the five primitives map to the $\Theta$-Trigger and $\Phi$-Execution layers of the Executable Knowledge Architecture.

The following four sections develop each perspective in turn. Together with the anatomy, they give you a complete picture of Stage 7 -- not just what the primitives are, but why they matter.

We begin with the mathematical perspective -- the logical foundation on which everything else rests.

## 26.5 Mathematical Perspective

Before examining each primitive, we need a shared mental model: **OWL is set theory.**

- A **class** is a set of individuals. `Pizza` is the set of all pizzas.
- An **individual** is an element of a set. `MargheritaPizza1` is an element of `MargheritaPizza`.
- A **(object) property** is a binary relation. `hasTopping` relates pizzas to toppings.
- A **restriction** is a set defined by a condition on the property's value.
- An **axiom** is a constraint on the sets: "the set `VegetarianPizza` is a subset of `Pizza`" is a constraint.

This is not an analogy. It is the formal semantics of OWL. Every OWL axiom has a precise set-theoretic interpretation, and the reasoner computes the logical consequences of those set-theoretic constraints.

With this model in mind, each of the five primitives becomes a specific *set operation* directly.

### 26.5.2 Closure as Set Restriction

The closure axiom:

```
Class: MargheritaPizza
   SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

is a **set restriction** on the property's range.

Formally, `hasTopping only X` says:

> For every individual `p` in the class, every value `t` related to `p` via `hasTopping` is an element of `X`.

In set-theoretic notation:

$$\forall p \in \text{MargheritaPizza} : \text{HasTopping}(p) \subseteq \text{MozzarellaTopping} \cup \text{TomatoTopping}$$

This is a **universal restriction**. It constrains the *range* of `hasTopping` for every instance of the class. It does not say anything about how many value exist -- only that whatever exists must be in the specified set.

**What closure does mathematically?**

It transforms an *open* set of possible values into a *closed* set. Before the closure, `hasTopping(MargheritaPizza1)` could contain any topping; after the closure, it can only contain mozzarella and tomato. The set of possible values is bounded.

**What closure does not do?**

It does not tell the reasoner *how many* values exist. The closure of `hasTopping(MargheritaPizza1)` to `{mozzarella, tomato}` is compatible with zero values (an empty subset), one value, or two values. It does not force the existence of any particular value.

> This is why the closure axiom alone is not sufficient for cardinality-based classification. The closure restricts the *range*, but the cardinality requires the *count*. The two are orthogonal dimensions of the property's extension.

## 26.5.3 Enumeration as Finite Extension

The enumerated class:

```
Class: Spiciness
   EquivalentTo: {Hot, Medium, Mild}
```

defines a class whose extension si a **finite set**.

Formally:

$$\text{Spiciness} = \{\text{Hot}, \text{Medium}, \text{Mild}\}$$

This is a *definition by listing*. It says: the class `Spiciness` contains exactly these three individuals, and no others.

**What enumeration does mathematically?**

It converts an *open class* (whose membership is unknown) into a *closed set* (whose membership is fully specified). Before the enumeration, the reasoner knows `Spiciness` is a class; after the enumeration, it knows the exact extension.

**Why enumeration is stronger than asserted membership?**

If we simply asserted that `Hot`, `Medium`, and `Mild` are instances of `Spiciness`, the reasoner would know they are *members*. But it would not know they are the *only members* -- under OWA, there could be other instances not yet asserted.

The enumeration axiom $\text{Spiciness} \equiv \{\text{Hot}, \text{Medium}, \text{Mild}\}$ closes this gap. It says: for any individual `x`, $\text{x} \in \text{Spiciness}$ if and only if $\text{x} \in \{\text{Hot}, \text{Medium}, \text{Mild}\}$. This is an *equivalence*, not just an implication. It excludes any individual outside the enumerated set.

**Mathematical consequence:**

Enumeration makes the class extension **decidable**. Given any individual `x`, the reasoner can determine in constant time whether $\text{x} \in \text{Spiciness}$: check whether `x` is one of the three enumerated individuals. Without enumeration, membership in `Spiciness` might require reasoning over other axioms.

**Why this matters for execution?**

An executable rule that fires on "any spiciness" can now enumerate all possible cases (`Hot`, `Medium`, `Mild`). The rule is guaranteed to be *exhaustive* -- every value it might encounter has been enumerated. This is a prerequisite for deterministic execution.

## 26.5.4 `hasValue` as Singleton Binding

The `hasValue` restriction:

```
Class: JalapenoPepperTopping
   SubClassOf: hasSpiciness value Hot
```

binds the property `hasSpiciness` to the specific individual `Hot`.

Formally:

$$\forall j \in \text{JalapenoPepperTopping} : \text{hasSpiciness}(j) = \{\text{Hot}\}$$

This is a **singleton binding** -- a restriction of the property's value to a set with exactly one element.

**What `hasValue` does mathematically?**

It fixes the property's value to a specific individual. Where the closure axiom restricted the *range* to a set, and enumeration fixed a *class's extension*, `hasValue` fixes a *property's value* to a singleton.

**Why `hasValue` is stronger than `only`?**

The restriction `hasSpiciness only Spiciness` would say "any spiciness value must be a Spiciness." But that's compatible with $\text{hasSpiciness}(\text{j}) = \emptyset$ (no value). The `hasValue` restriction says the value is `Hot` -- exactly that one individual, no ambiguity.

Formally, `p value v` is equivalent to `p only {v}` combined with `p some {v}` -- it requires both the range restriction and the existence.

**Mathematical consequence:**

`hasValue` makes the property's value **deterministic**. Given an individual $j \in \text{JalapenoPepperTopping}$, its `hasSpiciness` value is fully determined: it is `Hot`. This is what enables the trigger semantics.

**Why this matters for execution?**

A trigger that fires on "spiciness is Hot" requires not just that the property *could* be Hot, but that it *is* Hot. The `hasValue` restriction provides that certainty.

## 26.5.5 Cardinality as Bounded Extension

The cardinality restriction:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

bounds the **size** of the property's extension.

Formally:

$$\forall i \in \text{InteresitngPizza} : |\text{hasTopping}(i) \cap \text{PizzaTopping}| \geq 3$$

This says: for every instance of `InterestingPizza`, the set of its `PizzaTopping`-typed toppings has at least three elements.

**What cardinality does mathematically?**

It constrains the *number* of values a property takes. The three forms are:

- `min n`: $|\texttt{hasTopping(i)}| \geq \texttt{n}$ (at least `n`)
- `max n`: $|\texttt{hasTopping(i)}| \leq \texttt{n}$ (at most `n`)
- `exactly n`: $|\texttt{hsdTopping(i)} = \texttt{n}$ (exactly `n`)

**The qualified form.**

The restriction we wrote use `min 3 PizzaTopping` -- a *qualified* cardinality. It says "at least three toppings *of type* `PizzaTopping`." Because `hasTopping has range PizzaTopping`, every topping counts, and the qualification is trivially satisfied. But in general, the qualified form is more expressive -- it allows counting a subset of the property's values.

**The distinctness requirement.**

Counting is only meaningful if the elements being counted are **distinct**. The reasoner must be able to prove that the values of `hasTopping(i)` are pairwise different. This is what makes disjointness axioms (which we examined in §26.3.4) essential for cardinality: without them, the reasoner cannot prove that the toppings being counted are actually distinct individuals.

**Mathematical consequence.**

Cardinality requires that the ontology be able to prove **distinctness** of the values. This is where the set theory becomes subtle. Under the Open World Assumption, existential restrictions prove existence but not distinctness. Without additional axioms (disjointness, unique names, or explicit `differentFrom`), the reasoner cannot count.

This is precisely the lesson of **Exercise 26**: closure alone (which restricts *what*) is not sufficient for cardinality (which counts *how many*). Distinctness is what bridges the two.

## 26.5.6 The Mathematical Arc of Stage 7

Read across the five primitives as set operations, and Stage 7 becomes a single mathematical progression:

| Primitive | Set Operation | Effect |
|---|---|---|
| **Universal Restriction (`only`)** | Subset restriction | Constrains the *range* of a property |
| **Closure Axiom** | Complete subset restriction | Closes the *boundary* -- no values outside the set |
| **Enumerated Class** | Finite extension definition | Fixes a class's extension to a *finite set* |
| **`hasValue` Restriction** | Singleton binding | Fixes a property's value to a *single individual* |
| **Cardinality Restriction** | Bounded extension | Constrains the *size* of a property's extension |

Each operation constrains a different dimension of the same underlying structure:

- **Range**: what values are permitted (`only`, closure).
- **Extension**: what values a class contains (enumeration).
- **Binding**: what specific value a property takes (`hasValue`).
- **Count**: how many values a property has (cardinality).

Together, the four dimensions fully specify the executable conditions of a class.

**The mathematical foundation of the anatomy**. The three-layer structure of §26.4 is not arbitrary. It reflects the *logical dependency* between the operations:

1. **Boundary closure** (Layer 1) must precede value binding, because binding a value requires knowing what values are permitted.
2. **Value binding** (Layer 2) must precede count, because counting a property requires knowing which specific values are being counted.
3. **Cardinality** (Layer 3) closes the specification, because the count guarantees that the data required for the trigger is present.

The mathematics mirrors the engineering:

> each layer is a set of operation, and the operations compose in a specific order.

## 26.5.7 Distinctness -- The Underlying Condition

There is one mathematical condition that spans all five primitives, and it deserves explicit treatment: **distinctness.**

A set has distinct elements. An individual is not two individuals. But under the Open World Assumption, the reasoner does not automatically know that two named individuals are different -- **it must be told**.

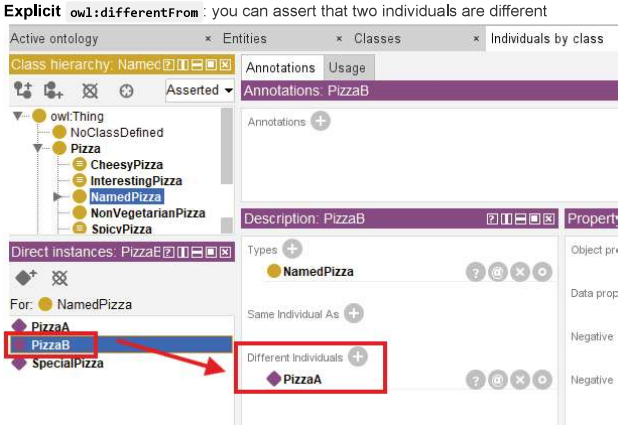
**Distinctness enters the picture at multiple points --**

- **Enumeration**: the enumeration `Spiciness` $\equiv$ `{Hot, Medium, Mild}` requires that `Hot`, `Medium`, and `Mild` are three distinct individuals. If they were the same, the enumeration would contain one element, not three.
- `hasValue` : the `hasValue` restriction binds a property to `Hot`. If `Hot` were the same individual as `Medium`, the binding would be to the same individual as if we had bound to `Medium`.
- **Cardinality**: the count "at least 3 toppings" requires that the toppings being counted are distinct. Without distinctness, three toppings could be one topping counted three times.

**How distinctness is established?**

There are three ways to prove distinctness in OWL:

1. **Unique Name Assumption (UNA)**: if UNA holds, every named individual is distinct. But OWL does not assume UNA.
2. **Explicit** `owl:differentFrom` : you can assert that two individuals are different




3. **Disjointness of classes**: if $A \perp B$, and $x \in A$, $y \in B$, then $x \neq y$. This is the form of distinctness we used in **Exercise 26**.

The disjointness approach is the most powerful for cardinality-based reasoning. By declaring $\text{MozzarellaTopping} \perp \text{PepperoniTopping}$, $\text{MozzarellaTopping} \perp \text{TomatoTopping}$, and $\text{PepperoniTopping} \perp \text{TomatoTopping}$, we make the reasoner able to prove that the three toppings in `AmericanaPizza` are distinct. Without those disjointness declarations, the reasoner would see three existential restrictions but would not be able to count them.

**Why distinctness is the mathematical heart of Stage 7?**

Every execution primitive depends on being able to prove distinctness -- because every primitive is a set operation, and sets contain *distinct* elements. Distinctness is what makes **set operations** well-defined.

This is why the closure and cardinality layers of the anatomy work together, and why the **Stage 1** taxonomy matters. The taxonomy declares the disjointness that makes distinctness provable. The closure and cardinality declare the set operations that *use* distinctness. The **Stage 7** primitives are not independent of **Stage 1** -- they are set operations *one the sets that **Stage 1** defines*.

**Key Insight:**

> "Every execution primitive is a set operation. Every set operation requires distinctness. Distinctness is what makes the sets well-defined. In an ontology, distinctness is what the taxonomy provides --and Stage 7 is what puts it to use."

## 26.5.8 What the Mathematics Teaches

The set-theoretic view of **Stage 7** is not just academic, it changes how you approach ontology engineering:

1. **Debugging is easier with the mathematical model**: when a classification does not fire, the first question is: *which set operation failed?* Is the range not closed (missing closure)? Is the extension not determined (missing enumeration)? Is the value not bound(missing `hasValue`)? Is the count not provable (missing cardinality or distinctness)? The mathematical model gives you a systematic checklist.

2. **Composition is compositional**: each primitive is a separate set operation. They compose without interfering -- but only if they *order* respects the dependency. Closure before binding, binding before count. This is why the anatomy's three layers must be built in order.
3. **Distinctness is the hidden requirement**: the most common failure model in **Stage 7** is a cardinality that doesn't fire because distinctness is not provable. When this happens, go back to **Stage 1** and check the taxonomy -- it is almost always the source of the missing disjointness.
4. **The distinctness between primitives are subtle but real**: `only` restricts the range. `value` binds the value. `some` asserts existence. `min` / `max` / `exactly` bound the count. These are four *different* set operations, and confusing them is the source of many modeling errors.

**Key Insight:**

> "OWL is set theory applied with engineering discipline. Every primitive restricts a set in a specific way. Understanding which set is being restricted, and how, is the difference between an ontology that works and one that merely looks correct."

### 26.5.9 The Mathematical Arc Across Stage 7

Read across the three layers of the anatomy (§26.4.2), the mathematics tells a complete story:

**Layer 1 -- Boundary.** The set-restriction operations (`only`, closure) constrain *what* values are permitted. They establish the *domain* of the property's extension.

**Layer 2 -- Value.** The extension and binding operations (enumeration, `hasValue`) fix *which* values the property takes. They establish the *range* of the extension to a specific individual.

**Layer 3 -- Integrity.** The count operation (cardinality) constrains *how many* values the property has. It establishes the *size* of the extension.

Each layer operates on the same underlying set -- the property's extension -- but constrains a different dimension of it. Together, they fully specify the property: its domain, its range, and its size.

The mathematics is what makes this composition rigorous. Without the set-theoretic view, the three layers look like three unrelated modeling patterns. With the set-theoretic view, they are three operations on the same mathematical object, and their composition is what makes the ontology executable.

This is the mathematical foundation of **Stage 7**. The remaining perspectives -- historical, industrial, and EKA -- build on this foundation. Each of them shows how the primitives have been used, in different eras and different domains, to solve the same class of problem: **how to encode knowledge with sufficient precision to act on it.**

**Key Insight:**

> "The five primitives are not five separate tools. They are five set operations on the same mathematical object -- the property's extension. Composed in the right order, they fully specify what it means to execute."

## 26.6 Historical Perspective

From expert systems and production rules to modern rule engines and executable knowledge

## 26.7 Industrial Perspective

ISA-95 / Rhize and the reasoning-to-execution pipeline in manufacturing

## 26.8 EKA Perspective -- Stage 7 Establishes $\Theta$ + $\Phi$

## 26.9 Engineering Guidelines for Executable Knowledge

## 26.10 Key Concepts

## 26.11 Chapter Summary

## 26.12 Looking Ahead -- Toward SKDL Synthesis

---

Last updated at 2026-10-26

# [Idea] Chapter 27 - SKDL Synthesis & Volume 4 Conclusion

1. Core Architectural Recap of Volume 4
    - From Ontology to Executable Models: Review the journey through Protégé, Noe4j, and graph prototyping, typing together the foundational architecture, semantic governance, validation, and reasoning.
    - The Semantic Stack: Synthesize how the layers -- ranging from schema definition and semantic enrichment to governance and automated reasoning -- form a cohesive, end-to-end knowledge architecture.
2. Comprehensive Summary of SKDL
    - Language & Paradigm Integration: Summarize how SKDL bridges conceptual ontologies with graph implementations
        - touch base the two different graph approach
    - Key Artifacts & Patterns: Highlight reusable modeling patterns, constraint structures, and rule sets established across chapter 17 (starting), then chapter 18 through 23
3. Best Practices in Semantic Engineering
    - Design Principles: Maintainability, scalability, and modularity in ontology and graph design
    - Governance & Quality Control: consolidate lessons learned from semantic validation (SHACL/constraints) and reasoning pipelines
4. Volume 4 Closure & Transition to Volume 5
    - Architectural Milestone: Reflect on the completion of the core semantic modeling lifecycle.
    - Bridge to Advanced Topics: Preview how Volume 5 will build upon this foundation to tackle advanced integration, enterprise tooling, and production-grade deployment scenarios (spanning Sections 5.1 through 10.0).

Last updated at 2026-09-30

# Appendix A -- Chapter Mapping with Exercises

| Volume # | Chapter | Exercise | Tutorial Section | Video | Notes |
|---|---|---|---|---|---|
| Vol-1 | Chapter 00~03 | N/A | N/A | N/A | Foundations & EKA Introduction |
| Vol-1 | Chapter 04 | Exercise 1, 2, 3 | Section 4.0 | 04 | Creating Named Classes |
| Vol-1 | Chapter 05 | Exercise 4 | Section 4.1 | 05 | Using a Reasoner |
| Vol-1 | Chapter 06 | Exercise 5 | Section 4.2 | 06 | Disjoint Classes |
| Vol-1 | Chapter 07 | Exercise 6 | Section 4.3 | 07 | Create Class Hierarchy |
| Vol-1 | Chapter 08 | N/A | N/A | N/A | Conceptual Modeling Foundations |
| Vol-2 | Chapter 09 | Exercise 7 | Section 4.4 | 09 | Object Properties |
| Vol-2 | Chapter 09 | Exercise 8 | Section 4.5 | 09 | Creating Property Hierarchies |
| Vol-2 | Chapter 10 | Exercise 9 | Section 4.6 | 10 | Inverse Properties |
| Vol-2 | Chapter 11 | Exercise 10 | Section 4.7 | 11 | Property Characteristics |
| Vol-2 | Chapter 12 | (Added by Author) | N/A | N/A | Property Characteristics Deep Dive |
| Vol-2 | Chapter 13 | Exercise 11, 12 | Section 4.9 | 13 | Domain & Range |

| Volume # | Chapter | Exercise | Tutorial Section | Video | Notes |
|---|---|---|---|---|---|
| Vol-3 | Chapter 14 | Exercise 13 | Section 4.10.2 | 14 | Existential Restrictions |
| Vol-3 | Chapter 15 | N/A | Section 4.10.2 | 14 | Semantic Description (Theory) |
| Vol-3 | Chapter 16 | N/A | Section 4.10.2 | 14 | Description Logic Foundations |
| Vol-4a | Chapter 17 | N/A | N/A | N/A | SKDL Introduction |
| Vol-4a | Chapter 18 | Exercise 14 | Section 4.10.3 | 15 | Stage 1 — Conceptual Modeling |
| Vol-4a | Chapter 19 | Exercise 15 | Section 4.10.3 | 16 | Stage 2 — Semantic Description |
| Vol-4a | Chapter 20 | Exercise 16 | Section 4.10.3 | 17 | Stage 3 — Knowledge Reuse (Cloning Pattern) |
| Vol-4a | Chapter 20 | Exercise 17 | Section 4.10.3 | 18 | Stage 3 — Knowledge Reuse (Practicing Specialization) |
| Vol-4a | Chapter 21 | Exercise 18 | Section 4.10.3 | 19 | Stage 4 — Semantic Governance (Disjointness) |
| Vol-4a | Chapter 22 | Exercise 19 | Section 4.10.4 | 20 | Stage 5 — Semantic Validation (Probe Class) |
| Vol-4b | Chapter 23 | Exercise 20 | Section 4.11 | 21 | Stage 6 — Semantic Reasoning (Primitive Classes) |
| Vol-4b | Chapter 23 | Exercise 21 | Section 4.11 | | Stage 6 — Semantic Reasoning (Defined Classes) |
| Vol-4b | Chapter 24-26 | Exercise 22-26 | N/A | | Stage 7 — Executable Knowledge (Introduction) |
| Vol-4b | Chapter 27 | N/A | N/A | | SKDL Synthesis & Volume 4 Closure |

| Volume # | Chapter | Exercise | Tutorial Section | Video | Notes |
|---|---|---|---|---|---|
| Vol-5 | | Exercise 28 | Section 5.2 | | Data Property Values & Classification |
| Vol-5 | | Exercise 29 | Section 9.1 | | DL Queries |
| Vol-5 | | Exercise 30 | Section 10.0 | | SWRL & SQwRL Rules |
| Vol-6 | | | | | Advanced Topics |
| Vol-7 | | | | | EKA & AI Integration |

Last Updated at: 2026-10-18

# Contributors

| Name / GitHub Handle | Contribution Type | Issue/Discussion | Chapter(s) | Date | Memo |
|---|---|---|---|---|---|
| Timothy Cook | Issue Report | #66 | 20 & 21 | 2026-09-06 | Continue technical review |
| 000wahab000 | Pull Request Contribution | PR #57 | Various | 2026-09-04 | Contributed updates and improvements via Pull Request #57. |
| Jah-yee | Pull Request Contribution | PR #54 | Various | 2026-09-04 | Submitted the repository's first community-contributed pull request, helping enhance project collaboration and open-source engagement. |
| sarcanon | Question / Issue Report | #39-#50 | Various | 2026-08-28 | Many detail issue reports |
| LaraAcuna | Question / Content Improvement | #26 | 13 | 2026-08-12 | Raised a thoughtful question about modeling OR semantics in Domain and Range (Chapter 13). Her question prompted the addition of a |

| Name / GitHub Handle | Contribution Type | Issue/Discussion | Chapter(s) | Date | Memo |
|---|---|---|---|---|---|
| | | | | | complete guide to `owl:unionOf`, separate properties, and local restriction patterns, including new screenshots, RDF examples, and a comparison table. This contribution significantly improved the practical guidance of Chapter 13. Fixed in v0.44.0. |
| Michael DeBellis | Refinement Suggestion | #19 | 15 | 2026-07-01 | Proposed revised definition for VegetarianPizza pattern to align with common real-world interpretation (include cheese/dairy). Pending. |
| Michael DeBellis | Terminology Clarification | #20 | Multiple | 2026-07-01 | Highlighted need to clearly distinguish |

| Name / GitHub Handle | Contribution Type | Issue/Discussion | Chapter(s) | Date | Memo |
|---|---|---|---|---|---|
| | | | | | "Annotation Properties" (reasoner-invisible) from logical property axioms. Pending. |
| Michael DeBellis | Critical Distinction / Error Report | #21 | 00, 08 | 2026-07-01 | Identified fundamental conflation between Property Graphs (e.g., Neo4j) and Native RDF Graphs (e.g., AllegroGraph, GraphDB, Stardog). Major architectural correction required. Pending. |
| Michael DeBellis | Question / Technical Review | #9 | 00, 06, 14 | 2026-06-13 | Clarified reasoner inference materialization distinction (Protégé UI behavior vs. production triplestores). Fixed in v0.39.0. |
| Michael DeBellis | Error Report | #10, #11, #12, #13, #14, #15 | Various | 2026-06-16 | Multiple error reports including |

| Name / GitHub Handle | Contribution Type | Issue/Discussion | Chapter(s) | Date | Memo |
|---|---|---|---|---|---|
| | | | | | OWL modeling errors, contributor attribution, proofreading, and other corrections. See individual issues for details. |
| mlungsta89 | Bug Report | #3 | — | 2025-09-15 | Reported reasoner getting stuck issue. Closed. |
| nikokaoya | Feedback | #2 | — | 2024-06-26 | Positive feedback and encouragement. Closed. |
| Your name here | Error report / Pull request / Question | — | X | YYYY-MM-DD | Brief Memo |

Last updated: 2026-09-11